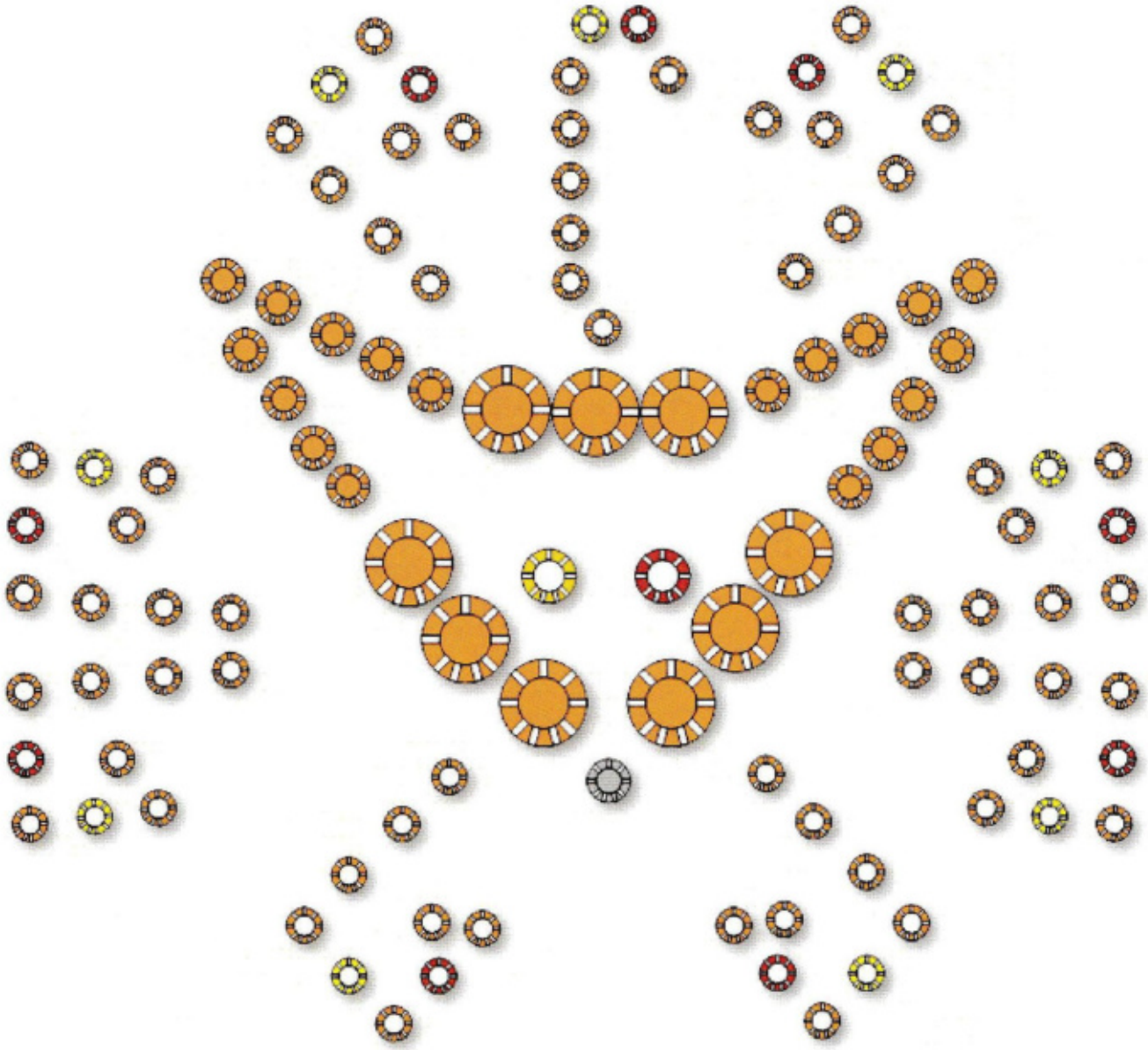
А.Н. Алфимцев

МУЛЬТИАГЕНТНОЕ ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ



BAUMANPRESS
ИЗДАТЕЛЬСТВО
МГТУ им. Н.Э. БАУМАНА

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»

А.Н. Алфимцев

Мультиагентное обучение с подкреплением

Учебное пособие

Москва
ИЗДАТЕЛЬСТВО
МГТУ им. Н.Э. Баумана
2021

УДК 004.85
ББК 32.813
А53

Издание доступно в электронном виде по адресу
https://bmstu.press/catalog/item/7198/

Факультет «Информатика и системы управления»
Кафедра «Информационные системы и телекоммуникации»

Рекомендовано Научно-методическим советом
МГТУ им. Н.Э. Баумана в качестве учебного пособия

Алфимцев, А. Н.

А53 Мультиагентное обучение с подкреплением : учебное пособие / А. Н. Алфимцев. — Москва : Издательство МГТУ им. Н. Э. Баумана, 2021. — 222, [2] с. : ил.

ISBN 978-5-7038-5616-1

Рассмотрены современные и классические алгоритмы одновременного машинного обучения множества агентов, основанные на теории игр, табличных, нейросетевых, эволюционных и роевых технологиях. Представлено последовательное развитие теоретической модели алгоритмов, базирующееся на марковских процессах принятия решений. Реализация алгоритмов выполнена на языке программирования Python с использованием библиотеки глубокого обучения PyTorch. Средой машинного обучения является компьютерная игра StarCraft II с интерфейсом кооперативного мультиагентного обучения SMAC.

Для магистрантов и аспирантов направления подготовки «Информатика и вычислительная техника».

УДК 004.85
ББК 32.813

Уважаемые читатели! Пожелания, предложения, а также сообщения о замеченных опечатках и неточностях Издательство просит направлять по электронной почте: info@baumanpress.ru

© МГТУ им. Н.Э. Баумана, 2021
© Оформление. Издательство МГТУ им. Н.Э. Баумана, 2021

ISBN 978-5-7038-5616-1

https://t.me/it_boooks

Предисловие

Учебное пособие предназначено для самостоятельного изучения магистрантами и аспирантами основных разделов дисциплины «Мультиагентные интеллектуальные системы» и имеет практико-ориентированный характер, обеспечивая возможность решения конкретных профессиональных задач.

Цель изучения дисциплины — формирование профессиональных компетенций, связанных с разработкой программного обеспечения для мультиагентного обучения, а также общепрофессиональных компетенций, необходимых для применения на практике новых научных принципов и методов машинного обучения.

В учебном пособии сформированы внутридисциплинарные и междисциплинарные связи, обеспечивающие преемственность знаний, полученных при изучении предшествующих дисциплин по основам программирования, а также непрерывность отдельных видов математической и технологической подготовки разработчика программного обеспечения.

Системность изложения материала обеспечена логической последовательностью и аргументированностью приводимых положений. Новые положения базируются на ранее изученных.

Учебное пособие состоит из пяти глав, каждая из которых включает девять типизированных разделов: «Классификация», «Модель», «Алгоритм», «Карта», «Технология», «Код», «Эксперимент», «Выводы», «Задачи для самоконтроля». В последовательном изложении по принципу «от простого к сложному» объединены теория и технология, поскольку для изучения и развития современного мультиагентного обучения недостаточно ограничиваться теоретическими исследованиями либо заниматься только программированием чужих изысканий.

В разделе «Классификация» рассматриваются общая идея главы и различные алгоритмы мультиагентного обучения, определяется место представляемых алгоритмов в системе научных координат.

В разделе «Модель» приводится основной используемый в главе математический аппарат.

В разделе «Алгоритм» анализируется основной алгоритм мультиагентного обучения, рассматриваемый в данной главе. Псевдокод алгоритма дается в авторской редакции, которая больше соответствует его реальной реализации.

В разделе «Карта» описывается карта игры StarCraft II, на материале которой проводится мультиагентное обучение в каждой главе. Раздел

«Технология» посвящен особенностям настройки компьютерной игры и используемых программных библиотек.

В разделе «Код» дается оригинальная реализация основного алгоритма, рассматриваемого в главе, на языке программирования Python.

В разделе «Эксперименты» приводятся результаты применения рассматриваемых в главе алгоритмов.

В разделе «Выводы» тезисно подводятся некоторые итоги.

В конце каждой главы в разделе «Задачи для самоконтроля» представлены задачи по программированию мультиагентного обучения, которые позволят закрепить рассмотренный в главе теоретический материал практическими навыками. Задачи даны по возрастанию сложности в порядке, соответствующем основным тезисам глав.

Гл. 1 учебного пособия посвящена независимому табличному Q-обучению. Алгоритмы обучения с подкреплением одними из первых были применены для мультиагентного обучения. Показаны основные преимущества и недостатки классического алгоритма Q-обучения.

В гл. 2 рассматривается мультиагентное обучение в матричных и стохастических играх с использованием теории игр. Несмотря на то что алгоритмы на основе теории игр объективно уступают современным алгоритмам, многие созданные в этот период фундаментальные концепции развиваются до сих пор.

В гл. 3 рассматриваются сильнейшие на сегодняшней день алгоритмы мультиагентного обучения, основанные на теории искусственных нейронных сетей. Показано, как алгоритмы IQN и MADDPG помогают добиться высоких результатов в микроменеджменте действий агентов в игре StarCraft II.

Считается, что агент может быть по-настоящему автономным, если он способен не просто достигать целей функционирования, но и создавать их. В гл. 4 анализируется современные принципы эволюционного и коэволюционного обучения.

Заключительная глава — гл. 5 — посвящена гомогенному кооперативному мультиагентному обучению, для которого иногда применяется термин «роевое обучение». Предполагается, что вдохновленные живой природой алгоритмы роевого обучения являются наиболее перспективной альтернативой алгоритмам нейросетевого обучения.

Каждую главу, а также введение и заключение предваряют эпиграфы, взятые из написанного в жанре фэнтези романа американского математика и писателя Юн Ха Ли «Гамбит девятихвостого лиса» (М.: Эксмо, 2018). В книге нестандартно спрогнозированы развитие мультиагентных технологий и роль человека в техногенном мире будущего. Главный герой романа учится эффективно использовать ресурсы и находить оптимальные стратегии действий, что отвечает основной идее представленного учебного пособия.

Введение

...Никто не устоял бы перед возможностью
использовать игру в качестве педагогического инструмента.
Юн Ха Ли. Гамбит девятихвостого лиса

В Национальной стратегии развития искусственного интеллекта одной из приоритетных целей является создание технологий распределенных коллективных систем. Для достижения этой цели необходимо решить задачи разработки программного обеспечения, математических моделей и алгоритмов мультиагентного обучения с подкреплением.

Мультиагентная система (*англ.* multi-agent system) — система, состоящая из нескольких взаимодействующих агентов.

Агент — компьютерная программа, которая расположена в некоторой среде и способна автономно действовать и обучаться в этой среде для достижения своих целей. Под средой обычно понимается все физическое и виртуальное пространство, находящееся вне агента.

Машинное обучение (*англ.* machine learning) — совокупность алгоритмов и моделей, которые позволяют агенту для достижения цели без предварительного программирования находить закономерности в данных, делать прогнозы, самостоятельно принимать и улучшать решения на основе полученного опыта взаимодействия.

Обучение с подкреплением (*англ.* reinforcement learning) — один из способов машинного обучения, особенностью которого является обучение агента в процессе взаимодействия со средой.

Мультиагентное обучение с подкреплением (*англ.* Multi-agent Reinforcement Learning, MARL) — совокупность алгоритмов и моделей машинного обучения с подкреплением, которые дают возможность обучать мультиагентные системы.

Мультиагентные системы как отдельная область дисциплины «Информатика и вычислительная техника» начала развиваться в конце XX в. В последние 20 лет интерес к этой области постоянно возрастает, о чем свидетельствуют более полумиллиона поисковых результатов в системе «Академия Google».

С одной стороны, этот рост связан с убеждением, что мультиагентные системы являются одной из перспективных парадигм разработки программного обеспечения. Смещение акцента с разработки приложений для персональных рабочих станций на разработку распределенных систем, взаимодействующих друг с другом дистанционно, привело к усложнению этих систем и приобретению ими свойств, характерных для мультиагентных систем.

С другой стороны, ключевым свойством агента, функционирующего в незнакомой среде, является способность к обучению на основе полученного опыта взаимодействия с этой средой или с другими агентами. Очевидные успехи глубокого одноагентного обучения в распознавательных и игровых задачах, достигнутые в последние годы, мотивируют исследователей и разработчиков использовать потенциал этих методов для мультиагентного обучения.

Мультиагентное обучение в настоящее время — фундаментальный компонент мультиагентной системы. С научной точки зрения исследование взаимодействия между множеством одновременно обучающихся агентов позволяет получить новые знания, которые могут быть полезны при анализе социальных структур общества, экономических конкурентных сред, в теории управления. С инженерной точки зрения одновременно обучающееся множество агентов может предоставить концептуально новый способ разработки коммуникационных систем, робототехники, транспортных систем, человеко-машинных интерфейсов, программного обеспечения.

В целом мультиагентный подход при построении системы в *сложной*, т. е. недетерминированной, динамической, непрерывной, *среде* дает следующие преимущества.

• *Робастность*. Мультиагентная система способна продолжить функционирование при сбоях в работе отдельных агентов.

• *Эффективность*. Параллельная обработка данных в мультиагентной системе автономными агентами ускоряет функционирование системы.

• *Адаптивность*. Децентрализованная мультиагентная система может динамически изменять свое поведение при изменениях в среде.

• *Инкапсулируемость*. Мультиагентная система модульна по своей природе, что позволяет гибко изменять ее структуру, абстрагировать данные и защищать внутренние реализации.

• *Масштабируемость*. При восприятии информации и принятии решения мультиагентная система не имеет ограничений по централизованному контролю данных, что дает возможность лучше справляться с увеличением рабочей нагрузки при добавлении ресурсов.

Однако применение методов одноагентного машинного обучения к мультиагентным системам приводит и к появлению новых вызовов.

Во-первых, мультиагентное обучение подвержено «проклятию размерности», вызванному экспоненциальным ростом пространства состояний-действий при увеличении числа агентов. Так, при табличном одноагентном обучении с подкреплением семи агентов с шестью возможными действиями для каждого состояния среды необходимо хранить 42 Q-значения. А при мультиагентном обучении совместным действиям нужно будет учесть 279 936 Q-значений.

Во-вторых, многие теоретические гарантии, которые существовали и формально доказывались при одноагентном обучении, при многоагентном обеспечении не действуют. Так, классическим алгоритмом Q-обучения

с подкреплением гарантируется сходимость к оптимальной стратегии для конечного марковского процесса принятия решений. Но при мультиагентном обучении агентам приходится довольствоваться не оптимальной стратегией, а некоторым усредненным равновесием, которого они достигли в действиях друг с другом и со средой.

В-третьих, выбор цели мультиагентного обучения, которую можно формально описать и автоматически проверить, не является столь очевидным решением, как при одноагентном обучении. На достижение цели максимизации награды агента при мультиагентном обучении с подкреплением могут влиять, например, результаты обучения других агентов. В связи с этим в практических задачах для формулирования цели мультиагентного обучения требуются отдельные, иногда уникальные, решения, касающиеся контроля стабильности обучения, адаптируемости к изменению поведения других агентов, устойчивости внутренней мотивации.

В-четвертых, нестационарность машинного обучения при мультиагентном обучении проявляется с иной стороны. Когда одновременно обучается несколько агентов, они постоянно подстраивают свои действия под действия других агентов, которые, в свою очередь, подстраиваются под них. Отдельной проблемой здесь является определение оптимальности стратегии, так как стратегия должна постоянно меняться в соответствии с изменяющимися стратегиями других агентов.

В-пятых, проблема эксплуатации-исследования также значительно усложняется при мультиагентном обучении. В случае, когда принять решение должен один агент, достаточно исследовать среду и можно начинать действовать, эксплуатировать полученный опыт. Но при обучении множества агентов этот индивидуальный переход одного агента от исследования к эксплуатации или наоборот неявно влияет на стратегии всех других агентов.

И наконец, шестой вызов связан с координацией действий агентов. Эффект от каждого действия агента зависит от действия или бездействия других агентов. В решении проблемы координации приходится постоянно балансировать между поиском эффективных совместных действий агентов и поиском оптимальных стратегий. Несмотря на то что эта проблема больше характерна для кооперативного мультиагентного обучения, отсутствие координации негативно влияет и на независимо обучающихся агентов.

В последние годы главные результаты машинного обучения обучающихся агентов достигнуты в компьютерных играх. Иногда результаты игроков-агентов сопоставимы с результатами игры профессиональных игроков-людей или даже их превосходят. Такие компьютерные игры, как стратегическая компьютерная игра StarCraft II, с точки зрения машинного обучения представляет собой сложную среду, в которой встречаются все актуальные вызовы мультиагентного обучения; в ней также можно безопасно промоделировать ситуации реального мира: частичный обзор информации, кооперативность или некооперативность взаимодействия,

децентрализованный микроменеджмент, большой объем входных данных, тысячи возможных действий, разнообразный набор сценариев, нестационарность обучения.

В связи с этим технологическая часть учебного пособия построена на базе компьютерной игры StarCraft II, разработанной кампанией Blizzard, и программной библиотеки мультиагентного обучения SMAC, разработанной исследовательской лабораторией Whiteson Оксфордского университета.

Это множество библиотек, которые выложены в открытом доступе на платформе Github. Примеры, приведенные в данном учебном пособии, также опубликованы в открытом виде, чтобы была возможность не только повторить рассматриваемые алгоритмы, но и провести модификацию предлагаемого решения, сравнить полученные результаты с результатами, приведенными в пособии.

Глава 1

Независимое табличное обучение

Календарная война — война сердец.
Однако числа способны влиять на сердца при условии,
что и те, и другие верные.
Юн Ха Ли. Гамбит девятихвостого лиса

1.1. Классификация

Первые методы мультиагентного обучения базировались на алгоритмах обучения с подкреплением. Особенностью *обучения с подкреплением* является обучение агента в процессе взаимодействия со средой. Алгоритмы обучения с подкреплением не требуют априорной информации о среде или какой-либо дополнительной модели среды. Агенты $i = 1, 2, ..., n$ обучаются в процессе взаимодействия, получая от среды положительные или отрицательные подкрепляющие сигналы $r_t^1, r_t^2, \ldots, r_t^n$ и новые состояния $s_{t+1}^1, s_{t+1}^2, \ldots, s_{t+1}^n$, зависящие от действий $a_t^1, a_t^2, \ldots, a_t^n$ агентов (рис. 1.1).

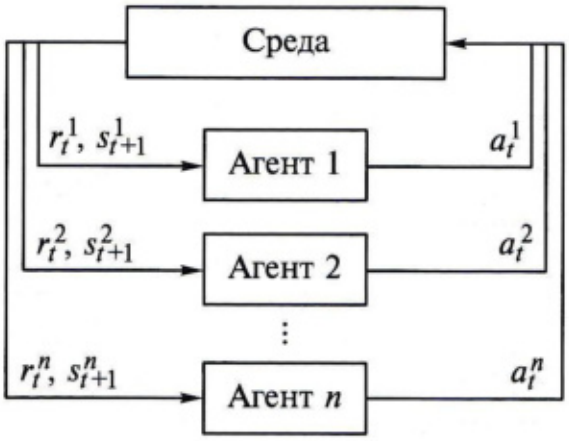


Рис. 1.1. Схема независимого мультиагентного обучения с подкреплением

Подкрепляющий сигнал менее информативен, чем в алгоритмах обучения с учителем, но все же дает некоторую информацию, в то время как в алгоритмах обучения без учителя такой сигнал отсутствует. При использовании алгоритмов обучения с подкреплением для мультиагентного обучения появляется возможность естественно совместить преимущество двух

подходов: автономное и адаптивное обучение множества агентов в сложных средах, способных децентрализованно достигать цели функционирования мультиагентной системы.

Несмотря на то что многие свойства одноагентного обучения не выполняются в случае мультиагентного обучения, в научной литературе были предложены и на практике апробированы разнообразные табличные алгоритмы мультиагентного независимого и совместного обучения (рис. 1.2). *Табличными* эти алгоритмы называют потому, что для хранения и обновления полученной в процессе обучения информации используют двухмерные таблицы данных.

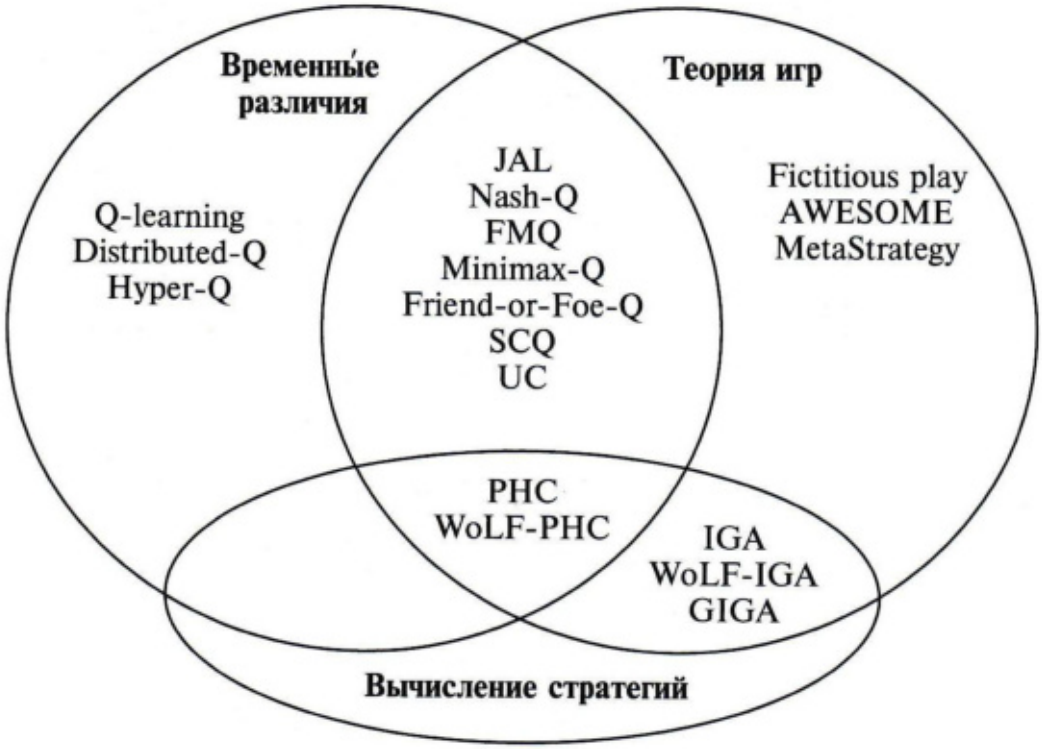


Рис. 1.2. Классификация табличных алгоритмов мультиагентного обучения с использованием принципов временны́х различий, теории игр и вычисления стратегий

В целом табличные алгоритмы мультиагентного обучения с подкреплением можно разделить на три класса: основанные на принципах временны́х различий; основанные на теории игр; позволяющие напрямую вычислять распределение вероятности для формирования стратегии действий.

Мультиагентное обучение с использованием теории игр, в том числе с вычислением стратегий, будет рассмотрено в гл. 2. В гл. 1 внимание уделено табличному алгоритму Q-обучения, в котором используется *принцип временны́х различий*. Этот принцип — один из центральных в безмодельном обучении с подкреплением, он определяет алгоритмы, с помощью которых

обучаются агенты на основе вычисления разности Q-значений или ценности состояний для смежных последовательных событий.

На принципе временны́х различий также основаны: алгоритм распределенного Q-обучения (*англ.* Distributed Q-learning), особенность которого — решение задач независимого мультиагентного обучения с исключительно положительными наградами от среды и невысокой вычислительной сложностью, сравнимой со сложностью классического алгоритма Q-обучения; алгоритм гипер-Q (*англ.* Hyper-Q), который позволяет строить модели действий других агентов и поэтому успешно адаптироваться к их нестационарному поведению, но именно вследствие этих дополнительных моделей подвержен «проклятию размерности».

В этой главе рассматривается полностью кооперативное мультиагентное обучение, при котором агенты имеют одинаковую функцию наград и награда для одного агента также является наградой для группы. Основное внимание уделено агентам, которые не взаимодействуют друг с другом и не имеют информации о действиях других агентов, — так называемым *независимо обучающимся* агентам. Такие агенты получили популярность в робототехнике в тех случаях, когда физически невозможно получить или обработать информацию о совместных действиях большого числа агентов.

1.2. Модель

Большинство алгоритмов одноагентного обучения с подкреплением изначально было разработано для математической модели МППР (марковских процессов принятия решений) (*англ.* Markov decision processes). *Марковский процесс принятия решений* — это кортеж (S, A, T, R), где S — конечное множество состояний (*англ.* States); A — конечное множество действий (*англ.* Actions) агента; $T : S \times A \times S \to [0;1]$ — функция переходов (*англ.* Transition), $R : S \to \mathbb{R}$ — функция наград (*англ.* Reward).

Марковский процесс принятия решений обладает марковским свойством, согласно которому будущие переходы и состояния полностью зависят только от текущего состояния. Вероятность перейти в состояние s' после выполнения действия a в состоянии s обозначают $T(s, a, s')$. Для всех действий a и всех состояний s и s' выполняется $0 \le T(s, a, s') \le 1$ и $\sum_{s' \in S} T(s, a, s') = 1$. Функция наград R возвращает награду $R(s, a, s')$ после выполнения действия a в состоянии s и перехода в состояние s'.

Применение МППР для мультиагентного обучения привело к появлению различных модификаций базового определения. В табл. 1.1. представлены определения МППР, разработанные с учетом стратегии агента π_i и множества стратегий агента Π_i для независимых и совместных состояний, действий и наград.

Таблица 1.1

Модифицированные МППР для мультиагентного обучения

Тип награды		
Тип действий	Независимая награда	Совместная награда
Независимые состояния		
Независимые действия	$(S_i, A_i, T, R_i, \Pi_i)$: $\pi_i \in \Pi_i, s_i \in S,$ $a_i \in A_i, r_i \in R_i$	(S_i, A_i, T, R, Π_i): $\pi_i \in \Pi_i, s_i \in S,$ $a_i \in A_i, r \in R$
Совместные действия	(S_i, A, T, R_i, Π_i): $\pi_i \in \Pi_i, s_i \in S,$ $a \in A, r_i \in R_i$	(S_i, A, T, R, Π): $\pi, s_i \in S,$ $a \in A, r \in R$
Совместные состояния		
Независимые действия	(S, A_i, T, R_i, Π_i): $\pi_i \in \Pi_i, s \in S,$ $a_i \in A_i, r_i \in R_i$	(S, A_i, T, R, Π_i): $\pi_i \in \Pi_i, s \in S,$ $a_i \in A_i, r \in R$
Совместные действия	(S, A, T, R_i, Π_i): $\pi_i \in \Pi_i, s \in S,$ $a \in A, r_i \in R_i$	(S, A, T, R, Π): $\pi, s \in S,$ $a \in A, r \in R$

Целью обучения в МППР является поиск оптимальной стратегии $\pi^*: S \to A$, которая задает действия агента, приводящие к получению максимальной награды. Качество стратегии определяется *функцией ценности* V^π (*англ.* Value). *Оптимальной стратегией* $\pi^*(s)$ называется стратегия, при которой $V^{\pi^*}(s) \geq V^\pi(s)$ для всех π и $s \in S$. *Функция ценности состояния для стратегии* π – $V^\pi(s)$ определяет общее количество награды, которую агент ожидает получить, начав из состояния s следовать стратегии π:

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma_t R(s_t) | s_0 = s\right],$$

где γ — параметр дисконтирования, $\gamma \in [0,1]$.

Однако функция ценности состояния не свидетельствует о том, какие конкретно действия следует выполнять агенту в определенном состоянии. Для этого задается *функция ценности действия для стратегии* π — $Q^\pi(s, a)$, определяющая общее количество награды, которую агент ожидает получить, если выполнит действие a в состоянии s, следуя стратегии π:

$$Q^\pi(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma_t R(s_t, a_t) | s_0 = s,\ a_0 = a\right],\ \gamma \in [0,1].$$

Функция ценности состояния $V^\pi(s)$ в обучении с подкреплением и в динамическом программировании обладает фундаментальным свойством,

заключающимся в том, что эту функцию можно вычислить рекурсивно с помощью уравнения Беллмана. Функция ценности состояния для любой стратегии π и любого состояния s вычисляется на основе полученной награды $R(s)$ и предполагаемой ценности следующего состояния $V^{\pi}(s')$ взвешенной вероятностью перехода в это состояние:

$$V_{t+1}^{\pi}(s) = R(s) + \gamma \sum_{s' \in S} T(s, \pi(s), s') V_t^{\pi}(s').$$

Рассмотрим результаты решения уравнения Беллмана на примере МППР, представленного в виде графа переходов на рис. 1.3. Марковский процесс принятия решений задает независимое поведение агента в некоторой среде, где агент перемещается по карте и ищет противников. В библиотеке SMAC для компьютерной игры StarCraft II у каждого агента информация о среде ограничена окружностью с радиусом девять квадратов игрового поля для наблюдения и радиусом шесть квадратов — для стрельбы. В связи с этим

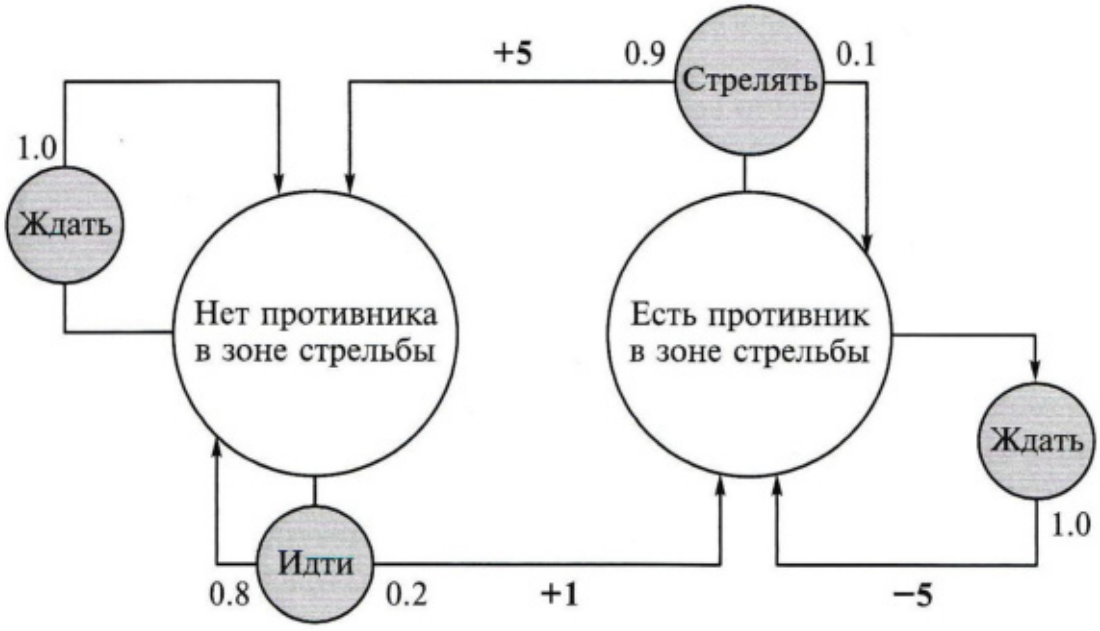


Рис. 1.3. Граф переходов марковского процесса принятия решений

зададим в МППР состояния S = {нет противника в зоне стрельбы, есть противник в зоне стрельбы}. Предположим, что если агент находится в первом состоянии s_1, то он может выполнить два действия A = {идти, ждать}. Если агент находится во втором состоянии s_2, то он может выполнить действия A = {стрелять, ждать}.

Вероятности переходов в первом состоянии s_1 при выполнении действий a_1 и a_2 определим как $T(s_1, a_1, s_2) = 0.2$; $T(s_1, a_1, s_1) = 0.8$; $T(s_1, a_2, s_1) = 1.0$. Вероятности переходов во втором состоянии s_2 при выполнении действий a_3 и a_4 определим как $T(s_2, a_3, s_1) = 0.9$; $T(s_2, a_3, s_2) = 0.1$; $T(s_2, a_4, s_2) = 1.0$.

Используя стандартный прием обучения с подкреплением, зададим цели агента с помощью дизайна наград МППР. Для того чтобы агент не просто искал противников, а вступал с ними в бой, определим награду как $R(s_1, a_1, s_2) = 1$ и $R(s_2, a_3, s_1) = 5$. В то же время, если агенту-морпеху противостоит зерглинг, то в качестве одной из целей необходимо предусмотреть сохранение дистанции с противником, потому что зерглинг — это наземный юнит ближнего боя, который может наносить урон, только находясь рядом с агентом. В данном МППР такую предосторожность можно задать лишь отрицательной наградой $R(s_2, a_4, s_2) = -5$.

В результате решения уравнения Беллмана на 10 итерациях с параметром $\gamma = 0.9$ ценность первого состояния МППР составит $V^{\pi}(s_1) = 0.243$, ценность второго состояния $V^{\pi}(s_2) = 20.804$. Ценность действий в первом состоянии рассчитана как $Q^{\pi}(s_1, a_1) = 6.868$; $Q^{\pi}(s_1, a_2) = 5.677$, во втором состоянии — как $Q^{\pi}(s_2, a_3) = 9.622$; $Q^{\pi}(s_2, a_4) = 3.317$. Таким образом, оптимальной стратегией π^* для данного МППР является в состоянии s_1 выполнение действия a_1 и в состоянии s_2 — выполнение действия a_3.

1.3. Алгоритм

Одним из прорывов в области одноагентного обучения с подкреплением стала разработка, основанная на принципах временны́х различий офлайнового алгоритма управления, больше известного в настоящее время как *алгоритм Q-обучения* (*англ.* Q-learning). Алгоритму Q-обучения не требуется модель среды в виде априорной информации о функции переходов и функции наград, алгоритм применим в средах со стохастическими переходами и наградами. Для любого конечного МППР в алгоритме Q-обучения находится оптимальная стратегия в отношении максимизации ожидаемой общей награды. Символ Q в названии алгоритма и его потомков происходит от *англ.* Quality — качество и означает качество действия, предпринятого в некотором состоянии.

Первой попыткой развития алгоритма Q-обучения применительно к мультиагентному обучению стала его реализация для независимого табличного обучения, при которой агент, использующий алгоритм, действовал рационально, но сходимость алгоритма к оптимальной стратегии не гарантировалась. При независимом Q-обучении агент игнорировал существование других агентов, предполагая, что все награды среды́ и переходы стационарны и обладают марковским свойством. Приобретаемый опыт агента i в таком случае имеет вид (a_i, r) где a_i — действие, выполненное агентом i; r — полученная награда.

Алгоритм независимого табличного Q-обучения для агента i, где α — скорость обучения; ε — параметр эксплуатации-исследования; Ne — число эпизодов, приведен ниже:

1. Инициализировать
   $\alpha \in (0, 1]$; $\gamma \in (0, 1]$; $\varepsilon \in (0, 1)$; $Ne \in [1, N]$;
   $i \leftarrow 1, \ldots, n$; $a \in A_i$;
   $Q(s, a) \leftarrow 0$, $\forall s \in S$.
2. Цикл
3. Получить состояние s.
4. Определить возможные действия $a \in A_i$ в состоянии s.
5. Выбрать возможное действие a в соответствии с максимальным значением $Q(s, a)$ и с учетом параметра ε.
6. Выполнить возможное действие a.
7. Получить награду r и следующее состояние s'.
8. Обновить значение $Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$.

Главной инновацией алгоритма Q-обучения стала вдохновленная уравнением Беллмана формула итерационного обновления Q-значений на основе полученной награды и максимальной ценности действия в следующем состоянии (шаг 8). При этом Q-значения $Q(s, a)$ обычно инициализируются нулями (шаг 1) и в процессе обучения хранятся в виде Q-таблиц. В Q-таблице строки соответствуют всем возможным состояниям МППР, столбцы — всем возможным действиям (см. разд. 1.4).

Однако несмотря на инновативность алгоритма Q-обучения, в него была заложена проблема «жадного» выбора действий a^* как максимального значения $Q(s, a^*) = \max Q(s, a_i)$ (шаг 5). Иначе говоря, в алгоритме не исследуются другие варианты действий и не проверяется их эффективность для работы в динамической среде, что особенно важно при длительном периоде обучения. Альтернативой постоянного «жадного» выбора действий стала концепция ε-«жадного» выбора действий, в соответствии с которой вводится дополнительный параметр эксплуатации-исследования $\varepsilon \in (0, 1)$. В результате с небольшой вероятностью $1 - \varepsilon$ в алгоритме Q-обучения выбираются случайные действия для исследования пространства действий и с вероятностью ε осуществляется «жадный» выбор действий, т. е. эксплуатируется полученный опыт (порядок значений ε и $1 - \varepsilon$ может быть инвертирован и зависит от реализации).

Еще одной проблемой длительного обучения, которую мультиагентное обучение получило в наследство от одноагентного, стала невозможность точно определить, результатом какого действия в будущем становится та или иная награда среды. В связи с этим следующей концепцией, активно используемой в алгоритме Q-обучения, стала концепция дисконтированной награды, определяемой параметром $\gamma \in [0, 1]$. Параметр дисконтирования γ задает степень влияния будущих наград на обновление Q-значений $Q(s, a)$. Если установить значение параметра дисконтирования γ близким к нулю,

то первые полученные награды будут существенно влиять на обновление Q-значений $Q(s, a)$. Если установить значение параметра дисконтирования γ близким к единице, то отдаленная во времени награда будет оказывать большее влияние на обновление Q-значений $Q(s, a)$. Но в целом попытка максимизации ближайших наград приводит к уменьшению общей ожидаемой награды.

Помимо этого, в структуру алгоритма Q-обучения заложен параметр скорости обучения α, который определяет, в какой степени вновь полученный опыт переопределяет старую информацию. Высокая скорость обучения (значение α близко к единице) позволяет обучаться быстрее, но может привести к пропуску глобального минимума. При низкой скорости обучения (значение α близко к нулю) агент обучается медленнее, что может привести к нахождению лишь локального минимума. По этой причине параметр скорости обучения α иногда задают в виде формулы, зависящей от эпизода обучения или от промежуточных результатов обучения.

1.4. Карта

Независимое табличное мультиагентное обучение в компьютерной стратегической игре StarCraft II с использованием программной библиотеки SMAC в гл. 1 проводилась на карте 2m2zFOX.SC2Map, представленной на рис. 1.4. В начале каждого эпизода на карте слева, в точке Team 1, автоматически создаются два агента юнита морпеха, которыми управляет обучающий алгоритм, справа, в точке Team 2, автоматически создаются два агента юнита зерглинга, которыми управляет компьютер, используя стандартные правила. Длительность одного полного эпизода игры составляет 3 с.

Юнит — игровая единица (персонаж) в компьютерных стратегических играх. Морпех — базовый военный юнит терранов. Имеет 45 единиц

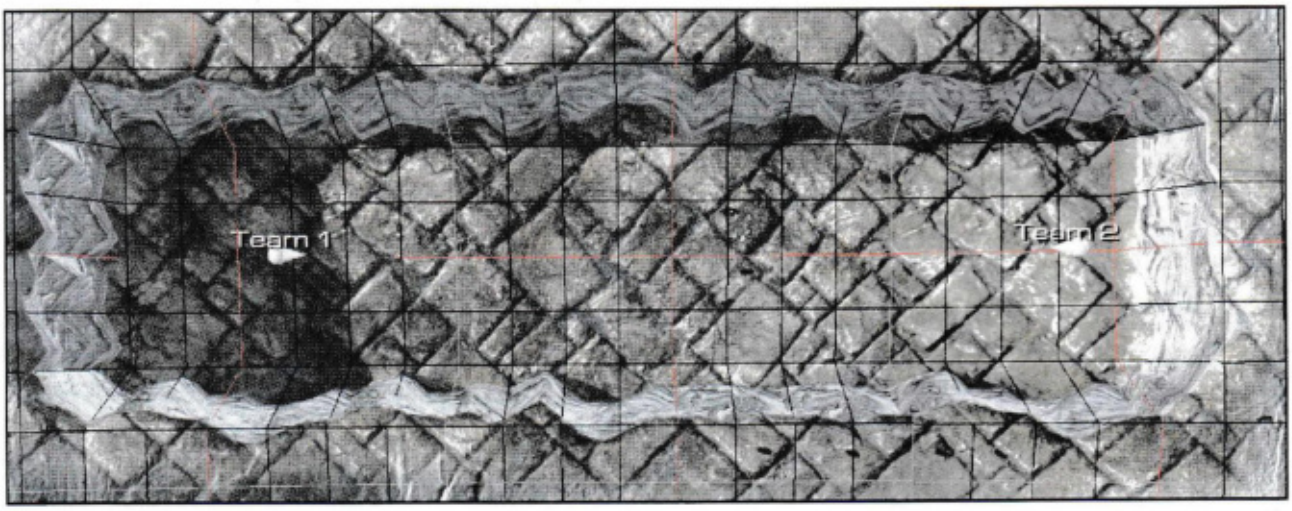


Рис. 1.4. Внешний вид карты 2m2zFOX.SC2Map в редакторе StarCraft II Editor

здоровья. Атакует наземные и воздушные юниты на расстоянии, нанося шесть единиц урона (дальность атаки — пять квадратов игрового поля). Стратегически используется при атаке пехотой и при защите от массовых атак противника. Изначально не имеет брони.

В отличие от морпеха, зерглинг расы зергов обладает меньшим количеством здоровья — 35 единиц и атакой пять единиц с дальностью 0.1. Стратегически применяется для ранних массовых атак на базы противника и разведки. Наиболее опасен для противника при использовании в комбинациях с другими юнитами.

Поскольку в этой главе рассматривается табличное обучение, для данной карты необходимо разработать МППР, представленный миром-сеткой SMAC.

Мир-сетка (*англ.* word-grid) — это двухмерное прямоугольное пространство, состоящее из ячеек одного размера (*Nx*, *Ny*). В мире-сетке агент может перемещаться только по горизонтали и по вертикали на одну клетку за одно действие. Агент всегда занимает только одну ячейку в мире-сетке и может взаимодействовать только с объектами, находящимися в данной клетке (но для действия «стрелять» в мире-сетке SMAC разрешено взаимодействие на нескольких клетках). Мир-сетка — довольно абстрактная среда, но она позволяет упростить мультиагентное обучение и пренебречь побочными факторами при проведении экспериментов. Кроме того, мир-сетка моделирует важные элементы сложных сред: зависимость действия от состояния, качественные и количественно-взвешенные переходы между состояниями, мгновенные и отложенные награды.

Мир-сетка SMAC 2×8 состоит из 16 клеток, соответствующих состояниям МППР $S = \{0, 1, 2, ..., 15\}$ (рис. 1.5). Соответствие координат карты состояниям МППР представлено в табл. 1.2. Для каждого агента задан свой МППР. Два агента $i = 1, 2$ начинают обучение в состояниях $s_1^3 = 3$ и $s_2^{11} = 11$, не имея информации о месторасположении противников и о существующих наградах. Для того чтобы применить алгоритм Q-обучения, необходимо для каждого агента также задать Q-таблицу для хранения выученных оценок действий в определенных состояниях МППР. В табл. 1.3 приведен пример обученной Q-таблицы первого агента, где реализована оптимальная стратегия (ячейки таблицы затемнены и пронумерованы, № 1–4).

Каждый агент может выполнить действия следующих типов: $A_i = \{\text{no} - \text{op}$, stop, NORTH, SOUTH, EAST, WEST, attack № 1, attack № 2$\}$; здесь действие «no – op» — («нет операций») означает, что агент погиб; действие stop задает полное бездействие агента; четыре действия NORTH, SOUTH, EAST, WEST определяют перемещения агента соответственно вверх, вниз, вправо, влево; два действия attack № 1, attack № 2 задают атаку агента — стрелять в первого или во второго противника (см. рис. 1.5). Каждый агент может совершить только одно действие в единицу игрового времени. Эпизод игры заканчивается, когда истекает время эпизода, погибают оба противника или оба агента.

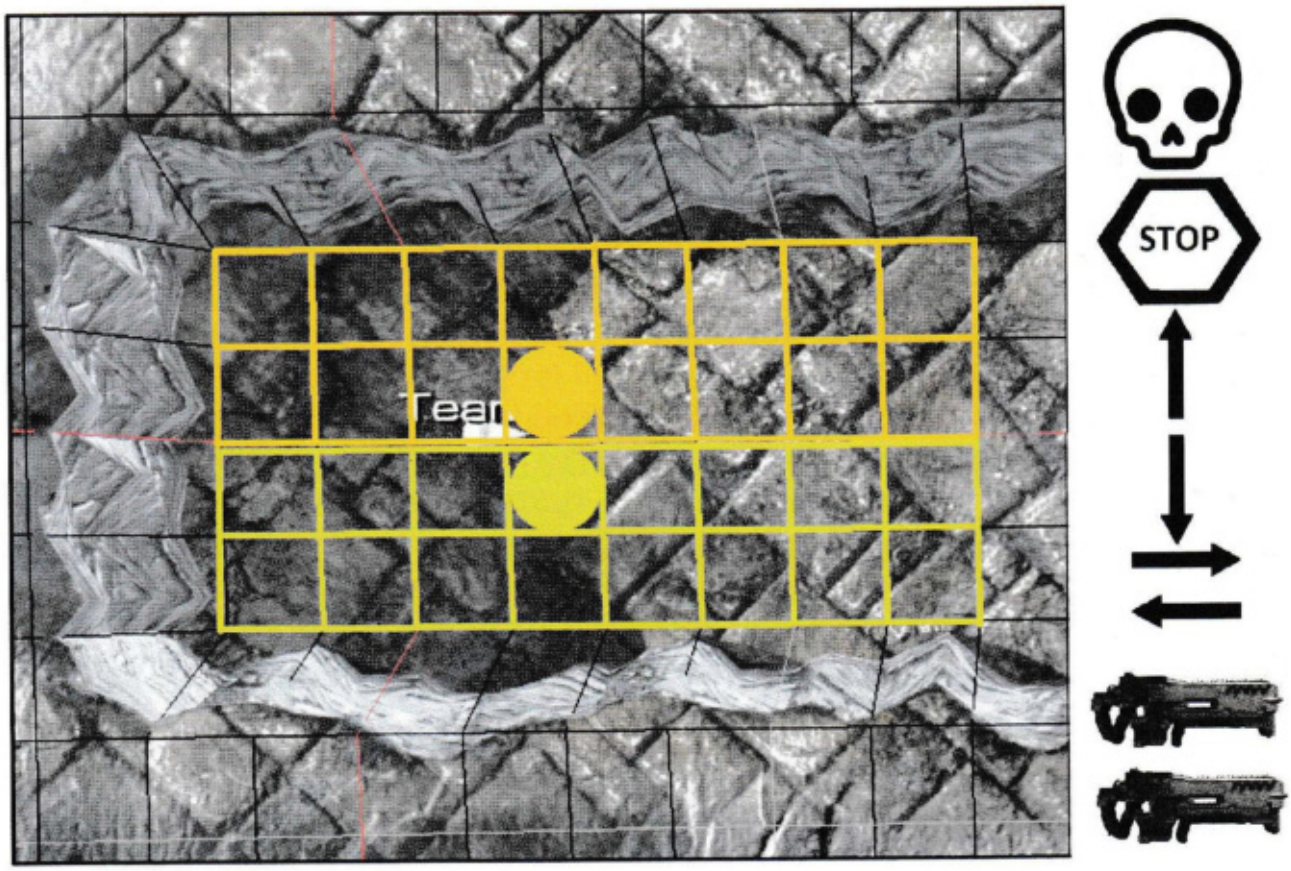


Рис. 1.5. Мир-сетка SMAC для двух агентов

Главными целями агентов в мире-сетке SMAC являются получение максимальной награды и достижение максимального процента побед. Оптимальной стратегией π_n^*, позволяющей достичь целей за отведенный промежуток времени, является стратегия «бежать вправо и стрелять при появлении противника».

Награда изначально задается программной библиотекой SMAC и вычисляется по формуле $R = \frac{(\text{reward_battle} + \text{reward_win})}{(\text{max_reward} / \text{reward_scale_rate})}$, где reward_win — награда +1 и −1 соответственно за победу и поражение в эпизоде; reward_scale_rate — коэффициент шкалы наград, равный 20.

Награда за битву reward_battle вычисляется по формуле reward_battle = = delta_enemy+delta_deaths–delta_ally, где delta_enemy — урон, нанесенный здоровью или броне противников; delta_deaths — отношение числа потерянных союзников к числу убитых противников; delta_ally — урон, нанесенный здоровью или броне агентов.

Максимальная награда карты max_reward вычисляется по формуле max_reward = n_enemies×reward_ death_value+reward_win, где n_enemies — число противников на данной карте; reward_ death_value — награда за уничтожение юнита противника, равная 10.

В итоге максимальная награда без дисконтирования для рассматриваемого мира-сетки SMAC составляет 20.381, максимальный процент побед равен 1.0, что соответствует 100 %.

Таблица 1.2

Соответствие координат x, y карты состояниям s МППР

s_2^0 $6 < x < 7$ $16.2 < y < 17$	s_2^1 $7 < x < 8$ $16.2 < y < 17$	s_2^2 $8 < x < 8.9$ $16.2 < y < 17$	s_2^3 $8.9 < x < 9.1$ $16.2 < y < 17$
s_2^4 $9.1 < x < 10$ $16.2 < y < 17$	s_2^5 $10 < x < 11$ $16.2 < y < 17$	s_2^6 $11 < x < 12$ $16.2 < y < 17$	s_2^7 $12 < x < 13.1$ $16.2 < y < 17$
s_2^8 $6 < x < 7$ $15.5 < y < 16.2$	s_2^9 $7 < x < 8$ $15.5 < y < 16.2$	s_2^{10} $8 < x < 8.9$ $15.5 < y < 16.2$	s_2^{11} $8.9 < x < 9.1$ $15.5 < y < 16.2$
s_2^{12} $9.1 < x < 10$ $15.5 < y < 16.2$	s_2^{13} $10 < x < 11$ $15.5 < y < 16.2$	s_2^{14} $11 < x < 12$ $15.5 < y < 16.2$	s_2^{15} $12 < x < 13.1$ $15.5 < y < 16.2$
s_1^0 $6 < x < 7$ $15 < y < 16.5$	s_1^1 $7 < x < 8$ $15 < y < 16.5$	s_1^2 $8 < x < 8.9$ $15 < y < 16.5$	s_1^3 $8.9 < x < 9.1$ $15 < y < 16.5$
s_1^4 $9.1 < x < 10$ $15 < y < 16.5$	s_1^5 $10 < x < 11$ $15 < y < 16.5$	s_1^6 $11 < x < 12$ $15 < y < 16.5$	s_1^7 $12 < x < 13.1$ $15 < y < 16.5$
s_1^8 $6 < x < 7$ $14 < y < 15$	s_1^9 $7 < x < 8$ $14 < y < 15$	s_1^{10} $8 < x < 8.9$ $14 < y < 15$	s_1^{11} $8.9 < x < 9.1$ $14 < y < 15$
s_1^{12} $9.1 < x < 10$ $14 < y < 15$	s_1^{13} $10 < x < 11$ $14 < y < 15$	s_1^{14} $11 < x < 12$ $14 < y < 15$	s_1^{15} $12 < x < 13.1$ $14 < y < 5$

Таблица 1.3

Q-таблица мира-сетки SMAC для первого агента

Состояния	Действия							
	0	1	2	3	4	5	6	7
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.433	0.343	0.332	0.532	0.245	0.201	0.249
3	0.0	0.515	0.520	0.518	0.557 № 1	0.522	0.231	0.523
4	0.0	0.528	0.531	0.530	0.589 № 2	0.528	0.532	0.531

Окончание табл. 1.3

Состояния	Действия							
	0	1	2	3	4	5	6	7
5	0.0	0.621	0.632	0.530	0.635 № 3	0.631	0.633	0.549
6	0.0	0.0	0.863	0.0	0.874 № 4	0.0	0.170	0.0
7	0.0	0.0	0.439	0.250	0.432	0.285	0.933 № 5	0.863
8	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
10	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
11	0.0	0.0	0.510	0.501	0.588	0.521	0.213	0.522
12	0.0	0.0	0.511	0.523	0.575	0.321	0.514	0.535
13	0.0	0.0	0.761	0.0	0.799	0.320	0.787	0.765
14	0.0	0.0	0.313	0.251	0.431	0.283	0.838	0.831
15	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

На рис. 1.6–1.10 проиллюстрированы различные ситуации в мире-сетке SMAC: начало эпизода игры, противники атакуют справа вне зоны стрельбы (см. рис. 1.6); противник достигает зоны стрельбы вначале первого, а затем и второго агента (см. рис. 1.7, 1.8); один из агентов погиб, и возникла проигрышная позиция «двое на одного» (см. рис. 1.9); агенты наконец-то выучили правильную стратегию (см. рис. 1.10).

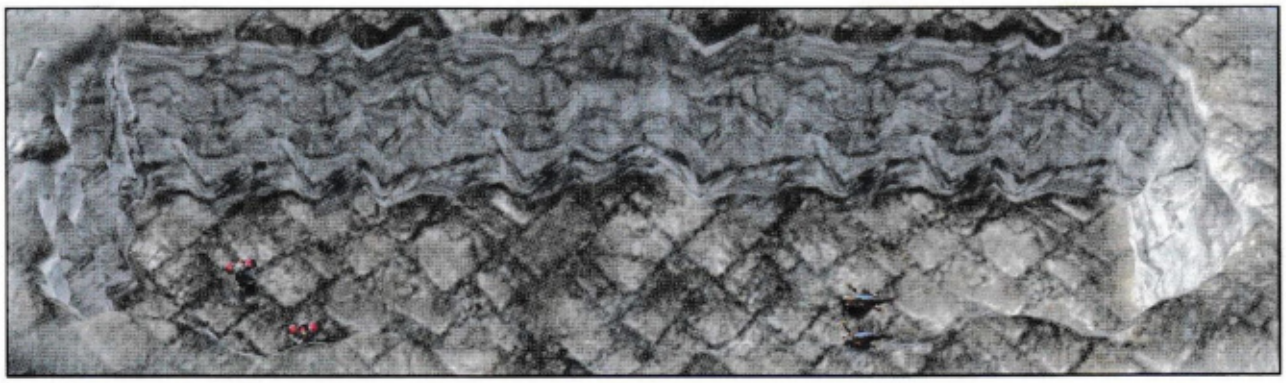


Рис. 1.6. Начало эпизода в мире-сетке SMAC

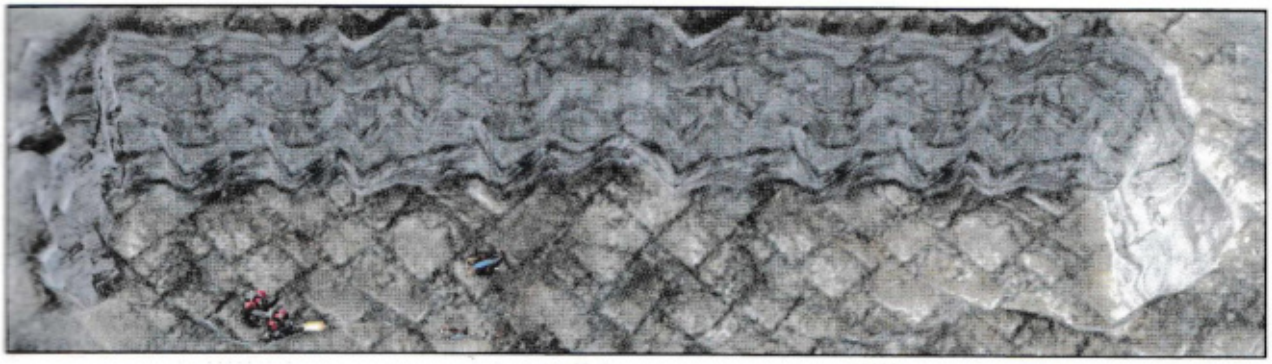


Рис. 1.7. Противник в зоне стрельбы первого агента

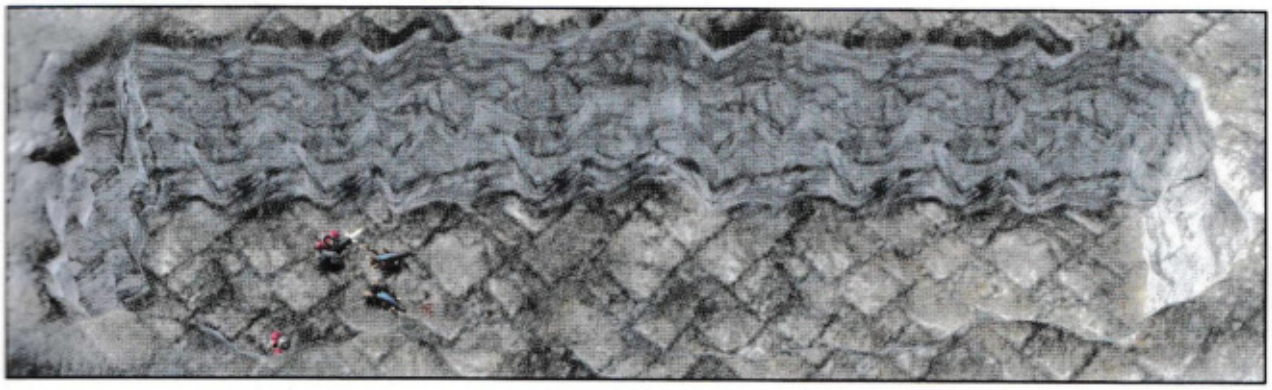


Рис. 1.8. Противник в зоне стрельбы второго агента

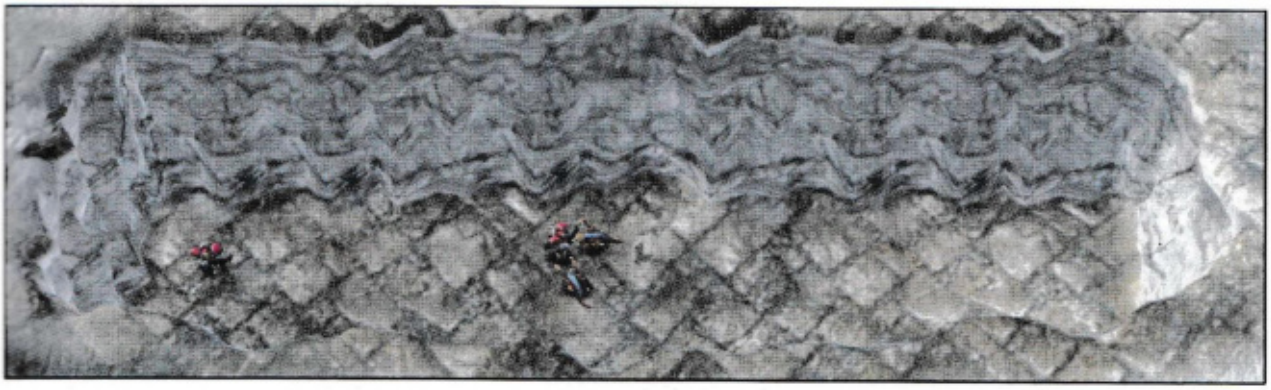


Рис. 1.9. Проигрышная позиция «двое на одного»

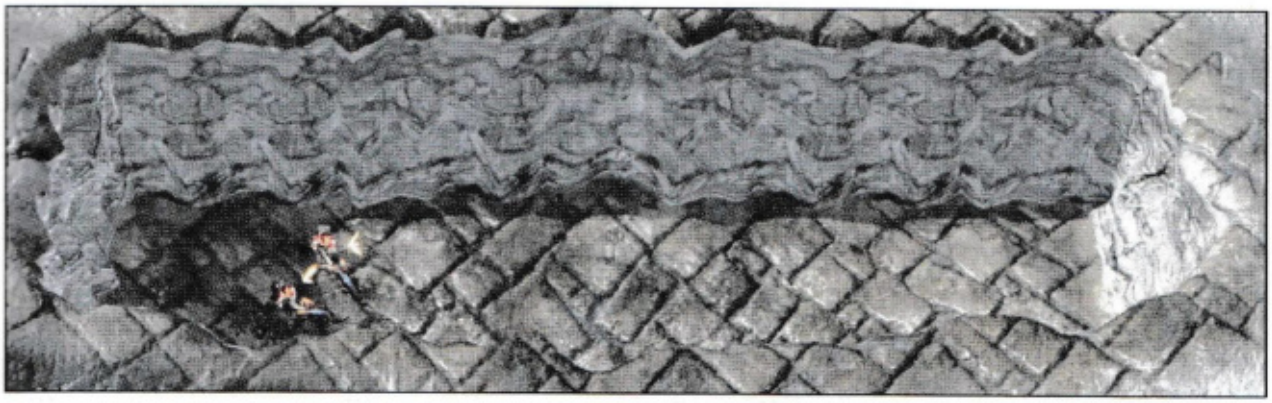


Рис. 1.10. Агенты реализуют оптимальную стратегию

Результаты экспериментального обучения в мире-сетке SMAC будут рассмотрены в разд. 1.7.

1.5. Технология

Рассматриваемый в этой главе алгоритм реализован на языке программирования Python версии 3.6. В представленном в разд. 1.6 программном коде используются стандартные библиотеки: numpy для работы с массивами, matplotlib для построения графиков, pickle для ввода/вывода данных в файл.

Основной программной библиотекой, используемой в данной главе и во всем учебном пособии, является библиотека кооперативного мультиагентного обучения SMAC (*англ.* StarCraft Multi-Agent Challenge). Кроме того, используются прикладной программный интерфейс машинного обучения StarCraft II от компании *Blizzard* и программная библиотека PySC2, разработанная компанией DeepMind. В отличие от аналогов, библиотека SMAC позволяет алгоритму машинного обучения управлять каждым агентом по отдельности и осуществлять децентрализованный микроменеджмент действий агентов.

В гл. 1 карта 2m2zFOXmatrix.SC2Map создана на базе карты SMAC в редакторе карт StarCraft II Editor. Ниже представлены изменения, которые необходимо внести в файл smac_maps.py, расположенный в папке «...\smac\env\starcraft2\maps», при создании новой карты или редактировании существующей:

```
«2m2zFOX»: { «n_agents»: 2, «n_enemies»: 2,
 «limit»: 240,
 «a_race»: «T»,
 «b_race»: «Z»,
«unit_type_bits»: 0, «map_type»: «marines»,}
```

Кроме того, файл с новой картой *.SC2Map должен быть добавлен в папку «...\StarCraft II\Maps\SMAC_Maps».

При обучении удобно ускорять визуализацию игры, а при анализе результатов нужен реальный масштаб времени. Существует возможность настройки скорости визуализации StarCraft II изменением значения переменной realtime с True на False в файле starcraft2.py, находящемся в папке «...\smac\env\starcraft2». После модификации настроек скорости или добавления новой карты необходимо перезапустить среду разработки программного обеспечения, чтобы загрузилась библиотека с изменениями.

Эксперименты по мультиагентному обучению проводились в операционной системе Windows 10. Для запуска примеров, приведенных в разд. 1.6, необходимо также установить бесплатный клиент компьютерной стратегической игры StarCraft II через приложение цифровой дистрибуции Blizzard Battle.net. Код, реализующий алгоритм независимого табличного Q-обучения, представлен в подразд. 1.6.1. Код, тестирующий выученную Q-таблицу, вынесен в отдельный файл и представлен в подразд. 1.6.2.

Примеры реализации алгоритмов в полном объеме представлены на платформе Github: https://github.com/Alekat13/FoxCommander

Видеопримеры работы алгоритмов приведены на YouTube: https://www.youtube.com/channel/UC33QPEjSlfA8P9rG0HtiH6g

1.6. Код

1.6.1. Алгоритм независимого табличного Q-обучения

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
import pickle
import matplotlib.pyplot as plt

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Получаем состояние агента как позицию на карте
def get_stateFox(agent_id, agent_posX, agent_posY):

    if agent_id == 0:
        state = 3

        if 6 < agent_posX < 7 and 15 < agent_posY < 16.5 :
            state = 0
        elif 7 < agent_posX < 8 and 15 < agent_posY < 16.5  :
            state = 1
        elif 8 < agent_posX < 8.9 and 15 < agent_posY < 16.5  :
            state = 2
        elif 8.9 < agent_posX < 9.1 and 15 < agent_posY < 16.5  :
            state = 3
        elif 9.1 < agent_posX < 10 and 15 < agent_posY < 16.5  :
            state = 4
        elif 10 < agent_posX < 11 and 15 < agent_posY < 16.5  :
            state = 5
        elif 11 < agent_posX < 12 and 15 < agent_posY < 16.5  :
            state = 6
        elif 12 < agent_posX < 13.1 and 15 < agent_posY < 16.5  :
            state = 7
        elif 6 < agent_posX < 7 and 14 < agent_posY < 15 :
            state = 8
        elif 7 < agent_posX < 8 and 14 < agent_posY < 15 :
            state = 9
        elif 8 < agent_posX < 8.9 and 14 < agent_posY < 15 :
            state = 10
        elif 8.9 < agent_posX < 9.1 and 14 < agent_posY < 15 :
            state = 11
        elif 9.1 < agent_posX < 10 and 14 < agent_posY < 15 :
            state = 12
        elif 10 < agent_posX < 11 and 14 < agent_posY < 15 :
            state = 13
        elif 11 < agent_posX < 12 and 14 < agent_posY < 15 :
            state = 14
        elif 12 < agent_posX < 13.1 and 14 < agent_posY < 15 :
            state = 15
```

```
    if agent_id == 1:
        state = 11

        if 6 < agent_posX < 7 and 16.2 < agent_posY < 17 :
            state = 0
        elif 7 < agent_posX < 8 and 16.2 < agent_posY < 17 :
            state = 1
        elif 8 < agent_posX < 8.9 and 16.2 < agent_posY < 17 :
            state = 2
        elif 8.9 < agent_posX < 9.1 and 16.2 < agent_posY < 17 :
            state = 3
        elif 9.1 < agent_posX < 10 and 16.2 < agent_posY < 17 :
            state = 4
        elif 10 < agent_posX < 11 and 16.2 < agent_posY < 17 :
            state = 5
        elif 11 < agent_posX < 12 and 16.2 < agent_posY < 17 :
            state = 6
        elif 12 < agent_posX < 13.1 and 16.2 < agent_posY < 17 :
            state = 7
        elif 6 < agent_posX < 7 and 15.5 < agent_posY < 16.2 :
            state = 8
        elif 7 < agent_posX < 8 and 15.5 < agent_posY < 16.2 :
            state = 9
        elif 8 < agent_posX < 8.9 and 15.5 < agent_posY < 16.2 :
            state = 10
        elif 8.9 < agent_posX < 9.1 and 15.5 < agent_posY < 16.2 :
            state = 11
        elif 9.1 < agent_posX < 10 and 15.5 < agent_posY < 16.2 :
            state = 12
        elif 10 < agent_posX < 11 and 15.5 < agent_posY < 16.2 :
            state = 13
        elif 11 < agent_posX < 12 and 15.5 < agent_posY < 16.2 :
            state = 14
        elif 12 < agent_posX < 13.1 and 15.5 < agent_posY < 16.2 :
            state = 15

    return state

#Выбираем действие
def select_actionFox(agent_id, state, avail_actions_ind, n_actionsFox, epsilon,
Q_table):

    if random.uniform(0, 1) < (1 - epsilon):
        #Исследуем пространство действий
        action = np.random.choice(avail_actions_ind)
    else:
        #Выбираем действие с использованием Q-таблицы
        qt_arr = np.zeros(len(avail_actions_ind))
        keys = np.arange(len(avail_actions_ind))
        act_ind_decode = dict(zip(keys, avail_actions_ind))
        stateFoxint = int(state)
        for act_ind in range(len(avail_actions_ind)):
            qt_arr[act_ind] = Q_table[agent_id, stateFoxint,
            act_ind_decode[act_ind]]

        action = act_ind_decode[np.argmax(qt_arr)]

    return action
#Главная функция программы обучения
def main():
    #Загружаем среду StarCraft II, карту и минимальную сложность противника
    env = StarCraft2Env(map_name="2m2zFOX", difficulty="1")
    #Получаем информацию о среде
```

```
env_info = env.get_env_info()

n_actions = env_info["n_actions"]
n_agents = env_info["n_agents"]

n_episodes = 120 #число эпизодов обучения
alpha = 0.1   #скорость обучения
gamma = 0.9   #параметр дисконтирования
epsilon = 0.7 #исследование пространства действий

n_statesFox = 16 #число состояний мира-сетки
n_actionsFox = 7 #вводим свое число действий
#задаем пустую Q-таблицу
Q_table = np.zeros([n_agents, n_statesFox, n_actions])
episode_reward_history = []
episode_reward_history_sum = 0
win_rate_history = []
battles_won_history = []

#Цикл по эпизодам
for e in range(n_episodes):
    #Обнуляем среду
    env.reset()
    #Параметр равен True, когда битва заканчивается
    terminated = False
    episode_reward = 0

    #Используем динамический параметр эпсилон
    if e % 15 == 0:
        epsilon += (1 - epsilon) * 10 / n_episodes
        print("epsilon = ", epsilon)
    #Цикл по шагам игры в эпизоде
    while not terminated:
        #Обнуляем промежуточные переменные
        actions = []
        action = 0
        #Сохраняем историю состояний на одном шаге для разных агентов
        stateFox= np.zeros([n_agents])
        #Сохраняем историю действий на одном шаге для разных агентов
        actionsFox = np.zeros([n_agents])

        #Цикл по агентам agent_id=0, agent_id=1
        for agent_id in range(n_agents):

            #Получаем характеристики юнита
            unit = env.get_unit_by_id(agent_id)
            #Получаем состояние по координатам юнита
            stateFox[agent_id] = get_stateFox(agent_id, unit.pos.x,
            unit.pos.y)

            #Получаем возможные действия агента
            avail_actions = env.get_avail_agent_actions(agent_id)
            avail_actions_ind = np.nonzero(avail_actions)[0]
            #Выбираем действие
            action = select_actionFox(agent_id, stateFox[agent_id],
            avail_actions_ind, n_actionsFox, epsilon, Q_table)
            #Собираем действия от разных агентов
            actions.append(action)
            actionsFox[agent_id] = action

        #Передаем действия агентов в среду, получаем награду
        #и прерывание игры от среды
        reward, terminated, _ = env.step(actions)
```

```
            #Суммируем награды за шаг для вычисления награды за эпизод
            episode_reward += reward


#################### Обучаем ##################################################

            for agent_id in range(n_agents):
                #Получаем характеристики юнита
                unit = env.get_unit_by_id(agent_id)
                #Получаем состояние по координатам юнита
                stateFox_next = get_stateFox(agent_id, unit.pos.x,
                                             unit.pos.y)
                stateFoxint= int(stateFox[agent_id])
                actionsFoxint = int(actionsFox[agent_id])
                #Вычисляем значения для Q-таблицы
                Q_table[agent_id, stateFoxint, actionsFoxint] =
                Q_table[agent_id, stateFoxint, actionsFoxint] + alpha * \
                (reward + gamma * np.max(Q_table[agent_id,stateFox_next, :])\
                - Q_table[agent_id, stateFoxint, actionsFoxint])

###############################################################################

        #Выводим общую награду за эпизод
        print("Total reward in episode {} = {}".format(e, episode_reward))
        episode_reward_history_sum += episode_reward
        episode_reward_history.append(episode_reward_history_sum)

        #Выводим результаты игр
        print ("get_stats()=", env.get_stats())
        stats5 = env.get_stats()
        #print (stats5["battles_won"])
        win_rate_history.append(stats5["win_rate"])
        battles_won_history.append(stats5["battles_won"])
    #Закрываем среду StarCraft II
    env.close()
    #Выводим на печать графики
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w', edgecolor='k')
    plt.plot(episode_reward_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Количество награды')
    plt.show()
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w', edgecolor='k')
    plt.plot(win_rate_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Процент побед')
    plt.show()
    #Выводим и сохраняем выученную Q-таблицу
    print(Q_table)
    with open("se25.pkl", 'wb') as f:
        pickle.dump(Q_table, f)

#Точка входа в программу
if __name__ == "__main__":
    main()
```

1.6.2. Тестирование Q-таблицы

```
#Функция выбора действий, в отличие от этапа обучения, не использует
#переменную эпсилон
 def select_actionFox(agent_id, state, avail_actions_ind, n_actionsFox, epsilon,
 Q_table):

    qt_arr = np.zeros(len(avail_actions_ind))
    keys = np.arange(len(avail_actions_ind))
    act_ind_decode = dict(zip(keys, avail_actions_ind))
    stateFoxint = int(state)

    for act_ind in range(len(avail_actions_ind)):
        qt_arr[act_ind] = Q_table[agent_id, stateFoxint,
                                  act_ind_decode[act_ind]]
    action = act_ind_decode[np.argmax(qt_arr)]

    return action


#Главная функция программы тестирования
def main():

    env = StarCraft2Env(map_name="2m2zFOX", difficulty="1")
    env_info = env.get_env_info()
    n_agents = env_info["n_agents"]
    n_episodes = 10 #число тестовых эпизодов
    n_actionsFox = 7 #вводим свое число действий
    total_reward = 0
    time_history = 0
    reward_history = np.zeros([n_episodes])

    #Загружаем обученную Q-таблицу
    with open("se25.pkl", 'rb') as f:
        Q_table = pickle.load(f)
        print (Q_table)

    for e in range(n_episodes):
        env.reset()
        terminated = False
        episode_reward = 0
        actions_history = []
        start_time = time.time()

        while not terminated:
            actions = []
            action = 0
            #Сохраняем историю состояний на одном шаге для разных агентов
            stateFox= np.zeros([n_agents])
            #Сохраняем историю действий на одном шаге для разных агентов
            actionsFox = np.zeros([n_agents])

            #Цикл agent_id= 0, agent_id= 1
            for agent_id in range(n_agents):

                #Получаем характеристики юнита
                unit = env.get_unit_by_id(agent_id)
                #Получаем состояние по координатам юнита
                stateFox[agent_id] = get_stateFox(agent_id, unit.pos.x,
                unit.pos.y)
```

```
                avail_actions = env.get_avail_agent_actions(agent_id)
                avail_actions_ind = np.nonzero(avail_actions)[0]
                #Выбираем действие
                action = select_actionFox(agent_id, stateFox[agent_id],
                avail_actions_ind, n_actionsFox, epsilon, Q_table)
                #Собираем действия от разных агентов
                actions.append(action)
                actions_history.append(action)

                actionsFox[agent_id] = action

            reward, terminated, _ = env.step(actions)
            episode_reward += reward

        total_reward +=episode_reward

        reward_history[e] = episode_reward

        finish_time = time.time() - start_time
        time_history += finish_time
        print("Total reward in episode {} = {}".format(e, episode_reward))
        print ("get_stats()=", env.get_stats())
        print("actions_history=", actions_history)

    print ("Average reward = ", total_reward/n_episodes)
    print("---------- %s seconds ----------" % (time_history/n_episodes))
    env.close()

    print ('-----Maximum from reward_history ---', np.max(reward_history))

#Точка входа в программу
if __name__ == "__main__":
    main()
```

1.7. Эксперимент

Для одноагентного машинного обучения в компьютерных играх, в том числе в аркадных играх компании Atari, было разработано несколько важных правил выбора и инициализации параметров обучения. Воспроизводимость этих правил для мультиагентного обучения является отдельной задачей, которая решена в этом разделе для мира-сетки SMAC.

Табличное машинное обучение чрезвычайно чувствительно к инициализации параметров обучения α, γ, ε, *Ne*. Для параметров скорости обучения α и дисконтирования γ в зависимости от типа игры рекомендуются значения 0.1, 0.5, 0.9. При малых значениях параметр скорости обучения α в некотором роде выступает как фильтр низких частот, тогда как близость значений α к единице в основном используется для детерминированных процессов. Но при мультиагентном обучении параметр скорости обучения α не рекомендуется приравнивать к единице, даже если среда детерминирована благодаря дополнительным исследовательским действиям кооперативных агентов. Кроме того, алгоритм может сходиться к определенной стратегии быстрее, чем это задается параметром скорости обучения α, что связано

именно с дополнительным взаимодействием агентов в процессе вычисления совместных действий или с влиянием механизма социальной координации. Таким образом в машинном обучении проявляется мультиагентный эффект.

Для параметра исследования пространства действий ε рекомендуется выбирать значения 0.1, 0.5, 0.7. Но только очень большая продолжительность эпизодов или их значительное число позволяет приравнивать параметр ε к 0.1, т. е. в 90 % случаев агенту разрешается выбирать случайные действия. При числе эпизодов обучения более 100 параметр ε рекомендуется итеративно увеличивать, чтобы перейти от исследования к эксплуатации полученного опыта. Для этого применяется динамическая формула вычисления значений ε_t для каждого 15-го эпизода: $\varepsilon_{t+1} = \varepsilon_t + 10\frac{1-\varepsilon_t}{Ne}$.

Число эпизодов *Ne* полностью зависит от продолжительности одного эпизода игры и от размера пространства состояний-действий. Так, для табличного одноагентного Q-обучения в игре FrozenLake, представленной миром-сеткой 4×4 с четырьмя возможными действиями, агенту для обучения достаточно 22 эпизода. С учетом того что рассматриваемый мир-сетка SMAC имеет две довольно скромные Q-таблицы размером 2×8 каждая и восемь возможных действий агента, число эпизодов не должно существенно превышать значения FrozenLake.

Основной эксперимент по подбору параметров обучения α, γ, ε, *Ne* проведем для рассматриваемого мира-сетки SMAC. С учетом трех возможных значений каждого из четырех параметров обучения требуется провести 81 тестовое обучение. Вследствие случайности обучения, которую вносят игровая среда и параметр исследования пространства действий ε, каждое тестовое обучение необходимо повторить 10 раз, чтобы усреднить найденные значения (выборочно результаты подбора параметров представлены в табл. 1.4).

Таблица 1.4

Результаты подбора параметров обучения для независимого табличного Q-обучения в мире-сетке SMAC

Параметры обучения	Результаты тестового обучения
$\alpha = 0.1,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 30$	$wr = 0,\ br = 6.6,\ avr = 2.9$
$\alpha = 0.1,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 60$	$wr = 0,\ br = 0,\ avr = 3.3$
$\alpha = 0.1,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 120$	$wr = 0.5,\ br = 0,\ avr = 11.3$
$\alpha = 0.1,\ \gamma = 0.5,\ \varepsilon = 0.5,\ Ne = 120$	$wr = 0,\ br = 4,\ avr = 1.6$
$\alpha = 0.1,\ \gamma = 0.1,\ \varepsilon = 0.1,\ Ne = 120$	$wr = 0,\ br = 9.5,\ avr = 4.9$
$\alpha = 0.5,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 30$	$wr = 0.2,\ br = 0.8,\ avr = 7.5$
$\alpha = 0.5,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 60$	$wr = 0,\ br = 8,\ avr = 3.1$
$\alpha = 0.5,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 120$	$wr = 0.4,\ br = 0.2,\ avr = 9.9$

Окончание табл. 1.4

Параметры обучения	Результаты тестового обучения
$\alpha = 0.5,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 120$	$wr = 0,\ br = 0,\ avr = 1.2$
$\alpha = 0.5,\ \gamma = 0.1,\ \varepsilon = 0.1,\ Ne = 120$	$wr = 0,\ br = 3.5,\ avr = 4.7$
$\alpha = 0.9,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 30$	$wr = 0,\ br = 0,\ avr = 1.3$
$\alpha = 0.9,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 60$	$wr = 0,\ br = 0,\ avr = 2.7$
$\alpha = 0.9,\ \gamma = 0.9,\ \varepsilon = 0.7,\ Ne = 120$	$wr = 0.1,\ br = 2.8,\ avr = 5$
$\alpha = 0.9,\ \gamma = 0.5,\ \varepsilon = 0.6,\ Ne = 120$	$wr = 0.3,\ br = 6.6,\ avr = 7.3$
$\alpha = 0.9,\ \gamma = 0.1,\ \varepsilon = 0.1,\ Ne = 120$	$wr = 0,\ br = 0,\ avr = 3$

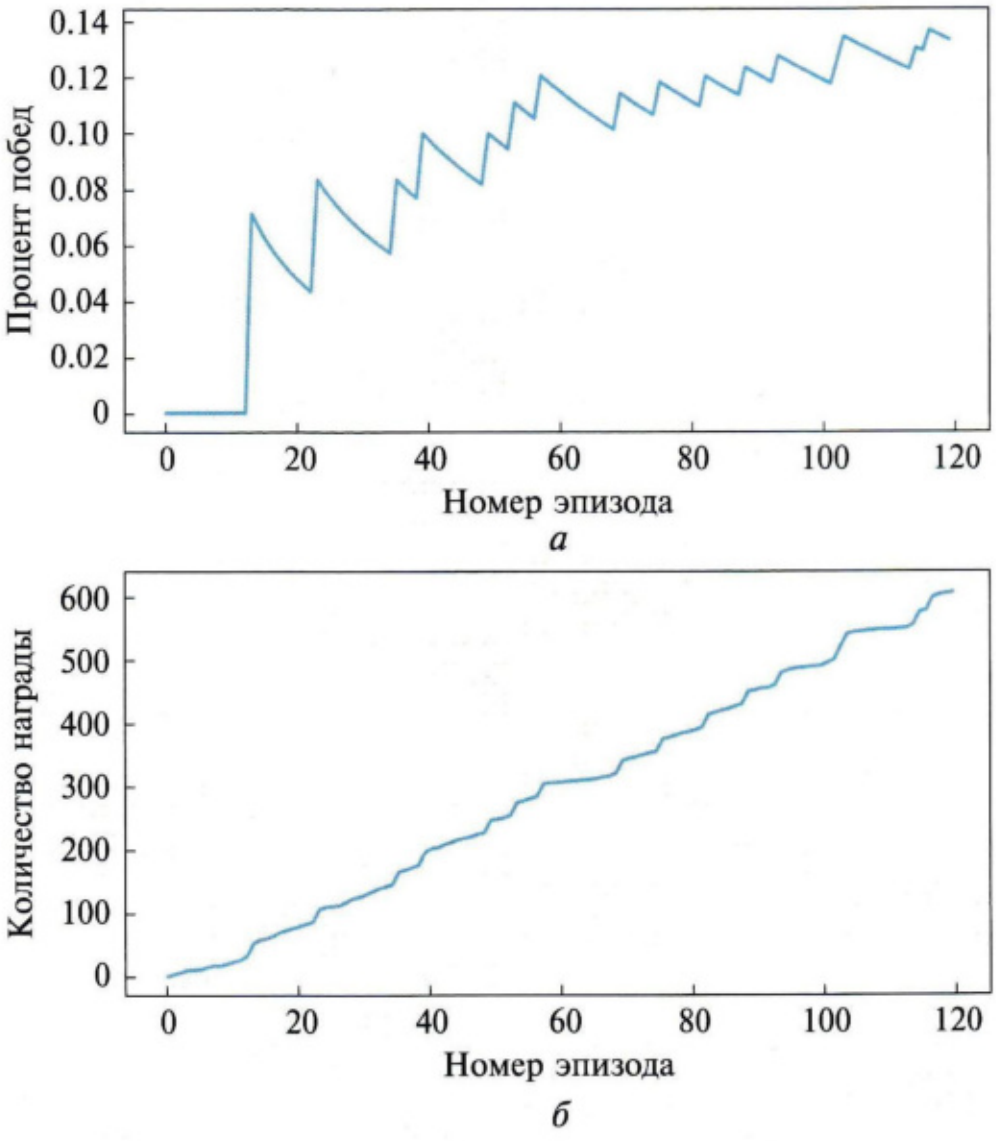


Рис. 1.11. Результаты независимого табличного Q-обучения двух агентов в мире-сетке SMAC:
а — процент побед; *б* — количество заработанной награды

Из данных табл. 1.4 следует, что существует сильная дивергенция результатов игры в зависимости от параметров обучения, однако негативные эффекты от полностью случайного поведения или переобучения отсутствуют. По критериям среднего процента побед *wr*, числа ничьих *br* и количества заработанной награды *avr* для мира-сетки SMAC лучшими параметрами обучения являются следующие: $\alpha = 0.1$, $\gamma = 0.9$, $\varepsilon = 0{,}7$, $Ne = 120$.

На рис. 1.11 показаны результаты использования найденных параметров при независимом табличном обучении двух агентов в мире-сетке SMAC.

Оба агента успешно выучивают оптимальную стратегию «бежать вправо и стрелять при появлении противника», что позволяет им достигать высоких показателей по числу побед и количеству заработанной награды при табличном подходе и данной карте.

1.8. Выводы

Независимое табличное Q-обучение было первой наивной попыткой применить методы обучения с подкреплением для мультиагентной среды. Однако для интеграции алгоритма Q-обучения в мультиагентное обучение требуется особое внимание к параметрам обучения, специальному дизайну наград и контролю за увеличением размерности Q-таблиц. Основной математической моделью, на которой базируется независимое табличное Q-обучение, служат марковские процессы принятия решений.

Одновременно и преимуществом, и недостатком МППР является концептуальная абстракция модели. Во-первых, непосредственно МППР может быть задан пятью разными способами: формулой, графом переходов, миром-сеткой, таблицей, правилами if-else. Во-вторых, временны́е интервалы МППР могут не соответствовать реальному времени, а составлять внутренние отношения мультиагентной системы. В-третьих, агент необязательно может воздействовать на среду, его действия могут быть вычислительными инструкциями или промежуточными решениями. В-четвертых, граница между средой и агентом в МППР не такая строгая, как в робототехнике. Среда представляет собой все пространство, которое находится вне агента, т. е. даже его сенсоры и актуаторы могут расцениваться как часть среды. Более того, награда, которая приходит из среды, может создаваться самим агентом и оставаться для него внешней.

В независимом табличном обучении вопросы мультиагентного обучения практически не рассматривались или подменялись искусственной координацией, агенты чаще всего не имели информации о других агентах. В связи с этим следующей попыткой улучшения мультиагентного обучения стало првлечение теории игр в качестве основной математической модели взаимозависимого обучения. Методика использования теории игр и основные алгоритмы матричных и стохастических игр рассматриваются в следующей главе.

1.9. Задачи для самоконтроля

1. Повторите результаты экспериментов для карты 2m2mFOXSC2Map «два юнита морпеха расы терран против двух юнитов морпехов».

2. Подберите наилучшие параметры α, γ, ε в алгоритме Q-обучения для карты 2m2m.

3. Проведите эксперимент по увеличению процента побед в зависимости от бонусной награды за выбор агентом действия «стрелять».

4. Проведите эксперимент по увеличению процента побед при бонусной награде за меньшее число шагов агента, достигающего цели игры.

5. Проведите эксперимент с увеличением числа агентов для карты 2m2z: «три против трех», «пять против пяти», «три против пяти».

6. Реализуйте мир-сетку «Минимакс» (рис. 1.12). Каждый агент может выполнять два действия в рамках своей зоны: двигаться вправо, двигаться влево. Агенты двигаются одновременно. При попадании агента 1 в целевую клетку он получает награду +5 и награду −10 при нахождении в одной клетке с агентом 2, который получает при этом награду +5. Сможет ли агент 1 найти оптимальную стратегию при независимом табличном Q-обучении обоих агентов?

Рис. 1.12. Мир-сетка «Минимакс»

7. Реализуйте мир-сетку «Координация» (рис. 1.13). Каждый агент может выполнить три действия: двигаться вниз, двигаться вправо, двигаться влево. Среда дает отрицательную награду −1, если агенты врезаются в камни или друг в друга. Смогут ли агенты выучить стратегию координирования своих действий с использованием алгоритма Q-обучения?

8. Разработайте и исследуйте реализацию алгоритма распределенного Q-обучения. Есть ли преимущества табличного обучения в мире-сетке SMAC для данного алгоритма по сравнению с классическим алгоритмом Q-обучения?

9. Разработайте и исследуйте реализацию алгоритма гипер-Q. Есть ли преимущества табличного обучения в мире-сетке SMAC для данного алгоритма по сравнению с классическим алгоритмом Q-обучения?

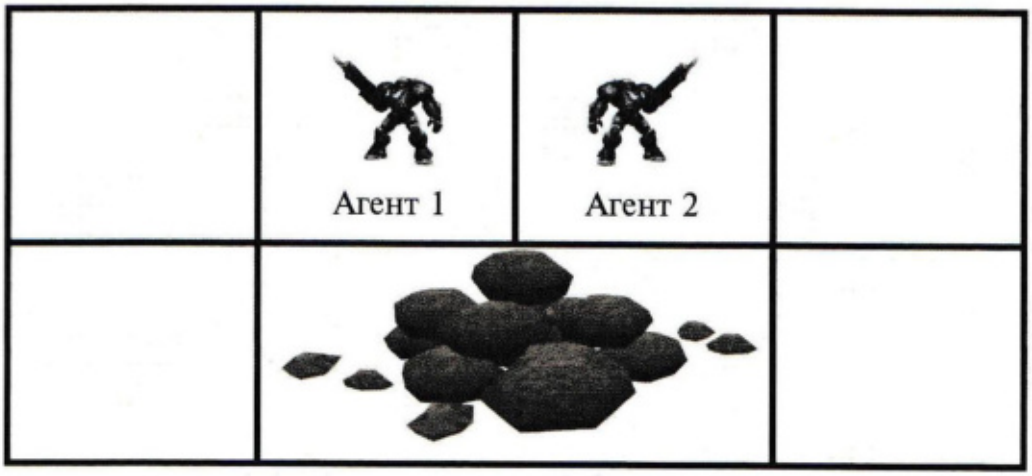


Рис. 1.13. Мир-сетка «Координация»

10. Разработайте сценарий игры и модификацию библиотеки SMAC, чтобы при независимом табличном обучении наилучшим образом раскрыть юнита расы терран мародера. Это пехотный наземный юнит, имеющий 125 единиц здоровья и одну единицу брони. Мародер атакует наземные юниты с расстояния шесть квадратов игрового поля, нанося 10 единиц урона. Эффективен при использовании совместно с морпехами благодаря способности замедлять быстрых юнитов ближнего боя. Уязвим в противостоянии с группой зилотов или зерглингов.

Глава 2

Обучение в матричных и стохастических играх

Суть игры заключается в моделировании поведения.
Правила ограничивают некоторые виды поведения
и вознаграждают другие...
...Чтобы выиграть календарную войну,
ты должен понять, как работают игровые системы.

Юн Ха Ли. Гамбит девятихвостого лиса

2.1. Классификация

Теория игр (*англ.* Game Theory) — это совокупность математических методов нахождения оптимальных стратегий в играх. Под *игрой* понимается математическая модель некоторого процесса, в котором участвуют два и более игрока-агента, кооперативно или некооперативно взаимодействующих для достижения своих целей. Действия агента в каждый момент времени определяются его стратегией. Стратегия позволяет агенту поставить в соответствие воспринимаемым состояниям окружающей среды — действия, которые должны быть предприняты. В зависимости от действий других агентов и реакций среды стратегия приводит агента к выигрышу или проигрышу. Машинное обучение позволяет агентам автоматически выбирать лучшие стратегии с учетом или без учета априорной информации о других участниках игры, их ресурсах и возможных действиях.

Теория игр вначале развивалась как платформа для исследования некооперативного стратегического взаимодействия в экономических науках. Но при появлении в области искусственного интеллекта мультиагентных систем сообщество обратило внимание на теорию игр как прикладную дисциплину, предоставляющую математические средства для описания и анализа процессов мультиагентого обучения в частности и процессов коммуникации информационных технологий в целом.

В табл. 2.1 представлены основные табличные алгоритмы мультиагентного обучения, классифицированные по типам игр — обучение в матричных играх или обучение в стохастических играх и по типу обучения — независимое обучение или совместное обучение.

В этой главе подробно рассматриваются принципы мультиагентного обучения с подкреплением в кооперативных матричных и стохастических

играх с использованием алгоритмов поиска экстремума стратегий (PHC), «выигрывай или учись быстро» (WoLF-PHC), Q-обучения Нэша (Nash-Q). Данные алгоритмы, как и большинство представленных в табл. 2.1, основаны на классическом алгоритме Q-обучения. Несмотря на преимущества при одноагентном обучении, в случае мультиагентного обучения алгоритм Q-обучения не гарантирует нахождения оптимальной стратегии, максимизирующей награду. В связи с этим были предложены следующие усовершенствования классического алгоритма Q-обучения с использованием принципов теории игр.

Таблица 2.1

Классификация алгоритмов мультиагентного обучения с использованием принципов теории игр

Тип обучения	Тип игр	
	Матричные игры	Стохастические игры
Независимое обучение	FMQ CS LA IGA	PHC WoLF-PHC LoC
Совместное обучение	–	JAL Nash-Q F-F-Q Minimax-Q SCQ UC

Алгоритм частотного максимума Q-значения (*англ.* Frequency Maximum Q-learning, FMQ), изначально разработанный для матричных игр, позволяет подсчитывать частоту получения максимальной награды при выполнении определенного действия. Эта частота напрямую прибавляется к Q-значению. При детерминированных наградах с помощью алгоритма FMQ агенты находят оптимальные действия в кооперативных играх.

Развитием алгоритма Q-обучения для подсчета стохастических наград стал алгоритм последовательных обязательств (*англ.* Commitment Sequences, CS). Последовательные обязательства представляют собой генерирующие по заданному протоколу временны́е интервалы, в которых агент обязан выбрать определенные действия. Такая эвристика дает возможность агенту преодолеть одну из проблем, характерных для мультиагентного обучения: влияние на получаемую награду действий других агентов.

Альтернативой алгоритму Q-обучения являются методы обучения, основанные на обучающихся автоматах (*англ.* Learning Automata, LA). В алгоритме LA используются распределение вероятностей выполнения действий, обычная схема обучения с подкреплением и оригинальные правила обновления вероятностей в зависимости от полученной положительной

или отрицательной награды. Несмотря на общую схожесть алгоритма LA и обучения с подкреплением, алгоритм LA позволяет без информации о других обучающихся агентах найти в играх асимптотически стабильную стратегию.

Другой альтернативой алгоритму Q-обучения является использование градиента при независимом обучении в матричных играх. Так, алгоритм бесконечно малого восхождения градиента (*англ.* Infinitesimal Gradient Ascent, IGA) гарантирует сходимость стратегий обучающих агентов к равновесию Нэша или сходимость средних наград к пределу ожидаемых наград в равновесии Нэша.

Равновесие Нэша, равновесие Штакельберга и коррелированное равновесие — это так называемые центральные концепции теории игр, которые дают агентам возможность найти стратегии, являющиеся лучшим ответом на стратегии других агентов. Несмотря на критику этих концепций, основанную на утверждениях, что равновесий в одной игре может быть несколько, что не гарантируется оптимальность, что отсутствует координация при выборе равновесий, равновесие Нэша остается важным элементов современного мультиагентного обучения с подкреплением. В этой главе анализ эффективности работы алгоритмов строится именно на равновесии Нэша. Кроме того, эта концепция органично включена в алгоритм Q-обучения Нэша (Nash-Q).

Еще двумя альтернативами алгоритму Q-обучения являются алгоритм AWESOME и алгоритм MetaStrategy, не включенные в табл. 2.1, так как они изначально были разработаны для повторяющихся игр. Основная идея алгоритма AWESOME заключается в постоянном мониторинге действий других агентов, чтобы в случае их нестационарного поведения использовать равновесие Нэша вместо стратегии максимизации награды. Похожая минимаксная стратегия используется и в гибридном алгоритме MetaStrategy: вначале исследуются действия оппонента и в случае невозможности максимизации награды выбирается безопасный вариант действий.

Рассмотренные расширения и альтернативы алгоритма Q-обучения в основном относятся к типу независимого обучения. В перечисленных алгоритмах не учитывается важная особенность совместных действий в мультиагентном обучении с подкреплением. Так, в случае одноагентного обучения с подкреплением агенту, для того чтобы оценить будущую награду в следующем состоянии, достаточно найти максимум Q-значения, зависящий только от его будущих действий. В случае мультиагентного обучения агент не может по-настоящему оценить будущую награду, потому что ему необходима информация о действиях других агентов, т. е. агент должен обучаться с учетом совместных действий агентов.

Первый алгоритм обучения совместным действиям (*англ.* Joint Action Learner, JAL) позволял оценить стратегии, используемые другими агентами, на основе простого подсчета числа выполненных совместных действий в определенном состоянии. Развитием этой идеи стал алгоритм «Минимакс» (*англ.* Minimax Q-learning, Minimax-Q), в котором предполагалось,

что в некооперативных играх обучающийся оппонент всегда будет стараться минимизировать награду агента. В дальнейшем алгоритм был преобразован в алгоритм Q-обучения «Друг или враг» (*англ.* Friend-or-Foe Q-learning, F-F-Q), в котором учитывались не только результаты игры, но и условия их достижения.

Необходимо обратить внимание еще на одну проблему, характерную именно для мультиагентного обучения. Эта проблема связана с размером пространства состояний-действий, которое увеличивается экспоненциально при увеличении числа агентов. В первых алгоритмах мультиагентного обучения эта проблема интуитивно решалась простым игнорированием других агентов при обучении. Но такой подход применим на практике, только если агенты физически разнесены в среде и им не требуется координации действий. Были разработаны два основных пути решения указанной проблемы: сокращение числа совместных действий или сокращение числа совместных состояний путем определения того, нужна ли в совместных действиях или состояниях мультиагентная координация или агент может обучаться индивидуально.

В алгоритме разреженного кооперативного Q-обучения (*англ.* Sparse Cooperative Q-learning, SCQ) используется графовая модель, для того чтобы представить пространство совместных действий более компактно и разрешить агентам обучаться совместным действиям только в случае, если требуется мультиагентная координация. Развитием этой идеи стал алгоритм полезной координации (*англ.* Utile Coordination, UC), который позволил не задавать графы экспертно, а строить их в процессе обучения.

В алгоритме обучения координации (*англ.* Learning of Coordination, LoC) используется активное действие по восприятию информации из среды, для того чтобы определить состояния, в которых находятся другие агенты. При этом учитывается уровень отрицательной награды, которая может быть получена при отсутствии координации.

2.2. Модель

2.2.1. Матричные игры

Матричная игра — это кортеж $(n, A_1, ..., A_n, R_1, ..., R_n)$, где n — число агентов; A_i — множество действий i-го агента, $R_i : A_1 \times ... \times A_n \to \mathbb{R}$ — функция наград i-го агента. Матричная игра имеет одно состояние и множество агентов-игроков. Каждый агент выбирает действие из множества действий A_i и получает награду R_i в зависимости от действий других агентов.

В матричной игре награда задается с помощью матрицы наград. Именно матрица наград является тем связующим звеном, которое позволяет при мультиагентном обучении учитывать действия нескольких агентов. Ниже приведены матрицы наград для популярных двухагентных матричных игр:

«Дилемма заключенного»

$$R_1 = \begin{bmatrix} 1 & 0 \\ 10 & 5 \end{bmatrix}, \; R_2 = \begin{bmatrix} 1 & 10 \\ 0 & 5 \end{bmatrix};$$

«Орлянка»

$$R_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \; R_2 = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}.$$

В матрице наград действия a_1^1, a_2^1 первого агента соответствуют строкам матрицы, действия a_1^2, a_2^2 второго агента соответствуют столбцам матрицы. Эти игры представляют основные типы матричных игр: «Дилемма заключенного» — игра с общей суммой, «Орлянка» — игра с нулевой суммой (агент получает ровно столько награды, сколько не получает ее другой агент).

В матричной игре целью агентов является нахождение чистой или смешанной стратегии, которая максимизирует их награды. *Чистая стратегия* — это стратегия, в соответствии с которой агенты выбирают действия детерминированно (агент будет постоянно выбирать одно и то же действие). При *смешанной стратегии* агенты выбирают действия, используя некоторое распределение вероятностей. Так, в игре «Дилемма заключенного» лучшим решением является применение чистой стратегии для обоих агентов, а в игре «Орлянка» — применение смешанной стратегии.

В процессе машинного обучения агенты ищут наилучшую стратегию, играя последовательно и итерационно. Стратегия π считается лучше стратегии π' или равной ей, если общее количество награды, полученной агентом, следующим стратегии π, больше или равно количеству награды, полученной агентом при использовании стратегии π'. Оптимальной называют стратегию π^*, которая лучше всех или равна всем другим стратегиям.

Для того чтобы оценить найденные стратегии нескольких агентов, используют равновесие Нэша. *Равновесие Нэша* в матричной игре — это множество оптимальных стратегий $\left(\pi_1^*, \ldots, \pi_n^*\right)$ агентов таких, что

$$V_i\left(\pi_1^*, \ldots, \pi_n^*\right) \geq V_i\left(\pi_1^*, \ldots, \pi_i, \ldots, \pi_n^*\right), \; \forall \pi_i \in \Pi_i, \; i = 1, \ldots, n,$$

где V_i — функция ценности i-го агента, которая является ожидаемой наградой агента с учетом стратегии всех агентов; π_i — это некоторая стратегия i-го агента из множества стратегий Π_i.

Следовательно, равновесие Нэша определяет множество таких стратегий агентов, в которых ни один агент не может получить большее количество награды при изменении своей стратегии с учетом того, что остальные агенты продолжат следовать своим стратегиям из равновесия Нэша. Так, в игре «Дилемма заключенного» у каждого агента есть всего два одинаковых действия: a_1 — «молчать», a_2 — «говорить». Равновесием Нэша $\left(\pi_1^*, \pi_2^*\right)$ в «Дилемме заключенного» будет являться постоянный выбор как

действия a_2^1 — «говорить» для первого агента, так и действия a_2^2 — «говорить» для второго агента. В этом и заключается дилемма игры: рациональные совместные действия агентов приводят к нерациональному решению.

Кроме того, в игре может возникнуть ситуация, когда количество награды агента не может быть увеличено без уменьшения количества награды любого другого агента. В этом случае множество стратегий $(\pi_1, ..., \pi_n)$ для n агентов называют *оптимальными по Парето*, если не существует другого множества стратегий $(\pi_1', ..., \pi_n')$, следуя которым каждый агент получит как минимум такую же награду R_i и как минимум один агент получит бóльшую награду R_j. Так, в «Дилемме заключенного» стратегии (π_1, π_2), оптимальные по Парето, будут задавать выбор действия a_1^1 — «молчать» для первого агента и a_1^2 — «молчать» для второго агента.

Таким образом, для игры «Дилемма заключенного» стратегии, оптимальные по Парето, не являются равновесием Нэша. Однако это не общее правило, и в полностью кооперативных играх стратегии, оптимальные по Парето, обычно являются равновесием Нэша. Из определений равновесия Нэша и оптимальности по Парето следует, что не существует стратегии, которая обеспечит большее количество награды любому агенту, поэтому невозможно получить что-либо и при одностороннем отклонении агента от этой стратегии.

2.2.2. Стохастические игры

Стохастическая игра — это кортеж $(n, S, A_1, ..., A_n, T, R_1, ..., R_n)$, где n — число агентов; S — множество состояний; A_i — множество действий i-го агента; $T : S \times A_1 \times ... \times A_n \times S \to [0,1]$ — функция переходов; $R_i : S \times A_1 \times ... \times A_n \to \mathbb{R}$ — функция наград i-го агента. В зависимости от функции наград стохастические игры могут быть классифицированы следующим образом: полностью кооперативные (командные) игры — функция наград одинакова для всех агентов; полностью некооперативные (конкурентные) игры — функция наград одного агента всегда имеет отрицательное значение относительно функций наград других агентов; игра с общей суммой — в игре присутствуют все типы функции наград.

В некотором состоянии s агенты независимо выполняют действия $a_1, ..., a_n$ и получают награды $r_i(s, a_1, ..., a_n)$, затем происходит переход в следующее состояние s' в соответствии с функцией перехода T, которая в стохастической игре может быть представлена распределением вероятностей на множестве состояний. Сумма вероятностей перехода из состояния s в состояние s' должна удовлетворять условию

$$\sum_{s' \in S} p(s' \mid s, a_1, ..., a_n) = 1.$$

Стохастическая игра включает множество состояний и множество агентов. Таким образом, стохастическая игра является, с одной стороны,

расширением матричных игр на множество состояний, а с другой стороны — расширением марковских процессов принятия решений для множества агентов (рис. 2.1).

Рис. 2.1. Взаимосвязь матричных игр, марковских процессов принятия решений и стохастических игр

Так же как и матричной игре, в стохастической игре целью агентов является нахождение чистой или смешанной стратегии, которая максимизирует их награды. В случае если все агенты, кроме одного, используют стационарные стратегии и не обучаются, стохастическую игру можно рассматривать как МППР, и поэтому алгоритм Q-обучения позволит найти оптимальную стратегию для обучающегося агента. Но в общем случае в стохастической игре для оценки найденной стратегии используется равновесие Нэша. *Равновесие Нэша в стохастической игре* — это множество оптимальных стратегий $\left(\pi_1^*, \ldots, \pi_n^*\right)$ агентов $i = 1, \ldots, n$ таких, что для всех состояний $s \in S$ выполняется неравенство

$$V_i\left(s, \pi_1^*, \ \ldots, \ \pi_n^*\right) \geq V_i\left(s, \pi_1^*, \ \ldots, \ \pi_i, \ \ldots, \ \pi_n^*\right), \ \forall \pi_i \in \Pi_i,$$

где V_i — функция ценности i-го агента, которая является ожидаемой наградой агента для данного состояния с учетом стратегий всех агентов; π_i — некоторая стратегия i-го агента из множества стратегий Π_i.

Каждое состояние стохастической игры можно рассматривать как матричную игру, где на вход подается матрица наград, в которой каждый элемент представляет собой Q-значения, зависящие от совместных действий агентов. В связи с этим процесс нахождения равновесий Нэша в стохастической игре представляет собой последовательное вычисление равновесий Нэша в матричных играх для каждого состояния стохастической игры.

Рассмотрим стохастическую игру на примере игры «Мир-сетка 3×3» (рис. 2.2). Мир-сетка 3×3 состоит из девяти клеток, соответствующих состояниям игры $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ (рис. 2.3, *а*). Два агента $i = 1, 2$ начинают игру в верхних клетках слева и справа, не имея информации о месторасположении целевой клетки и о существующих наградах. Каждый агент может совершить только одно действие в единицу времени. Действия

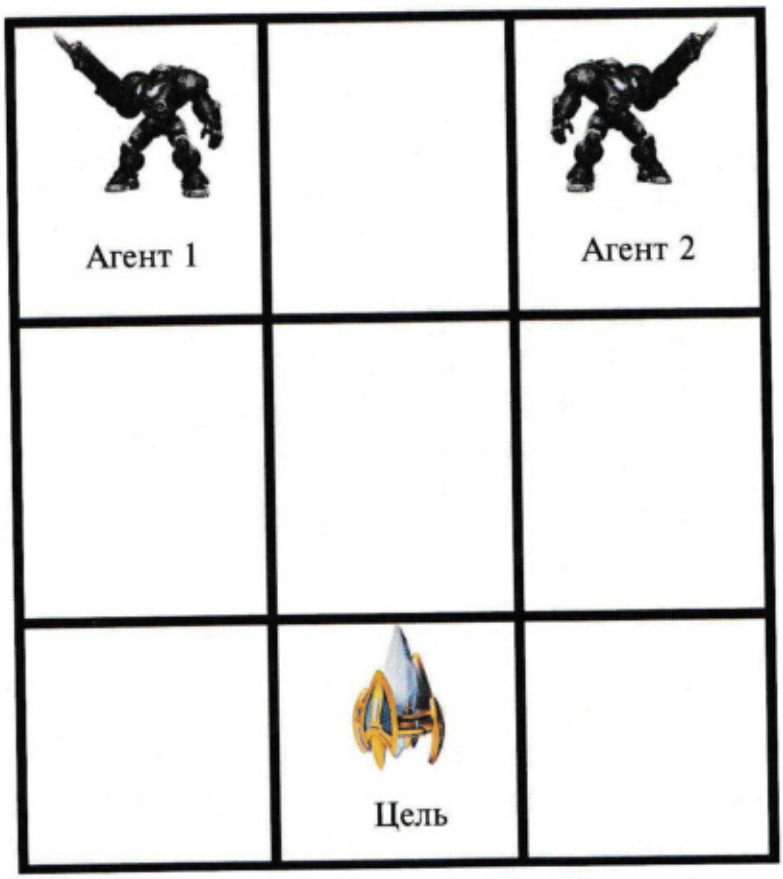
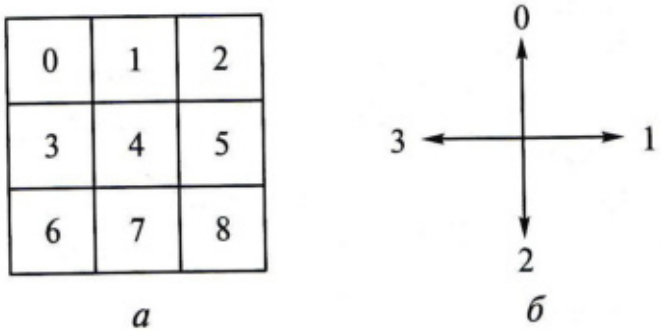


Рис. 2.2. Игра для двух агентов «Мир-сетка 3×3»



а *б*

Рис. 2.3. Нумерация в игре «Мир-сетка 3×3»:
а — состояний; *б* — действий

агентов представляют собой четыре типа перемещения: A_i = {вверх, вправо, вниз, влево} (рис. 2.3, *б*). Если агенты одновременно перемещаются в одну и ту же клетку (не считая целевой клетки), то они получают отрицательную награду $r_i = -1$. Игра заканчивается, когда один из агентов или оба агента достигают целевой клетки и получают награду $r_i = 100$. Целью агента в данной игре является достижение целевой клетки за минимальное число действий.

В этой игре возможность совместного достижения целевой клетки и наказание за перемещение в одну и ту же клетку неявно мотивируют агентов координировать свои действия. Стратегией π_i в данной игре по сути является путь в мире-сетке от начальной клетки до целевой клетки. Два кратчайших пути без пересечений друг с другом в данной игре и будут представлять

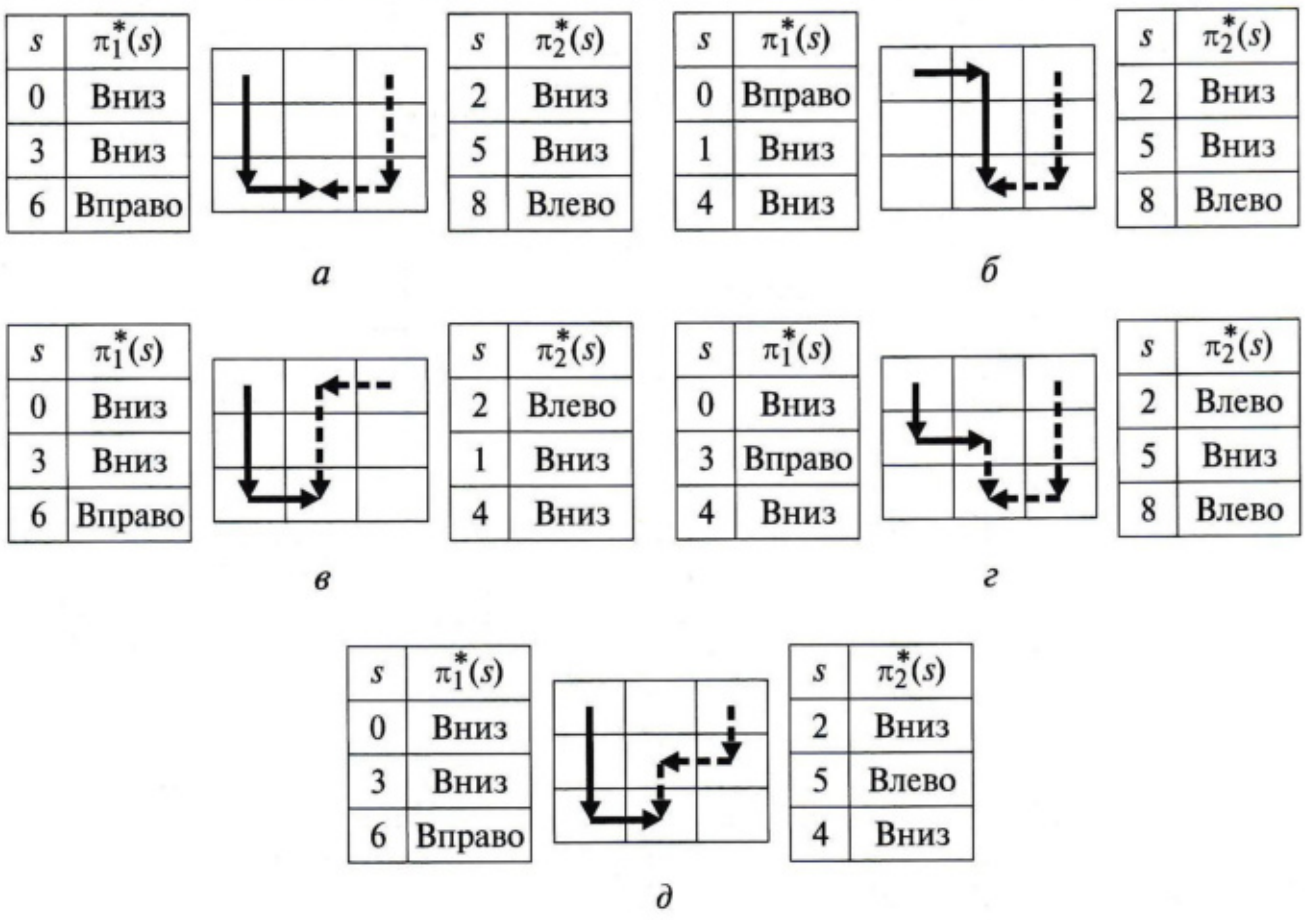


Рис. 2.4. Возможные равновесия Нэша $\left(\pi_1^*, \pi_2^*\right)$ в мире-сетке 3×3

собой равновесие Нэша $\left(\pi_1^*, \pi_2^*\right)$. На рис. 2.4 в виде путей и стратегий действий в мире-сетке 3×3 представлены пять возможных равновесий Нэша (*а* — *д*) для данной игры.

В следующем разделе будут рассмотрены алгоритмы мультиагентного обучения с подкреплением в матричных и стохастических играх.

2.3. Алгоритм

2.3.1. Поиск экстремума стратегий (PHC)

Алгоритм поиска экстремума стратегий (*англ.* Policy Hill Climbing, PHC) является рациональным алгоритмом, основанным на классическом алгоритме Q-обучения. *Рациональным* называется алгоритм, который способен сходиться к стратегии, лучше всего отвечающей стратегиям других агентов, сходящимся во время обучения к стационарным стратегиям. *Стационарной* называется стратегия, которая не изменяется со временем.

Алгоритм обладает *свойством сходимости* в том случае, если стратегия обучающегося агента обязательно сходится к стационарной стратегии. Стратегия $\pi_t(s_t, a)$ агента i *сходится* к стратегии $\pi^*(s_t, a)$

тогда и только тогда, когда для некоторого $e > 0$ существует $T > 0$ такое, что $\left|\pi_t(s_t, a) - \pi^*(s_t, a)\right| < e$, $\forall t > T$, $\forall a \in A_i$. Стратегия агента i сходится к стационарной стратегии тогда и только тогда, когда стратегия агента i сходится к π^* для некоторой $\pi^* \in \Pi_i$.

Свойство рациональности алгоритма — базовое свойство алгоритма мультиагентного обучения с подкреплением в матричных и стохастических играх, фактически определяющее, что обучающийся агент может найти и использовать наилучшую стратегию (если такая существует), которая отвечает стратегиям других агентов. В случае когда другие агенты используют только стационарные стратегии, стохастическая игра становится обычным марковским процессом принятия решений. Таким образом, алгоритм, обладающий свойством рациональности, дает возможность найти оптимальную стратегию для некоторого МППР. Именно поэтому в основу алгоритма PHC был положен алгоритм Q-обучения, который доказал свою эффективность при решении задач обучения для МППР.

В совокупности свойствами рациональности и сходимости гарантируется, что алгоритм обучения сходится к стационарной стратегии, оптимальной с учетом игры других агентов. Существует связь между этими свойствами и равновесием Нэша. Если все агенты используют рациональный алгоритм и эти алгоритмы сходятся, то стратегии должны сходиться к равновесию Нэша. Поскольку все алгоритмы сходятся к стационарной стратегии, каждый алгоритм, будучи рациональным, должен сходиться к лучшему ответу на эти стратегии. Это утверждение верно для каждого алгоритма, поэтому стратегии агентов, по определению, должны находиться в равновесии. Кроме того, если все алгоритмы рациональны и сходятся по отношению к алгоритмам других агентов, то гарантируется сходимость к равновесию Нэша.

Не все алгоритмы обладают свойствами рациональности и сходимости. Свойство рациональности может несколько видоизменяться, как, например, в эволюционной теории игр, где допускается возможность «мутаций» стратегий. В результате некоторые агенты оказываются «генетически» склонны к одним стратегиям, а другие — к другим, и приверженцы более выгодных стратегий лучше выживают в длительном периоде игры. Таким образом, стратегии все равно сходятся к равновесию Нэша, но при этом агенты придерживаются данных стратегий без явного рационального объяснения.

Главными преимуществами алгоритма PHC являются способность обучать агента нахождению смешанных стратегий, а также отсутствие требований к априорным знаниям о стратегии оппонента или о среде, в которой агент обучается. Однако алгоритм PHC не обладает свойством сходимости и сходится к оптимальным стратегиям только в случае, если все другие агенты не обучаются и используют стационарные стратегии.

Алгоритм PHC позволяет осуществлять поиск восхождением к вершине в пространстве смешанных стратегий.

Алгоритм PHC для агента i приведен ниже:

1. Инициализировать
   $\alpha \in (0, 1]$; $\delta \in (0, 1]$; $\gamma \in (0, 1)$; $\varepsilon \in (0,1)$;
   $i \leftarrow 1, \ldots, n$; $a \in A_i$;
   $$Q(s,a) \leftarrow 0;\ \pi(s,a) \leftarrow \frac{1}{|A_i|},\ \forall s \in S.$$
2. Цикл
3. Выбрать действие a в соответствии со стратегией $\pi(s, a)$ и с учетом параметра ε.
4. Получить награду r и следующее состояние s'.
5. Обновить значение $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a')\right)$.
6. Вычислить значение $\delta_{sa} \leftarrow \min\left(\pi(s,a),\ \frac{\delta}{|A_i|-1}\right)$.
7. Выбрать значение $\Delta_{sa} \leftarrow \begin{cases} -\delta_{sa}, \text{ если } a \neq \operatorname{argmax}_{a'} Q(s,a'); \\ \sum_{a' \neq a} \delta_{sa'}. \end{cases}$
8. Обновить значение $\pi(s,a) \leftarrow \pi(s,a) + \Delta_{sa}$.

В алгоритме PHC значения $Q(s, a)$ вычисляются так же, как и в обычном алгоритме Q-обучения. Кроме того, на каждом шаге вычисляется текущая стратегия $\pi(s, a)$. Именно в соответствии со стратегией $\pi(s, a)$ и параметром исследования пространства действий ε агент i выбирает действия a. Стратегия $\pi(s, a)$ улучшается при увеличении вероятности того, что агент i в соответствии с этой стратегией выберет наиболее ценные действия a с учетом параметра обучения $\delta \in (0,1]$.

2.3.2. «Выигрывай или учись быстро» (WoLF-PHC)

Модификация алгоритма PHC на основе принципа «выигрывай или учись быстро» (*англ.* Win or Learn Fast, WoLF) представляет собой новый рациональный алгоритм WoLF-PHC, обладающий свойством сходимости. Идея принципа «выигрывай или учись быстро» состоит в динамическом изменении параметра обучения δ алгоритма в зависимости от игровой ситуации: агент выигрывает — обучение замедляется (параметр δ уменьшается), агент проигрывает — обучение ускоряется (параметр δ увеличивается). В целом алгоритм имеет два параметра обучения, отвечающих за скорость обучения, причем $\delta_l > \delta_w$ (индексы l и w от *англ.* lose — терять и win — выигрывать).

Алгоритм WoLF-PHC для агента i приведен ниже:

1. Инициализировать
   $\alpha \in (0, 1]$; $\delta_l > \delta_w \in (0, 1]$; $\gamma \in (0, 1)$; $\varepsilon \in (0,1)$;
   $i \leftarrow 1, \ldots, n$; $a \in A_i$;
   $$Q(s,a), C(s), \bar{\pi}(s,a') \leftarrow 0;\ \pi(s,a) \leftarrow \frac{1}{|A_i|},\ \forall s \in S.$$
2. Цикл
3. Выбрать действие a в соответствии со стратегией $\pi(s, a)$ и с учетом параметра ε.
4. Получить награду r и следующее состояние s'.
5. Обновить значение $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a')\right)$.
6. Обновить значение $C(s) \leftarrow C(s) + 1$.
7. Вычислить $\bar{\pi}(s,a') \leftarrow \bar{\pi}(s,a') + \frac{1}{C(s)}\left(\pi(s,a) - \bar{\pi}(s,a')\right)$.
8. Выбрать значение $\delta \leftarrow \begin{cases} \delta_w, & \text{если } \sum_{a'} \pi(s,a')Q(s,a') > \sum_{a'} \bar{\pi}(s,a')Q(s,a'); \\ \delta_l & \text{в другом случае.} \end{cases}$
9. Вычислить $\delta_{sa} \leftarrow \min\left(\pi(s,a), \frac{\delta}{|A_i| - 1}\right)$.
10. Выбрать значение $\Delta_{sa} \leftarrow \begin{cases} -\delta_{sa}, & \text{если } a \neq \operatorname{argmax}_{a'} Q(s,a'); \\ \sum_{a' \neq a} \delta_{sa'}. \end{cases}$
11. Обновить значение $\pi(s,a) \leftarrow \pi(s,a) + \Delta_{sa}$.

Алгоритм WoLF-PHC состоит из трех основных частей: алгоритм Q-обучения (шаги 3–5), алгоритм PHC (шаги 9–11) и правило определения ситуации, в которой находится агент, — проигрывает или выигрывает (шаг 8). Определение ситуации основывается на вычислении разности между текущей стратегией $\pi(s, a)$ и средней стратегией $\bar{\pi}(s,a')$. Это еще одно, новое по отношению к алгоритму PHC решение, которое позволяет уравновесить возможные скачки при обучении и избавиться от осцилляции.

Средняя стратегия $\bar{\pi}(s,a')$ вычисляется с помощью дополнительного параметра $C(s)$. Интересно, что в реализациях данного алгоритма на

платформе Github параметр $C(s)$ иногда просто игнорируется. Однако ссылка автора алгоритма WoLF-PHC на принципы *фиктивных игр* (*англ.* fictitious play) дает подсказку, что параметр $C(s)$ обязательно должен присутствовать в правильной реализация алгоритма WoLF-PHC. Ведь именно принципы фиктивных игр позволяют неявно учитывать взаимные действия агентов при мультиагентном обучении на основе подсчета частоты попаданий в некоторое состояние (в стохастических играх) или частоты выполнения некоторого действия (в матричных играх).

В подразд. 2.6.1 представлена программная реализация алгоритма WoLF-PHC для матричных игр. Основными отличиями программной реализации от рассмотренного алгоритма являются отсутствие состояний и вычисление только ценности действий $Q(a)$ и стратегии действий $\pi(a)$.

2.3.3. Q-обучение Нэша (Nash-Q)

Алгоритм Q-обучения Нэша (*англ.* Nash Q-Learning, Nash-Q), обладающий свойством сходимости, представляет собой расширение классического алгоритма Q-обучения применительно к мультиагентному обучению.

Первым фундаментальным отличием алгоритма Q-обучения Нэша от классического алгоритма Q-обучения, в котором учитываются только индивидуальные действия агентов, является использование совместных действий агентов. Для стохастической игры с n агентами Q-функция некоторого агента $Q(s, a)$ будет иметь вид $Q(s, a_1, ..., a_n)$.

Вторым отличием является введение нового понятия — «Q-значение Нэша» (шаг 8), которое является ожидаемой суммой дисконтированных наград при условии, что все агенты будут следовать стратегиям из равновесия Нэша. В классическом алгоритме Q-обучения будущие награды зависят только от индивидуальной стратегии агента. В Q-значении Нэша используются равновесия Нэша $\left(\pi_1^*(s'), ..., \pi_n^*(s')\right)$. Для их вычисления могут быть применены различные методы нахождения равновесия в матричных играх, например алгоритм Лемке — Хоусона.

В рассматриваемой реализации алгоритма Q-обучения Нэша в подразд. 2.6.2 используется программная библиотека nashpy. Однако в результате работы этих методов может быть найдено несколько равновесий Нэша для одной и той же матричной игры, и тогда алгоритм Q-обучения Нэша можно модифицировать в алгоритм с выбором первого равновесия, второго равновесия, лучшего равновесия (по критериям получаемой в матрице максимальной награды).

Третье отличие алгоритма Q-обучения Нэша заключается в том, что в нем обновление Q-значений зависит от Q-значения Нэша, т. е. от равновесных действий других агентов (шаг 10). В классическом алгоритме Q-обучения обновление Q-значений зависит только от нахождения максимальных значений собственной Q-таблицы индивидуального агента.

Алгоритм Q-обучения Нэша для агента i приведен ниже:

1. Инициализировать
   $\gamma \in (0, 1)$; $\varepsilon \in (0,1)$;
   $i \leftarrow 1, \ldots, n$; $a_i \in A_i$; $|A_1| \leftarrow k$; $|A_n| \leftarrow m$;
   $Q_i(s, a_1, \ldots, a_n)$, $C_i(s, a_1, \ldots, a_n)$, $\alpha_i(s, a_1, \ldots, a_n) \leftarrow 0$, $\forall s \in S$.
2. Цикл
3. Выбрать действие a_i из состояния s в соответствии с максимальным значением $Q_i(s, a_1, \ldots, a_n)$ и с учетом параметра ε.
4. Обновить значение $C_i(s, a_1, \ldots, a_n) \leftarrow C_i(s, a_1, \ldots, a_n) + 1$.
5. Получить награду r_i и следующее состояние s'.
6. Построить матрицы наград $R_1\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right]$,
   $\ldots, R_n\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right]$ для состояния s'.
   6.1. Получить возможные действия $(a_1^1 \ldots a_1^k), \ldots, (a_1^1 \ldots a_n^m)$ агентов в состоянии s'.
   6.2. Заполнить матрицы наград
   $R_1\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right], \ldots, R_n\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right]$
   значениями из $Q_1(s', a_1, \ldots, a_n) \ldots Q_n(s', a_1, \ldots, a_n)$.
7. Вычислить равновесие Нэша $\left(\pi_1^*(s') \ldots \pi_n^*(s')\right)$ с использованием матриц наград $R_1\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right]$, $\ldots$, $R_n\left[\left[a_1^1 \ldots a_1^k\right]\right] \ldots \left[\left[a_n^1 \ldots a_n^m\right]\right]$.
8. Вычислить $\mathrm{Nash}Q_i(s') \leftarrow \pi_1^*(s'), \ldots, \pi_n^*(s') Q_i(s', a_1, \ldots, a_n)$.
9. Вычислить $\alpha_i(s, a_1, \ldots, a_n) \leftarrow \dfrac{1}{C_i(s, a_1, \ldots, a_n)}$.
10. Обновить $Q_i(s, a_1, \ldots, a_n) \leftarrow (1 - \alpha_i(s, a_1, \ldots, a_n)) Q_i(s, a_1, \ldots, a_n) +$
    $+ \alpha_i(s, a_1, \ldots, a_n)\left(r_i + \gamma \mathrm{Nash}Q_i(s')\right)$.

В представленной реализации алгоритма Q-обучения Нэша (см. подразд. 2.6.2) матрицы наград формируются на основе значений $Q_1(s', a_1, \ldots, a_n)$, $\ldots$, $Q_n(s', a_1, \ldots, a_n)$. Но агент может не иметь доступа к Q-таб-

лицам других агентов. В таких случаях ему требуются Q-таблица для себя и Q-таблицы для всех других агентов, участвующих в игре. Таким образом, агент должен сопровождать n Q-таблиц. При этом, если принять число состояний стохастической игры $|S|$, число действий агента i — $|A_i|$ и допустить, что число возможных действий всех агентов примерно одинаковое, т. е. $|A| \approx ... \approx |A_n| \approx |A|$, то сложность алгоритма Q-обучения Нэша можно оценить как линейную относительно числа состояний, полиномиальную относительно числа действий и экспоненциальную относительно числа агентов: $n|S||A|^n$.

2.4. Карта

Мультиагентное обучение в гл. 2 для матричных игр проводилось на карте 2m2zFOXmatrix.SC2Map (рис. 2.5). Как и в гл. 1, в начале каждого эпизода на карте слева, в точке Team 1, автоматически создаются два агента юнита морпеха расы терран, которыми управляет обучающий алгоритм, справа, в точке Team 2, автоматически создаются два агента юнита зерглинга расы зергов, которыми управляет компьютер, используя стандартные правила. Длительность одного эпизода игры составляет 3 с. На карте, по сравнению с картой в гл. 1 (см. рис. 1.4), появилось непроходимое препятствие, поэтому агент, который собирается атаковать, должен вначале сдвинуться вниз и только затем — вправо.

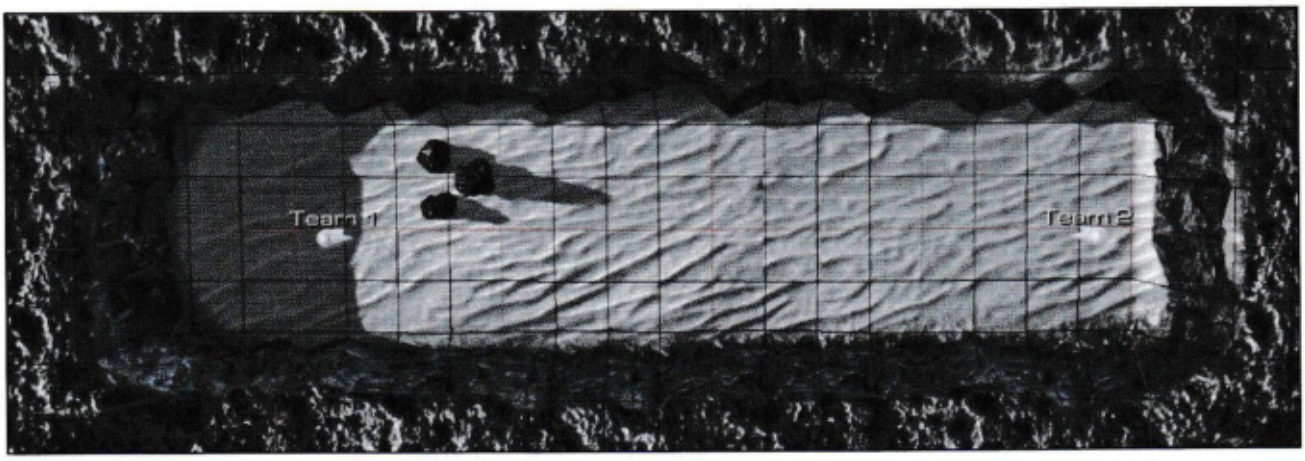


Рис. 2.5. Внешний вид карты 2m2zFOXmatrix.SC2Map в редакторе StarCraft II Editor

Для матричной игры SMAC зададим одинаковые матрицы наград первого и второго агентов морпехов, мотивирующие их выучивать совместные стратегии действий. Матрицы наград двух агентов матричной игры:

$$R_1 = \begin{bmatrix} 10 & 0 \\ -10 & 1 \end{bmatrix}, \; R_2 = \begin{bmatrix} 10 & -10 \\ 0 & 1 \end{bmatrix}.$$

В данной матричной игре у каждого агента заданы два действия: «вперед», «прятаться». В соответствии с матрицей наград оптимальной стратегией для данной карты будет стратегия «идти вперед вместе», реализуемая действиями «вперед», «вперед» первого и второго агентов. В результате этой стратегии каждый агент получит по 10 единиц награды. Однако если атаку одного агента не поддержит его напарник, то награды не будет (нуль), так как агенты не продемонстрировали совместных действий. Если оба агента будут выполнять действия «прятаться», «прятаться», то награда составит единицу. В случае если один агент в некотором смысле предаст другого и в ответ на действие напарника «вперед» выполнит действие «прятаться», то агент будет наказан отрицательной наградой −10. Таким образом, в соответствии с матрицей наград агенты должны выучить оптимальную стратегию действий «вперед», «вперед».

На рис. 2.6–2.9 продемонстрированы позиции агентов в начале и в конце выполнения каждой стратегии: «вперед», «вперед» (см. рис. 2.6); «вперед», «прятаться» (см. рис. 2.7); «прятаться», «вперед» (см. рис. 2.8); «прятаться», «прятаться» (см. рис. 2.9).

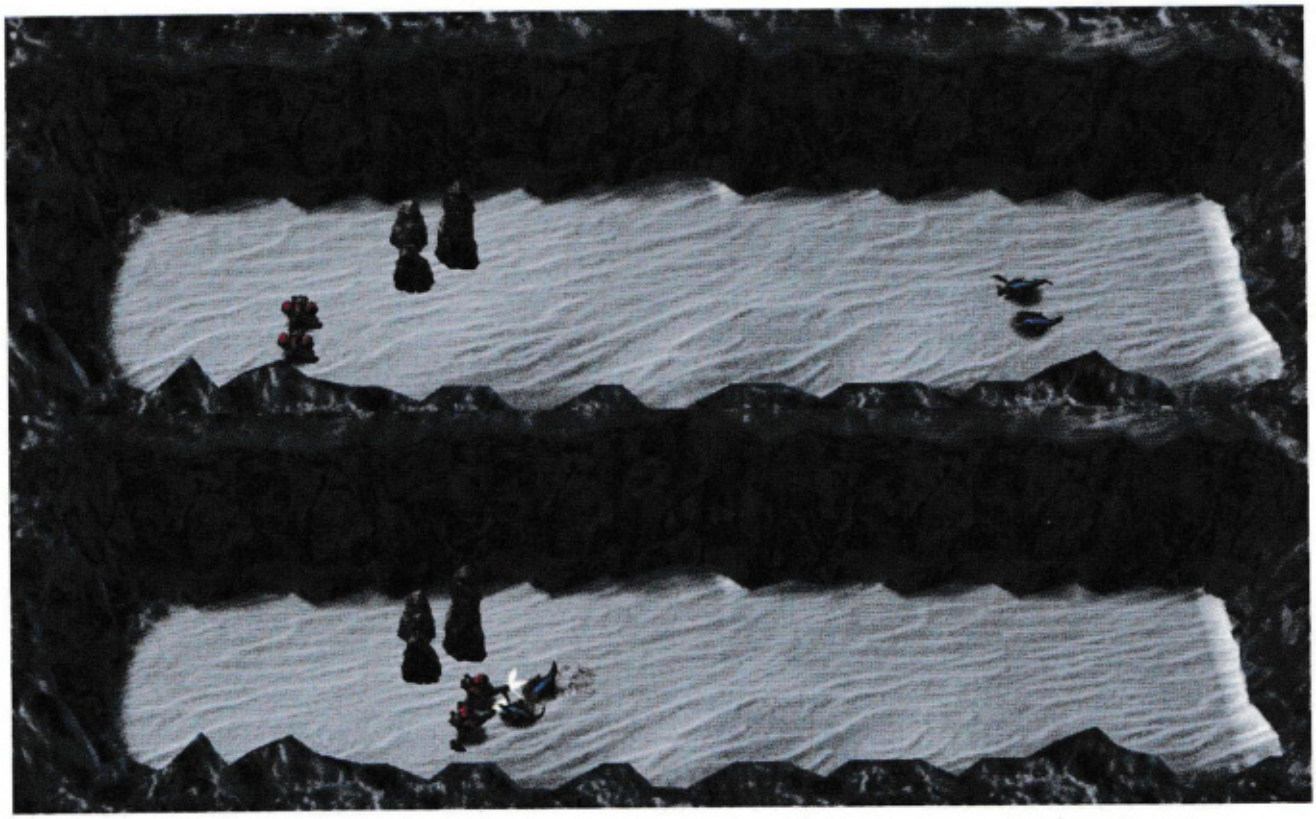


Рис. 2.6. Позиции агентов в начале и в конце выполнения стратегии «вперед», «вперед»

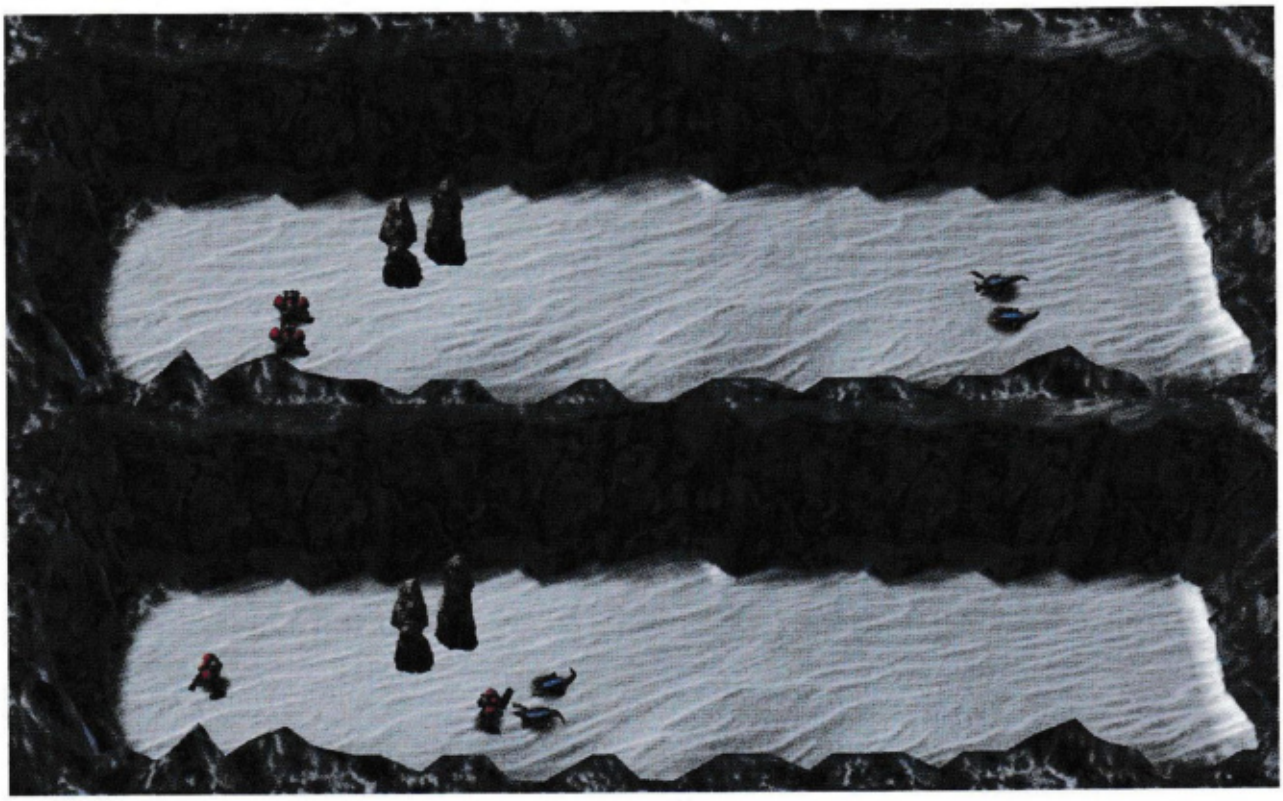


Рис. 2.7. Позиции агентов в начале и в конце выполнения стратегии «вперед», «прятаться»

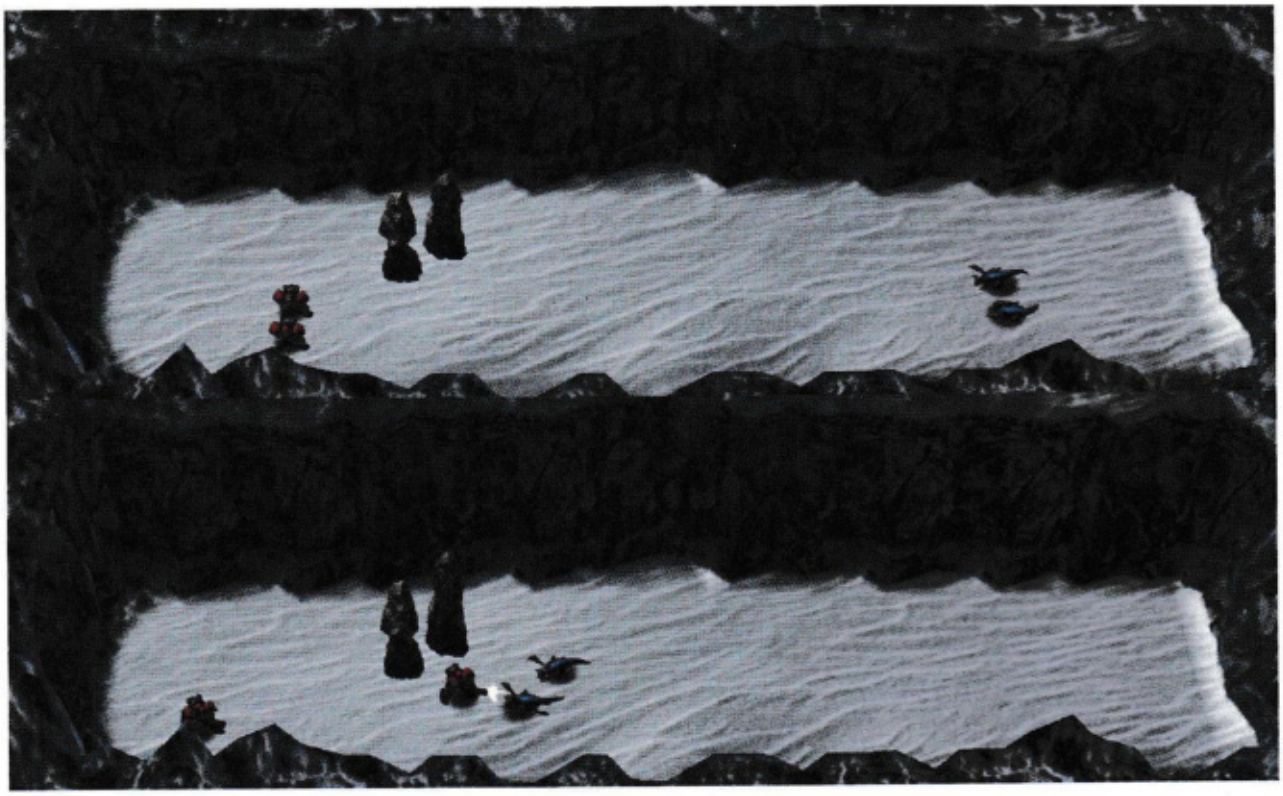


Рис. 2.8. Позиции агентов в начале и в конце выполнения стратегии «прятаться», «вперед»

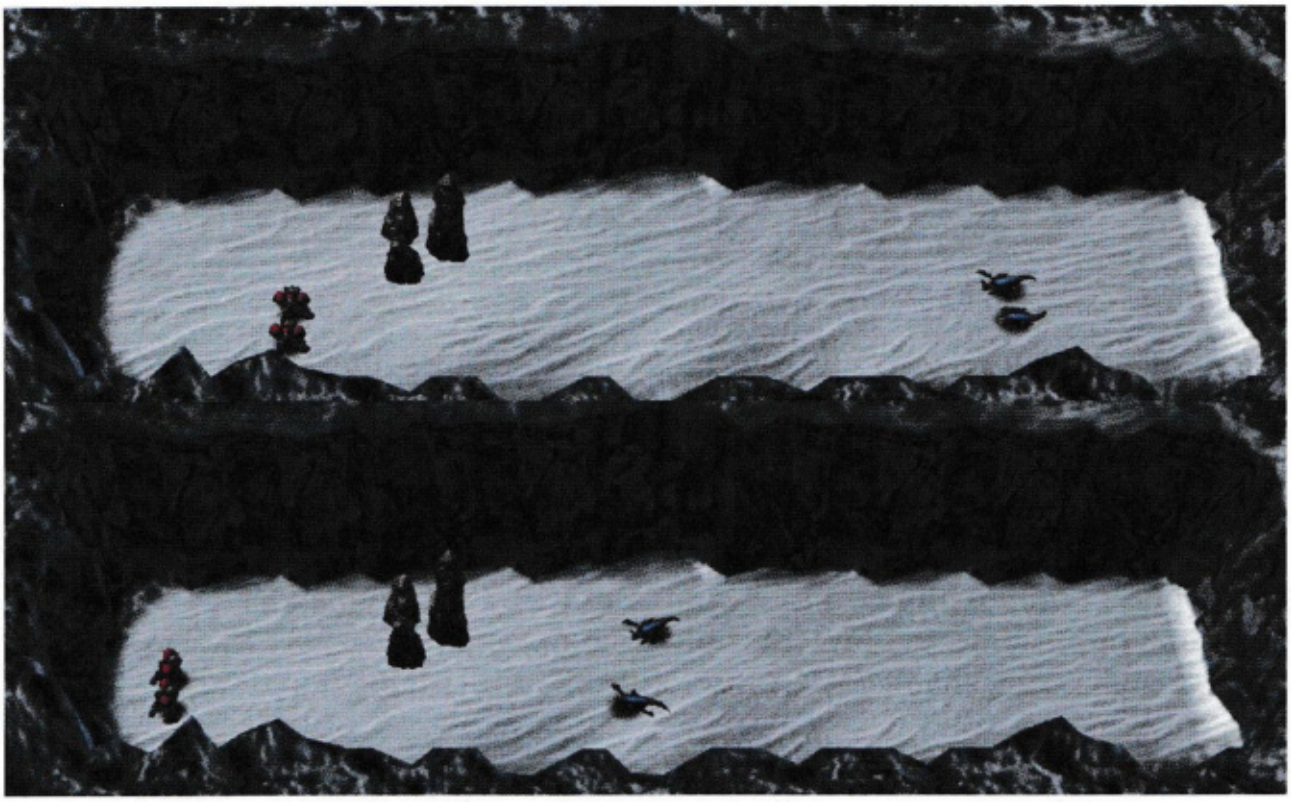


Рис. 2.9. Позиции агентов в начале и в конце выполнения стратегии «прятаться», «прятаться»

Результаты обучения агентов с помощью алгоритма WoLF-PHC будут продемонстрированы в подразд. 2.7.1.

В следующем разделе будут рассмотрены библиотеки, необходимые для программной реализации карты 2m2zFOXmatrixSC2Map в матричной игре.

2.5. Технология

Рассматриваемые в гл. 2 алгоритмы реализованы на языке программирования Python версии 3.6. В представленном в разд. 2.6 программном коде используются стандартные библиотеки: numpy для работы с массивами, random для генерации случайных чисел, matplotlib для построения графиков, pickle для ввода-вывода данных в файл, nashpy для вычисления равновесий Нэша. Библиотека SMAC фактически выступает в качестве визуализатора в среде StarCraft II принятых в игровых алгоритмах решений.

Карта 2m2zFOXmatrix.SC2Map создана на базе карты SMAC в редакторе StarCraft II Editor.

Ниже представлены изменения, которые необходимо внести в файл smac_maps.py, расположенный в папке «...\smac\env\starcraft2\maps», при создании новой карты или редактировании существующей:

```
«2m2zFOXmatrix»: {
    «n_agents»: 2,
    «n_enemies»: 2,
    «limit»: 240,
    «a_race»: «T»,
    «b_race»: «Z»,
    «unit_type_bits»: 0,
    «map_type»: «marines»,}
```

Эксперименты с алгоритмами проводились в операционной системе Windows 10, но для мультиагентного обучения с подкреплением в матричных и стохастических играх данное требование не является обязательным.

Примеры реализации алгоритмов в полном объеме представлены на платформе Github: https://github.com/Alekat13/FoxCommander

Видеопримеры работы алгоритмов приведены на YouTube: https://www.youtube.com/channel/UC33QPEjSlfA8P9rG0HtiH6g

2.6. Код

2.6.1. Алгоритм WoLF-PHC

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
import random
import matplotlib.pyplot as plt

#Функция выбора действия в зависимости от стратегии и параметра исследования
def select_actionMatrix(pi_agent, epsilon):
    if random.uniform(0, 1) < (1 - epsilon):
        #Исследуем пространство действий
        action = np.random.choice([0,1])
    else:
        #В зависимости от стратегии выбираем действия
        if pi_agent [0] > pi_agent [1]:
            action = 0
        elif pi_agent [0] < pi_agent [1]:
            action = 1
        #Если вероятность действий 0.5, то выбираем случайное действие
        elif pi_agent [0] == pi_agent [1]:
            action = np.random.choice([0,1])
    #Возвращаем выбранное действие
    return action
#Функция задает матрицу наград для матричной игры SMAC
def rewardMatrix (action_1, action_2):
    reward_1 = 0
    reward_2 = 0
    if (action_1, action_2) == (0, 0):
        reward_1 = 10
        reward_2 = 10
```

```
    elif (action_1, action_2) == (1, 0):
        reward_1 = -10
        reward_2 = -10
    elif (action_1, action_2) == (0, 1):
        reward_1 = 0
        reward_2 = 0
    elif (action_1, action_2) == (1, 1):
        reward_1 = 1
        reward_2 = 1
    return (reward_1, reward_2)

#Главная функция программы, содержащая реализацию алгоритма WoLF-PHC
#и визуализацию в среде SMAC
def main():
    #Определяем число агентов-игроков
    n_agents = 2

    n_actions_agent1 = 2 # число действий первого агента
    n_actions_agent2 = 2 # число действий второго агента

    #Определяем число эпизодов игры
    n_episodes = 1000
    #Задаем параметр для модификации обучающих параметров
    timeStep = 0
    #Инициализируем параметры, управляющие обучением
    alpha =  1 / (10 + 0.00001 * timeStep)
    gamma = 0.9
    epsilon = 0.5 / (1 + 0.0001 * timeStep)
    delta = 0
    delta_win = 0
    delta_lose = 0

    deltaAction = 0 # параметр для вычисления стратегии pi первого агента
    deltaAction1 = 0 # параметр действия 1 для вычисления deltaAction
    deltaAction2 = 0 # параметр действия 2 для вычисления deltaAction

    #Задаем пустую Q-таблицу для первого агента
    q_table_agent1 = np.zeros([n_actions_agent1])
    #Задаем пустую Q-таблицу для второго агента
    q_table_agent2 = np.zeros([n_actions_agent2])

    #Задаем пустую стратегию для первого агента
    pi_agent1 = np.zeros([n_actions_agent1])
    #Задаем пустую стратегию для второго агента
    pi_agent2 = np.zeros([n_actions_agent2])
    #Задаем пустую среднюю стратегию для первого агента WoLF-PHC
    pi_agent1_average = np.zeros([n_actions_agent1])
    #Задаем пустую среднюю стратегию для второго агента WoLF-PHC
    pi_agent2_average = np.zeros([n_actions_agent2])

    #Частота выполнения действия первым агентом
    countAction__agent1 = np.zeros([n_actions_agent1])
    #Частота выполнения действия вторым агентом
    countAction__agent2 = np.zeros([n_actions_agent2])
    #Сохраняем историю действий для вывода графика
    pi_agent1_History = []
    pi_agent2_History = []

    pi_agent1 [0] = 0.5 # инициализируем стратегию первого агента
    pi_agent1 [1] = 0.5 # инициализируем стратегию первого агента

    pi_agent2 [0] = 0.5 # инициализируем стратегию второго агента
    pi_agent2 [1] = 0.5 # инициализируем стратегию второго агента
```

```
#Цикл по эпизодам игры
for e in range(n_episodes):

    #Если эпизод закончен, то параметр = True
    terminated = False
    #Инициализируем награду за эпизод
    episode_reward = 0

    #Цикл внутри эпизода игры
    while not terminated:

        #Обнуляем вспомогательные параметры
        actions = []
        action = 0
        reward_agent1=0
        reward_agent2=0
        actionsFox = np.zeros([n_agents])
        #Вычисляем обучающие параметры алгоритма
        timeStep+=1
        alpha = 1 / (10 + 0.00001 * timeStep)
        epsilon = 0.5 / (1 + 0.0001 * timeStep)
        delta_win = 1.0 / (20000 + timeStep)
        delta_lose = 2.0 * delta_win

        #Цикл по множеству агентов, участвующих в игре
        for agent_id in range(n_agents):

            #Выбираем действие агента в соответствии со стратегией
            if agent_id == 0:
                action = select_actionMatrix(pi_agent1, epsilon)
            elif agent_id == 1:
                action = select_actionMatrix(pi_agent2, epsilon)

            #Собираем действия разных агентов для передачи в среду
            actions.append(action)
            actionsFox[agent_id] = action


        #Завершаем эпизод игры и получаем награду из матрицы наград
        terminated = True
        reward_agent1,reward_agent2 = rewardMatrix (int(actionsFox[0]),
                                                    int(actionsFox[1]))
        episode_reward += reward_agent1

        #Обновляем значения в Q-таблице
        q_table_agent1 [int(actionsFox[0])] = (1-alpha)*
                       q_table_agent1[int(actionsFox[0])] + alpha*
                       (reward_agent1 + gamma*np.max(q_table_agent1[:]))

        q_table_agent2 [int(actionsFox[1])] = (1-alpha)*
                       q_table_agent2[int(actionsFox[1])] + alpha*\
                       (reward_agent2 + gamma*np.max(q_table_agent2[:]))

        #Вычисляем среднюю стратегию
        countAction__agent1[int(actionsFox[0])]+=1
        countAction__agent2[int(actionsFox[1])]+=1

        pi_agent1_average[int(actionsFox[0])]=
              Pi_agent1_average[int(actionsFox[0])]+
              (1/CountAction__agent1[int(actionsFox[0])])*
              (Pi_agent1 [int(actionsFox[0])]-
               Pi_agent1_average[int(actionsFox[0])])
```

```
        Pi_agent2_average[int(actionsFox[1])]=
               pi_agent2_average[int(actionsFox[1])]+
               (1/countAction__agent2[int(actionsFox[1])])*
               (pi_agent2 [int(actionsFox[1])]-
                pi_agent2_average[int(actionsFox[1])])

        #Вычисляем быстрое или медленное delta для первого агента
        expected_value = 0
        expected_value_average = 0

        for aidx, _ in enumerate(pi_agent1):
            expected_value += pi_agent1[aidx]*q_table_agent1[aidx]
            expected_value_average += pi_agent1_average[aidx]*
                                      q_table_agent1[aidx]

        if expected_value > expected_value_average:
            delta = delta_win
        else:
            delta = delta_lose

        #Обновляем стратегию Pi первого агента

        deltaAction1 = np.min([pi_agent1 [0],
                               delta /(n_actions_agent1 - 1)])
        deltaAction2 = np.min([pi_agent1 [1],
                               delta / (n_actions_agent1 - 1)])


        if  int(actionsFox[0]) != np.argmax (q_table_agent1[:]):
            deltaAction = (-1)*deltaAction1
        else:
            deltaAction = deltaAction2

        pi_agent1 [0] = pi_agent1 [0] + deltaAction
        #Проверка — значение вероятности в границах [0;1]
        if pi_agent1 [0]>1: pi_agent1 [0]= 1
        if pi_agent1 [0]<0: pi_agent1 [0]= 0
        #Сумма вероятностей выбора действий всегда должна быть равна единице
        pi_agent1 [1] = 1 - pi_agent1 [0]
        pi_agent1_History.append(pi_agent1 [0])


        #Обновляем стратегию Pi второго агента по аналогии со стратегией
        #первого
        #...

    #Выводим на печать выученную стратегию действий в эпизоде
    print ("------------------------------------------------------------")
    print("Strategy Pi_agent1 in episode {} = {}".format(e, pi_agent1))
    print("Strategy Pi_agent2 in episode {} = {}".format(e, pi_agent2))

#Выводим на печать выученные Q-таблицы
print("Q_table_agent1=", q_table_agent1)
print("Q_table_agent2=", q_table_agent2)

#Выводим на печать графики
plt.figure(num=None, figsize=(6, 3), dpi=100, facecolor='w',
           edgecolor='k')
plt.plot(pi_agent1_History)
plt.xlabel('Номер итерации')
plt.ylabel('Вероятность выполнить действие 1 агентом 2')
plt.show()
```

```
plt.figure(num=None, figsize=(6, 3), dpi=100, facecolor='w',
           edgecolor='k')
plt.plot(pi_agent2_History)
plt.xlabel('Номер итерации')
plt.ylabel('Вероятность выполнить действие 1 агентом 2')
plt.show()

#Визуализируем выученные стратегии в Starcraft II с помощью SMAC
env = StarCraft2Env(map_name="2m2zFOXmatrix")
env_info = env.get_env_info()
n_agents = env_info["n_agents"]
n_episodes = 3 # число повторов
#Визуализация стратегии «вперед», «вперед» (realtime=False)
forward_both = np.array([
[1, 1, 1, 1, 3, 1, 3, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1],
[1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 1, 1, 1, 1, 1, 1, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1]])
#Визуализация стратегии «прятаться», «прятаться»
hide_both = np.array([
[5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1,
 5, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
[5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1,
 5, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])
#Визуализация стратегии «вперед», «прятаться»
forward_hide = np.array([
[3, 3, 1, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1],
[5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1,
 5, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])
#Визуализация стратегии «прятаться», «вперед»
hide_forward = np.array([
[5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1, 5, 1,
 5, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1,
 4, 1, 4, 1, 4, 1, 4, 1, 4, 1, 4, 1]])

for e in range(n_episodes):

    env.reset()
    terminated = False
    episode_reward = 0
    n_steps = 1

    if (pi_agent1[0]>pi_agent1[1]) and (pi_agent2[0]>pi_agent2[1]):
        strategy = forward_both
    elif (pi_agent1[1]>pi_agent1[0]) and (pi_agent2[1]>pi_agent2[0]):
        strategy = hide_both
    elif (pi_agent1[0]>pi_agent1[1]) and (pi_agent2[0]<pi_agent2[1]):
        strategy = forward_hide
    elif (pi_agent1[0]<pi_agent1[1]) and (pi_agent2[0]>pi_agent2[1]):
        strategy = hide_forward

    while not terminated:
```

```
            actions = []

            for agent_id in range(n_agents):
                if n_steps < len(strategy[0]):
                    action = strategy[agent_id][n_steps - 1]
                else:
                    avail_actions = env.get_avail_agent_actions(agent_id)
                    if avail_actions[6] == 1:
                        action = 6
                    elif avail_actions[7] == 1:
                        action = 7
                    else:
                        avail_actions_ind = np.nonzero(avail_actions)[0]
                        action = np.random.choice(avail_actions_ind)
                actions.append(action)

            reward, terminated, _ = env.step(actions)
            reward = reward / n_steps
            n_steps += 1
            episode_reward += reward

    env.close()


#Точка входа в программу
if __name__ == "__main__":
    main()
```

2.6.2. Алгоритм Nash-Q

```
#Подключаем библиотеки
import numpy as np
import random
import matplotlib.pyplot as plt
import nashpy as nash
import pickle

#Вывод массива целиком
np.set_printoptions(threshold=np.inf)

#Создаем глобальную переменную для подсчета совместных действий
def globja():
    if not hasattr(globja, '_state'):
        #Инициализация значения
        globja._state = 0
    print(globja._state)
    globja._state = globja._state + 1

#Выбираем действия в зависимости от параметра эпсилон и значения Q-таблицы
def select_actionFox(agent_id, state, avail_actions_ind1, avail_actions_ind2,
n_actionsFox, epsilon, Q_table):

    action = 0

    #Исследуем пространство действий
    if random.uniform(0, 1) < (1 - epsilon):
        if agent_id == 0:
            action = np.random.choice(avail_actions_ind1)
        elif agent_id == 1:
            action = np.random.choice(avail_actions_ind2)
```

```
    else:
        maxelement = -1000
        maxaction1 = 0
        maxaction2 = 0

        avail_actions_ind1int = 0
        avail_actions_ind2int = 0
        stateFoxint = int(state)

        #На основе действий выбираем значения из Q-таблицы
        if agent_id == 0:

            for i in range(len(avail_actions_ind1)):
                for j in range(len(avail_actions_ind2)):
                    avail_actions_ind1int = int (avail_actions_ind1[i])
                    avail_actions_ind2int = int (avail_actions_ind2[j])
                    if maxelement < Q_table[stateFoxint,
                     avail_actions_ind1int, avail_actions_ind2int]:
                        maxelement = Q_table[stateFoxint,
                     avail_actions_ind1int, avail_actions_ind2int]
                        maxaction1 = avail_actions_ind1int

            action = maxaction1

        elif agent_id == 1:

            for i in range(len(avail_actions_ind2)):
                for j in range(len(avail_actions_ind1)):
                    avail_actions_ind2int = int (avail_actions_ind2[i])
                    avail_actions_ind1int = int (avail_actions_ind1[j])
                    if maxelement < Q_table[stateFoxint,
                     avail_actions_ind2int, avail_actions_ind1int]:
                        maxelement = Q_table[stateFoxint,
                     avail_actions_ind2int, avail_actions_ind1int]
                        maxaction2 = avail_actions_ind2int

            action = maxaction2

    return action

#Получаем состояние как позицию агента в клетке мира-сетки
def get_stateFox(agent_id, gridWorld, gridIndexes):

    state = 0

    if agent_id == 0:
        for i in range(len(gridWorld)):
            for j in range(len(gridWorld[i])):
                if gridWorld[i][j] == 1:
                    state = gridIndexes[i][j]

    if agent_id == 1:
        for i in range(len(gridWorld)):
            for j in range(len(gridWorld[i])):
                if gridWorld[i][j] == 2:
                    state = gridIndexes[i][j]

    return state
#Получаем возможные действия агента в зависимости от состояния
def get_avail_agent_actionsFox(stateFox):

    agent_actions = []
    if stateFox == 0:
```

```
        agent_actions = [1, 2]
    elif stateFox == 1:
        agent_actions = [1, 2, 3]
    elif stateFox == 2:
        agent_actions = [2, 3]
    elif stateFox == 3:
        agent_actions = [0, 1, 2]
    elif stateFox == 4:
        agent_actions = [0, 1, 2, 3]
    elif stateFox == 5:
        agent_actions = [0, 2, 3]
    elif stateFox == 6:
        agent_actions = [0, 1]
    elif stateFox == 7:
        agent_actions = [0, 1, 3]
    elif stateFox == 8:
        agent_actions = [0, 3]

    return agent_actions

#Выполняем действие агента в мире-сетке
def stepFox(actionsFox, stateFox, gridIndexes, gridWorld, Joint_Goalstate):

    terminated = False
    #Инициализируем награду, отдельную для каждого агента
    reward = np.zeros([2])
    stateFoxOld = np.zeros([2])
    stateFoxOld[0] = stateFox[0]
    stateFoxOld[1] = stateFox[1]

    stateFoxNew = np.zeros([2])

    gridWorldOld = np.zeros([3, 3])

    for i in range(len(gridWorld)):
            for j in range(len(gridWorld[i])):
                gridWorldOld[i][j] = gridWorld[i][j]

    #Перемещаем первого агента
    for i in range(len(gridIndexes)):
        for j in range(len(gridIndexes[i])):
            if gridIndexes[i][j] == stateFox[0]:
                if actionsFox[0] == 0:
                    gridWorld[i-1][j]=1
                    gridWorld[i][j]=0
                elif actionsFox[0] == 1:
                    gridWorld[i][j+1]=1
                    gridWorld[i][j]=0
                elif actionsFox[0] == 2:
                    gridWorld[i+1][j]=1
                    gridWorld[i][j]=0
                elif actionsFox[0] == 3:
                    gridWorld[i][j-1]=1
                    gridWorld[i][j]=0

    #Находим новое состояние первого агента
    stateFoxNew[0] = get_stateFox(0, gridWorld, gridIndexes)

    #Перемещаем второго агента
    for i in range(len(gridIndexes)):
        for j in range(len(gridIndexes[i])):
            if gridIndexes[i][j] == stateFox[1]:
                if actionsFox[1] == 0:
```

```
                    gridWorld[i-1][j]=2
                    gridWorld[i][j]=0
                elif actionsFox[1] == 1:
                    gridWorld[i][j+1]=2
                    gridWorld[i][j]=0
                elif actionsFox[1] == 2:
                    gridWorld[i+1][j]=2
                    gridWorld[i][j]=0
                elif actionsFox[1] == 3:
                    gridWorld[i][j-1]=2
                    gridWorld[i][j]=0

    #Находим новое состояние второго агента
    stateFoxNew[1] = get_stateFox(1, gridWorld, gridIndexes)

    if (stateFoxNew[0] == 7) and (stateFoxNew[1] == 7):
        Joint_Goalstate += 1
        print ('Число совместных попаданий в целевое состояние: ')
        globja()
    #Проверяем; если агенты в целевом состоянии, то заканчиваем эпизод
    #с наградой
    if (stateFoxNew[0] == 7):
        terminated = True
        reward[0] = 100
    elif (stateFoxNew[1] == 7):
        terminated = True
        reward[1] = 100
    #Проверяем, нет ли совпадений в одной клетке, в этом случае делаем откат
    elif (stateFoxNew[0] == stateFoxNew[1]) and (stateFoxNew[0] != 7) and
         (stateFoxNew[1] != 7):
        reward[0] = -1
        reward[1] = -1
        stateFoxNew = stateFoxOld
        gridWorld = gridWorldOld

    return gridWorld, stateFoxNew, reward, terminated, Joint_Goalstate

#Главная функция программы, содержащая реализацию алгоритма Q-обучения Нэша
def main():

    #Задаем размер мира-сетки
    world_height = 3
    world_width = 3

    #Определяем индексы клеток мира-сетки
    gridIndexes = np.array([[0, 1, 2],
                            [3, 4, 5],
                            [6, 7, 8]])

    #Инициализируем мир-сетку нулями
    gridWorld = np.zeros([world_height, world_width])

    #Задаем начальную позицию первого агента
    gridWorld [0][0] = 1

    #Задаем начальную позицию второго агента
    gridWorld [0][2] = 2

    #Задаем обозначение цели в состоянии 7
    gridWorld [2][1] = 3

    #Определяем число агентов-игроков
    n_agents = 2
```

```
n_actions_agent1 = 4 # число действий первого агента
n_actions_agent2 = 4 # число действий второго агента
#Определяем число эпизодов игры
n_episodes = 1000

#Число состояний для одного агента в мире-сетке
statesNumber = world_height*world_width

#Инициализируем параметры, управляющие обучением
Alpha1 = np.zeros([statesNumber, n_actions_agent1, n_actions_agent2])
Alpha2 = np.zeros([statesNumber, n_actions_agent2, n_actions_agent1])
gamma = 0.99
epsilon = 0.1

#Задаем свою пустую Q-таблицу для первого агента
Q_table_agent1_own = np.zeros([statesNumber, n_actions_agent1,
                               n_actions_agent2])

#Задаем свою пустую Q-таблицу для второго агента
Q_table_agent2_own = np.zeros([statesNumber, n_actions_agent2,
                               n_actions_agent1])

#Задаем пустую стратегию для первого агента (свою и оппонента)
Pi_agent1 = np.zeros([n_actions_agent1*n_actions_agent2,
                      n_actions_agent1])
Pi_agent1_op = np.zeros([n_actions_agent1*n_actions_agent2,
                         n_actions_agent1])

#Задаем пустую стратегию для второго агента (свою и оппонента)
Pi_agent2 = np.zeros([n_actions_agent1*n_actions_agent2,
                      n_actions_agent2])
Pi_agent2_op = np.zeros([n_actions_agent1*n_actions_agent2,
                         n_actions_agent2])

#Задаем пустые значения Nash-Q для первого агента
NashQ1 = np.zeros([n_actions_agent1, n_actions_agent2])
#Задаем пустые значения Nash-Q для второго агента
NashQ2 = np.zeros([n_actions_agent2, n_actions_agent1])

#Подсчитываем число попаданий в состояние и выполнение совместного
#действия
CountStateJAc__agent1 = np.ones([statesNumber, n_actions_agent1,
                                 n_actions_agent2])
#Подсчитываем число попаданий в состояние и выполнение совместного
#действия
CountStateJAc__agent2 = np.ones([statesNumber, n_actions_agent2,
                                 n_actions_agent1])

#Сохраняем историю наград для вывода графика
Reward_history_agent1 = []
Reward_history_agent2 = []

#Сохраняем число совместных попаданий в целевое состояние
Joint_Goalstate = 0
Joint_Goalstate_history = []

#Цикл по эпизодам игры
for e in range(n_episodes):

    #Инициализируем мир-сетку нулями
    gridWorld = np.zeros([world_height, world_width])
```

```
#Задаем начальную позицию первого агента
gridWorld [0][0] = 1

#Задаем начальную позицию второго агента
gridWorld [0][2] = 2

#Задаем обозначение цели в состоянии 7
gridWorld [2][1] = 3

print ("---------------Эпизод---------------")
print ("-----------------", e,"-----------------")
print ("Начальный мир-сетка")
print (gridWorld)

#Если эпизод закончен то параметр = True
terminated = False
#Инициализируем награду за эпизод
episode_reward = 0
#Подсчитываем число шагов в эпизоде
stepnumber = 0

#Цикл внутри эпизода игры
while not terminated:

    #Обнуляем вспомогательные параметры
    reward = np.zeros([n_agents])
    avail_actions_ind1 = []
    avail_actions_ind2 = []
    #Сохраняем историю действий на одном шаге для разных агентов
    actionsFox = np.zeros([n_agents])
    #Сохраняем историю состояний на одном шаге для разных агентов
    stateFox= np.zeros([n_agents])

    #Получаем состояния агентов как позицию в клетке мира-сетки
    stateFox[0] = get_stateFox(0, gridWorld, gridIndexes)
    stateFox[1] = get_stateFox(1, gridWorld, gridIndexes)

    #Получаем возможные действия агентов
    avail_actions_ind1 = get_avail_agent_actionsFox(stateFox[0])
    avail_actions_ind2 = get_avail_agent_actionsFox(stateFox[1])

    #Выбираем действия агентов в зависимости от параметра эпсилон
    #и значений Q-таблицы
    actionsFox[0] = select_actionFox(0, stateFox[0],
     avail_actions_ind1, avail_actions_ind2, n_actions_agent1,
     epsilon, Q_table_agent1_own)
    actionsFox[1] = select_actionFox(1, stateFox[1],
     avail_actions_ind1, avail_actions_ind2, n_actions_agent2,
     epsilon, Q_table_agent2_own)

    #Подсчитываем число совместных действий в определенном
    #состоянии
    CountStateJAc__agent1[int (stateFox[0]), int (actionsFox[0]), int
                          (actionsFox[1])] += 1
    CountStateJAc__agent2[int (stateFox[1]), int (actionsFox[1]), int
                          (actionsFox[0])] += 1

    #Завершаем эпизод игры и получаем награду от среды
    gridWorld, stateFoxNew, reward, terminated, Joint_Goalstate =
     stepFox(actionsFox, stateFox, gridIndexes, gridWorld,
             Joint_Goalstate)
    stepnumber += 1
```

```
            if terminated == True:
                Reward_history_agent1.append(reward [0])
                Reward_history_agent2.append(reward [1])
                Joint_Goalstate_history.append(Joint_Goalstate)

            episode_reward += reward[0]+reward[1]

            print ("Шаг: ", stepnumber)
            print (gridWorld)
            print ('Награда за эпизод: ', episode_reward)


###################_Обучаем_################################################

            #Получаем возможные действия агентов в новом состоянии
            avail_actions_ind1 = []
            avail_actions_ind2 = []
            avail_actions_ind1 = get_avail_agent_actionsFox(stateFoxNew[0])
            avail_actions_ind2 = get_avail_agent_actionsFox(stateFoxNew[1])

            #Инициализируем матрицу наград
            rewardMatrix1 = np.zeros([n_actions_agent1, n_actions_agent2])
            rewardMatrix2 = np.zeros([n_actions_agent2, n_actions_agent1])

            #Выбираем данные из первой Q-таблицы для первого агента
            avail_actions_ind1index = 0
            avail_actions_ind2index = 0
            stateFoxNewint = int (stateFoxNew[0])
            for i in range(len(avail_actions_ind1)):
                for j in range(len(avail_actions_ind2)):
                    avail_actions_ind1index = int (avail_actions_ind1[i])
                    avail_actions_ind2index = int (avail_actions_ind2[j])

rewardMatrix1[avail_actions_ind1index][avail_actions_ind2index] =
 Q_table_agent1_own[stateFoxNewint, avail_actions_ind1index,
                    avail_actions_ind2index]

            #Выбираем данные из второй Q-таблицы для второго агента
            avail_actions_ind2index = 0
            avail_actions_ind1index = 0
            stateFoxNewint = int (stateFoxNew[1])
            for i in range(len(avail_actions_ind2)):
                for j in range(len(avail_actions_ind1)):
                    avail_actions_ind2index = int (avail_actions_ind2[i])
                    avail_actions_ind1index = int (avail_actions_ind1[j])

rewardMatrix2[avail_actions_ind2index][avail_actions_ind1index] =
 Q_table_agent2_own[stateFoxNewint, avail_actions_ind2index,
                    avail_actions_ind1index]
#############################################################################
            #Вычисляем стратегии pi как равновесия Нэша матричной игры
            #с помощью библиотеки nashpy
            #Равновесия Нэша для первого агента
            rps = nash.Game(rewardMatrix1, rewardMatrix2)

            equilibria = rps.support_enumeration()

            #Передаем данные из генератора в массив
            tempi = 0
            tempj = 0
            for eq in equilibria:
                for tempj in range(4):
                    Pi_agent1[tempi][tempj] = eq[0][tempj]
```

```
            Pi_agent1_op[tempi][tempj] = eq[1][tempj]
            tempj += 1
        tempi += 1
        tempj = 0

#Равновесия Нэша для второго агента
rps2 = nash.Game(rewardMatrix2, rewardMatrix1)

equilibria2 = rps2.support_enumeration()

tempi = 0
tempj = 0
for eq in equilibria2:
    for tempj in range(4):
        Pi_agent2[tempi][tempj] = eq[0][tempj]
        Pi_agent2_op[tempi][tempj] = eq[1][tempj]
        tempj += 1
    tempi += 1
    tempj = 0

#####################################################################
#Извлекаем индексы ненулевых действий из равновесных стратегий
indexAgent1Action1 = []
indexAgent1Action2 = []

for i in range(len(Pi_agent1)):
    for j in range(len(Pi_agent1[i])):
        if Pi_agent1[i][j] != 0:
            indexAgent1Action1.append(j)

for k in range(len(Pi_agent1_op)):
    for l in range(len(Pi_agent1_op[k])):
        if Pi_agent1_op[k][l] != 0:
            indexAgent1Action2.append(l)

#####################################################################
#Вычисляем Nash-Q и обновляем Q-таблицу для первого агента
stateFoxNewint = int (stateFoxNew[0])
NashQ1 = np.zeros([n_actions_agent1, n_actions_agent2])
for i in range(len(indexAgent1Action1)):
    for j in range(len(indexAgent1Action2)):
        #Вычисляем Nash-Q
        NashQ1[indexAgent1Action1[i], indexAgent1Action2[j]] =
         Q_table_agent1_own [stateFoxNewint,
           indexAgent1Action1[i], indexAgent1Action2[j]]
        #Вычисляем Alpha
        Alpha1[stateFoxNewint, indexAgent1Action1[i],
         indexAgent1Action2[j]] = 1 /
         CountStateJAc__agent1[stateFoxNewint,
         indexAgent1Action1[i], indexAgent1Action2[j]]
        #Обновляем Q-таблицу
        Q_table_agent1_own [stateFoxNewint,
         indexAgent1Action1[i], indexAgent1Action2[j]] = (1-
         Alpha1[stateFoxNewint, indexAgent1Action1[i],
         indexAgent1Action2[j]])*Q_table_agent1_own
         [stateFoxNewint,indexAgent1Action1[i],
         indexAgent1Action2[j]] + \
         Alpha1[stateFoxNewint, indexAgent1Action1[i],
         indexAgent1Action2[j]]*(reward[0] +
         gamma*NashQ1[indexAgent1Action1[i],
         indexAgent1Action2[j]])

indexAgent2Action1 = []
```

```
            indexAgent2Action2 = []
            for i in range(len(Pi_agent2)):
                for j in range(len(Pi_agent2[i])):
                    if Pi_agent2[i][j] != 0:
                        indexAgent2Action1.append(j)

            for k in range(len(Pi_agent2_op)):
                for l in range(len(Pi_agent2_op[k])):
                    if Pi_agent2_op[k][l] != 0:
                        indexAgent2Action2.append(l)

            ######################################################################
            #Вычисляем Nash-Q и обновляем Q-таблицу для второго агента
            stateFoxNewint = int (stateFoxNew[1])
            NashQ2 = np.zeros([n_actions_agent2, n_actions_agent1])
            for i in range(len(indexAgent2Action1)):
                for j in range(len(indexAgent2Action2)):

                    NashQ2[indexAgent2Action1[i], indexAgent2Action2[j]] =
                     Q_table_agent2_own [stateFoxNewint,
                     indexAgent2Action1[i], indexAgent2Action2[j]]

                    Alpha2[stateFoxNewint, indexAgent2Action1[i],
                     indexAgent2Action2[j]] = 1 /
                     CountStateJAc__agent2[stateFoxNewint,
                     indexAgent2Action1[i], indexAgent2Action2[j]]

                Q_table_agent2_own [stateFoxNewint,
                 indexAgent2Action1[i], indexAgent2Action2[j]] = (1-
                 Alpha2[stateFoxNewint, indexAgent2Action1[i],
                 indexAgent2Action2[j]])*Q_table_agent2_own
                 [stateFoxNewint,indexAgent2Action1[i], indexAgent2Action2[j]]
                 + \
                 Alpha2[stateFoxNewint, indexAgent2Action1[i],
                 indexAgent2Action2[j]]*(reward[0] +
                 gamma*NashQ2[indexAgent2Action1[i], indexAgent2Action2[j]])

    #Выводим на печать и сохраняем в файл выученные Q-таблицы
    print("Q_table_agent1_own=", Q_table_agent1_own)
    print("Q_table_agent2_own=", Q_table_agent2_own)

    with open("se31_1.pkl", 'wb') as f:
        pickle.dump(Q_table_agent1_own, f)

    with open("se31_2.pkl", 'wb') as f:
        pickle.dump(Q_table_agent2_own, f)

    #Выводим на печать график
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(Joint_Goalstate_history)
    plt.xlabel('Номер итерации')
    plt.ylabel('Число равновесий Нэша')
    plt.show()

#Точка входа в программу
if __name__ == "__main__":
    main()
```

2.7. Эксперимент

2.7.1. Матричные игры

На рис. 2.10, 2.11 представлены результаты обучения первого агента с помощью алгоритма PHC в двухагентной матричной игре «Орлянка» при заданных параметрах обучения $\alpha = 0.1$, $\delta = 0.5$, $\gamma = 0.9$, $\varepsilon = 0.7$ на 100 эпизодах игры. Показано, что алгоритм PHC сходится к оптимальной стратегии $\pi_1^*\left(a_1^1, a_2^1\right) = [1,0]$ при стационарной стратегии второго агента, который всегда выполняет первое действие $\pi_2\left(a_1^2, a_2^2\right) = [1,0]$ (см. рис. 2.10). При одновременном обучении двух агентов алгоритм PHC сходится к оптимальной стратегии $\pi_1^*\left(a_1^1, a_2^1\right) = [0.5, 0.5]$ (см. рис. 2.11). На графиках заметна сильная осцилляция алгоритма PHC рядом с оптимальной стратегией.

В более сложных случаях мультиагентного обучения проблема сходимости обостряется еще сильнее, не давая возможности широкого практического применения алгоритма PHC. Чтобы устранить недостаток сходимости алгоритма PHC, был разработан алгоритм WoLF-PHC.

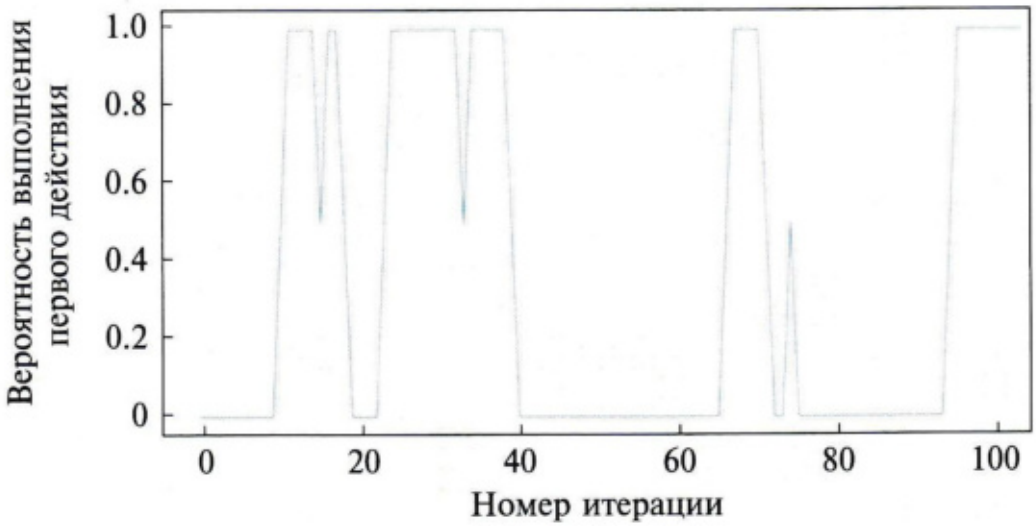


Рис. 2.10. Результаты обучения первого агента с помощью алгоритма PHC в матричной игре «Орлянка» в случае стационарной стратегии выбора первого действия вторым агентом

На рис. 2.12 представлены результаты обучения первого агента с помощью алгоритма WoLF-PHC в двухагентной матричной игре «Орлянка» при заданном параметре $\gamma = 0.9$ и динамически вычисляемых (t — номер итерации) параметрах обучения $\alpha = \frac{1}{10 + 0.00001t}$, $\varepsilon = \frac{0.5}{1 + 0.0001t}$, $\delta_w = \frac{1}{20\,000 + t}$, $\delta_l = 2\delta_w$ на 100 000 эпизодов игры. Алгоритм WoLF-PHC,

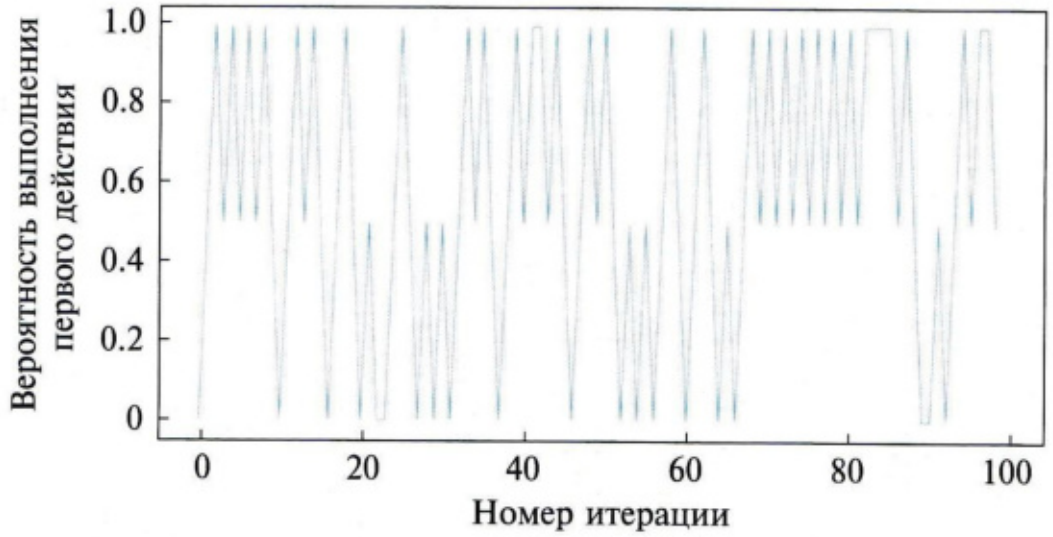
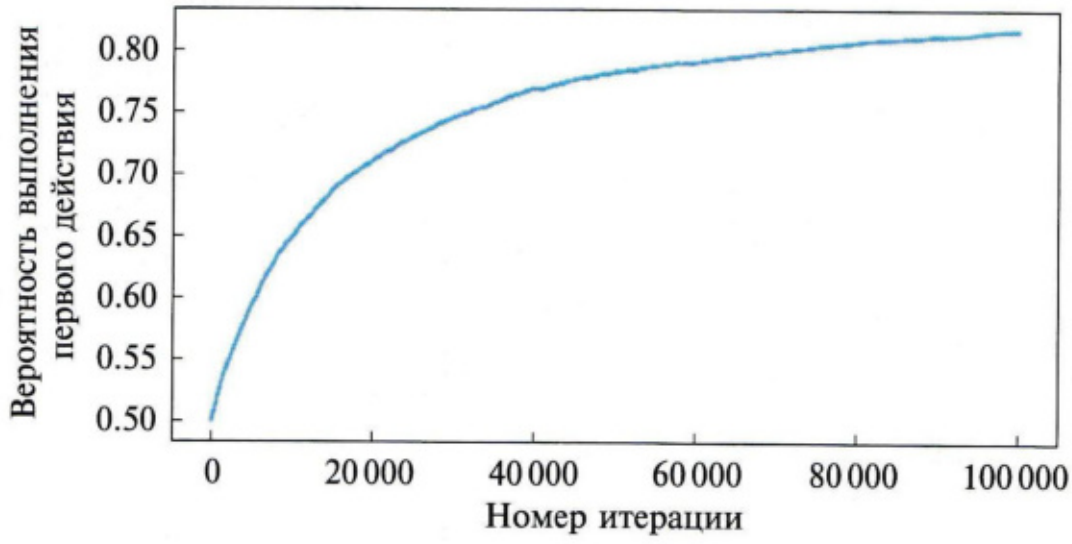
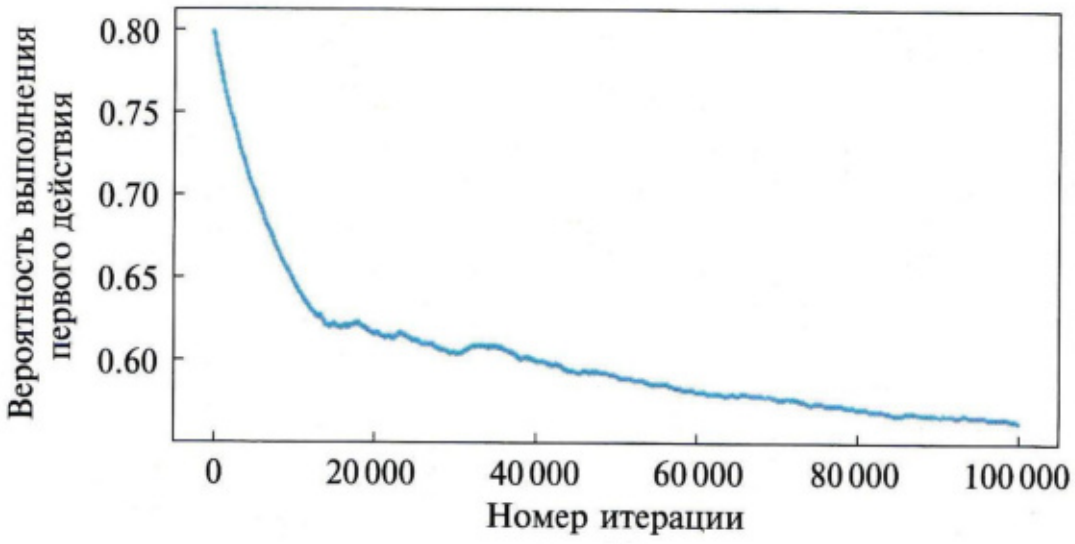


Рис. 2.11. Результаты обучения первого агента с помощью алгоритма PHC в матричной игре «Орлянка» в случае обучения первого и второго агентов



а



б

Рис. 2.12. Результаты обучения первого агента с помощью алгоритма WoLF-PHC в матричной игре «Орлянка» в случае:
а — стационарной стратегии выбора первого действия вторым агентом;
б — обучения первого и второго агентов

инициализированный стратегией $\pi_1\left(a_1^1, a_2^1\right)=[0.5, 0.5]$, сходится к оптимальной стратегии $\pi_1^*\left(a_1^1, a_2^1\right)=[1, 0]$ при стационарной стратегии второго агента, который всегда выполняет первое действие $\pi_2\left(a_1^2, a_2^2\right)=[1, 0]$ (рис. 2.12, *а*). При одновременном обучении двух агентов алгоритм WoLF-PHC, инициализированный стратегией $\pi_1\left(a_1^1, a_2^1\right)=[0.8, 0.2]$, сходится к оптимальной стратегии $\pi_1^*\left(a_1^1, a_2^1\right)=[0.5, 0.5]$ (рис. 2.12, *б*).

На рис. 2.13 представлены результаты обучения агентов стратегии «вперед», «вперед» с помощью алгоритма WoLF-PHC в матричной игре SMAC (см. раздел 2.4).

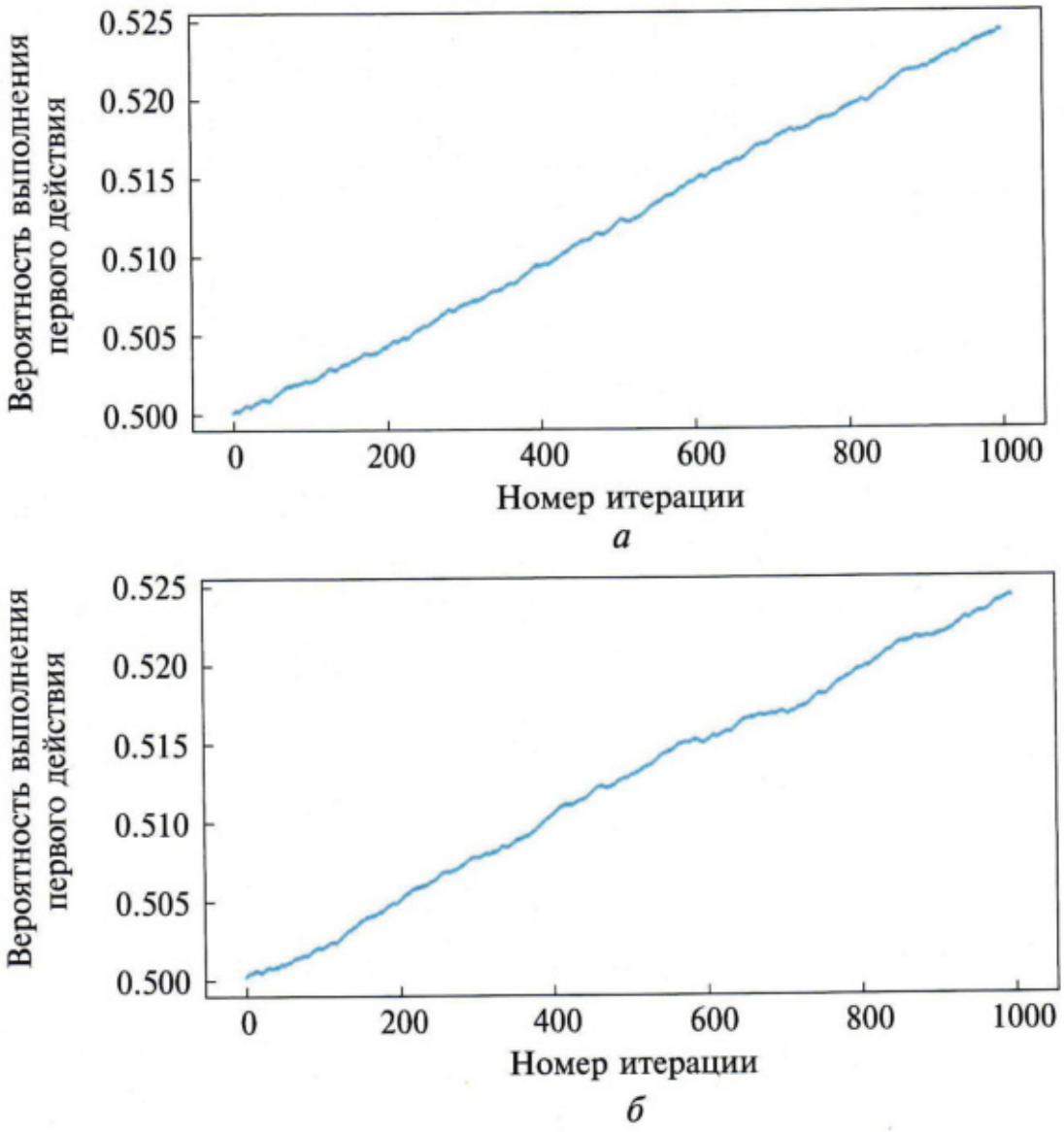


Рис. 2.13. Результаты обучения первого агента с помощью алгоритма WoLF-PHC в матричной игре SMAC в случае:
а — стационарной стратегии выбора первого действия вторым агентом;
б — обучения первого и второго агентов

Как и предполагалось, проблема сходимости, характерная для алгоритма PHC, успешно преодолена в алгоритме WoLF-PHC на основе реализации принципа «выигрывай или учись быстро» и использования усредненной стратегии. Кроме того, в данной реализации алгоритма WoLF-PHC использованы динамические параметры обучения α, ε, δ_w, δ_l, подробно исследованные в научной литературе.

2.7.2. Стохастические игры

На рис. 2.14 представлены результаты одновременного обучения двух агентов с помощью алгоритма Q-обучения Нэша в двухагентной стохастической игре «Мир-сетка 3×3» при заданных параметрах обучения $\gamma = 0.99$, $\varepsilon = 0.1$ на 5000 эпизодов игры.

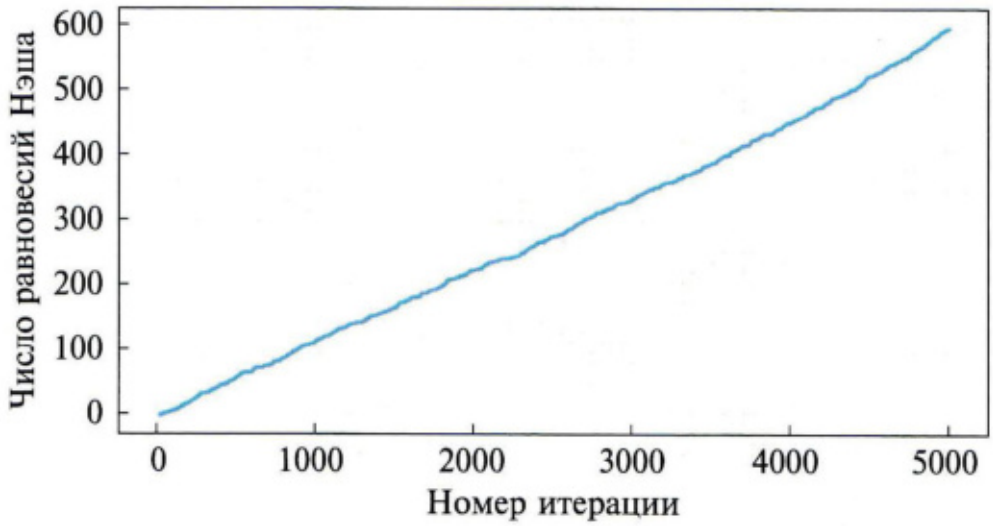


Рис. 2.14. Результаты мультиагентного обучения алгоритмом Q-обучения Нэша в двухагентной стохастической игре «Мир-сетка 3×3»

В каждом эпизоде игры агент совершает в среднем шесть действий, т. е. эксперимент в среднем требует выполнения 30 000 действий. При этом в данной стохастической игре существует 324 кортежа состояния-действия $(s, a_1, ..., a_n)$, поэтому каждый кортеж появляется в игре в среднем 93 раза. В научной литературе для алгоритма Q-обучения Нэша рекомендуется сделать параметр скорости обучения α инвертированным относительно числа C_i появлений в игре кортежей состояния-действия $\alpha_i(s, a_1, \ldots, a_n) \leftarrow \frac{1}{C_i(s, a_1, \ldots, a_n)}$. Таким образом, параметр скорости обучения $\alpha \approx 0.01$, что позволяет надежно фиксировать выученные стратегии в Q-таблице вследствие более сложного изменения уже выученных Q-значений при новых появлениях в игре редких кортежей состояния-действия, что, в свою очередь, не допускает осцилляции при мультиагентном обучении.

2.8. Выводы

Теория игр долгое время являлась главным инструментом мультиагентного обучения с подкреплением, позволяя получать теоретически обоснованные и практически эффективные решения небольших задач. В алгоритмах, основанных на принципах теории игр, явно учитываются взаимозависимости агентов в кооперативных и некооперативных играх для достижения равновесия Нэша.

В гл. 2 для матричных игр были рассмотрены алгоритм PHC и алгоритм WoLF-PHC, обладающий свойствами рациональности и сходимости. Недостаток алгоритма PHC, заключающийся в отсутствии свойства сходимости при стохастически действующем агенте-оппоненте и приводящий к осцилляции, преодолевается в алгоритме WoLF-PHC введением дополнительных параметров обучения и усреднения.

Стохастические игры рассмотрены на примере алгоритма Q-обучения Нэша, который является развитием классического алгоритма Q-обучения применительно к мультиагентному обучению. В алгоритме предложены три фундаментально новых решения, расширяющие теоретические границы и общее понимание принципов мультиагентного обучения: использование совместных действий; прямое вычисление Q-значения Нэша; обновление Q-значений в зависимости от действий других агентов. В то же время алгоритм довольно сложен и относительно медленно вычисляет равновесные взаимозависимые Q-значения в мультиагентном обучении.

На современном этапе развития мультиагентного обучения принципы и алгоритмы, основанные на теории игр, расширяют путем введения в подкрепляющий контур нейронных сетей, которые будут рассмотрены в следующей главе.

2.9. Задачи для самоконтроля

1. Обучите агентов играть в матричную игру с использованием алгоритма WoLF-PHC в условиях стационарной и стохастической стратегии противника, используя приведенные ниже матрицы наград матричных игр:
«Координация»

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \; R_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix};$$

«Битва полов»

$$R_1 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \; R_2 = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix};$$

«Камень–ножницы–бумага»

$$R_1 = \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{bmatrix}, \; R_2 = \begin{bmatrix} 0 & 1 & -1 \\ -1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix};$$

«Восхождение»

$$R_1 = \begin{bmatrix} 11 & -30 & 0 \\ -30 & 7 & 6 \\ 0 & 0 & 5 \end{bmatrix}, \; R_2 = \begin{bmatrix} 11 & -30 & 0 \\ -30 & 7 & 6 \\ 0 & 0 & 5 \end{bmatrix};$$

«Полковник Блотто»

$$R_1 = \begin{bmatrix} 4 & 2 & 1 & 0 \\ 1 & 3 & 0 & -1 \\ -2 & 2 & 2 & -2 \\ -1 & 0 & 3 & 1 \\ 0 & 1 & 2 & 4 \end{bmatrix}, \; R_2 = \begin{bmatrix} -4 & -2 & -1 & 0 \\ -1 & -3 & 0 & 1 \\ 2 & -2 & -2 & 2 \\ 1 & 0 & -3 & -1 \\ 0 & -1 & -2 & -4 \end{bmatrix}.$$

2. Проведите эксперимент и выберите наилучшие значения α, δ_w, δ_l в алгоритме WoLF-PHC для матричной игры «Камень–ножницы–бумага» сравнив следующие формулы:

а) $\alpha = \dfrac{1}{100 + 0.0001t}$, $\delta_w = \dfrac{1}{20\,000 + t}$, $\delta_l = 2\delta_w$;

б) $\alpha = \dfrac{1}{10 + 0.0001t}$, $\delta_w = \dfrac{1}{20\,000 + t}$, $\delta_l = 2\delta_w$;

в) $\alpha = \dfrac{1}{100 + 0.002t}$, $\delta_w = \dfrac{1}{1000 + 0.1t}$, $\delta_l = 4\delta_w$.

3. Проведите эксперимент с увеличением числа агентов (3, 5, 7) для матричной игры SMAC (см. разд. 2.4).

4. Разработайте в редакторе StarCraft II Editor карту с дополнительными физическими препятствиями для перемещения агентов, обучите агентов и сравните работу алгоритмов WoLF-PHC и Q-обучения Нэша в стохастической игре SMAC.

5. Разработайте стохастическую игру «Защита территории», представленную миром-сеткой 3×3 (рис. 2.15), и обучите агентов с использованием алгоритма Q-обучения Нэша. За один ход агенты могут перемещаться только по вертикали или по горизонтали на одну клетку. Награды в игре назначаются за следующие действия: –10 защитнику при достижении нападающим клетки-цели; +10 защитнику при нахождении в одной клетке с нападающим; +10 нападающему при достижении клетки-цели; –10 нападающему при нахождении в одной клетке с защитником.

Рис. 2.15. Мир-сетка стохастической игры «Защита территории»

6. Разработайте и исследуйте реализацию алгоритма JAL для стохастической игры SMAC.

7. Разработайте и исследуйте реализацию алгоритма Minimax-Q для стохастической игры SMAC.

8. Разработайте и исследуйте реализацию алгоритма Q-обучения «друг или враг» F-F-Q для стохастической игры SMAC.

9. Сравните алгоритмы WoLF-PHC, Nash-Q, F-F-Q и выберите лучший алгоритм для стохастической игры с общей суммой SMAC.

Разработайте сценарий игры и модификацию библиотеки SMAC, чтобы при мультиагентном обучении в стохастической игре наилучшим образом раскрыть юнита расы терран «медэвак». Этот бронированный транспортный корабль обладает функцией исцеления биологических единиц, имеет 150 единиц здоровья и одну единицу брони. Может использоваться для быстрого перемещения морпехов в ключевые точки карты.

Глава 3

Нейросетевое обучение

Любой сумеет загрузить числа в сеть —
но чтобы понять, какие числа имеют значение,
надо разбираться в базовых системах.

Юн Ха Ли. Гамбит девятихвостого лиса

3.1. Классификация

Астрономическое количество возможных состояний реальной среды до последнего времени ограничивало область реализации алгоритмов обучения с подкреплением только простыми задачами в табличном мире-сетке. Однако недавние работы компании Google DeepMind позволили применять эти алгоритмы для высокоразмерных и сложных искусственных сред — компьютерных видеоигр. Новые алгоритмы продемонстрировали достижение человеческого уровня и даже превосходство над ним в аркадных играх Atari, стратегических играх Dota 2 и StarCraft II. Один и тот же алгоритм обучался в самых разных играх с почти одинаковым успехом, что свидетельствует о его высокой способности обобщать данные и знания, полученные из среды. Более того, алгоритм зачастую использовал только необработанную визуальную информацию об игровой среде по аналогии с биологическими агентами, получающими основную информацию об окружающей среде с помощью зрения.

Все эти объективные успехи связаны с использованием в качестве главной математической модели машинного обучения — *искусственных нейронных сетей* (*англ.* artificial neural networks), которые являются системами соединенных и взаимодействующих между собой простых процессоров обработки данных, построенных по принципу организации и функционирования биологических нейронных сетей — сетей нервных клеток живого организма. Нейросетевое обучение, достигшее доминирующего положения в области систем искусственного интеллекта за последние 10 лет в одноагентных сценариях, было результативно расширено и на мультиагентные системы.

В табл. 3.1 представлены основные алгоритмы мультиагентного нейросетевого обучения, классифицированные по архитектурам нейронных сетей: полносвязная, сверточная, рекуррентная, исполнитель-критик — и по принципу обучения с подкреплением: вычисление ценности состояний-действий, вычисление стратегий.

В гл. 3 подробно рассматриваются принципы мультиагентного нейросетевого обучения на основе алгоритма независимого глубокого обучения с использованием полносвязной нейронной сети (IQN), алгоритма централизованного обучения с использованием сверточной нейронной сети (CDQN), алгоритма декомпозиции Q-значений с использованием рекуррентной нейронной сети (VDN), алгоритма мультиагентного глубокого детерминированного градиента стратегий (MADDPG).

Таблица 3.1

Классификация алгоритмов мультиагентного нейросетевого обучения

Принцип обучения с подкреплением	Архитектура нейронной сети			
	Полносвязная	Сверточная	Рекуррентная	Исполнитель-критик
Вычисление ценности состояний-действий	IQN	CDQN DRON LDQN DPIQN DBPR+	VDN DPIRQN RIAL DIAL DRQN HDRQN QMIX	MADDPG COMA
Вычисление стратегий	ENL	PS-TRPO	LOLA	

К настоящему времени разработано большое число разнообразных алгоритмов мультиагентного нейросетевого обучения, но все они основываются на комбинации трех базовых технологий: обучение с подкреплением, глубокое обучение и мультиагентное обучение.

В традиционном обучении с подкреплением агенты обучаются через взаимодействие со средой, при этом многие характерные признаки среды для алгоритмов обучения задаются вручную программистом (см. гл. 1). Для автоматического создания характерных признаков используется глубокое обучение. *Глубокое обучение* (*англ.* Deep Learning, DL) — разновидность нейросетевого обучения, при котором применяются сложные архитектуры нейронных сетей с тремя и более слоями. В *глубоком обучении с подкреплением* (*англ.* Deep Reinforcement Learning, DRL) глубокие нейронные сети обучаются для того, чтобы аппроксимировать оптимальные стратегии агентов или ценности состояний-действий.

Иногда разработчиков алгоритмов глубокого обучения обвиняют в амнезии в том смысле, что многим классическим алгоритмам машинного обучения, разработанным до эпохи глубоких нейронных сетей, не уделяется должного внимания или алгоритмы используются без упоминания авторов. В этом отношении несколько лучше выглядит *мультиагентное глубокое обучение с подкреплением* (*англ.* Multi-Agent Deep Reinforcement Learning, MDRL). В нем DRL применяется для обучения множества агентов с помощью как классических, так и новых мультиагентных алгоритмических

приемов: «прямая коммуникация», «разделение параметров», «централизованное обучение с децентрализованным исполнением», «дистилляция стратегий», «моделирование оппонента или пропонента», «вспомогательные задачи», «концепция толерантности», «гистерезис системы», «разница наград», «декомпозиция Q-значений».

В классических алгоритмах кооперативного мультиагентного обучения редко использовалась коммуникация между агентами. Но одним из современных трендов мультиагентного нейросетевого обучения является добавление агентам возможностей прямой коммуникации, которая должна упростить и ускорить процесс обучения. В алгоритме ENL (*англ.* Emergence of Natural Language) используется простая трехслойная нейронная сеть агента — отправителя сообщений и четырехслойная нейронная сеть агента — получателя сообщений для решения задачи координации. Необходимость коммуницировать мотивировала агентов tabula rasa (изначально не имеющих никаких знаний о среде или друг о друге) присвоить собственные смысловые значения получаемым и передаваемым сообщениям. При этом сгенерированные агентами «смыслы» находились на уровне абстракции, который поддавался интерпретации человеком.

Новым мультиагентным алгоритмическим приемом, тесно связанным с коммуникацией агентов, является разделение параметров. При разделении параметров используется одна глобальная нейронная сеть для всех агентов, тем самым весá и гиперпараметры нейронной сети обобщают наблюдения всех агентов сразу. Разделение параметров обычно применяется при гомогенном мультиагентном обучении, когда действия у всех агентов одинаковые. В алгоритме DIAL (*англ.* Differentiable Inter-Agent Learning), основанном на рекуррентных нейронных сетях с выходом в виде Q-значений, используется не только разделение параметров, но и передача через коммуникационные каналы градиентов. Таким образом сообщениям агентов присваиваются приоритеты, что снимает проблему задержек при обработке большого числа запросов. Похожая концепция реализована в алгоритме PS-TRPO (*англ.* Parameter Sharing-Trust Region Policy Optimization), который является обобщением популярного в глубоком обучении с подкреплением алгоритма TRPO.

В алгоритме DIAL и его аналоге — алгоритме RIAL (*англ.* Reinforced Inter-Agent Learning) используется также алгоритмический прием централизованного обучения с децентрализованным исполнением. Классические алгоритмы мультиагентного обучения были либо полностью централизованными, либо полностью децентрализованными. Однако в MDRL часто используется именно новая смешанная парадигма: обучение агентов проводится под контролем единой нейронной сети, а во время реального взаимодействия со средой агенты действуют децентрализованно, не имея единого контроля. Основное преимущество алгоритмического приема централизованного обучения с децентрализованным исполнением состоит в том, что во время обучения можно использовать дополнительную информацию (глобальное состояние, совместные действие или групповые

награды), а во время исполнения эта информация удаляется, что экономит ресурсы и ускоряет функционирование агентов. Необходимо отметить, что не для всех проблем реального мира пригоден этот алгоритмический прием, он характерен скорее для компьютерных игр, где доступна качественная модель среды.

В контексте классического обучения с подкреплением методы многозадачного обучения предназначены для разработки агентов, которые способны обучаться выполнению сразу нескольких задач в среде. В DRL для этой цели используется алгоритмический прием — дистилляция стратегий: после обучения нескольким задачам множество стратегий агрегируется в одну и затем создается более компактная нейронная сеть. В алгоритме DBPR+ (*англ.* Deep Bayesian Policy Reuse) с использованием сверточной нейронной сети дистилляция стратегий применяется для моделирования множества стратегий оппонентов.

Важным свойством агента, обучающегося в группе, является способность моделировать поведение других агентов — как оппонентов, так и пропонентов. Для этой цели в алгоритме DRON (*англ.* Deep Reinforcement Opponent Network) используются две нейронные сети, основанные на стандартной архитектуре DQN (*англ.* Deep Q-Network): первая нейронная сеть — для нахождения своих Q-значений, вторая — для моделирования действий оппонента. В этом алгоритме очень многое заимствовано из классической теории игр, идеи совместных действий формализованы в виде архитектур нейронных сетей. Однако в алгоритме DRON для описания оппонента применялись характерные признаки, созданные вручную. Улучшением стали алгоритм DPIQN (*англ.* Deep Policy Inference Q-Network) и его рекуррентная версия — алгоритм DRPIQN (*англ.* Deep Recurrent Policy Inference Q-Network), автоматизирующий процесс создания характерных признаков на основе наблюдений за оппонентом. В отличие от предшественников, алгоритм LOLA (*англ.* Learning with Opponent-Learning Awareness) позволял не просто строить модель действий других агентов, но и вычислять стратегии оппонентов с учетом их способности к обучению.

В алгоритме DPIQN также применялся новый алгоритмический прием MDRL, называемый «вспомогательные задачи». С помощью этого приема алгоритму добавляются дополнительные, второстепенные цели для обучения, что позволяет точнее и быстрее достигать главной цели обучения. Так, вспомогательной задачей в алгоритме DPIQN является обучение действиям оппонентов, знание о которых позволило агенту в компьютерной игре «Футбол» заработать количество награды на 30 % больше, чем получил агент, использующий классический алгоритм DQN. Кроме того, вспомогательные задачи являются эффективным инструментом для преодоления локальных минимумов в глубоком обучении.

В алгоритме LDQN (*англ.* Lenient-DQN) применена концепция толерантности в MDRL, первоначально представленная в классическом мультиагентном обучении. Идея толерантности заключается в том, что на ранних этапах обучения ни один агент не знает, какие совместные действия с другими

агентами являются лучшими, поэтому в алгоритме должны изначально учитываться слабые и зачастую случайные действия пропонентов. Дополнительной целью применения концепции толерантности является преодоление стандартной мультиагентной проблемы затенения действий. Затенение действий возникает, когда объединение действий разных агентов в кортежи совместных действий приводит к финальному выбору неоптимальных действий, имеющих наивысшую ценность только в кортеже. Проблема затенения действий — частный случай проблемы сверхобобщения в глубоком обучении.

Проблема сверхобобщения является обратной проблеме переобучения, только весá нейронной сети настраиваются на слишком широкие и абстрактные границы данных, которые не присутствовали в обучающей выборке. Если для преодоления проблемы переобучения в мультиагентном обучении используются ансамбли стратегий, то для преодоления сверхобобщения в алгоритме LDQN стремятся уменьшить шум от стохастических наград среды, предотвращая вовлечение агентов в субоптимальные стратегии. Однако, как и в других алгоритмах MDRL, в алгоритме LDQN возникают проблемы реализации буфера воспроизведения для мультиагентного обучения. По этой причине коэффициент толерантности используется и для управления выборкой опыта из буфера воспроизведения.

Общим недостатком алгоритмов, основанных на концепции толерантности, является то, что у них слишком высок «уровень оптимизма» — алгоритмы завышают Q-значения. В связи с этим в MDRL вместе с концепцией толерантности используется классический мультиагентный алгоритмический прием — гистерезис системы. В отличие от инерционности поведения мультиагентной системы, которое означает монотонное сопротивление системы изменению ее состояния, при гистерезисе системы мгновенный отклик на воздействие зависит от текущего состояния, а поведение системы на интервале времени во многом определяется совокупностью предыдущих ее состояний.

В алгоритме HDRQN (*англ.* Hysteretic Deep Recurrent Q-Network) гистерезис реализован в виде двух параметров скорости обучения, результатом использования которых является запаздывание в обновлении Q-значений для действий, связанных с положительной наградой, полученной при кооперации агентов. Следовательно, агенты устойчивы к негативному обучению, возникающему в результате исследовательских действий пропонентов. Несмотря на простую реализацию, гистерезис системы позволяет значительно улучшить стабильность кооперативного мультиагентного обучения. Примечательно, что в отличие от распределенного Q-обучения в гистерезисном Q-обучении допускается возможная деградация Q-значений, которые были переоценены вследствие действий, не связанных с совместными действиями агентов.

В кооперативном мультиагентном обучении предпочтение чаще всего отдается глобальной награде для группы вместо локальных наград для каждого агента. Однако при этом возникает проблема предоставления кредита,

заключающаяся в том, что бывает довольно сложно определить, какое действие какого агента внесло наибольший вклад в получение глобальной награды. Классическим приемом решением этой проблемы является алгоритмический прием «разница наград». Самой простой реализацией этого приема может быть непосредственное вычисление разницы между частотой выполнения агентом определенного действия и средней частотой выполнения этого действия во всей группе. В алгоритме COMA (англ. Counterfactual Multi-Agent) в этом случае используется функция преимущества, которая основывается на вкладе каждого агента в общую награду и эффективно вычисляется с помощью глубоких нейронных сетей с централизованной архитектурой исполнитель-критик. В данной архитектуре критик вычисляет ценности состояний-действий, а исполнитель вычисляет стратегии.

Алгоритмы типа COMA, в которых используется централизованное обучение, не имеют проблемы мультиагентной нестационарности, характерной для алгоритмов независимого обучения IQL. Но централизованное обучение подвержено ограничениям по гибкому масштабированию, что негативно сказывается на реализации мультиагентной системы. В гибридных алгоритмах типа QMIX и VDN противоречие централизованного и независимого обучения стараются смягчить, используя новый алгоритмический прием «декомпозиция Q-значений». В мультиагентном обучении декомпозиция — это разделение объекта на части, которые, будучи объединенными по некоторому критерию, дают исходный объект. В алгоритме QMIX используется отдельная полносвязная нейронная сеть для нелинейного агрегирования Q-значений от множества агентов, построенных на базе рекуррентных нейронных сетей DRQN. Особенно высокие показатели процента побед и количества награды при использовании алгоритма QMIX продемонстрированы в гетерогенном микроменеджменте компьютерной игры StarCraft II.

3.2. Модель

3.2.1. Глубокое Q-обучение

Целью обучения с подкреплением на основе математической модели марковских процессов принятия решений является нахождение оптимальной стратегии $\pi^* : S \to A$, которая задает действия агента, приводящие к получению максимального количества награды. Рассмотренный в гл. 1 алгоритм Q-обучения позволяет достигнуть этой цели, вычислив оценку Q ожидаемой награды $(r_1 + r_2 + ... + r_T)$ с параметром дисконтирования $\gamma \in [0, 1]$, которую агент может получить в некотором состоянии s_t, если выполнит действия a_t в соответствии со стратегией π на временно́м интервале длительностью T:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \ldots + \gamma_{T-1} r_T | \pi\right].$$

Оптимальной функцией ценности действия $Q^*(s_t, a_t)$ для стратегии π является функция

$$Q^*(s_t, a_t) = \max Q^{\pi}(s_t, a_t).$$

Следовательно, оптимальную стратегию π^* можно найти простым последовательным выбором $Q^*(s_t, a_t)$ в каждом состоянии s_t.

Для некоторого состояния s, действия a, награды r и следующего состояния s' значения $Q^*(s, a)$ вычисляются с помощью итеративного решения уравнения Беллмана при теоретически обоснованной сходимости $Q_i \to Q^*$ и $i \to \infty$:

$$Q_{i+1}(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q_i(s', a') \right].$$

Для непосредственного вычисления значений $Q(s, a)$ требуются точное знание динамики среды, достаточные вычислительные ресурсы и строгое выполнение марковского свойства. В большинстве реальных задач одновременно выполнить все эти условия затруднительно, поэтому в современных алгоритмах обучения с подкреплением используются аппроксимированные значения $Q(s, a; \theta)$.

В глубоком Q-обучении для нелинейной аппроксимации Q-значений используются глубокие нейронные сети. Глубокую нейронную сеть, параметризированную весами θ, методом стохастического градиентного спуска (и его аналогов) обучают минимизировать функцию потерь $L(\theta)$:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta') - Q(s, a; \theta) \right)^2 \right].$$

В этой формуле напрямую используется итеративное обновление из уравнения Беллмана. По существу, по формуле вычисляется среднеквадратичная ошибка между текущим значением и обновленным значением $Q(s', a'; \theta')$, которое включает награду и оценку качества следующего состояния. В архитектуре нейронной Q-сети значение $Q(s, a; \theta)$ является выходным значением нейронной сети (прогнозом), а значение $Q(s', a'; \theta')$ — целевым значением. Причем в отличие от обучения с учителем, где целевое значение задается до обучения, в глубоком Q-обучении целевое значение зависит от весов нейронной сети.

3.2.2. Децентрализованные частично наблюдаемые марковские процессы принятия решений (Dec-POMDP)

Зашумленные и лимитированные сенсоры могут ограничить возможности агента по восприятию окружающей среды. Неопределенность, связанная с неспособностью агента получить точное состояние среды, делает невозможным применение для машинного обучения марковских процессов принятия решений. Для преодоления неопределенности состояния среды

была разработана математическая модель частично наблюдаемых марковских процессов принятия решений (*англ.* Partially Observable Markov Decision Processes, POMDP). В модели POMDP вместо точных состояний среды используются наблюдения, зависящие от этих состояний. На основе истории наблюдений за некоторый период времени агент может найти вероятность возникновения некоторого состояния и применить эти данные для принятия решения о выполнении того или иного действия.

Обобщением модели POMDP для множества агентов является модель децентрализованных частично наблюдаемых МППР (*англ.* Decentralized POMDP, Dec-POMDP), представляющая собой кортеж $(n, S, \mathbb{A}, T, R, \mathbb{O}, O, h, b^0)$, где n — множество агентов; S — конечное множество состояний; $\mathbb{A}$ — множество совместных действий; T — функция переходов; R — функция немедленной награды; $\mathbb{O}$ — множество совместных наблюдений; O — функция наблюдений; h — временной горизонт задачи; b^0 — начальное распределение состояний.

В модели Dec-POMDP агент имеет информацию только о своих действиях. По сравнению с моделью POMDP в модель Dec-POMDP добавлены совместные действия и совместные наблюдения агентов. Множество совместных действий $\mathbb{A}$ состоит из $\mathbb{A} = \times_{i \in n} A_i$, где A_i — множество действий агента i, которое может быть разным для разных агентов. В каждый момент времени t агент i выполняет некоторое действие a_i, приводящее к формированию совместного действия $\breve{a} = \langle a_1, a_2, \ldots, a_n \rangle$. Результат воздействия совместного действия $\breve{a}$ на среду определяется функций переходов T как вероятность $T(s, \breve{a}, s')$. Динамика функционирования модели Dec-POMDP проиллюстрирована на рис. 3.1.

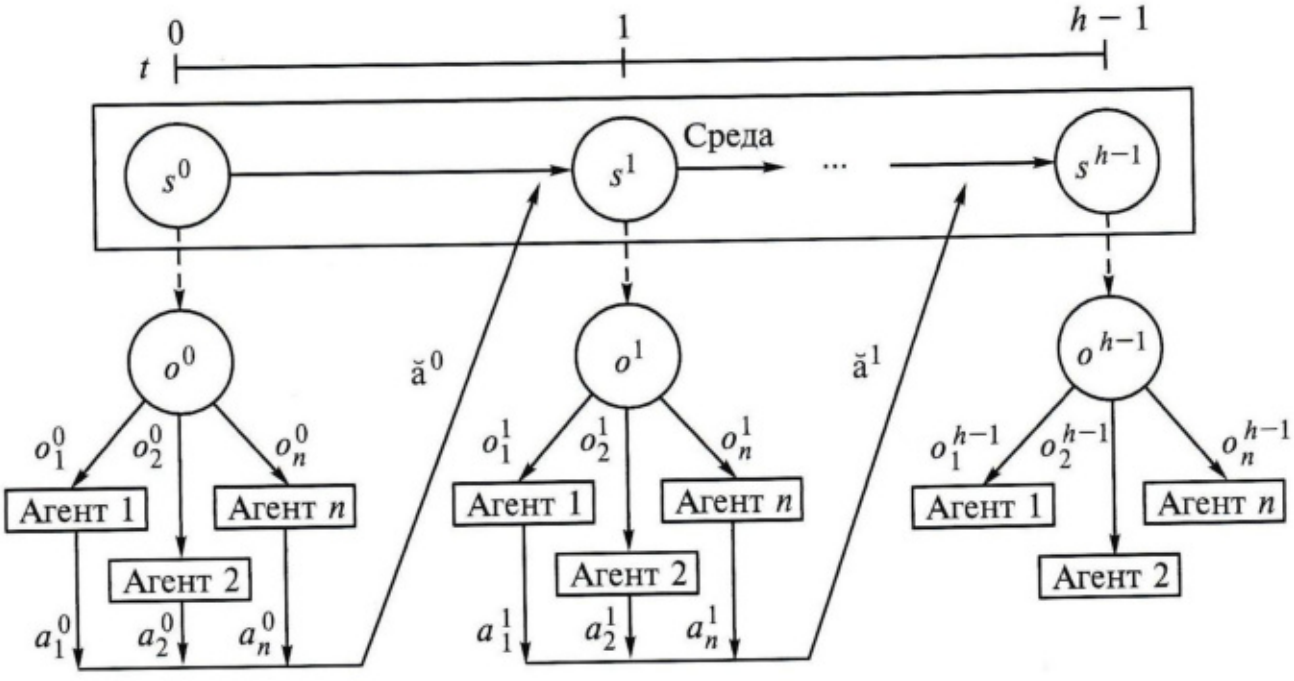


Рис. 3.1. Динамика функционирования модели Dec-POMDP

Множество совместных наблюдений $\mathbb{O}$ состоит из $\mathbb{O}=\times_{i\in n}\mathbb{O}_i$, где $\mathbb{O}_i$ — множество наблюдений, доступных агенту i. В каждый момент времени t агент i получает некоторое наблюдение o_i, которое входит в совместное наблюдение $o = \langle o_1, o_2, ..., o_n\rangle$. Функция наблюдений O определяет вероятность восприятия агентами состояния среды как совместного наблюдения $O(o', \breve{a}, s')$. В гл. 3 совместное наблюдение o используется в алгоритме CDQN и задано в виде цифрового изображения псевдокарты (см. разд. 3.4). В алгоритме CDQN каждый агент i получает одинаковое наблюдение, поэтому $o = o_1 = o_2 = o_3$.

В модели Dec-POMDP агент, не имея информации о реальном состоянии среды, не может использовать немедленную функцию награды $R: S\times\mathbb{A}\to\mathbb{R}$, получаемую из среды сразу после выполнения совместного действия $\breve{a}$. В связи с этим множество агентов обучается оптимизировать свои действия путем максимизации ожидаемой кумулятивной награды $\breve{R}_t = \sum_{t=0}^{h-1}\gamma^{t-1}R_t$ с параметром дисконтирования γ на некотором промежутке времени h с учетом начального распределения состояний $b^0 \in \mathcal{P}(S)$ в момент времени $t = 0$, где $\mathcal{P}(\cdot)$ — множество распределений вероятностей.

3.2.3. Двойная декомпозиция Q-значений

В некоторых средах для нахождения оптимальной стратегии агенту недостаточно знаний о приближенных наблюдениях o_i или о точных состояниях среды s в данный момент времени t. Так, в среде SMAC для победы над противником может быть недостаточно знаний только о статических позициях пропонентов и оппонентов на карте, необходимы также знания о скоростях и направлениях их движений. Эти знания невозможно получить только из одного состояния s или наблюдения o_i, необходимо хранить и обрабатывать историю H_i состояний s или наблюдений o_i. Впервые для этой цели были использованы рекуррентные нейронные сети LSTM (*англ.* Long Short Term Memory) в алгоритме DRQN.

В алгоритме VDN при декомпозиции Q-значений на первом мультиагентном уровне идея обработки истории состояний была развита следующим образом. В обучении с подкреплением на основе функции ценности действий $Q: S\times A\to\mathbb{R}$ можно вычислить «жадную» стратегию действий $\pi(a|s) = \arg\max_{a\in A} Q(s,a)$. При этом, если существует возможность сохранять и обрабатывать историю состояний среды $\bar{H} = (H_1, H_2, ..., H_n)$ нескольких агентов n, в общем случае стратегию можно представить в виде отображения $\pi: H_i \to \mathcal{P}(A_i)$. Если каждый агент i имеет независимую историю H_i, т. е. агенты функционируют в децентрализованном режиме, то $\pi: H_i \to \mathcal{P}_i(A)$.

Главное допущение, которое делается при декомпозиции Q-значений на первом мультиагентном уровне, заключается в том, что совместная функция

ценности действий $Q\big((H_1,\ldots,H_n),\ (a_1,\ldots,a_n)\big)$ может быть аддитивно разделена на отдельные функции ценности действий $\tilde{Q}_i(H_i,a_i)$ каждого агента i:

$$Q\big((H_1,\ldots,H_n),\ (a_1,\ldots,a_n)\big)\approx\sum_{i=1}^{n}\tilde{Q}_i(H_i,a_i).$$

Функция ценности действий $\tilde{Q}_i(H_i,a_i)$ зависит от состояний s или локальных наблюдений o_i каждого отдельного агента i. В алгоритме VDN функция ценности действий $\tilde{Q}_i(H_i,a_i;\theta)$ представляется нейронными сетями, весá θ которых настраиваются методом стохастического градиентного спуска (и его аналогов). Таким образом, при мультиагентном обучении с использованием декомпозиции Q-значений не требуется никаких дополнительных коммуникационных каналов между агентами, а также не нужно проводить экспертный дизайн наград для каждого отдельного агента.

В случае, например, трех агентов, суммарно получающих награду $r(s,a)=$ $=r_1(s_1,a_1)+r_2(s_2,a_2)+r_3(s_3,a_3)$, декомпозицию Q-значений можно представить в виде

$$Q^{\pi}(s,a;\theta)=\mathbb{E}\left[\sum_{t=1}^{\infty}\gamma_{t-1}r\left(s^t,a^t;\pi,\theta\right)\right]=$$

$$=\mathbb{E}\left[\sum_{t=1}^{\infty}\gamma_{t-1}r_1\left(s_1^t,a_1^t;\pi,\theta_1\right)\right]+\mathbb{E}\left[\sum_{t=1}^{\infty}\gamma_{t-1}r_2\left(s_2^t,a_2^t;\pi,\theta_2\right)\right]+\mathbb{E}\left[\sum_{t=1}^{\infty}\gamma_{t-1}r_1\left(s_3^t,a_3^t;\pi,\theta_3\right)\right]\approx$$

$$\approx\tilde{Q}_1^{\pi}(H_1,a_1;\theta_1)+\tilde{Q}_2^{\pi}(H_2,a_2;\theta_2)+\tilde{Q}_3^{\pi}(H_3,a_3;\theta_3).$$

Под вторым одноагентным уровнем декомпозиции Q-значений в алгоритме VDN понимается разделение локальных Q-значений независимых агентов на функцию ценности истории состояний $V_i^{\pi}(H_i;\theta_i)$ и функцию преимущества действия $A_i^{\pi}(a_i;\theta_i)$:

$$\tilde{Q}_i^{\pi}(H_i,a_i;\theta_i)=V_i^{\pi}(H_i;\theta_i)+A_i^{\pi}(a_i;\theta_i).$$

Такая декомпозиции Q-значений, реализованная в виде дуэльной архитектуры нейронной сети, позволяет точнее оценивать Q-значения. В случае большого числа действий в каждом состоянии функция ценности истории состояний $V_i^{\pi}(H_i;\theta_i)$ снимает необходимость дополнительно оценивать каждое действие. В тех случаях, когда сложно определить преимущество одного действия перед другими действиями, например, вследствие редкой положительной награды, функция преимущества $A_i^{\pi}(a_i;\theta_i)$ дает дополнительную информацию о предпочтительных действиях.

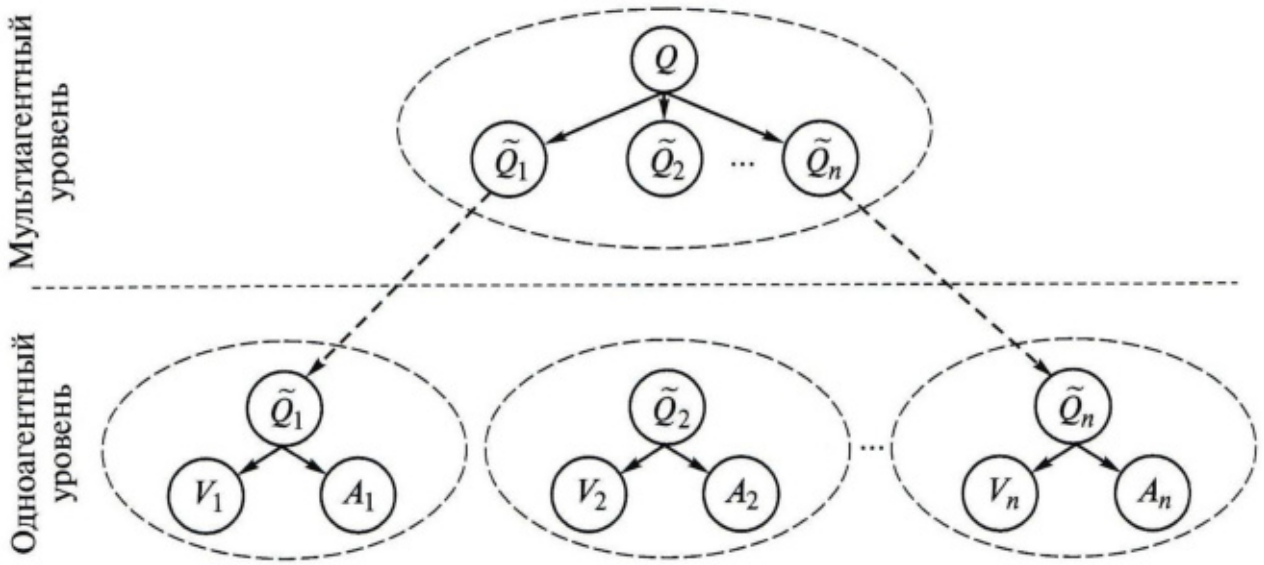


Рис. 3.2. Схема двойной декомпозиции Q-значений

Обобщенная схема двойной декомпозиции Q-значений на первом, мультиагентном, и втором, одноагентном, уровнях представлена на рис. 3.2.

Таким образом, завершаем рассмотрение методов нахождения ценности и переходим к методам вычисления стратегии.

3.2.4. Глубокий детерминированный градиент стратегий

В случае когда мультиагентное нейросетевое обучение происходит в непрерывной среде, где сложно получить отдельные состояния, или в случае огромного числа действий, с которыми не может справиться даже дуэльная архитектура, вместо методов нахождения ценностей состояний-действий используются методы вычисления стратегий. Наиболее известные алгоритмы одногоагентного обучения с подкреплением — TRPO, PPO, REINFORCE, в которых используются градиенты стратегий, имеют серьезный недостаток по накоплению минимально необходимого объема опыта для полноценного решения задачи. Этот недостаток имеют все алгоритмы с вычислением текущей стратегии действий, так как после каждого обновления стратегии приходится накапливать новый опыт. В связи с этим были разработаны алгоритмы, основанные на глубоком детерминированном градиенте стратегий и использовании нейросетевой архитектуры исполнитель-критик.

Поведение агента может определяться стохастической стратегией $\pi : S \to P(A)$, которая отображает множество состояний среды S на распределение вероятностей выполнения действий A. Общее количество награды или доход агента в некотором состоянии s_t определяется как кумулятивная награда $R_t = \sum_{t=0}^{T} \gamma_{t-1} r_t(s_t, a_t)$ с параметром дисконтирования $\gamma \in [0,1]$. Целью обучения агента является нахождение оптимальной стохастической стратегии π^*, которая максимизирует доход агента. В связи с этим целевая функция может быть задана в виде $J(\theta) = \mathbb{E}_{\pi_\theta}[R_t]$, где θ — вектор параметров

стохастической стратегии π, который в глубоком обучении является весами нейронной сети. Для максимизации целевой функции в процессе машинного обучения параметры стохастической стратегии θ изменяются в направлении ее градиента $\nabla_\theta J(\theta)$:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\pi_\theta}[R_t].$$

В соответствии с теоремой о градиенте стратегии аналитическим выражением для вычисления градиента целевой функции $\nabla_\theta J(\theta)$ является следующее:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) Q_{\pi_\theta}(s,a)\right].$$

Теорема о градиенте стратегии имеет важное практическое значение, потому что вычисление градиента целевой функции $\nabla_\theta J(\theta)$ не зависит от распределения состояний $\rho_\pi(s)$, для нахождения которого требуется наличие модели среды. Вычисление градиента сведено к нахождению математического ожидания $\mathbb{E}_{\pi_\theta}$, которое можно аппроксимировать выборкой данных за несколько шагов обучения N:

$$\nabla_\theta J(\theta) = \frac{1}{N}\sum_{t=0}^{N}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) Q_{\pi_\theta}(s_t,a_t)\right].$$

С помощью теоремы о градиенте стратегии было разработано несколько эффективных алгоритмов, в которых используется вычисление градиентов стохастичских стратегий, различающихся только способом нахождения значения $Q_{\pi_\theta}(s,a)$. Так, если аппроксимировать значения $Q_{\pi_\theta}(s,a)$ простым доходом агента R_t, то получится одна из модификаций алгоритма REINFORCE. Альтернативным вариантом является использование нейросетевой архитектуры исполнитель-критик, в которой нейронная сеть критика аппроксимирует значения $Q_{\pi_\theta}(s,a;\theta^{cr})$, а нейронная сеть исполнителя — значения стохастической стратегии $\pi_\theta(a|s;\theta^{ac})$.

Однако стохастическая стратегия $\pi_\theta(a|s)$ остается зависимой как от множества состояний S, так и от множества действий A, что требует большого объема обучающих данных. В связи с этим была предложена детерминированная стратегия $\mu_\theta(s): S \to A$, зависящая только от множества состояний S. Доказано, что для некоторого параметра дисперсии $\sigma \to 0$ стохастический градиент стратегии π_θ сходится к детерминированному градиенту стратегии μ_θ: $\lim_{\sigma\to 0}\nabla_\theta J_{\pi_{\theta,\sigma}}(\theta) = \nabla_\theta J_{\mu_\theta}(\theta)$. Поэтому для целевой функции $J(\theta) = \mathbb{E}_\mu\left[r\left(s,\mu_\theta(s)\right)\right]$ детерминированный градиент стратегии может быть вычислен по формуле

$$\nabla_\theta J(\theta) = \mathbb{E}_\mu\left[\nabla_\theta \mu_\theta(s) \nabla_a Q_\mu(s,a)|_{a=\mu_\theta(s)}\right].$$

В оффлайновом безмодельном алгоритме глубокого детерминированного градиента стратегий (*англ.* Deep Deterministic Policy Gradient, DDPG) детерминированный градиент стратегий $\mu_\theta\left(s;\theta^{ac}\right)$ и значение $Q_\mu\left(s,a;\theta^{cr}\right)$ аппроксимировались глубокими нейронными сетями, в которых используется архитектура исполнитель-критик.

Детерминированный градиент стратегий $\nabla_{\theta_i} J(\theta_i)$ агента i из множества одновременно обучающихся агентов n, использующих алгоритм DDPG, может быть вычислен по формуле

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{\mu_i}\left[\nabla_{\theta_i} \mu_{\theta_i}(s_i) \nabla_{a_i} Q_i^\mu\left((s_1,\ldots,s_n),\ (a_1,\ldots,a_n)\right)|_{a_i=\mu_{\theta_i}(s_i)}\right].$$

В этой формуле $Q_i^\mu\left((s_1,\ldots,s_n),\ (a_1,\ldots,a_n)\right)$ — совместная функция ценности действий, которая зависит от выполненных действий всех агентов (a_1, ..., a_n) и всех локальных состояний (s_1, ..., s_n), полученных каждым агентом отдельно. Таким образом, при централизованном мультиагентном обучении, реализованном нейронной сетью критика, снимается проблема нестационарности среды, поскольку известны действия всех агентов:

$$p\left(s' \mid s, a_1,\ldots,a_n,\mu_{\theta_1},\ldots,\mu_{\theta_n}\right) = p\left(s' \mid s, a_1,\ldots,a_n,\mu'_{\theta_1},\ldots,\mu'_{\theta_n}\right),\ \forall \mu_{\theta_i} \neq \mu'_{\theta_i}.$$

В следующем разделе рассмотрим алгоритмы мультиагентного нейросетевого обучения, основанные на проанализированных математических моделях.

3.3. Алгоритмы

3.3.1. Независимое глубокое обучение с использованием полносвязной нейронной сети (IQN)

Алгоритм независимого глубокого обучения с использованием полносвязной нейронной сети (*англ.* Independent Q-Network, IQN) основан на алгоритме DQN. Тремя главными инновациями, заимствованными IQN в глубоком Q-обучении, являются: применение глубоких нейронных сетей, использование буфера воспроизведения и целевой нейронной сети.

Табличные алгоритмы обучения с подкреплением не могли обучаться при большом числе состояний, не были способны обобщать и автоматически выделять характерные признаки среды. Для преодоления этих проблем использовались аппроксимации Q-функций на базе деревьев решений, радиальных базисных функций, регрессий. Для аппроксимации Q-функций

применялись также обычные нейронные сети в алгоритмах TD-gammon и NEAT-Q. Но именно глубокие нейронные сети в качестве нелинейных аппроксиматоров $Q(s, a; \theta)$ позволили преодолеть «проклятие размерности» благодаря свойству $|\theta| \ll |S \times A|$, начали полномасштабно работать с непрерывными действиями агентов, позволили автоматизировать создание характерных признаков.

Однако расширение глубокого обучения на обучение с подкреплением привело к появлению нового вызова — отсутствию независимости и одинаковой распределенности данных (н.о.р.). Многие алгоритмы глубокого обучения с учителем основываются именно на том, что данные обладают свойством н.о.р. Но при обучении с подкреплением обучающие данные состоят из сильно коррелированных последовательных взаимодействий агент-среда, а это нарушает условие независимости данных. Более того, при обучении с подкреплением агент может исследовать различные части пространства состояний, чем нарушается условие одинаковой распределенности данных.

Вследствие этих ограничений теоретическое обоснование сходимости алгоритмов глубокого обучения с подкреплением до сих пор остается слабо проработанным. Возникают и проблемы стабильности глубокого Q-обучения, характеризующиеся сильной дисперсией результатов работы алгоритма при незначительном изменении входных данных.

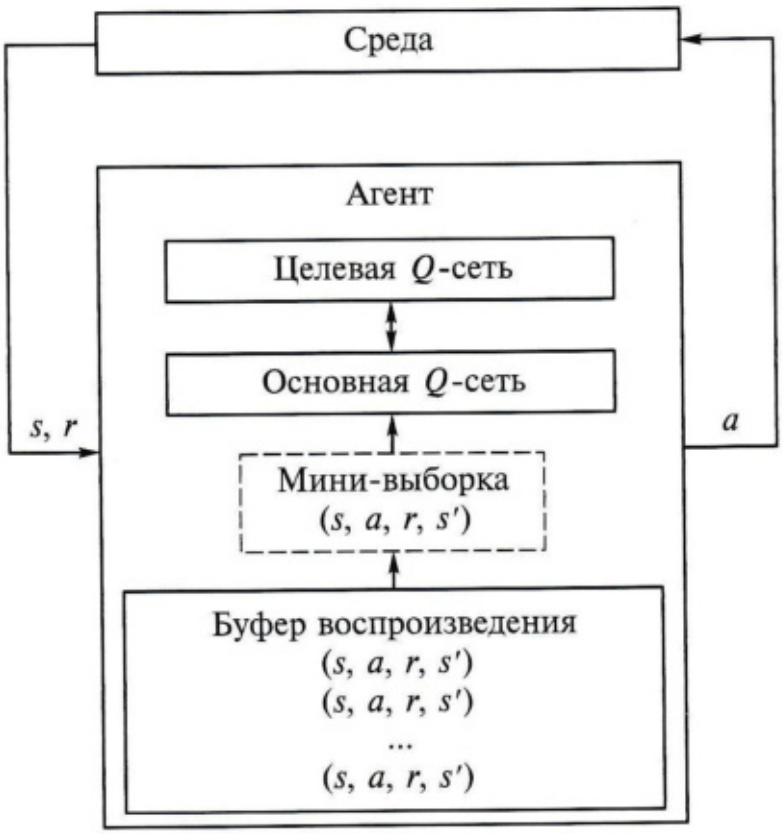


Рис. 3.3. Схема обучения с подкреплением в алгоритме DQN

Для того чтобы улучшить сходимость и стабильность обучения, в алгоритме DQN используется буфер воспроизведения, где сохраняются все кортежи (s, a, r, s') в качестве будущих обучающих данных (рис. 3.3). Буфер воспроизведения позволяет обеспечить алгоритм независимыми данными для обучения и снизить вероятность попадания в локальный минимум. Поведение агента за некоторый промежуток времени может усредняться, что дополнительно снижает осцилляцию обучения. Буфер воспроизведения также значительно повышает эффективность использования данных за счет их повторного использования. Это особенно важно в средах обучения с подкреплением, где награда бывает очень редкой и каждое такое событие сохраняется в буфере, чтобы несколько раз обучать алгоритм на успешном взаимодействии агент-среда.

Кроме того, для улучшения стабильности в алгоритме DQN использовалась дополнительная целевая нейронная сеть, которая дублировала основную нейронную сеть, но весá в целевой нейронной сети $Q(s', a'; \theta')$ заменялись на весá основной нейронной сети $Q(s, a; \theta)$ с определенной задержкой. Между обновлениями весá целевой нейронной сети оставались неизменными, что и привело к повышению стабильности обучения.

Существует несколько адаптаций алгоритма DQN для мультиагентного нейросетевого обучения. В гл. 3 рассматриваются три из них: алгоритм независимого глубокого обучения с использованием полносвязной нейронной сети; алгоритм централизованного обучения с использованием сверточной нейронной сети; алгоритм декомпозиции Q-значений с использованием рекуррентной нейронной сети.

В алгоритме IQL идея независимого обучения реализуется путем встраивания в каждого отдельного агента одинаковой нейросетевой архитектуры с учетом отсутствия явной коммуникации между агентами.

Основная и целевая нейронные сети инициализируются случайными весами $\theta \leftarrow \xi$ и $\theta' \leftarrow \xi$ (шаг 1). Буфер воспроизведения *Eb* инициализируется пустым, его объем Bl устанавливается равным 10 000. Объем мини-выборки *Bs* из буфера воспроизведения принят равным 32. Период копирования весов *Cs* основной и целевой нейронной сети выбран равным 100 эпизодам. Для того чтобы осуществлять исследование пространства возможных действий агента, в алгоритме IQL применяется концепция ε-«жадного» выбора действий (шаг 5). Значение ε линейно убывает с течением времени от значения 1.0 до 0.1. Главными преимуществами алгоритма IQL являются простота реализации, скорость работы, децентрализованная архитектура мультиагентной системы и устойчивость в получении хороших результатов при применении в разных задачах.

В гл. 1 и 2 раздел «Алгоритм» в основном представлял собой инструкцию по порядку применения математических формул. Но в нейросетевом обучении эта инструкция дополняется описанием архитектуры сети. При правильной настройке гиперпараметров архитектура нейронной сети может улучшать работу алгоритма или, наоборот, нивелировать его преимущества и заложенные в нем фундаментальные идеи.

Алгоритм IQL для агента i приведен ниже:

1. Инициализировать
   $\alpha \in (0,1]$; $\gamma \in [0,1]$; $\varepsilon \in [0.1,1]$; $Ne \in [1,\mathrm{N}]$;
   $Eb \leftarrow 0$; $El \leftarrow 10\,000$; $Bs \leftarrow 32$; $Cs \leftarrow 100$;
   $i \leftarrow 1, \ldots, n$; $a \in A_i$; $j \leftarrow 1, \ldots, Bs$; $K \leftarrow |A_i|$;
   $\theta \leftarrow \xi$; $\theta' \leftarrow \xi$; $\forall s \in S$.
2. Цикл
3. Получить состояние s.
4. Определить возможные действия $a \in A_i$ в состоянии s.
5. Передать состояние s в основную нейронную сеть и выбрать возможное действие a в соответствии с максимальным значением $Q_1(s, a; \theta)$, $Q_2(s, a; \theta)$, ..., $Q_K(s, a; \theta)$ и с учетом параметра ε.
6. Выполнить возможное действие a.
7. Получить награду r и следующее состояние s'.
8. Сохранить кортеж (s, a, r, s') в буфере воспроизведения Eb. Если объем El превышен, заменить значение по принципу очереди FIFO.
9. Случайно получить мини-выборку (s, a, r, s') объемом Bs из буфера воспроизведения Eb.
10. Для каждого j-го кортежа (s, a, r, s') из мини-выборки вычислить значение $Q_j(s', a'; \theta')$, передав в целевую нейронную сеть s'.
11. Вычислить $y^j \leftarrow \begin{cases} r, & \text{если } t = T \\ r + \gamma \max\limits_{a'} Q^j(s', a'; \theta'), & \text{если } t \neq T. \end{cases}$
12. Для каждого j-го кортежа (s, a, r, s') из мини-выборки вычислить $Q_j(s, a; \theta)$, передав в основную нейронную сеть s.
13. Вычислить функцию потерь $L^j(\theta) \leftarrow \frac{1}{K} \sum\limits_{p=1}^{K} \left(y_p^j - Q_p^j(s, a; \theta) \right)^2$.
14. Обновить весá θ основной нейронной сети с использованием $L^j(\theta)$ и скорости обучения α.
15. Заменить весá θ' целевой нейронной сети на веса θ основной нейронной сети, если закончился очередной период Cs.

На рис. 3.4 представлена архитектура нейронной сети алгоритма IQL для рассматриваемой в гл. 3 задачи мультиагентного обучения трех агентов. В отличие от нейронной сети стандартного алгоритма DQN, в рассматриваемой нейронной сети алгоритма IQL используются только линейные слои. Архитектура нейронной сети алгоритма IQL включает три полносвязных линейных слоя, с входными и выходными размерами соответственно 23×66, 66×60, 60×7. На вход нейронная сеть получает состояние среды *s*, заданное 23 отсчетами, на выходе возвращает семь Q-значений.

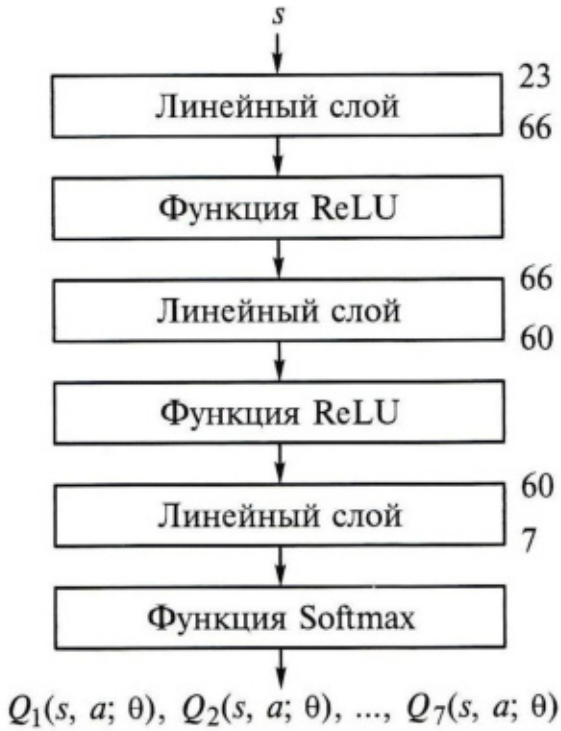


Рис. 3.4. Архитектура нейронной сети алгоритма IQL

Функциями активации первых двух слоев являются функции выпрямителя ReLU (*англ.* Rectified Linear Unit). В настоящее время в глубоких нейронных сетях функция ReLU — одна из самых популярных нелинейных функций активации, ускоряющая обучение и улучшающая распространение градиентов по нейронной сети (НС). Выход третьего слоя обрабатывается обобщенной логистической функцией Softmax для получения нормализованного распределения вероятностей действий. Схема мультиагентного обучения с использованием алгоритма IQL представлена на рис. 3.5.

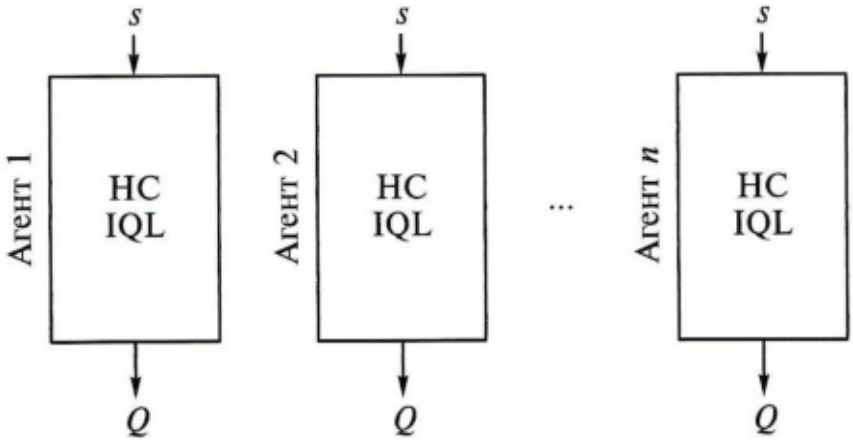


Рис. 3.5. Схема мультиагентного обучения с использованием алгоритма IQL

Далее рассмотрим алгоритм с использованием сверточной нейронной сети.

3.3.2. Централизованное обучение с использованием сверточной нейронной сети (CDQN)

Алгоритм централизованного обучения с использованием сверточной нейронной сети (*англ.* Central Deep Q-Network, CDQN) основан на алгоритме DQN.

Алгоритм CDQN для агента i приведен ниже:

1. Инициализировать
   $\alpha \in (0,1]$; $\gamma \in [0,1]$; $\varepsilon \in [0.1,1]$; $Ne \in [1, N]$;
   $Eb \leftarrow 0$; $El \leftarrow 10\,000$; $Bs \leftarrow 32$; $Cs \leftarrow 100$;
   $i \leftarrow 1, ..., n$; $a_i \in A_i$; $j \leftarrow 1, ..., Bs$; $K_i \leftarrow |A_i|$;
   $\theta \leftarrow \xi$; $\theta' \leftarrow \xi$; $\forall o \in \mathbb{O}$.
2. Цикл
3. Получить состояние o.
4. Передать наблюдение o в основную нейронную сеть и получить совместную оценку $Q(o, a_1, ..., a_n; \theta)$.
5. Выделить в $Q(o, a_1, ..., a_n; \theta)$ оценки действий $Q_{1...K_i}(o, a_1; \theta)$, $..., Q_{1...K_i}(o, a_n; \theta)$ для каждого агента i.
6. Выбрать возможное действие a_i в соответствии с максимальным значением $Q_1(o, a_i; \theta), Q_2(o, a_i; \theta), ..., Q_{K_i}(o, a_i; \theta)$ и с учетом параметра ε.
7. Выполнить возможное действие a_i.
8. Получить награду r и следующее наблюдение o'.
9. Сохранить кортеж $(o, a_1, ..., a_n, r, o')$ в буфере воспроизведения Eb. Если объем Bl превышен, заменить значение по принципу очереди FIFO.
10. Случайно получить мини-выборку $(o, a_1, ..., a_n, r, o')$ объемом Bs из буфера воспроизведения Eb.
11. Для каждого j-го кортежа $(o, a_1, ..., a_n, r, o')$ из мини-выборки вычислить значение $Q^j(o', a'_1, ..., a'_n; \theta)$, предварительно передав в целевую нейронную сеть o'.
12. Вычислить $$y^j \leftarrow \begin{cases} r, & \text{если } t = T; \\ r + \gamma \max\limits_{a'_1, ..., a'_n} Q^j(o', a'_1, ..., a'_n; \theta'), & \text{если } t \neq T. \end{cases}$$
13. Для каждого j-го кортежа $(o, a_1, ..., a_n, r, o')$ из мини-выборки вычислить значение $Q^j(o, a_1, ..., a_n; \theta)$, предварительно передав в основную нейронную сеть o.

14. Вычислить функцию потерь $L^j(\theta) \leftarrow \frac{1}{K}\sum_{p=1}^{K}\left(y_p^j - Q_p^j(o, a_1, \ldots, a_n; \theta)\right)^2$.

15. Обновить весá θ основной нейронной сети с использованием $L^j(\theta)$ и скорости обучения α.

16. Заменить весá θ′ целевой нейронной сети на весá θ основной нейронной сети, если закончился очередной период C*s*.

В алгоритме CDQN, так же как и в алгоритме IQL, используются буфер воспроизведения (шаг 9) и целевая нейронная сеть (шаг 11). Но буфер воспроизведения, целевая нейронная сеть и основная нейронная сеть являются едиными для всех обучающихся агентов. Таким образом параметры нейронных сетей совместно используются всеми агентами. Кроме того, в буфере воспроизведения сохраняется наблюдение, а не точное состояние среды, заданное цифровым изображением псевдокарты (см. разд. 3.4). Главным преимуществом алгоритма CDQN является преодоление проблемы нестационарности мультиагентного обучения, но при этом нивелируются сильные стороны распределенной системы агентов.

На рис. 3.6 представлена архитектура нейронной сети алгоритма CDQN для рассматриваемой в гл. 3 задачи мультиагентного обучения трех агентов. В отличие от нейронной сети алгоритма IQN, в нейронной сети алгоритма

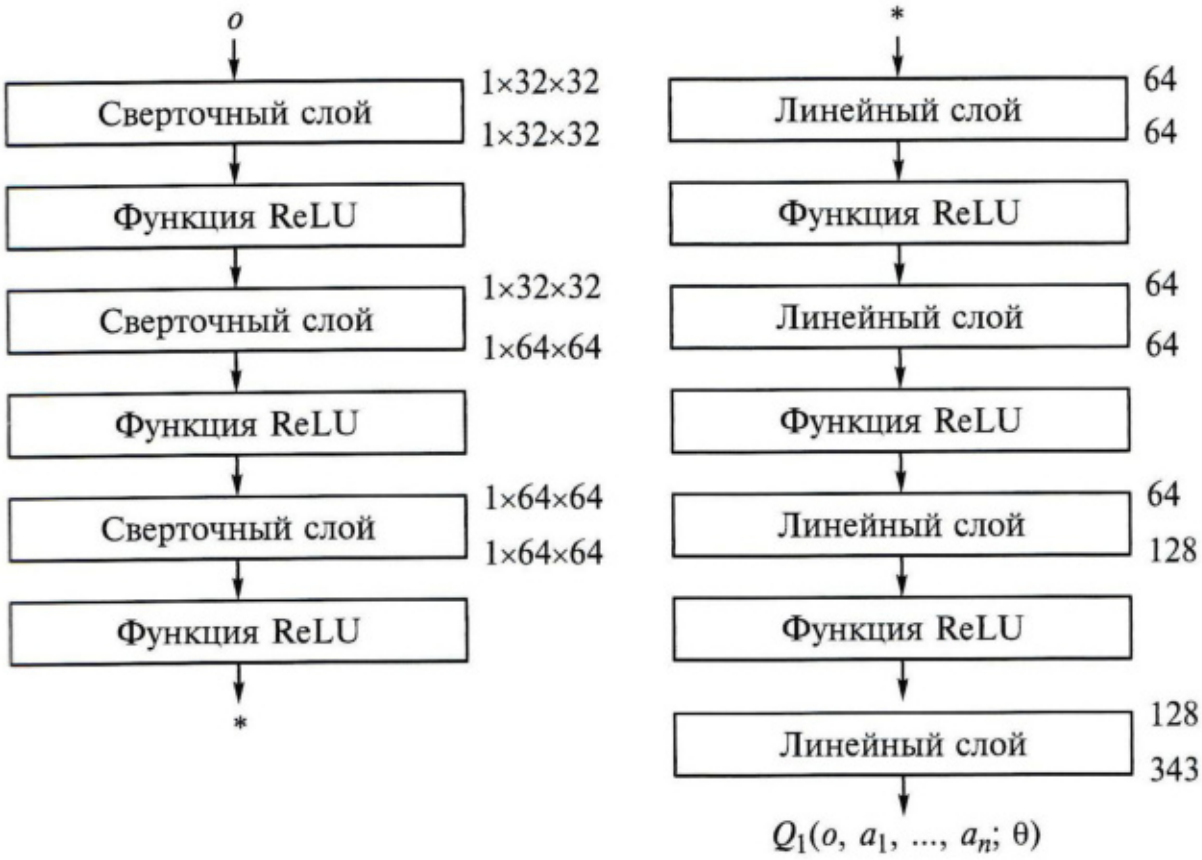


Рис. 3.6. Архитектура нейронной сети алгоритма CDQN

CDQN дополнительно используются сверточные слои для автоматической обработки цифрового изображения, определяющего наблюдение агентов, и извлечения из него характерных признаков, которые затем обрабатываются линейными слоями. Архитектура нейронной сети алгоритма CDQN включает три сверточных слоя с входными размерами соответственно 1×32×32, 1×32×32, 1×64×64 и выходными размерами 1×32×32, 1×64×64, 1×64×64; четыре полносвязных линейных слоя имеют входные и выходные размеры соответственно 64×64, 64×64, 64×128, 128×343. Функциями активации сверточных и полносвязных линейных слоев являются функции выпрямителя ReLU.

Рис. 3.7. Схема мультиагентного обучения с использованием алгоритма CDQN

На вход нейронная сеть получает наблюдение o, заданное полутоновым цифровым изображением размером 32×32 пикселя. На выходе нейронная сеть возвращает 343 Q-значения, задающих совместную функцию ценности действий трех агентов, каждый из которых способен выполнить семь действий. Схема мультиагентного обучения с использованием алгоритма CDQN приведена на рис. 3.7.

Далее рассмотрим алгоритм с использованием рекуррентной нейронной сети.

3.3.3. Декомпозиция Q-значений с использованием рекуррентной нейронной сети (VDN)

Приведенный ниже алгоритм декомпозиции Q-значений с использованием рекуррентной нейронной сети (*англ.* Value-Decomposition Network, VDN) создавался компанией Google DeepMind для преодоления проблем независимого и централизованного обучения.

Алгоритм VDN для агента i:

1. Инициализировать
   $\alpha \in (0,1]$; $\gamma \in [0,1]$; $\varepsilon \in [0.1,1]$; $Ne \in [1, N]$;
   $Eb \leftarrow 0$; $El \leftarrow 10\,000$; $Bs \leftarrow 32$; $Cs \leftarrow 100$;
   $i \leftarrow 1, ..., n$; $a_i \in A_i$; $j \leftarrow 1, ..., Bs$; $K \leftarrow |A_i|$;
   $\theta \leftarrow \xi$; $\theta' \leftarrow \xi$; $\forall s \in S; H_i \in \bar{H}$.
2. Цикл
3. Получить состояние s.
4. Определить возможные действия a_i в состоянии s.

5. Передать состояние s в основную нейронную сеть и выбрать возможное действие a_i в соответствии с максимальным значением $\tilde{Q}_1(H_i, a_i; \theta_i), \tilde{Q}_2(H_i, a_i; \theta_i), \ldots, \tilde{Q}_{K_i}(H_i, a_i; \theta_i)$ и с учетом параметра ε.

6. Выполнить возможное действие a_i.

7. Получить награду r и следующее состояние s'.

8. Сохранить кортеж (s, a, r, s') в буфере воспроизведения *Eb*. Если объем *Bl* превышен, заменить значение по принципу очереди FIFO.

9. Получить последовательную мини-выборку(s, a, r, s') объемом *Bs* из буфера воспроизведения *Eb*.

10. Для каждого агента i и j-го кортежа (s, a, r, s') из последовательной мини-выборки вычислить $\tilde{Q}_i^j(H_i', a_i'; \theta_i')$, предварительно передав в целевую нейронную сеть состояние s'.

11. Вычислить целевое совместное Q-значение
$$Q^j\left((H_1', \ldots, H_n'), (a_1', \ldots, a_n'); \theta'\right) \leftarrow \tilde{Q}_1^j(H_1', a_1'; \theta_1') + \ldots + \tilde{Q}_n^j(H_n', a_n'; \theta_n').$$

12. Вычислить $y^j \leftarrow \begin{cases} r, & \text{если } t = T; \\ r + \gamma \max\limits_{a_i'} Q^j\left((H_1', \ldots, H_n'), (a_1', \ldots, a_n'); \theta'\right), & \text{если } t \neq T. \end{cases}$

13. Для каждого агента i и j-го кортежа (s, a, r, s') из последовательной мини-выборки вычислить $\tilde{Q}_i^j(H_i, a_i; \theta_i)$, предварительно передав в основную нейронную сеть состояние s.

14. Вычислить основное совместное Q-значение
$$Q^j\left((H_1, \ldots, H_n), (a_1, \ldots, a_n); \theta\right) \leftarrow \tilde{Q}_1^j(H_1, a_1; \theta_1) + \ldots + \tilde{Q}_n^j(H_n, a_n; \theta_n).$$

15. Вычислить функцию потерь
$$L^j(\theta) \leftarrow \frac{1}{K} \sum_{p=1}^{K} \left(y_p^j - Q_p^j\left((H_1, \ldots, H_n), (a_1, \ldots, a_n); \theta\right)\right)^2.$$

16. Обновить весá θ основных нейронных сетей каждого агента i с использованием $L^j(\theta)$ и скорости обучения α.

17. Заменить весá θ′ целевых нейронных сетей на весá θ основных нейронных сетей каждого агента i, если закончился очередной период *Cs*.

В алгоритме VDN применяются следующие мультиагентные алгоритмические приемы: «декомпозиция Q-значений», «разница наград» и «централизованное обучение с децентрализованным исполнением». Помимо этого, в алгоритме VDN используются идеи целого ряда популярных алгоритмов: основная схема обучения с буфером воспроизведения и целевой нейронной сетью основана на алгоритме DQN; дуэльная архитектура заимствована из алгоритма Dueling DQN; идея обучения совместным действиям взята из алгоритма JAL; эксплуатация истории состояний основана на рекуррентных нейронных сетях алгоритма DRQN.

В целом алгоритм VDN открывает новое направление в мультиагентном обучении по автоматической декомпозиции задачи машинного обучения на аддитивные локальные задачи. Но в алгоритме VDN явно не указывается, что этот механизм реализуется через процедуру агрегирования (шаги 11, 14), поэтому, как и в алгоритме QMIX, декомпозиция или факторизация глобально относится к решаемой агентами задаче машинного обучения, а непосредственно мультиагентная составляющая реализуется через объединение, или агрегирование, Q-значений по некоторому критерию. В алгоритме VDN критерием агрегирования выступает обычный оператор суммы, хотя в области мягких вычислений разработано несколько эффективных нелинейных операторов агрегирования, например, нечеткий интеграл Шоке или нечеткий интеграл Суджено.

Первые восемь шагов алгоритма VDN полностью повторяют независимое обучение алгоритма IQL. Отличие появляется на шаге 9 при получении последовательной мини-выборки (s, a, r, s') объемом Bs из буфера воспроизведения Eb.

В оригинальном алгоритме DRQN исследовались два способа получения мини-выборки из буфера воспроизведения: классический случайный набор данных и последовательный набор данных, который создается для удобства обучения рекуррентной нейронной сети на основе истории состояний. Однако здесь возникает противоречие: с одной стороны, случайность полезна для схемы обучения DQN из-за свойства н.о.р., но не позволяет создать историю состояний; с другой стороны, последовательная мини-выборка полезна для обучения реккурентной нейронной сети LSTM, но нарушает свойство н.о.р. В рассматриваемой реализации алгоритма VDN предпочтение отдано последовательной мини-выборке для полного раскрытия потенциала LSTM.

Другое отличие алгоритма VDN от алгоритма IQL заключается в том, что алгоритм VDN реализует мультиагентную составляющую через нахождение совместного Q-значения на шагах 11 и 14. И весá независимых нейронных сетей обновляются с помощью значения функции потерь $L_Q(\theta)$, вычисленной на основе совместного значения $Q\big((H_1,\ldots,H_n),(a_1,\ldots,a_n);\theta\big)$.

На рис. 3.8 представлена архитектура нейронной сети алгоритма VDN для рассматриваемой в гл. 3 задачи мультиагентного обучения трех агентов. На вход нейронная сеть получает состояние среды s, заданное 23 отсчетами, на выходе возвращает семь Q-значений. Первый слой нейронной сети

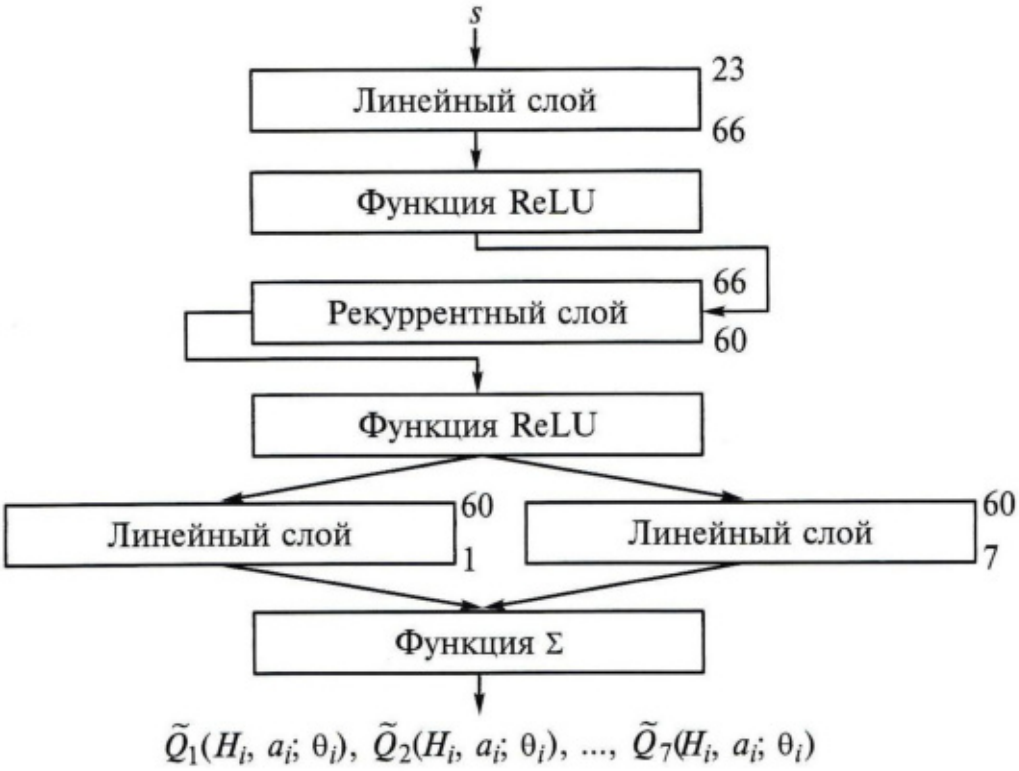


Рис. 3.8. Архитектура нейронной сети алгоритма VDN

является полносвязным линейным слоем с входным и выходным размерами 23×66. Второй слой нейронной сети является рекуррентным слоем LSTM с входным и выходным размерами 66×60. Функциями активации первых двух слоев являются функции выпрямителя ReLU.

Третий слой нейронный сети представляет собой дуэльную архитектуру, заданную полносвязным линейным слоем с входным и выходным размерами 60×1, моделирующим функцию ценности истории состояний $V_i^{\pi}(H_i; \theta_i)$, и полносвязным линейным слоем с входным и выходным размерами 60×7, моделирующим функцию преимущества действия $A_i^{\pi}(a_i; \theta_i)$. Выход третьего

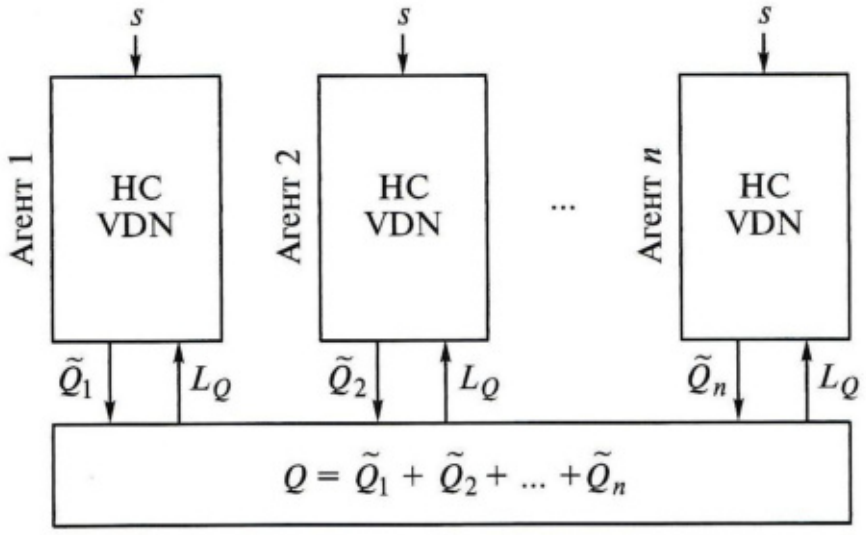


Рис. 3.9. Схема мультиагентного обучения с использованием алгоритма VDN

слоя обрабатывается простой функцией суммирования Σ для нахождения значения $\tilde{Q}_i^{\pi}(H_i, a_i; \theta_i)$. Схема мультиагентного обучения с использованием алгоритма VDN представлена на рис. 3.9.

В заключение рассмотрим алгоритм с архитектурой исполнитель-критик.

3.3.4. Мультиагентный глубокий детерминированный градиент стратегий (MADDPG)

Алгоритм мультиагентного глубокого детерминированного градиента стратегий (*англ.* Multiagent Deep Deterministic Policy Gradient, MADDPG) представляет собой расширение алгоритма DDPG для мультиагентного обучения. Алгоритм MADDPG создавался компанией OpenAI с целью преодоления проблем координации, нестационарности и нестабильности мультиагентного обучения. В алгоритме MADDPG используется мультиагентный алгоритмический прием «централизованное обучение с децентрализованным исполнением».

Алгоритм MADDPG для агента *i* приведен ниже:

1. Инициализировать

   $\alpha_{ac}, \alpha_{cr} \in (0,1]$; $\gamma \in [0,1]$; $Ne \in [1, N]$;
   $\mu_{\theta_i^{ac}} \leftarrow 0$; $Eb \leftarrow 0$; $Bl \leftarrow 10\,000$; $Bs \leftarrow 32$;
   $i \leftarrow 1, ..., n$; $a_i \in A_i$; $j \leftarrow 1, ..., Bs$; $K_i \leftarrow |A_i|$;
   $\theta \leftarrow \xi$; $\theta' \leftarrow \xi$; $\mathcal{N} \leftarrow \xi$; $\forall s_i \in S_i$.

2. Цикл

3. Получить состояние s_i.

4. Передать состояние s_i в основную нейронную сеть исполнителя и получить стратегию $\mu_{\theta_i^{ac}}(s_i)$.

5. Выбрать возможное действие a_i в соответствии со стратегией $\mu_{\theta_i^{ac}}(s_i)$ и случайным шумом $\mathcal{N}$: $a_i \leftarrow \mu_{\theta_i^{ac}}(s_i) + \mathcal{N}$.

6. Выполнить возможное действие a_i.

7. Получить награду r и следующее состояние s_i'.

8. Сохранить кортеж $((s_1, ..., s_n), a_1, ..., a_n, r, (s_1', ..., s_n'))$ в буфере воспроизведения *Eb*. Если объем *Bl* превышен, заменить значение по принципу очереди FIFO.

9. Случайно получить мини-выборку $((s_1, ..., s_n), a_1, ..., a_n, r, (s_1', ..., s_n'))$ объемом *Bs* из буфера воспроизведения *Eb*.

10. Для каждого j-го кортежа $\left((s_1,\ldots,s_n),a_1,\ldots,a_n,r,(s'_1,\ldots,s'_n)\right)$ из мини-выборки вычислить значение $Q^j\left((s'_1,\ldots,s'_n),a'_1,\ldots,a'_n;\theta_i'^{cr}\right)$, передав в целевую нейронную сеть критика состояния $(s'_1,\ldots,s'_n)$.

11. Вычислить $y^j \leftarrow \begin{cases} r, & \text{если } t=T; \\ r+\gamma Q^j\left((s'_1,\ldots,s'_n),a'_1,\ldots,a'_n;\theta_i'^{cr}\right), & \text{если } t\neq T. \end{cases}$

12. Для каждого j-го кортежа $\left((s_1,\ldots,s_n),a_1,\ldots,a_n,r,(s'_1,\ldots,s'_n)\right)$ из мини-выборки вычислить значение $Q^j\left((s_1,\ldots,s_n),a_1,\ldots,a_n;\theta_i^{cr}\right)$, передав в основную нейронную сеть критика $(s'_1,\ldots,s'_n)$.

13. Вычислить функцию потерь критика:
$$L_i^j\left(\theta_i^{cr}\right)\leftarrow\frac{1}{K}\sum_{p=1}^{K}\left(y_p^j-Q_p^j\left((s_1,\ldots,s_n),a_1,\ldots,a_n;\theta_i^{cr}\right)\right)^2.$$

14. Обновить весá θ_i^{cr} основной нейронной сети критика с использованием $L_i^j\left(\theta_i^{cr}\right)$ и скорости обучения α_{cr}.

15. Вычислить функцию потерь исполнителя:
$$L_i^j\left(\theta_i^{ac}\right)\leftarrow-\left(\frac{1}{K}\sum_{p=1}^{K}Q_p^j\left((s_1,\ldots,s_n),a_1,\ldots,a_n;\theta_i^{cr}\right)\right).$$

16. Обновить весá θ_i^{ac} основной нейронной сети исполнителя с использованием $L_i^j\left(\theta_i^{ac}\right)$ и скорости обучения α_{cr}.

17. Заменить весá θ'^{cr}_i целевой нейронной сети критика и весá θ'^{ac}_i целевой нейронной сети исполнителя на веса θ_i^{cr} и θ_i^{ac} основных нейронных сетей, используя принципа мягкой замены:
$$\theta'^{cr}_i\leftarrow\tau\theta_i^{cr}+(1-\tau)\theta_i^{cr};$$
$$\theta'^{ac}_i\leftarrow\tau\theta_i^{ac}+(1-\tau)\theta_i^{ac}.$$

Шесть главных отличий алгоритма MADDPG от рассмотренных алгоритмов мультиагентного нейросетевого обучения заключаются в следующем.

Первое отличие алгоритма MADDPG состоит в том, что он относится как к классу алгоритмов вычисления стратегий, так и к классу алгоритмов вычисления ценности состояний-действий (см. табл. 3.1). В алгоритме MADDPG используется нейросетевая архитектура исполнитель-критик,

в которой нейронная сеть исполнителя с весами аппроксимирует стратегию $\mu_{\theta_i^{ac}}$, а нейронная сеть критика с весами аппроксимирует совместную функцию ценности действий $Q_i^{\mu}\left((s_1,...,s_n),(a_1,...,a_n);\theta_i^{cr}\right)$. В связи с этим в алгоритме MADDPG применяются два параметра скорости обучения — отдельно для исполнителя α_{ac} и для критика α_{cr} (шаг 1). Кроме того, в алгоритме MADDPG используется несколько нейронных сетей критиков для каждого агента.

Авторы алгоритма MADDPG доказали, что вероятность нахождения правильного градиента для обновления весов уменьшается экспоненциально при увеличении числа агентов вследствие нестационарности мультиагентного обучения. И именно архитектура нейронной сети исполнитель-критик позволила преодолеть эту проблему. Роль централизованного критика заключается в устранении неопределенности, возникающей вследствие того, что агенты не имеют информации о действиях других агентов. По этой причине все действия агентов (a_1, ..., a_n) подаются в нейронные сети критиков.

Вторым важным отличием алгоритма MADDPG является выбор действий агентами на основе стратегии $\mu_{\theta_i^{ac}}$, а не на основе оценки Q-значений. При использовании Q-значений выбирается действие a_i, которое имеет максимальное Q-значение. А при использовании стратегии $\mu_{\theta_i^{ac}}$, которая представляет собой распределение вероятностей выполнения того или иного действия, выбирается действие a_i с максимальной вероятностью. Но это только вероятность, а не гарантия выполнения данного действия, как при использовании Q-значений. Таким образом может быть выполнено и другое действие, с меньшей вероятностью. Поэтому выбор возможного действия на шаге 5 алгоритма MADDPG, несмотря на внешнюю схожесть с выбором действий в предыдущих алгоритмах, имеет кардинально другое значение.

Третье отличие заключается в том, что в алгоритме MADDPG используется схема обучения алгоритма DQN в виде буфера воспроизведения и целевой нейронной сети только для обучения нейронных сетей критиков (шаги 9–14). Буфер воспроизведения здесь один и хранит совместные состояния и совместные действия всех агентов сразу (шаг 8). Как и в алгоритме CDQN, в алгоритме MADDPG в буфере воспроизведения хранится общее для всех агентов наблюдение или состояние. Схема обучения алгоритма DQN была изменена: на шаге 11 целевое значение y вычислялось без нахождения максимума Q-значений, чтобы не переоценивать целевые значения.

Четвертым отличием является использование в алгоритме MADDPG случайного шума $\mathcal{N}$ для исследования действий вместо ε-«жадного» алгоритма (шаг 5). В оригинальной версии алгоритма MADDPG для генерации шума используется процесс Орнштейна — Уленбека, но последние исследования показывают, что даже обычного гауссова шума достаточно, чтобы алгоритм успешно обучался. Этот вопрос остается открытым.

Пятое отличие алгоритма MADDPG состоит в принципе мягкой замены весов целевых нейронных сетей на веса основной нейронной сети с параметром τ (шаг 17). Этим алгоритм MADDPG отличается от всех рассмотренных алгоритмов, где замена весов происходит одномоментно через каждые Cs шагов.

Ну и наконец, основным отличием алгоритмов, в которых используется архитектура нейронной сети исполнитель-критик, является обновление весов нейронных сетей исполнителей с помощью значения выхода нейронной сети критика. Точнее, усредненное по мини-выборке отрицательное выходное Q-значение критика используется в качестве значения функции потерь (шаг 15) для обучения нейронных сетей исполнителей (шаг 16). Отрицательное значение здесь чрезвычайно важно, так как весá нейронных сетей исполнителей обновляются методом градиентного спуска в противоположном градиенту направлении, но для максимизации значения целевой функции веса должны изменяться в направлении градиента $\nabla_{\theta_i^{ac}} j\left(\theta_i^{ac}\right)$.

Главными преимуществами алгоритма MADDPG являются его универсальность и способность функционировать в разных средах для кооперативного, конкурентного и смешанного мультиагентного обучения. Алгоритм MADDPG не требует дифференцируемой модели среды и явной коммуникации между агентами при решении задачи физической или информационной координации.

Недостатком алгоритма является сложность разработки и сопровождения вследствие особенностей передачи данных между всеми нейронными

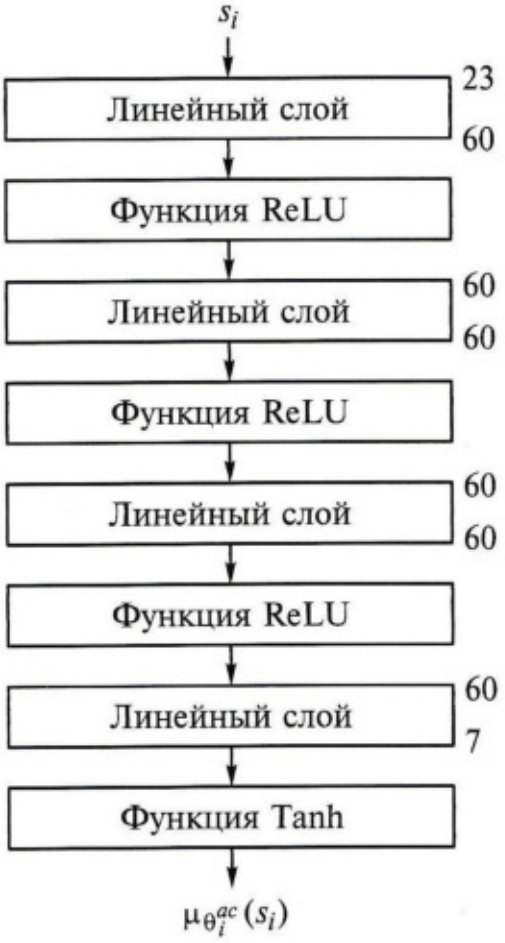


Рис. 3.10. Архитектура нейронной сети исполнителя алгоритма MADDPG

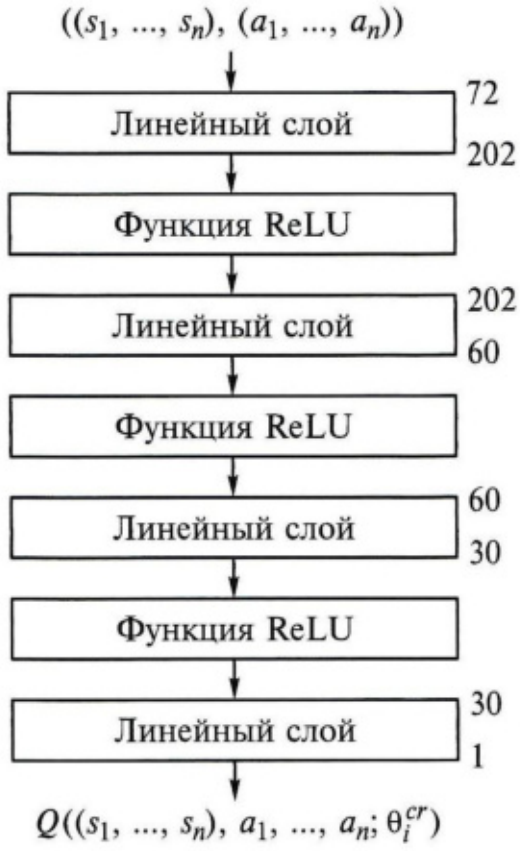


Рис. 3.11. Архитектура нейронной сети критика алгоритма MADDPG

сетями алгоритма. Кроме того, в алгоритме MADDPG не преодолена линейная зависимость Q-значений критика от числа обучающихся агентов.

На рис. 3.10 представлена архитектура нейронной сети исполнителя алгоритма MADDPG для рассматриваемой в гл. 3 задачи мультиагентного обучения трех агентов. Архитектура нейронной сети исполнителя включает четыре полносвязных линейных слоя с входными и выходными размерами соответственно 23×60, 60×60, 60×60, 60×7. На вход нейронная сеть исполнителя получает локальное состояние среды s_i каждого отдельного агента i, на выходе — возвращает стратегию $\mu_{\theta_i^{\mu}}(s_i)$.

Функциями активации первых трех слоев являются функции выпрямителя ReLU. Выход четвертого слоя обрабатывается гиперболической тангенциальной функцией Tanh, которая повышает точность оценки стратегии $\mu_{\theta_i^{\mu}}(s_i)$. Кроме того, вследствие симметричности относительно начала координат и выходных значений, близких к нулю, функция Tanh часто применяется в качестве функции активации на последних слоях глубокой нейронной сети для предотвращения быстрого достижения максимальных значений их весов.

На рис. 3.11 представлена архитектура нейронной сети критика алгоритма MADDPG для рассматриваемой в гл. 3 задачи мультиагентного обучения

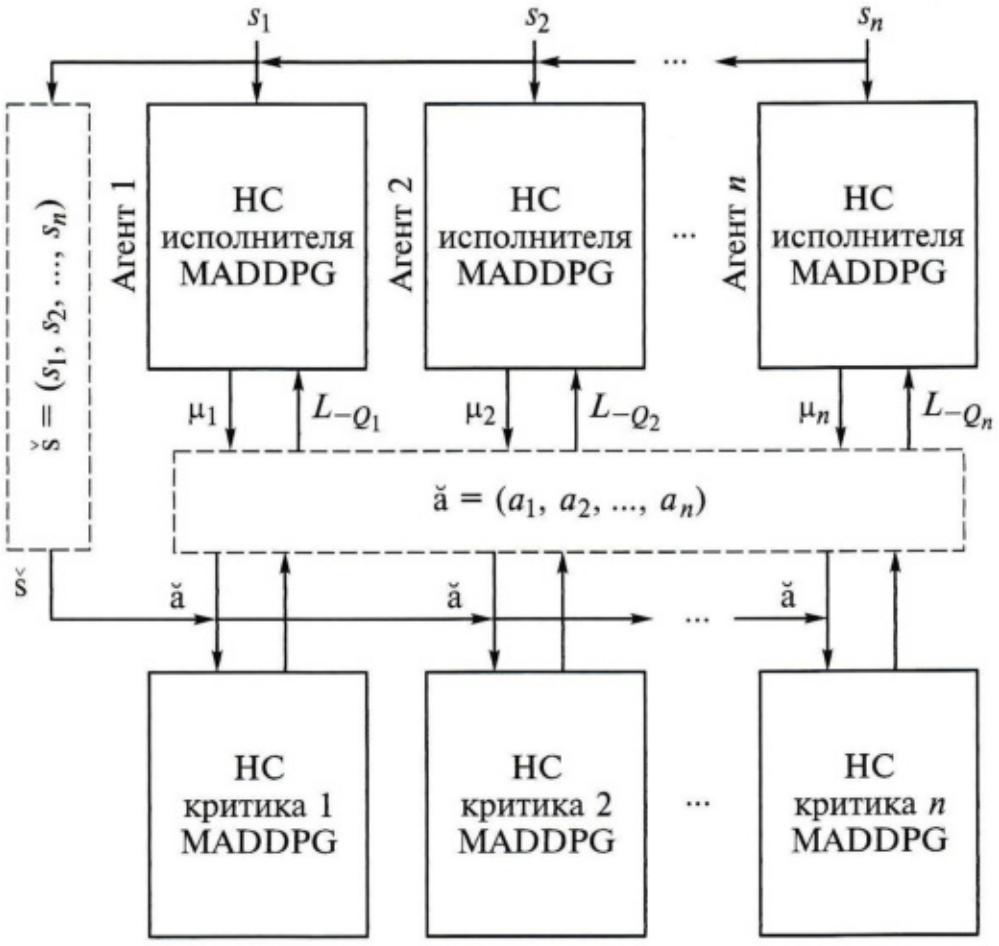


Рис. 3.12. Схема мультиагентного обучения с использованием алгоритма MADDPG

трех агентов. Архитектура нейронной сети критика включает четыре полносвязных линейных слоя с входными и выходными размерами соответственно 72×202, 202×60, 60×30, 30×1. На вход нейронная сеть критика получает кортеж $((s_1,...,s_n),(a_1,...,a_n))$ совместных состояний и совместных действий агентов, на выходе — возвращает одно Q-значение. Функциями активации первых трех слоев являются функции выпрямителя ReLU. Схема мультиагентного обучения с использованием алгоритма MADDPG представлена на рис. 3.12.

Рассмотрим описание среды для мультиагентного нейросетевого обучения.

3.4. Карта

Мультиагентное нейросетевое обучение в гл. 3 проводится на карте 3ps1zgWallFOX.SC2Map (рис. 3.13). В начале каждого эпизода на карте слева, в точке Team 1, автоматически создаются три агента юнита сталкера расы протоссов, которыми управляет обучающий алгоритм, справа, в точке Team 2, автоматически создается один агент юнит гидралиск **расы зергов**, которым управляет компьютер, используя стандартные правила. Длительность

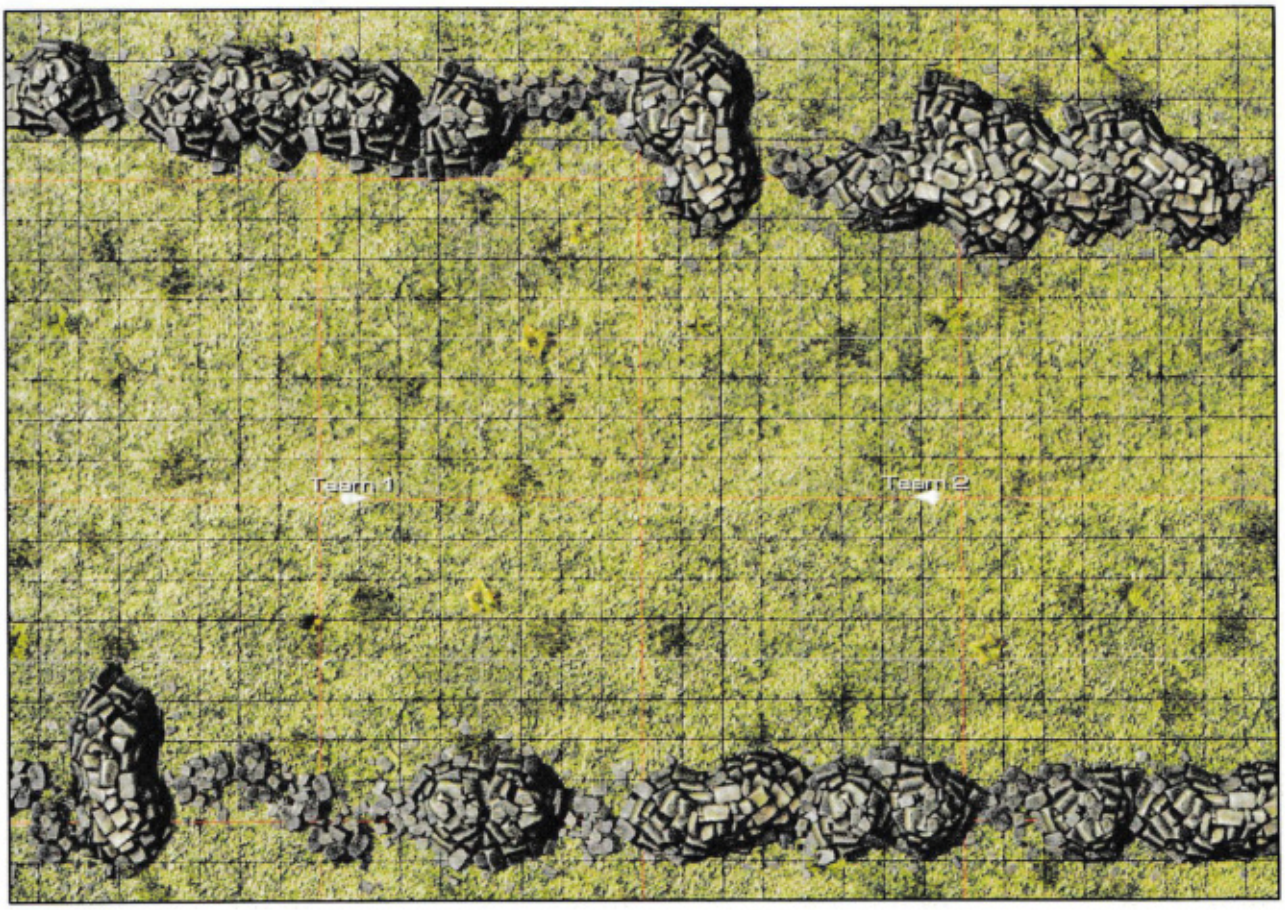


Рис. 3.13. Внешний вид карты 3ps1zgWallFOX.SC2Map в редакторе StarCraft II Editor

одного полного эпизода игры составляет 6 с. На карте, по сравнению с картой в гл. 1 (см. рис. 1.4), появилось значительно больше пространства для перемещения юнитов.

Сталкер — наземный боевой юнит протоссов, обладающий атакой дальнего боя, способный поражать как наземные, так и воздушные юниты на расстоянии шести квадратов игрового поля, нанося 10 единиц урона. Название «сталкер» (преследователь, охотник) связано с часто используемой стратегией мобильных атак (для исследования ресурсов оппонента). Сталкер имеет 80 единиц здоровья и одну единицу брони. Гидралиск — наземный боевой юнит зергов, обладающий атакой ближнего и дальнего боя, способный поражать как наземные, так и воздушные юниты на расстоянии пяти квадратов игрового поля, нанося 12 единиц урона. Гидралиск имеет 90 единиц здоровья. При столкновении с протоссами гидралиск эффективно используется против сталкеров и зилотов.

На рис. 3.14–3.17 проиллюстрированы различные ситуации в среде SMAC: начало эпизода игры, противник атакует справа вне зоны

Рис. 3.14. Начало эпизода

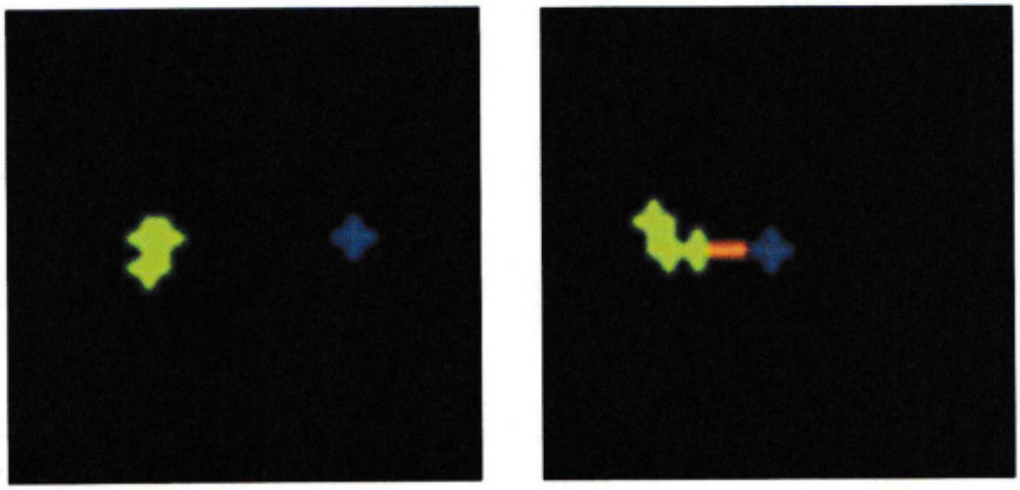
Рис. 3.15. Агенты спасаются бегством

Рис. 3.16. Два агента выстроились для одновременной атаки

Рис. 3.17. Все агенты выучили оптимальную стратегию

стрельбы (см. рис. 3.14); агенты спасаются бегством, стараясь уменьшить потери от столкновения с противником (см. рис. 3.15); два агента выстроились для одновременной атаки, а третий агент пытается преодолеть преграду пропонентов (см. рис. 3.16); все агенты выучили оптимальную стратегию (см. рис. 3.17). Результаты экспериментального обучения в среде SMAC будут продемонстрированы в разд. 3.7.

На рис. 3.18 представлено схематическое изображение агентов на псевдокарте, которая используется в централизованном обучении агентов с помощью алгоритма CDQN. Для целей визуализации цифровое изображение псевдокарты приведено к размеру 320×320 пикселей. Зеленым цветом обозначены сталкеры, синим — гидралиск. На рис. 3.18, *а* представлено начало



а *б*

Рис. 3.18. Схематическое изображение агентов на псевдокарте:
а — начало эпизода; *б* — боевое столкновение

эпизода игры, на рис. 3.18, *б* — боевое столкновение, когда один из агентов выполняет действие «стрелять», показанное оранжевым отрезком.

Для использования карты необходимо подключить определенные программные библиотеки.

3.5. Технология

Рассматриваемые в гл. 3 алгоритмы реализованы на языке программирования Python версии 3.6. В представленном в разд. 3.6 программном коде, кроме библиотеки кооперативного мультиагентного обучения SMAC, используются стандартные библиотеки: numpy для работы с массивами, collections для использования специализированного типа данных обобщенной очереди deque, time для измерения времени, random для выбора случайных элементов из списка, matplotlib для построения графиков, cv2 для цифровой обработки изображений.

Глубокие нейронные сети разработаны с помощью библиотеки PyTorch версии 1.5.1, свободно распространяемой исследовательской лабораторией FAIR (*англ.* Facebook's AI Research lab). Библиотека PyTorch занимает некоторое промежуточное положение между довольно сложной библиотекой глубокого обучения TensorFlow и недостаточно детализированной библиотекой Keras. Библиотека PyTorch позволяет создавать нейронные сети логически ясным способом с достаточной конкретизацией, используя «питоновский» стиль программирования. Современная библиотека PyTorch в большей степени ориентирована на проведение исследований в области глубокого обучения и создание новых интеллектуальных технологий.

Компания Google DeepMind при обучении агентов с помощью нейросетевого алгоритма AlphaStar использовала полные игровые данные компьютерной игры StarCraft II, длительность которых в пересчете на астрономическое время составляет 200 лет. Компания OpenAI при обучении агентов с использованием нейросетевого алгоритма OpenAI Five в компьютерной игре Dota 2 использовала игровые данные длительностью 180 лет. Обучение требовало значительных вычислительных ресурсов — 128 тыс. центральных процессоров и 256 графических процессоров. Рассматриваемые в гл. 3 алгоритмы при решении локальной задачи микроменеджмента в среде SMAC разрабатывались на персональном компьютере с центральным процессором Intel i5 с видеокартой NVIDIA GeForce RTX 2060.

Карта 3ps1zgWallFOX.SC2Map создана на базе карты SMAC в редакторе StarCraft II Editor. Ниже представлены изменения, которые необходимо внести в файл smac_maps.py, расположенный в папке «...\smac\env\starcraft2\maps» при создании новой карты или редактировании существующей:

```
«3ps1zgWallFOX»: {
    «n_agents»: 3,
    «n_enemies»: 1,
    «limit»: 240,
    «a_race»: «P»,
    «b_race»: «Z»,
    «unit_type_bits»: 0,
    «map_type»: «stalkers»,}
```

Эксперименты с алгоритмами проводились в операционной системе Windows 10. Код, реализующий нейросетевые алгоритмы обучения, представлен в подразд. 3.6.1–3.6.3. Код, тестирующий обученные нейронные сети, вынесен в отдельные файлы, которые можно найти на платформе Github.

Примеры реализации алгоритмов представлены в полном объеме на Github: https://github.com/Alekat13/FoxCommander

Видеопримеры работы алгоритмов приведены на YouTube https://www.youtube.com/channel/UC33QPEjSlfA8P9rG0HtiH6g

3.6. Код

3.6.1. Алгоритм IQN

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
from collections import deque
import matplotlib.pyplot as plt
import time

import torch
import torch.nn as nn
import torch.optim as optim

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Определяем архитектуру нейронной сети
class Q_network(nn.Module):
    #На вход нейронная сеть получает состояние среды
    #На выходе нейронная сеть возвращает оценку действий в виде Q-значений
    def __init__(self, obs_size, n_actions):
        super(Q_network, self).__init__()
        self.Q_network = nn.Sequential(
            #Первый линейный слой обрабатывает входные данные состояния среды
            nn.Linear(obs_size, 66),
            nn.ReLU(),
            #Второй линейный слой обрабатывает внутренние данные
            nn.Linear(66, 60),
            nn.ReLU(),
            #Третий линейный слой обрабатывает данные для оценки действий
            nn.Linear(60, n_actions)
        )
        #Применение к выходным данным функции Softmax
        self.sm_layer = nn.Softmax(dim=1)
    #Вначале данные x обрабатываются полносвязной сетью с функцией ReLU
    #На выходе происходит обработка функцией Softmax
    def forward(self, x):
        q_network_out = self.Q_network(x)
        sm_layer_out = self.sm_layer(q_network_out)
        #Финальный выход нейронной сети
        return sm_layer_out
```

```
#Выбираем возможное действие с максимальным Q-значением в зависимости
#от эпсилон
def select_actionFox(action_probabilities, avail_actions_ind, epsilon):
    p = np.random.random(1).squeeze()
    #Исследуем пространство действий
    if np.random.rand() < epsilon:
        return np.random.choice (avail_actions_ind)
    else:
        #Находим возможное действие
        #Проверяем, есть ли действие в доступных действиях агента
        for ia in action_probabilities:
            action = np.argmax(action_probabilities)
            if action in avail_actions_ind:
                return action
            else:
                action_probabilities[action] = 0

#Создаем мини-выборку определенного объема из буфера воспроизведения
def sample_from_expbuf(experience_buffer, batch_size):
    #Функция возвращает случайную последовательность заданной длины
    perm_batch = np.random.permutation(len(experience_buffer))[:batch_size]
    #Мини-выборка
    experience = np.array(experience_buffer)[perm_batch]
    #Возвращаем значения мини-выборки по частям
    return experience[:,0], experience[:,1], experience[:,2],
           experience[:,3], experience[:,4]

#Основная функция программы
def main():

    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="3ps1zgWallFOX", reward_only_positive=False,
reward_scale_rate=200, difficulty="1")
    #Получаем и выводим на печать информацию о среде
    env_info = env.get_env_info()
    print ('env_info=',env_info)
    #Получаем и выводим на печать размер локальных состояний среды для агента
    obs_size =  env_info.get('obs_shape')
    print ("obs_size=",obs_size)
    #Число действий агента
    n_actions = env_info["n_actions"]
    #Число дружественных агентов
    n_agents = env_info["n_agents"]

    #Определяем основные параметры нейросетевого обучения
    ##########################################################################
    #Определим динамический эпсилон с затуханием
    eps_max = 1.0 #начальное значение эпсилон
    eps_min = 0.1 #финальное значение эпсилон
    eps_decay_steps = 15000 #шаг затухания эпсилон
    #Основные переходы в алгоритме IQL зависят от шагов игры
    global_step = 0  #подсчитываем общее число шагов в игре
    copy_steps = 100 #через каждые 100 шагов синхронизируем нейронные сети
    start_steps = 1000 #начинаем обучать через 1000 шагов
    steps_train = 4  #после начала обучения продолжаем обучать каждый
    #четвертый шаг
    #Размер мини-выборки
    batch_size = 32
    #Общее число эпизодов игры
    n_episodes = 300
    #Параметр дисконтирования
    gamma = 0.99
```

```
#Скорость обучения
alpha = 0.01
#Объем буфера воспроизведения
buffer_len = 10000
#########################################################################

#Создаем буфер воспроизведения для каждого агента на основе
#обобщенной очереди deque
experience_buffer0 = deque(maxlen=buffer_len)
experience_buffer1 = deque(maxlen=buffer_len)
experience_buffer2 = deque(maxlen=buffer_len)

#Pytorch определяет возможность использования графического процессора
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#Определяем выходной размер нейронной сети
qofa_out = n_actions

#Создаем основную нейронную сеть
q_network = Q_network(obs_size, n_actions).to(device)
#Создаем целевую нейронную сеть
tgt_network = Q_network(obs_size, n_actions).to(device)

#Создаем списки для мультиагентного обучения
q_network_list = []
tgt_network_list = []
optimizer_list = []
objective_list = []
for agent_id in range(n_agents):
    #Создаем список основных нейронных сетей для трех агентов
    q_network_list.append(q_network)
    #Создаем список целевых нейронных сетей для трех агентов
    tgt_network_list.append(tgt_network)
    #Создаем список оптимизаторов нейронных сетей для трех агентов
    optimizer_list.append(optim.Adam
    (params=q_network_list[agent_id].parameters(), lr=alpha))
    #Создаем список функций потерь для трех агентов
    objective_list.append(nn.MSELoss())

#Выводим на печать списки основных нейронных сетей
print ('q_network_list=', q_network_list)

#Определяем вспомогательные параметры
Loss_History = []
Reward_History = []
winrate_history = []
total_loss = []
m_loss = []

#Основной цикл по эпизодам игры
################_цикл for по эпизодам_###################################
for e in range(n_episodes):

    #Перезагружаем среду
    env.reset()
    #Флаг окончания эпизода
    terminated = False
    #Награда за эпизод
    episode_reward = 0
    #Обновляем и выводим динамический эпсилон
    epsilon = max(eps_min, eps_max - (eps_max-eps_min) *
                  global_step/eps_decay_steps)
    print ('epsilon=',epsilon)
```

```
#Шаги игры внутри эпизода
######################_цикл while_###################################
while not terminated:

    #Обнуляем промежуточные переменные
    actions = []
    action = 0
    #Сохраняем историю действий на одном шаге для разных агентов
    actionsFox = np.zeros([n_agents])
    #Сохраняем историю состояний среды на одном шаге для разных агентов
    obs_agent = np.zeros([n_agents], dtype=object)
    obs_agent_next = np.zeros([n_agents], dtype=object)

    ###########_Цикл по агентам для выполнения действий в игре_#####
    for agent_id in range(n_agents):

        #Получаем состояние среды для независимого агента IQL
        obs_agent[agent_id] = env.get_obs_agent(agent_id)
        #Конвертируем данные в тензор
        obs_agentT = torch.FloatTensor([obs_agent[agent_id]])
                                       .to(device)
        #Передаем состояние среды в основную нейронную сеть
        #и получаем Q-значения для каждого действия
        action_probabilitiesT = q_network_list[agent_id](obs_agentT)
        #Конвертируем данные в numpy
        action_probabilitiesT = action_probabilitiesT.to("cpu")
        action_probabilities = action_probabilitiesT.data.numpy()[0]

        #Находим возможные действия агента в данный момент времени
        avail_actions = env.get_avail_agent_actions(agent_id)
        avail_actions_ind = np.nonzero(avail_actions)[0]
        #Выбираем возможное действие агента с учетом
        #максимального Q-значения и параметра эпсилон
        action = select_actionFox(action_probabilities,
                                  avail_actions_ind, epsilon)
        #Обрабатываем исключение при ошибке в возможных действиях
        if action is None:
           action = np.random.choice (avail_actions_ind)
        #Собираем действия от разных агентов
        actions.append(action)
        actionsFox[agent_id] = action
    ####_конец цикла по агентам для выполнения действий в игре_######

    #Передаем действия агентов в среду, получаем награду
    #и прерывание игры от среды
    reward, terminated, _ = env.step(actions)
    #Суммируем награды за этот шаг для вычисления награды за эпизод
    episode_reward += reward

    ####################_Цикл по агентам для обучения_##############
    for agent_id in range(n_agents):

        #Получаем новое состояние среды
        obs_agent_next[agent_id] = env.get_obs_agent(agent_id)
        #Сохраняем переход в буфере воспроизведения для каждого
        #агента
        if agent_id == 0:
            experience_buffer0.append([obs_agent[agent_id],
             actionsFox[agent_id], obs_agent_next[agent_id], reward,
             terminated])
        elif agent_id == 1:
            experience_buffer1.append([obs_agent[agent_id],
```

```
            actionsFox[agent_id], obs_agent_next[agent_id], reward,
            terminated])
        elif agent_id == 2:
            experience_buffer2.append([obs_agent[agent_id],
            actionsFox[agent_id], obs_agent_next[agent_id], reward,
            terminated])

        #Если буфер воспроизведения наполнен, начинаем обучать сеть
        ########################_начало if обучения_################
        if (global_step % steps_train == 0) and (global_step >
                                                start_steps):
            #Получаем мини-выборку из буфера воспроизведения
            if agent_id == 0:
                exp_obs, exp_act, exp_next_obs, exp_rew, exp_termd =
                sample_from_expbuf(experience_buffer0, batch_size)
            elif agent_id == 1:
                exp_obs, exp_act, exp_next_obs, exp_rew, exp_termd =
                sample_from_expbuf(experience_buffer1, batch_size)
            elif agent_id == 2:
                exp_obs, exp_act, exp_next_obs, exp_rew, exp_termd =
                sample_from_expbuf(experience_buffer2, batch_size)

            #Конвертируем данные в тензор
            exp_obs = [x for x in exp_obs]
            obs_agentT = torch.FloatTensor([exp_obs]).to(device)

            #Подаем мини-выборку в основную нейронную сеть,
            #чтобы получить Q(s,a)
            action_probabilitiesT = q_network_list[agent_id]
                                                (obs_agentT)
            action_probabilitiesT = action_probabilitiesT.to("cpu")
            action_probabilities = action_probabilitiesT.
                                        data.numpy()[0]

            #Конвертируем данные в тензор
            exp_next_obs = [x for x in exp_next_obs]
            obs_agentT_next = torch.FloatTensor([exp_next_obs])
                                                .to(device)

            #Подаем мини-выборку в целевую нейронную сеть,
            #чтобы получить Q'(s',a')
            action_probabilitiesT_next = tgt_network_list[agent_id]
                                                (obs_agentT_next)
            action_probabilitiesT_next = action_probabilitiesT_next.
                                                to("cpu")
            action_probabilities_next = action_probabilitiesT_next.
                                                data.numpy()[0]

            #Вычисляем целевое значение y
            y_batch = exp_rew + gamma *
            np.max(action_probabilities_next, axis=-1)*(1 - exp_termd)

            #Переформатируем y_batch размером batch_size
            y_batch64 = np.zeros([batch_size, qofa_out])
            for i in range (batch_size):
                for j in range (qofa_out):
                    y_batch64[i][j] = y_batch[i]
            #Конвертируем данные в тензор
            y_batchT = torch.FloatTensor([y_batch64])

            #Обнуляем градиенты
            optimizer_list[agent_id].zero_grad()
```

```
                    #Вычисляем функцию потерь
                    loss_t = objective_list[agent_id](action_probabilitiesT,
                                                      y_batchT)
                    #Сохраняем данные для графиков
                    Loss_History.append(loss_t)
                    loss_n=loss_t.data.numpy()
                    total_loss.append(loss_n)
                    m_loss.append(np.mean(total_loss[-1000:]))

                    #Выполняем обратное распространение ошибки
                    loss_t.backward()

                    #Выполняем оптимизацию нейронных сетей
                    optimizer_list[agent_id].step()

                ######################_конец if обучения_####################

                #Синхронизируем веса основной и целевой нейронных сетей
                #через каждые 100 шагов
                if (global_step+1) % copy_steps == 0 and global_step >
                                                    start_steps:
                  tgt_network_list[agent_id].
                  load_state_dict(q_network_list[agent_id].state_dict())

            ######################_Конец цикла по агентам для обучения_#######

            #Обновляем счетчик общего числа шагов
            global_step += 1

        ######################_конец цикла while_##############################

        #Выводим на печать счетчик шагов игры и общую награду за эпизод
        print('global_step=', global_step, "Total reward in episode {} =
                                        {}".format(e, episode_reward))

        #Собираем данные для графиков
        Reward_History.append(episode_reward)
        status = env.get_stats()
        winrate_history.append(status["win_rate"])

    ################_конец цикла по эпизодам игры_############################

    #Закрываем среду StarCraft II
    env.close()

    #Сохраняем параметры обученных нейронных сетей
    for agent_id in range(n_agents):

torch.save(q_network_list[agent_id].state_dict(),"qnet_%.0f.dat"%agent_id)

    #Выводим на печать графики
    #Средняя награда
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                                                  edgecolor='k')
    plt.plot(Reward_History)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Количество награды за эпизод')
    plt.show()
    #Процент побед
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                                                  edgecolor='k')
    plt.plot(winrate_history)
    plt.xlabel('Номер эпизода')
```

```
    plt.ylabel('Процент побед')
    plt.show()
    #Значения функции потерь
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                                                  edgecolor='k')
    plt.plot(m_loss)
    plt.xlabel('Номер каждой 1000-й итерации')
    plt.ylabel('Функция потерь')
    plt.show()

#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    #Выводим на печать время обучения алгоритма IQL
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

3.6.2. Алгоритм VDN

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
from collections import deque
import matplotlib.pyplot as plt
import time

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Размер выхода первого полносвязного линейного слоя
HIDDEN_SIZE = 66
#Размер выхода рекуррентного слоя LSTM
LSTM_HIDDEN_LAYER_SIZE = 60

#Определяем архитектуру нейронной сети
class VDN_network(nn.Module):
    #На вход нейронная сеть получает состояние среды
    #На выходе нейронная сеть возвращает оценку действий в виде Q-значений
    def __init__(self, batch_size, obs_size, lstm_hidden_layer_size,
                 hidden_size, n_actions):
        super(VDN_network, self).__init__()
        #Первый линейный слой обрабатывает входные данные состояния среды
        self.first_layer = nn.Sequential(
            nn.Linear(obs_size, hidden_size),
            nn.ReLU())
        #Определяем выход рекуррентной сети LSTM
        self.hidden_layer_size = lstm_hidden_layer_size
        #Инициализируем внутреннее состояние LSTM
        self.hidden_cell = (torch.zeros(1, batch_size,
                            self.hidden_layer_size).to("cuda"),
                            torch.zeros(1, batch_size,
                            self.hidden_layer_size).to("cuda"))
        #Второй рекуррентный слой LSTM нейронной сети
        self.second_layer = nn.LSTM(input_size=hidden_size,
        hidden_size=self.hidden_layer_size, num_layers=1, batch_first=True)
        #Линейный слой, вычисляющий функцию преимущества
```

```
        self.linear_advantage =
        nn.Sequential(nn.Linear(self.hidden_layer_size, n_actions))
        #Линейный слой, вычисляющий функцию ценности состояния
        self.linear_value = nn.Sequential(nn.Linear(self.hidden_layer_size,
                                                    1))
    #Вначале данные x обрабатываются линейным, затем рекуррентным слоем
    #На выходе данные суммируются после дуэльной обработки линейными слоями
    def forward(self, x):
        #Результат после обработки данных первым слоем
        first_layer_out = self.first_layer(x)
        #Результат после обработки данных вторым слоем
        lstm_out, self.hidden_cell =
        self.second_layer(first_layer_out.view(len(first_layer_out), 1, -1),
                          self.hidden_cell)
        #Применение функции ReLu
        lstm_out = F.relu(lstm_out)
        #Результат первого дуэльного слоя
        aofa_out = self.linear_advantage(lstm_out.view(len(first_layer_out),
                                                       -1))
        #Результат второго дуэльного слоя
        vofs_out = self.linear_value(lstm_out.view(len(first_layer_out), -1))
        #На финальном выходе нейронной сети суммируются результаты дуэльной
        #обработки
        return vofs_out + (aofa_out - aofa_out.mean(dim=1, keepdim=True))

#Выбираем возможное действие с максимальным Q-значением в зависимости
#от эпсилон
def select_actionFox(action_probabilities, avail_actions_ind, epsilon):

    p = np.random.random(1).squeeze()
    #Исследуем пространство действий
    if np.random.rand() < epsilon:
        return np.random.choice (avail_actions_ind)
    else:
        #Находим возможное действие
        #Проверяем, есть ли действие в доступных действиях агента
        for ia in action_probabilities:
            action = np.argmax(action_probabilities)
            if action in avail_actions_ind:
                return action
            else:
                action_probabilities[action] = 0

#Создаем последовательную мини-выборку определенного объема из буфера
#воспроизведения
def sample_sequence_from_expbuf(ind_begin, experience_buffer, big_batch_size):

    #Мини-выборку инициализируем пустой
    experience_buffer_sequence = []

    #Последовательно выбираем значения (s,a,r,s') из буфера воспроизведения
    for i in range(big_batch_size):
        experience_buffer_sequence.append (experience_buffer[ind_begin])
        ind_begin+=1

    return experience_buffer_sequence

#Основная функция программы
def main():
    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="3ps1zgWallFOX", reward_only_positive=False,
                        reward_scale_rate=200, difficulty="1")
```

```
#Получаем и выводим на печать информацию о среде
env_info = env.get_env_info()
print ('env_info=',env_info)
#Получаем и выводим на печать размер наблюдений агента
obs_size =  env_info.get('obs_shape')
print ("obs_size=",obs_size)
#Число действий агента
n_actions = env_info["n_actions"]
#число дружественных агентов
n_agents = env_info["n_agents"]

#Определяем основные параметры нейросетевого обучения
#############################################################################
#Определим динамический эпсилон с затуханием
eps_max = 1.0 #начальное значение эпсилон
eps_min = 0.1 #финальное значение эпсилон
eps_decay_steps = 15000 #шаг затухания эпсилон
#Основные переходы в алгоритме VDN зависят от шагов игры
global_step = 0 #подсчитываем общее количество шагов в игре
copy_steps = 100 #через каждые 100 шагов синхронизируем нейронные сети
start_steps = 1000 #начинаем обучать через 1000 шагов
steps_train = 4 #после начала обучения продолжаем обучать через каждые
                #четыре шага
#Размер мини-выборки
batch_size = 1
#Размер мини-выборки, моделируемый обучением с помощью цикла
BIG_batch_size = 32
#Общее число эпизодов игры
n_episodes = 350
#Параметр дисконтирования
gamma = 0.99
#Скорость обучения
alpha = 0.01
#Объем буфера воспроизведения
buffer_len = 10000
#############################################################################

#Pytorch определяет возможность использования графического процессора
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#Определяем выходной размер нейронной сети
qofa_out = n_actions

#Создаем буферы воспроизведения на основе deque для трех агентов
experience_buffer0 = deque(maxlen=buffer_len)
experience_buffer1 = deque(maxlen=buffer_len)
experience_buffer2 = deque(maxlen=buffer_len)

#Создаем основные нейронные сети для трех агентов
q_network0 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                         HIDDEN_SIZE, n_actions).to(device)
q_network1 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                         HIDDEN_SIZE, n_actions).to(device)
q_network2 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                         HIDDEN_SIZE, n_actions).to(device)
#Создаем целевые нейронные сети для трех агентов
tgt_network0 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                           HIDDEN_SIZE, n_actions).to(device)
tgt_network1 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                           HIDDEN_SIZE, n_actions).to(device)
tgt_network2 = VDN_network(batch_size, obs_size, LSTM_HIDDEN_LAYER_SIZE,
                           HIDDEN_SIZE, n_actions).to(device)
#Выключаем обучение целевой сети
```

```
tgt_network0.eval()
tgt_network1.eval()
tgt_network2.eval()

#Создаем оптимизаторы нейронных сетей для трех агентов
optimizer0 = optim.Adam(params=q_network0.parameters(), lr=alpha)
optimizer1 = optim.Adam(params=q_network1.parameters(), lr=alpha)
optimizer2 = optim.Adam(params=q_network1.parameters(), lr=alpha)

#Создаем функции потерь для трех агентов
objective0= nn.MSELoss()
objective1= nn.MSELoss()
objective2= nn.MSELoss()

#Выводим на печать архитектуру созданной нейронной сети
print ('VDN_network=', q_network0)

#Определяем вспомогательные параметры
Loss_History = []
Reward_History = []
winrate_history = []
total_loss = []
m_loss = []

#Основной цикл по эпизодам игры
################_цикл for по эпизодам_###################################
for e in range(n_episodes):

    #Перезагружаем среду
    env.reset()
    #Флаг окончания эпизода
    terminated = False
    #Награда за эпизод
    episode_reward = 0
    #Обновляем и выводим динамический эпсилон
    epsilon = max(eps_min, eps_max - (eps_max-eps_min) *
                  global_step/eps_decay_steps)
    print ('epsilon=',epsilon)

    #Шаги игры внутри эпизода
    ######################_цикл while_#################################
    while not terminated:

        #Обнуляем промежуточные переменные
        actions = []
        action = 0
        #Сохраняем историю действий на одном шаге для разных агентов
        actionsFox = np.zeros([n_agents])
        #Сохраняем историю состояний среды на одном шаге для разных агентов
        obs_agent = np.zeros([n_agents], dtype=object)
        obs_agent_next = np.zeros([n_agents], dtype=object)

        ##########_Цикл по агентам для выполнения действий в игре_#####
        for agent_id in range(n_agents):

            #Получаем состояние среды для каждого агента VDN
            obs_agent[agent_id] = env.get_obs_agent(agent_id)
            #Конвертируем данные в тензор
            obs_agentT =
             torch.FloatTensor([obs_agent[agent_id]]).to(device)

            #Передаем состояние среды в основную нейронную сеть
            #и получаем Q-значения для каждого действия
```

```
        if agent_id == 0:
            #Выключаем обучение
            q_network0.eval()
            #Инициализируем внутреннее состояние LSTM
            q_network0.hidden_cell = (torch.zeros(1, batch_size,
             LSTM_HIDDEN_LAYER_SIZE).to(device), torch.zeros(1,
             batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device))
            #Получаем результат обработки данных нейронной сетью
            action_probabilitiesT = q_network0(obs_agentT)
        elif agent_id == 1:
            q_network1.eval()
            q_network1.hidden_cell = (torch.zeros(1, batch_size,
             LSTM_HIDDEN_LAYER_SIZE).to(device), torch.zeros(1,
             batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device))
            action_probabilitiesT = q_network1(obs_agentT)
        elif agent_id == 2:
            q_network2.eval()
            q_network2.hidden_cell = (torch.zeros(1, batch_size,
             LSTM_HIDDEN_LAYER_SIZE).to(device), torch.zeros(1,
             batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device))
            action_probabilitiesT = q_network2(obs_agentT)

        #Конвертируем данные в numpy
        action_probabilitiesT = action_probabilitiesT.to("cpu")
        action_probabilities = action_probabilitiesT.data.numpy()[0]

        #Находим возможные действия агента в данный момент времени
        avail_actions = env.get_avail_agent_actions(agent_id)
        avail_actions_ind = np.nonzero(avail_actions)[0]
        #Выбираем возможное действие агента с учетом
        #максимального Q-значения и параметра эпсилон
        action = select_actionFox(action_probabilities,
                                  avail_actions_ind, epsilon)
        #Обрабатываем исключение при ошибке в возможных действиях
        if action is None:
            action = np.random.choice (avail_actions_ind)

        #Собираем действия от разных агентов
        actions.append(action)
        actionsFox[agent_id] = action
    ####_конец цикла по агентам для выполнения действий в игре_######

    #Передаем действия агентов в среду, получаем награду
    #и прерывание игры от среды
    reward, terminated, _ = env.step(actions)
    #Суммируем награды за этот шаг для вычисления награды за эпизод
    episode_reward += reward

    ####################_Цикл по агентам для обучения_##############
    for agent_id in range(n_agents):

        #Получаем новое состояние среды
        obs_agent_next[agent_id] = env.get_obs_agent(agent_id)

        #Сохраняем переход в буфере воспроизведения
        if agent_id == 0:
            experience_buffer0.append([obs_agent[agent_id],
             actionsFox[agent_id], obs_agent_next[agent_id], reward,
             terminated])
        elif agent_id == 1:
            experience_buffer1.append([obs_agent[agent_id],
             actionsFox[agent_id], obs_agent_next[agent_id], reward,
             terminated])
```

```
            elif agent_id == 2:
                experience_buffer2.append([obs_agent[agent_id],
                 actionsFox[agent_id], obs_agent_next[agent_id], reward,
                 terminated])

        #Если буфер воспроизведения наполнен, начинаем обучать сеть
        ########################_начало if обучения_#################
        if (global_step % steps_train == 0) and (global_step >
                                                 start_steps):
            #Инициализируем переменную для хранения
            #последовательной мини-выборки из буфера воспроизведения
            experience_buffer_sequence = []
            #Находим текущий размер буфера воспроизведения
            lengthofexpbuf = len(experience_buffer0)
            #Создаем случайное число от 1 до размера буфера
            ind_begin = np.random.randint (1, (lengthofexpbuf -
                                               BIG_batch_size))

            for agent_id in range(n_agents):
                if agent_id == 0:
                    #Выбираем из буфера воспроизведения
                    #последовательную мини-выборку
                    experience_bufferTemp =
                     sample_sequence_from_expbuf(ind_begin,
                    experience_buffer0, BIG_batch_size)

                    experience_buffer_sequence.
                     append(experience_bufferTemp)
                elif agent_id == 1:
                    experience_bufferTemp =
                     sample_sequence_from_expbuf(ind_begin,
                      experience_buffer1, BIG_batch_size)
                    experience_buffer_sequence.
                     append(experience_bufferTemp)
                elif agent_id == 2:
                    experience_bufferTemp =
                     sample_sequence_from_expbuf(ind_begin,
                      experience_buffer2, BIG_batch_size)
                    experience_buffer_sequence.
                     append(experience_bufferTemp)

            #Моделируем мини-выборку циклом
            #######_Цикл по модели мини-выборки_#########################
            for batch_id in range(BIG_batch_size):

                #Объявляем вспомогательные переменные
                #Сумма Q-значений от трех нейронных сетей
                action_probabilities_next_sum = np.zeros([n_actions])
                #Сумма выходов, в которой будут отслеживаться градиенты
                #определенного агента, чтобы выполнить обучение
                action_probabilities_sumT0 = torch.zeros(1, n_actions)
                action_probabilities_sumT1 = torch.zeros(1, n_actions)
                action_probabilities_sumT2 = torch.zeros(1, n_actions)
                #Храним общую сумму наград
                exp_rew_sum = 0.0

                #########_Цикл по агентам для суммирования Q-значений_###
                for agent_id in range(n_agents):
                    #Разбираем на части последовательную мини-выборку
                    exp_obs =
                     experience_buffer_sequence[agent_id][batch_id][0]
                    exp_act =
                     experience_buffer_sequence[agent_id][batch_id][1]
```

```
exp_next_obs =
 experience_buffer_sequence[agent_id][batch_id][2]
exp_rew =
 experience_buffer_sequence[agent_id][batch_id][3]
exp_termd =
 experience_buffer_sequence[agent_id][batch_id][4]

exp_rew_sum = exp_rew

#Конвертируем данные в тензор
exp_obs = [x for x in exp_obs]
obs_agentT = torch.FloatTensor([exp_obs]).to(device)

#Подаем мини-выборку в основную нейронную сеть
if agent_id == 0:
    #Включаем обучение
    q_network0.train()
    #Инициализируем внутреннее состояние LSTM
    #основной сети
    q_network0.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    #Инициализируем внутреннее состояние LSTM
    #целевой сети
    tgt_network0.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    #Подаем состояние в основную сеть
    action_probabilitiesT = q_network0(obs_agentT)
elif agent_id == 1:
    q_network1.train()
    q_network1.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    tgt_network1.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    action_probabilitiesT = q_network1(obs_agentT)
elif agent_id == 2:
    q_network2.train()
    q_network2.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    tgt_network2.hidden_cell = (torch.zeros(1,
     batch_size, LSTM_HIDDEN_LAYER_SIZE).to(device),
     torch.zeros(1, batch_size,
     LSTM_HIDDEN_LAYER_SIZE).to(device))
    action_probabilitiesT = q_network2(obs_agentT)

#Конвертируем данные в numpy
action_probabilitiesT =
                    action_probabilitiesT.to("cpu")
action_probabilities =
              action_probabilitiesT.data.numpy()[0]

#Находим сумму Q-значений: Q=Q1+Q2+Q3,
#но при этом создаем три копии суммы, зависящие
#только от градиентов определенного агента
```

```
if agent_id == 0:
    action_probabilities_sumT0 =
     action_probabilities_sumT0 +
     action_probabilitiesT
elif agent_id == 1:
    with torch.no_grad():
        action_probabilities_sumN0 =
         action_probabilities_sumT0.data.numpy()[0]
        action_probabilities_sumTnograd0 =
      torch.FloatTensor([action_probabilities_sumN0])
    action_probabilities_sumT1 =
     action_probabilities_sumTnograd0 +
     action_probabilitiesT

    with torch.no_grad():
        action_probabilities_sumN1 =
         action_probabilitiesT.data.numpy()[0]
        action_probabilities_sumTnograd1 =
    torch.FloatTensor([action_probabilities_sumN1])
    action_probabilities_sumT0 =
     action_probabilities_sumT0 +
     action_probabilities_sumTnograd1

elif agent_id == 2:
    with torch.no_grad():
        action_probabilities_sumN0 =
        action_probabilities_sumT0.data.numpy()[0]
        action_probabilities_sumTnograd0 =
     torch.FloatTensor([action_probabilities_sumN0])

    action_probabilities_sumT2 =
     action_probabilities_sumTnograd0+
     action_probabilitiesT

    with torch.no_grad():
        action_probabilities_sumN2 =
         action_probabilitiesT.data.numpy()[0]
        action_probabilities_sumTnograd2 =
     torch.FloatTensor([action_probabilities_sumN2])
    action_probabilities_sumT0 =
     action_probabilities_sumT0+
     action_probabilities_sumTnograd2
    action_probabilities_sumT1 =
     action_probabilities_sumT1+
     action_probabilities_sumTnograd2

#Конвертируем данные в тензор
exp_next_obs = [x for x in exp_next_obs]
obs_agentT_next =
 torch.FloatTensor([exp_next_obs]).to(device)

#Подаем мини-выборку в целевую нейронную сеть
if agent_id == 0:
    action_probabilitiesT_next =
     tgt_network0(obs_agentT_next)
elif agent_id == 1:
    action_probabilitiesT_next =
     tgt_network1(obs_agentT_next)
elif agent_id == 2:
    action_probabilitiesT_next =
     tgt_network2(obs_agentT_next)

#Конвертируем данные в numpy
```

```
                action_probabilitiesT_next =
                  action_probabilitiesT_next.to("cpu")
                action_probabilities_next =
                 action_probabilitiesT_next.data.numpy()[0]

                #Находим сумму Q-значений для выходов целевых
                #нейронных сетей
                for i in range (n_actions):
                    action_probabilities_next_sum[i] +=
                      action_probabilities_next[i]
            ## Конец цикла по агентам для суммирования Q-значений ###

            #Вычисляем целевое значение y
            y_batch = exp_rew_sum + gamma*
                      np.max(action_probabilities_next_sum,axis=-1)*
                      (1 - exp_termd)


            #Переформатируем y_batch размером batch_size
            y_batch64 = np.zeros([batch_size, qofa_out])
            for i in range (batch_size):
                for j in range (qofa_out):
                    y_batch64[i][j] = y_batch

            y_batchT = torch.FloatTensor([y_batch64])


            for agent_id in range(n_agents):

                if agent_id == 0:
                    #Обнуляем градиенты
                    optimizer0.zero_grad()
                    #Вычисляем функцию потерь
                    loss_t0 = objective0(action_probabilities_sumT0,
                                         y_batchT.squeeze(0))
                    #Выполняем обратное распространение ошибки
                    loss_t0.backward()
                    #Выполняем оптимизацию нейронных сетей
                    optimizer0.step()
                elif agent_id == 1:
                    optimizer1.zero_grad()
                    loss_t1 = objective1(action_probabilities_sumT1,
                                         y_batchT.squeeze(0))
                    loss_t1.backward()
                    optimizer1.step()
                elif agent_id == 2:
                    optimizer2.zero_grad()
                    loss_t2 = objective2(action_probabilities_sumT2,
                                         y_batchT.squeeze(0))
                    loss_t2.backward()
                    optimizer2.step()

                #Сохраняем данные для графиков
                Loss_History.append(loss_t0)
                loss_n=loss_t0.data.numpy()
                total_loss.append(loss_n)
                m_loss.append(np.mean(total_loss[-10000:]))
        ######## Конец цикла по модели мини-выборки ###################
    ######################## конец if обучения ########################

    #Синхронизируем веса основной и целевой нейронных сетей
    #через каждые 100 шагов
    if (global_step+1) % copy_steps == 0 and global_step >
```

```
                                                    start_steps:
                    tgt_network0.load_state_dict(q_network0.state_dict())
                    tgt_network1.load_state_dict(q_network1.state_dict())
                    tgt_network2.load_state_dict(q_network2.state_dict())

                ######################_Конец цикла по агентам для обучения_#######

                #Обновляем счетчик общего числа шагов
                global_step += 1

            ######################_конец цикла while_############################
            #Выводим счетчик шагов игры и общую награду за эпизод
            print('global_step=', global_step, "Total reward in episode {} =
                  {}".format(e, episode_reward))

            #Собираем данные для графиков
            Reward_History.append(episode_reward)
            status = env.get_stats()
            winrate_history.append(status["win_rate"])

        ################_конец цикла по эпизодам игры_############################

        #Закрываем среду StarCraft II
        env.close()

        #Сохраняем параметры обученных нейронных сетей
        torch.save(q_network0.state_dict(),"qnet_0.dat")
        torch.save(q_network1.state_dict(),"qnet_1.dat")
        torch.save(q_network2.state_dict(),"qnet_2.dat")

        #Выводим на печать графики
        #Средняя награда
        plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                   edgecolor='k')
        plt.plot(Reward_History)
        plt.xlabel('Номер эпизода')
        plt.ylabel('Количество награды за эпизод')
        plt.show()
        #Процент побед
        plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                   edgecolor='k')
        plt.plot(winrate_history)
        plt.xlabel('Номер эпизода')
        plt.ylabel('Процент побед')
        plt.show()
        #Значения функции потерь
        plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
                   edgecolor='k')
        plt.plot(m_loss)
        plt.xlabel('Номер каждой 1000-й итерации')
        plt.ylabel('Функция потерь')
        plt.show()

#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

3.6.3. Алгоритм MADDPG

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
from collections import deque
import matplotlib.pyplot as plt
import time
import random

import torch
import torch.nn as nn
import torch.optim as optim

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Определяем архитектуру нейронной сети исполнителя
class MADDPG_Actor(nn.Module):
    def __init__(self, obs_size, n_actions):
        super(MADDPG_Actor, self).__init__()
        #На вход нейронная сеть получает состояние среды для отдельного
        #агента
        #На выходе нейронная сеть возвращает стратегию действий
        self.MADDPG_Actor = nn.Sequential(
            #Первый линейный слой обрабатывает входные данные состояния среды
            nn.Linear(obs_size, 60),
            nn.ReLU(),
            #Второй линейный слой обрабатывает внутренние данные
            nn.Linear(60, 60),
            nn.ReLU(),
            #Третий линейный слой обрабатывает внутренние данные
            nn.Linear(60, 60),
            nn.ReLU(),
            #Четвертый линейный слой обрабатывает данные для стратегии
            #действий
            nn.Linear(60, n_actions)
            )
        #Финальный выход нерйонной сети обрабатывается функцией Tanh()
        self.tanh_layer = nn.Tanh()
    #Вначале данные x обрабатываются полносвязной сетью с функцией ReLU
    #На выходе происходит обработка функцией Tanh()
    def forward(self, x):
        #Обработка полносвязными линейными слоями
        network_out = self.MADDPG_Actor(x)
        #Обработка функцией Tanh()
        tanh_layer_out = self.tanh_layer(network_out)
        #Выход нейронной сети
        return tanh_layer_out

#Определяем архитектуру нейронной сети критика
class MADDPG_Critic(nn.Module):
    def __init__(self, full_obs_size, n_actions_agents):
        super(MADDPG_Critic, self).__init__()
        #На вход нейронная сеть получает состояние среды,
        #включающее все локальные состояния среды от отдельных агентов
        #и все выполненные действия отдельных агентов
        #На выходе нейронная сеть возвращает корректирующее значение
        self.network = nn.Sequential(
            #Первый линейный слой обрабатывает входные данные
            nn.Linear(full_obs_size+n_actions_agents, 202),
            nn.ReLU(),
            #Второй линейный слой обрабатывает внутренние данные
```

```
            nn.Linear(202, 60),
            nn.ReLU(),
            #Третий линейный слой обрабатывает внутренние данные
            nn.Linear(60, 30),
            nn.ReLU(),
            #Четвертый линейный слой обрабатывает выходные данные
            nn.Linear(30, 1)
            )
    #Данные x последовательно обрабатываются полносвязной сетью с функцией
    #ReLU
    def forward(self, state, action):
        #Объединяем данные состояний и действий для передачи в сеть
        x = torch.cat([state, action], dim=2)
        #Результаты обработки
        Q_value = self.network(x)
        #Финальный выход нейронной сети
        return Q_value

#Выбираем возможное действие с максимальной вероятностью из стратегии
#действий с учетом дополнительного случайного шума
def select_actionFox(act_prob, avail_actions_ind, n_actions, noise_rate):
    p = np.random.random(1).squeeze()
    #Добавляем случайный шум к действиям для исследования
    #разных вариантов действий
    for i in range(n_actions):
        #Создаем шум заданного уровня
        noise = noise_rate*(np.random.rand())
        #Добавляем значение шума к значению вероятности выполнения действия
        act_prob [i] =  act_prob [i] + noise

    #Выбираем действия в зависимости от вероятностей их выполнения
    for j in range(n_actions):
        #Выбираем случайный элемент из списка
        actiontemp =  random.choices(['0','1','2','3','4','5','6'],
                      weights=[act_prob[0],act_prob[1],act_prob[2],
                               act_prob[3],act_prob[4],
                               act_prob[5],act_prob[6]])
        #Преобразуем тип данных
        action = int (actiontemp[0])
        #Проверяем наличие выбранного действия в списке действий
        if action in avail_actions_ind:
            return action
        else:
            act_prob[action] = 0

#Создаем мини-выборку определенного объема из буфера воспроизведения
def sample_from_expbuf(experience_buffer, batch_size):
    #Функция возвращает случайную последовательность заданной длины
    perm_batch = np.random.permutation(len(experience_buffer))[:batch_size]
    #Мини-выборка
    experience = np.array(experience_buffer)[perm_batch]
    #Возвращаем значения мини-выборки по частям
    return experience[:,0], experience[:,1], experience[:,2],
           experience[:,3], experience[:,4], experience[:,5]

#Основная функция программы
def main():
    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="3ps1zgWallFOX", reward_only_positive=False,
                        reward_scale_rate=200, difficulty="1")
    #Получаем и выводим на печать информацию о среде
    env_info = env.get_env_info()
```

```
print ('env_info=',env_info)
#Получаем и выводим на печать размер локальных состояний среды для агента
obs_size =  env_info.get('obs_shape')
print ("obs_size=",obs_size)
#Число действий агента
n_actions = env_info["n_actions"]
#Число дружественных агентов
n_agents = env_info["n_agents"]

#Определяем основные параметры нейросетевого обучения
#################################################################################
#Некоторые переходы в алгоритме MADDPG зависят от шагов игры
global_step = 0 #подсчитываем общее число шагов в игре
start_steps = 1000 #начинаем обучать через 1000 шагов
steps_train = 4 #после начала обучения продолжаем обучать через каждые
                #четыре шага
#Размер мини-выборки
batch_size = 32
#Общее число эпизодов игры
n_episodes = 510
#Параметр дисконтирования
gamma = 0.99
#Скорость обучения исполнителя
alpha_actor = 0.01
#Скорость обучения критика
alpha_critic = 0.01
#Уровень случайного шума
noise_rate = 0.01
#Начальное значение случайного шума
noise_rate_max = 0.9
#Финальное значение случайного шума
noise_rate_min = 0.01
#Шаг затухания уровня случайного шума
noise_decay_steps = 15000
#Параметр мягкой замены
tau = 0.01
#Объем буфера воспроизведения
buffer_len = 10000
#################################################################################

#Создаем буфер воспроизведения на основе deque
experience_buffer = deque(maxlen=buffer_len)

#Pytorch определяет возможность использования графического процессора
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

#Реализуем модифицированный алгоритм MADDPG
#с одной нейронной сетью критика и тремя нейронными сетями исполнителей
#Создаем основную нейронную сеть исполнителя
actor_network = MADDPG_Actor(obs_size, n_actions).to(device)
#Создаем целевую нейронную сеть исполнителя
tgtActor_network = MADDPG_Actor(obs_size, n_actions).to(device)
#Синхронизуем веса нейронных сетей исполнителей
tgtActor_network.load_state_dict(actor_network.state_dict())

#Создаем основную нейронную сеть критика
critic_network = MADDPG_Critic(obs_size*n_agents, n_agents).to(device)
#Создаем целевую нейронную сеть критика
tgtCritic_network = MADDPG_Critic(obs_size*n_agents, n_agents).to(device)
#Синхронизуем веса нейронных сетей критиков
tgtCritic_network.load_state_dict(critic_network.state_dict())

#Создаем списки для мультиагентного случая
```

```
actor_network_list = []
tgtActor_network_list = []
optimizerActor_list = []
objectiveActor_list = []

for agent_id in range(n_agents):
    #Создаем список основных нейронных сетей исполнителей трех агентов
    actor_network_list.append(actor_network)
    #Создаем список целевых нейронных сетей исполнителей
    tgtActor_network_list.append(tgtActor_network)
    #Создаем список оптимизаторов нейронных сетей исполнителей
    optimizerActor_list.append(optim.Adam(params=
            actor_network_list[agent_id].parameters(), lr=alpha_actor))
    #Создаем список функций потерь исполнителей
    objectiveActor_list.append(nn.MSELoss())

#Создаем оптимизатор нейронной сети критика
optimizerCritic = optim.Adam(params=
                            critic_network.parameters(), lr=alpha_critic)
#Создаем функцию потерь критика
objectiveCritic = nn.MSELoss()

#Выводим на печать архитектуру нейронных сетей
print ('Actor_network_list=', actor_network_list)
print ('Critic_network_list=', critic_network)

#Определяем вспомогательные параметры
Loss_History = []
Loss_History_actor = []
Reward_History = []
winrate_history = []
total_loss = []
total_loss_actor = []
m_loss = []
m_loss_actor = []

#Основной цикл по эпизодам игры
################_цикл for по эпизодам_#####################################
for e in range(n_episodes):

    #Перезагружаем среду
    env.reset()
    #Флаг окончания эпизода
    terminated = False
    #Награда за эпизод
    episode_reward = 0
    #Обновляем и выводим динамический уровень случайного шума
    noise_rate = max(noise_rate_min, noise_rate_max - (noise_rate_max-
                    noise_rate_min) * global_step/noise_decay_steps)
    print ('noise_rate=', noise_rate)

    #Шаги игры внутри эпизода
    ######################_цикл while_#####################################
    while not terminated:
        #Обнуляем промежуточные переменные
        actions = []
        observations = []
        action = 0
        #Сохраняем историю действий на одном шаге для разных агентов
        actionsFox = np.zeros([n_agents])
        #Сохраняем историю состояний среды на одном шаге для разных агентов
        obs_agent = np.zeros([n_agents], dtype=object)
        obs_agent_next = np.zeros([n_agents], dtype=object)
```

```
########## Цикл по агентам для выполнения действий в игре ######
for agent_id in range(n_agents):
    #Получаем состояние среды для независимого агента
    obs_agent[agent_id] = env.get_obs_agent(agent_id)
    #Конвертируем данные в тензор
    obs_agentT = torch.FloatTensor([obs_agent[agent_id]])
                                                .to(device)
    #Передаем состояние среды в основную нейронную сеть
    #и получаем стратегию действий
    action_probabilitiesT =
      actor_network_list[agent_id](obs_agentT)
    #Конвертируем данные в numpy
    action_probabilitiesT = action_probabilitiesT.to("cpu")
    action_probabilities = action_probabilitiesT.data.numpy()[0]

    #Находим возможные действия агента в данный момент времени
    avail_actions = env.get_avail_agent_actions(agent_id)
    avail_actions_ind = np.nonzero(avail_actions)[0]

    #Выбираем возможное действие агента с учетом
    #стратегии действий и уровня случайного шума
    action = select_actionFox(action_probabilities,
              avail_actions_ind, n_actions, noise_rate)
    #Обрабатываем исключение при ошибке в возможных действиях
    if action is None:
        action = np.random.choice (avail_actions_ind)

    #Собираем действия от разных агентов
    actions.append(action)
    actionsFox[agent_id] = action
    #Собираем локальные состояния среды от разных агентов
    for i in range(obs_size):
        observations.append(obs_agent[agent_id][i])
###### конец цикла по агентам для выполнения действий в игре ####

#Передаем действия агентов в среду, получаем награду
#и прерывание игры от среды
reward, terminated, _ = env.step(actions)
#Суммируем награды за этот шаг для вычисления награды за эпизод
episode_reward += reward
#Подготовляем данные для сохранения в буфере воспроизведения
actions_next = []
observations_next = []
#Если эпизод не завершился, то можно найти новые действия
#и состояния
if terminated == False:
    for agent_id in range(n_agents):
        #Получаем новое состояние среды для независимого агента
        obs_agent_next[agent_id] = env.get_obs_agent(agent_id)
        #Собираем от разных агентов новые состояния
        for i in range(obs_size):
            observations_next.append(obs_agent_next[agent_id][i])
        #Конвертируем данные в тензор
        obs_agent_nextT = torch.FloatTensor
                          ([obs_agent_next[agent_id]]).to(device)
        #Получаем новые действия агентов для новых состояний
        #из целевой сети исполнителя
        action_probabilitiesT = tgtActor_network_list[agent_id]
                                                (obs_agent_nextT)
        #Конвертируем данные в numpy
        action_probabilitiesT = action_probabilitiesT.to("cpu")
        action_probabilities = action_probabilitiesT.
```

```
                                                    data.numpy()[0]
            #Находим новые возможные действия агента
            avail_actions = env.get_avail_agent_actions(agent_id)
            avail_actions_ind = np.nonzero(avail_actions)[0]
            #Выбираем новые возможные действия
            action = select_actionFox(action_probabilities,
                      avail_actions_ind, n_actions, noise_rate)
            if action is None:
                action = np.random.choice (avail_actions_ind)
            #Собираем новые действия от разных агентов
            actions_next.append(action)
    elif terminated == True:
        #Если эпизод на этом шаге завершился, то новых действий
        #не будет
        actions_next = actions
        observations_next = observations

    #Сохраняем переход в буфере воспроизведения
    experience_buffer.append([observations, actions,
      observations_next, actions_next, reward, terminated])

    #Если буфер воспроизведения наполнен, начинаем обучать сеть
    ########################_начало if обучения_######################
    if (global_step % steps_train == 0) and (global_step >
                                             start_steps):
        #Получаем мини-выборку из буфера воспроизведения
        exp_obs, exp_acts, exp_next_obs, exp_next_acts, exp_rew,
        exp_termd = sample_from_expbuf(experience_buffer, batch_size)

        #Конвертируем данные в тензор
        exp_obs = [x for x in exp_obs]
        obs_agentsT = torch.FloatTensor([exp_obs]).to(device)
        exp_acts = [x for x in exp_acts]
        act_agentsT = torch.FloatTensor([exp_acts]).to(device)

        ###############_Обучаем нейронную сеть критика_##############

        #Получаем значения из основной сети критика
        action_probabilitieQT = critic_network(obs_agentsT,
                                               act_agentsT)
        action_probabilitieQT = action_probabilitieQT.to("cpu")

        #Конвертируем данные в тензор
        exp_next_obs = [x for x in exp_next_obs]
        obs_agents_nextT = torch.FloatTensor([exp_next_obs]).
                                                 to(device)
        exp_next_acts = [x for x in exp_next_acts]
        act_agents_nextT = torch.FloatTensor([exp_next_acts])
                                                .to(device)

        #Получаем значения из целевой сети критика
        action_probabilitieQ_nextT =
         tgtCritic_network(obs_agents_nextT,act_agents_nextT)
        action_probabilitieQ_nextT =
         action_probabilitieQ_nextT.to("cpu")
        action_probabilitieQ_next =
         action_probabilitieQ_nextT.data.numpy()[0]

        #Переформатируем y_batch размером batch_size
        y_batch = np.zeros([batch_size])
        action_probabilitieQBT = torch.empty(1, batch_size,
                                             dtype=torch.float)
```

```
for i in range (batch_size):
    #Вычисляем целевое значение y
    y_batch[i] = exp_rew[i] +
     (gamma*action_probabilitieQ_next[i])*(1 - exp_termd[i])
    action_probabilitieQBT[0][i] =
     action_probabilitieQT[0][i]

y_batchT = torch.FloatTensor([y_batch])

#Обнуляем градиенты
optimizerCritic.zero_grad()

#Вычисляем функцию потерь критика
loss_t_critic = objectiveCritic(action_probabilitieQBT,
                                y_batchT)

#Сохраняем данные для графиков
Loss_History.append(loss_t_critic)
loss_n_critic = loss_t_critic.data.numpy()
total_loss.append(loss_n_critic)
m_loss.append(np.mean(total_loss[-1000:]))

#Выполняем обратное распространение ошибки для критика
loss_t_critic.backward()

#Выполняем оптимизацию нейронной сети критика
optimizerCritic.step()
###################_Закончили обучать критика_###############

##############_Обучаем нейронные сети исполнителей_##########
#Разбираем совместное состояние на локальные состояния
obs_local1 = np.zeros([batch_size, obs_size])
obs_local2 = np.zeros([batch_size, obs_size])
obs_local3 = np.zeros([batch_size, obs_size])
for i in range (batch_size):
    for j in range (obs_size):
        obs_local1[i][j] = exp_obs[i][j]
for i in range (batch_size):
    k=0
    for j in range (obs_size, obs_size*2):
        obs_local2[i][k] = exp_obs[i][j]
        k = k + 1
for i in range (batch_size):
    k=0
    for j in range (obs_size*2, obs_size*3):
        obs_local3[i][k] = exp_obs[i][j]
        k = k + 1
#Конвертируем данные в тензор
obs_agentT1 = torch.FloatTensor([obs_local1]).to(device)
obs_agentT2 = torch.FloatTensor([obs_local2]).to(device)
obs_agentT3 = torch.FloatTensor([obs_local3]).to(device)

#Обнуляем градиенты
optimizerActor_list[0].zero_grad()
optimizerActor_list[1].zero_grad()
optimizerActor_list[2].zero_grad()

#Подаем в нейронные сети исполнителей локальные состояния
action_probabilitiesT1 = actor_network_list[0](obs_agentT1)
action_probabilitiesT2 = actor_network_list[1](obs_agentT2)
action_probabilitiesT3 = actor_network_list[2](obs_agentT3)
```

```
#Конвертируем данные в numpy
action_probabilitiesT1 = action_probabilitiesT1.to("cpu")
action_probabilitiesT2 = action_probabilitiesT2.to("cpu")
action_probabilitiesT3 = action_probabilitiesT3.to("cpu")
action_probabilities1 =
 action_probabilitiesT1.data.numpy()[0]
action_probabilities2 =
 action_probabilitiesT2.data.numpy()[0]
action_probabilities3 =
 action_probabilitiesT3.data.numpy()[0]

#Вычисляем максимальные значения с учетом объема мини-выборки
act_full = np.zeros([batch_size, n_agents])
for i in range (batch_size):
    act_full[i][0] = np.argmax(action_probabilities1[i])
    act_full[i][1] = np.argmax(action_probabilities2[i])
    act_full[i][2] = np.argmax(action_probabilities3[i])
act_fullT = torch.FloatTensor([act_full]).to(device)

#Конвертируем данные в тензор
exp_obs = [x for x in exp_obs]
obs_agentsT = torch.FloatTensor([exp_obs]).to(device)

#Задаем значение функции потерь для нейронных сетей
#исполнителей как отрицательный выход критика
actor_lossT = -critic_network(obs_agentsT, act_fullT)

#Усредняем значение по числу элементов мини-выборки
actor_lossT = actor_lossT.mean()

#Выполняем обратное распространение ошибки
actor_lossT.backward()

#Выполняем оптимизацию нейронных сетей исполнителей
optimizerActor_list[0].step()
optimizerActor_list[1].step()
optimizerActor_list[2].step()

#Собираем данные для графиков
actor_lossT = actor_lossT.to("cpu")
Loss_History_actor.append(actor_lossT)
actor_lossN = actor_lossT.data.numpy()
total_loss_actor.append(actor_lossN)
m_loss_actor.append(np.mean(total_loss_actor[-1000:]))
############### Закончили обучать исполнителей ###############

#Реализуем механизм мягкой замены
#Обновляем целевую сеть критика
for target_param, param in
 zip(tgtCritic_network.parameters(),
     critic_network.parameters()):
    target_param.data.copy_((1 - tau) * param.data +
                            tau * target_param.data)
#Обновляем целевые сети акторов
for agent_id in range(n_agents):
    for target_param, param in
 zip(tgtActor_network_list[agent_id].parameters(),
     actor_network_list[agent_id].parameters()):
        target_param.data.copy_((1 - tau) * param.data +
                            tau * target_param.data)

#####################_конец if обучения_###################
```

```
            #Обновляем счетчик общего числа шагов
            global_step += 1
        ######################_конец цикла while_##############################

        #Выводим на печать счетчик шагов игры и общую награду за эпизод
        print('global_step=', global_step,
              "Total reward in episode {} = {}".format(e, episode_reward))
        #Собираем данные для графиков
        Reward_History.append(episode_reward)
        status = env.get_stats()
        winrate_history.append(status["win_rate"])

    ################_конец цикла по эпизодам игры_##############################
    #Закрываем среду StarCraft II
    env.close()

    #Сохраняем параметры обученных нейронных сетей
    for agent_id in range(n_agents):
        torch.save(actor_network_list[agent_id].state_dict(),
                   "actornet_%.0f.dat"%agent_id)

    #Выводим на печать графики
    #Средняя награда
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(Reward_History)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Количество награды за эпизод')
    plt.show()
    #Процент побед
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(winrate_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Процент побед')
    plt.show()
    #Значения функции потерь исполнителя
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(m_loss_actor)
    plt.xlabel('Номер каждой 1000-й итерации')
    plt.ylabel('Функция потерь исполнителя')
    plt.show()
    #Значения функции потерь критика
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(m_loss)
    plt.xlabel('Номер каждой 1000-й итерации')
    plt.ylabel('Функция потерь критика')
    plt.show()

#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

3.7. Эксперимент

3.7.1. Алгоритм IQN

На рис. 3.19 и 3.20 представлены результаты обучения трех независимых агентов с помощью алгоритма IQL при заданных параметрах обучения $\gamma = 0.99$, $\alpha = 0.01$ на 300 эпизодах игры. Все агенты успешно выучивают оптимальную стратегию «бежать вправо и стрелять при появлении противника», демонстрируя правильное кооперативное поведение. Процент побед в процессе обучения практически постоянно монотонно увеличивается, значение функции потерь после небольшого увеличения идет на спад, что свидетельствует об успешной динамике обучения. Непосредственно визуализация процесса игры также демонстрирует постепенное приближение

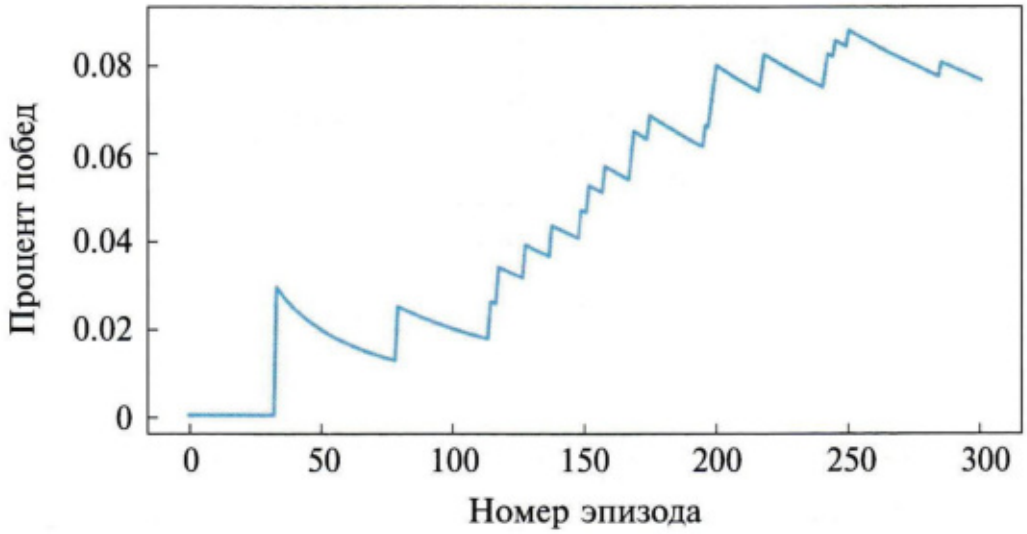


Рис. 3.19. Процент побед при обучении трех независимых агентов с помощью алгоритма IQL в среде SMAC

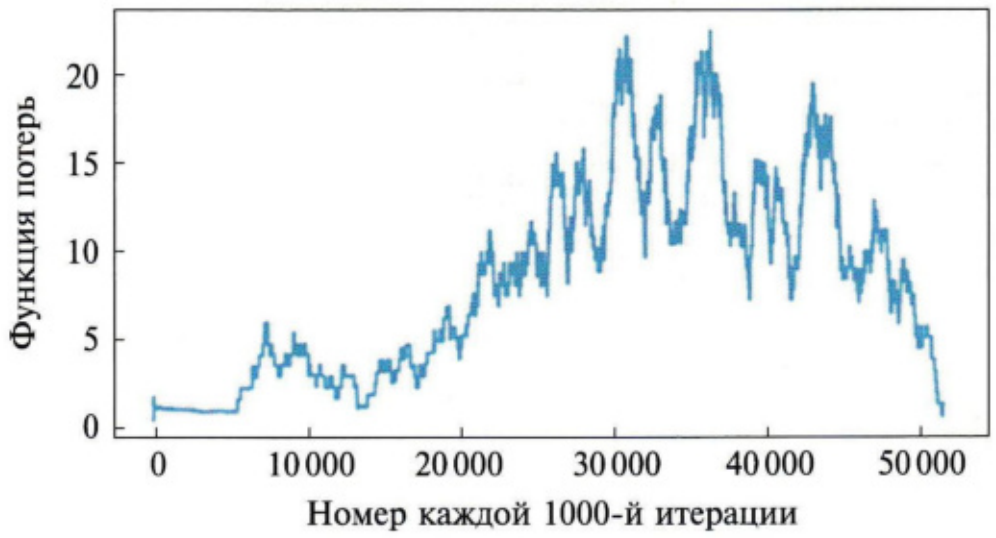


Рис. 3.20. Функция потерь при обучении трех независимых агентов с помощью алгоритма IQL в среде SMAC

агентов к оптимальным действиям и в итоге их успешное использование (см. рис. 3.17).

В процессе обучения алгоритмом IQL использовались нестандартные награды среды SMAC. Диапазон наград был расширен до отрицательных значений −200 и до положительных значений +200, т. е. была предпринята успешная попытка явно выделить и оценить редкие действия агентов, соответствующие оптимальной стратегии.

В нейросетевом обучении рекомендуется итеративно уменьшать параметр исследования пространства действий ε, проводя процедуру «отжига», или «затухания», параметра ε. Динамическое значение ε_t в алгоритме IQL вычислялось по формуле $\varepsilon_t = \max\left(\varepsilon_{\min}, \varepsilon_{\max} - \left(\varepsilon_{\max} - \varepsilon_{\min}\right) * c_t / \varepsilon_{\text{decay}}\right)$, где c_t — счетчик количества шагов игры; $\varepsilon_{\min} = 0.1$; $\varepsilon_{\max} = 1$; $\varepsilon_{\text{decay}} = 15\,000$.

На рис. 3.21 представлены результаты тестирования алгоритма IQL. Тестирование проводилась при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 100 %, среднее количество награды — 20.210.

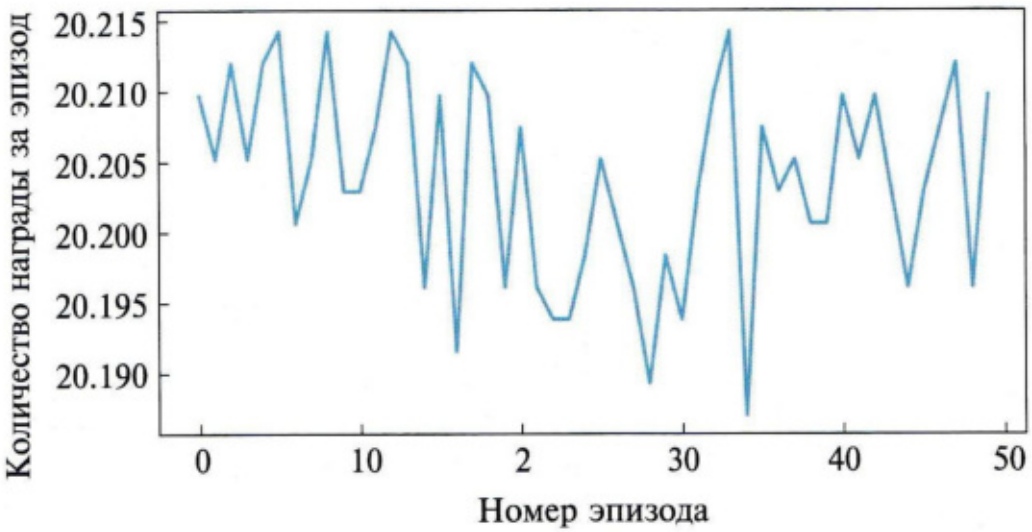


Рис. 3.21. Результаты тестирования алгоритма IQL в среде SMAC

В качестве одного из недостатков алгоритма IQL выделяют завышенные оценки Q-значений. Завышенная оценка не указывает на то, что стратегия, основанная на этом значении, будет неверной. Базирующиеся на DQN новые алгоритмы, например алгоритм Double DQN, благодаря двойному вычислению позволяют решить проблему завышенных Q-значений. В целом использование новых алгоритмов, разрабатываемых в глубоком обучении, для независимого нейросетевого обучения представляется главным направлением развития алгоритма IQL. В некоторых научных работах выдвигается гипотеза, что нет необходимости в специальных алгоритмах мультиагентного обучения, достаточно разработать мощный одноагентный алгоритм DQN, который по умолчанию решит вопрос машинного обучения как одного, так и нескольких агентов.

3.7.2. Алгоритм CDQN

На рис. 3.22 и 3.23 представлены результаты централизованного обучения трех агентов с помощью алгоритма CDQN при заданных параметрах обучения $\gamma = 0.99$, $\alpha = 0.01$ на 210 эпизодах игры. Из графиков видно, что функция потерь монотонно убывает, что хорошо для обучения, а процент побед постепенно уменьшается, что крайне нежелательно.

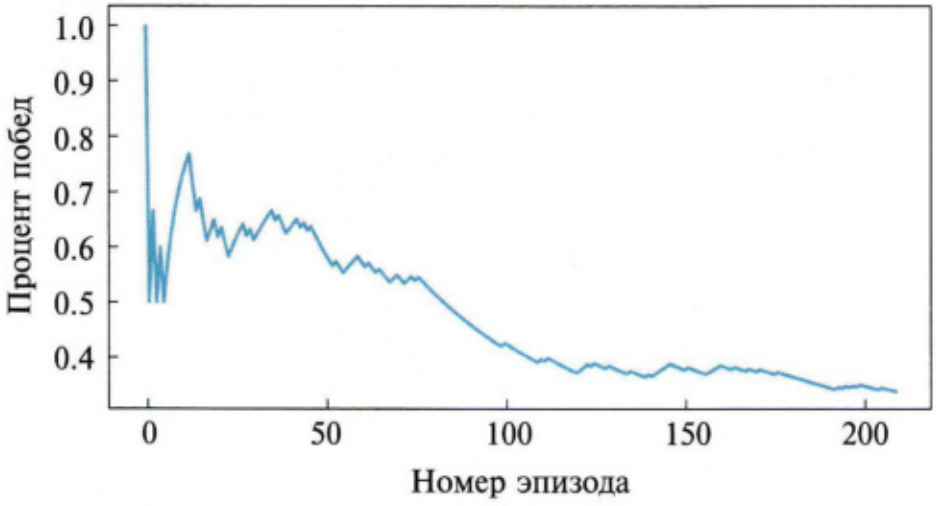


Рис. 3.22. Процент побед при централизованном обучении трех агентов с помощью алгоритма CDQN в среде SMAC

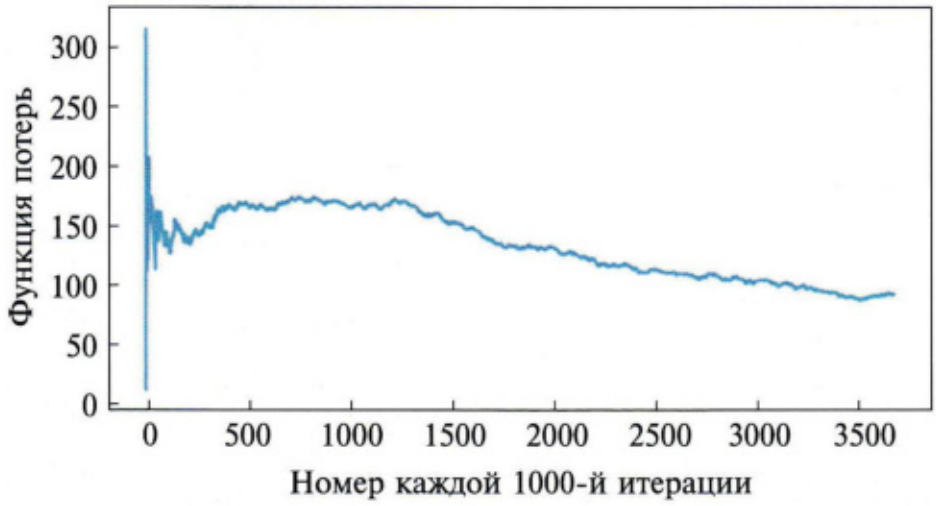


Рис. 3.23. Функция потерь при централизованном обучении трех агентов с помощью алгоритма CDQN в среде SMAC

На рис. 3.24 представлены результаты централизованного тестирования алгоритма CDQN. Тестирование проводилась при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 35 %, среднее количество награды — 8.

Невысокий процент побед агентов, обученных с помощью алгоритма CDQN, в данном случае связан с возникновением проблемы «ленивого агента». Именно при централизованном обучении может возникнуть ситуации, когда агенты выучивают неоптимальные стратегии действий. Бывает, что один агент, который в начале игры приносит награду всему коллективу, продолжает активно развиваться и действовать, но другой агент, действия которого в начале игры были неэффективными или даже убыточными с точки зрения количества заработанной награды, перестает исследовать новые действия и практически ничему не обучается.

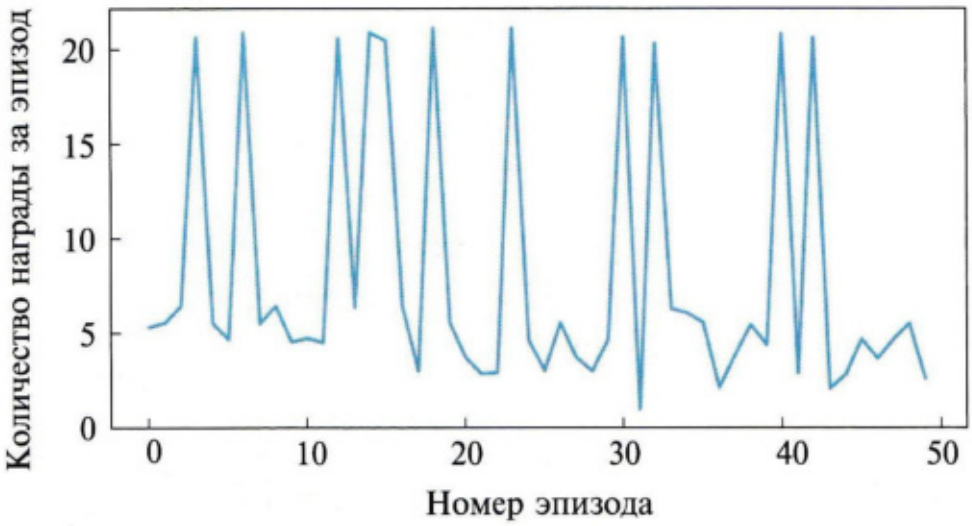


Рис. 3.24. Результаты централизованного тестирования алгоритма CDQN в среде SMAC

В среде SMAC эту проблему можно назвать проблемой «трусливого агента». Вместо того чтобы начать двигаться в сторону противника и, возможно, погибнуть, но при этом выучить действие «стрелять», агенты разбегаются по карте, стараясь держаться как можно дальше от противника и при этом тактически не теряют в награде по причине гибели, но стратегически проигрывают игру из-за неспособности выучить атакующие действия (см. рис. 3.15).

3.7.3. Алгоритм VDN

На рис. 3.25 и 3.26 представлены результаты централизованного обучения трех агентов алгоритмом VDN при заданных параметрах обучения $\gamma = 0.99$, $\alpha = 0.01$ на 350 эпизодах игры. Из графиков видно, что функция потерь, осциллируя, убывает, но процент побед вместо увеличения выходит на устойчивое плато на уровне 50 %.

Изначально преимущества алгоритма VDN перед алгоритмами IQL и CDQN были продемонстрированы компанией Google DeepMind

в маленьком мире-сетке 5×5 для двух агентов, имеющих по восемь действий каждый и кооперативно решающих задачу координации. На картах гл. 1 и 2 (см. рис. 1.4 и 2.5) алгоритм VDN также достиг максимальных

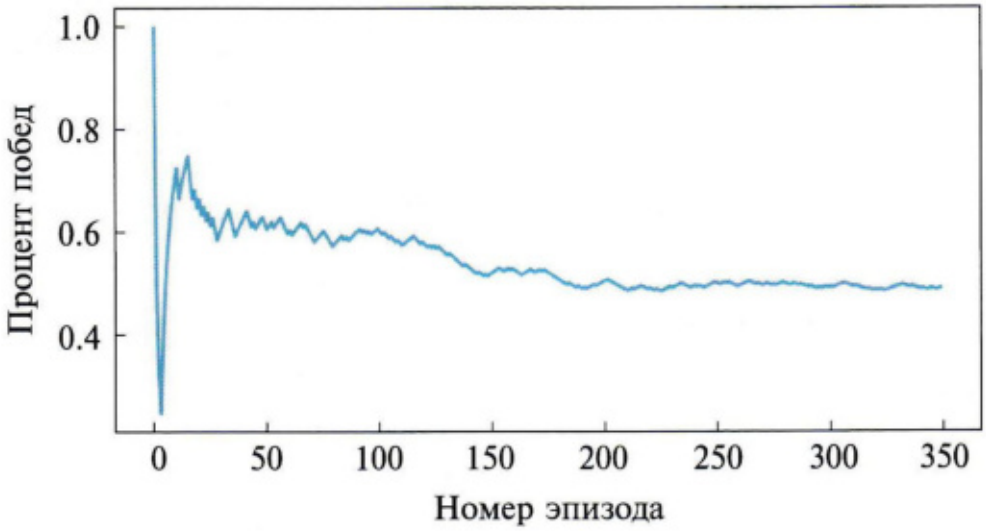


Рис. 3.25. Процент побед при централизованном обучении трех агентов с помощью алгоритма VDN в среде SMAC

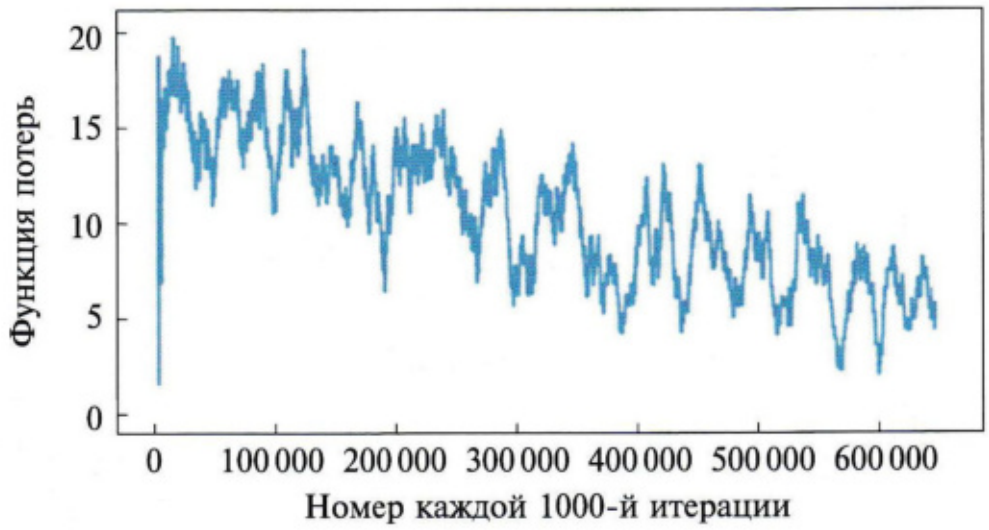


Рис. 3.26. Функция потерь при централизованном обучении трех агентов с помощью алгоритма VDN в среде SMAC

результатов по проценту побед и количеству заработанной награды. Однако для рассматриваемого в главе случая трех агентов при стандартной награде в среде SMAC процент побед на 50 тестовых эпизодах игры равен всего 55 %, а среднее количество наград составило 11.2 (рис. 3.27).

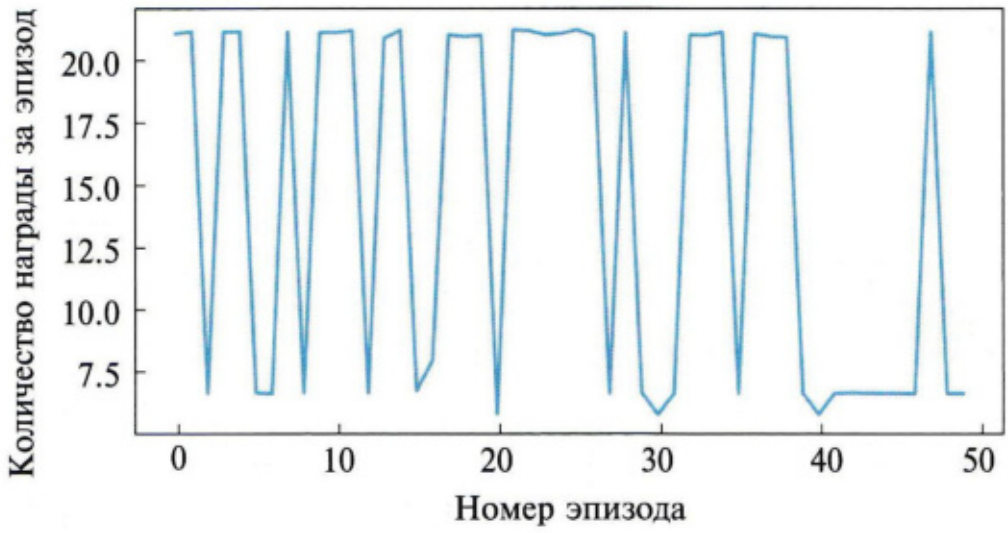


Рис. 3.27. Результаты децентрализованного тестирования алгоритма VDN в среде SMAC

В заключение рассмотрим результаты лучшего на сегодняшний день алгоритма мультиагентного нейросетевого обучения.

3.7.4. Алгоритм MADDPG

На рис. 3.28–3.30 представлены результаты обучения трех агентов с помощью модифицированного алгоритма MADDPG с одним критиком и тремя исполнителями при заданных параметрах обучения $\gamma = 0.99$, $\alpha_{ac} = 0.01$,

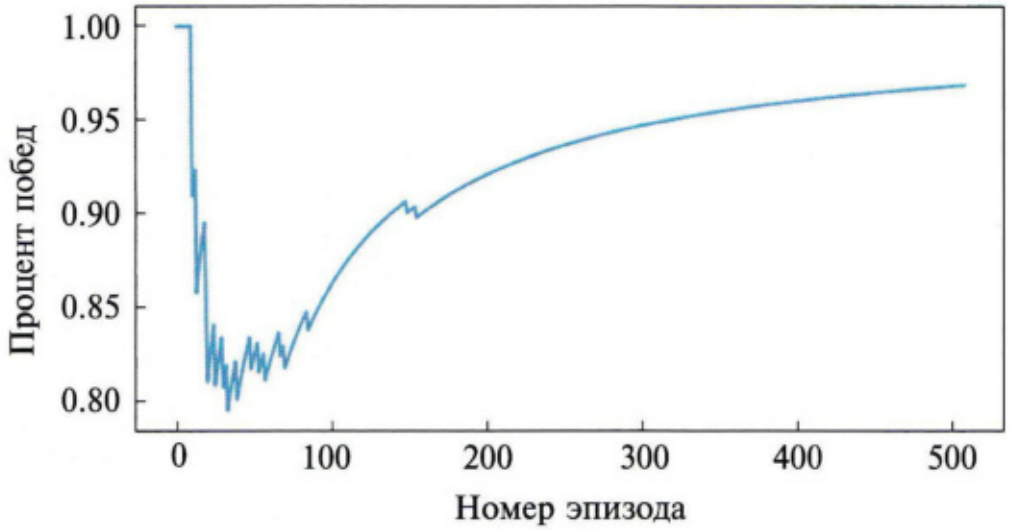


Рис. 3.28. Процент побед при обучении трех агентов с помощью алгоритма MADDPG в среде SMAC

$\alpha_{cr} = 0.01$, $\mathcal{N} = 0.01$, $\tau = 0.01$ на 510 эпизодах игры. Из графиков видно, что функции потерь исполнителя и критика монотонно убывают, процент побед после небольшого выброса обретает стабильность и монотонно увеличивается.

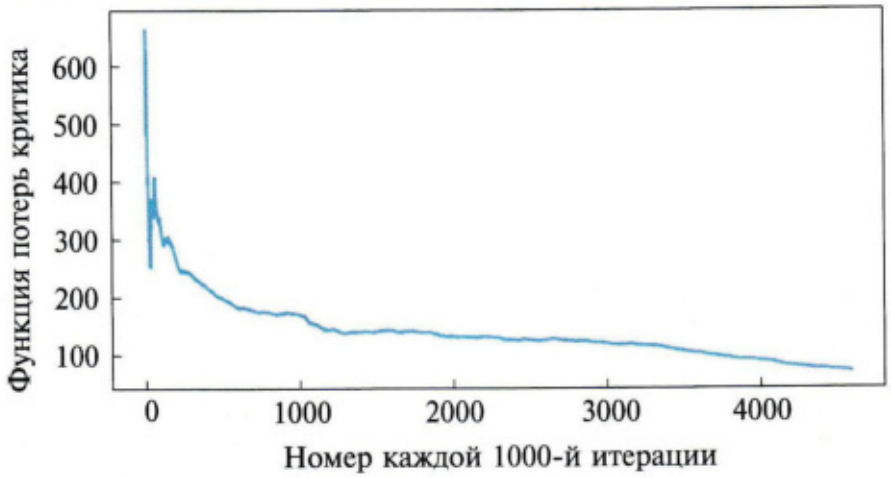


Рис. 3.29. Функция потерь критика при обучении трех агентов с помощью с помощью алгоритма MADDPG в среде SMAC

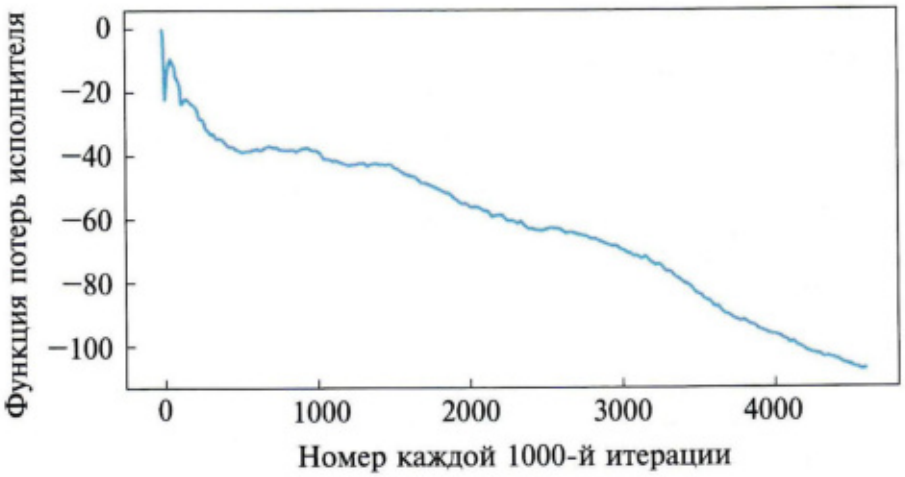


Рис. 3.30. Функция потерь исполнителя при обучении трех агентов с помощью алгоритма MADDPG в среде SMAC

На рис. 3.31 представлены результаты тестирования алгоритма MADDPG. Тестирование проводилась при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 100 %, среднее количество награды — 20.25.

Из всех алгоритмов, рассмотренных в гл. 3, лучшими по проценту побед и количеству полученной награды стали алгоритмы IQL и MADDPG. В среднем алгоритм MADDPG позволяет получить несколько большее количество награды, чем алгоритм IQL, если увеличить число тестовых примеров и обучающих сессий. Кроме того, только алгоритм MADDPG сохраняет процент побед при удвоении числа противников, обладающих атакой дальнего боя.

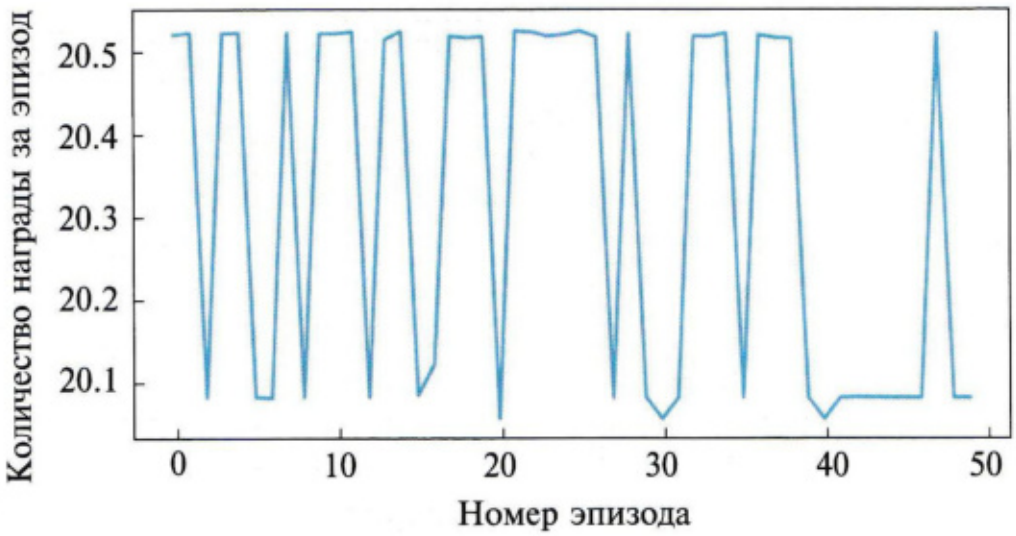


Рис. 3.31. Результаты тестирования алгоритма MADDPG в среде SMAC

Все рассмотренные алгоритмы позволяют проводить мультиагентное обучение с разными нейросетевыми архитектурами.

3.8. Выводы

В гл. 3 были проанализированы алгоритмы мультиагентного нейросетевого обучения IQL, CDQN, VDN и MADDPG, демонстрирующие основные тренды современного глубокого мультиагентого обучения: централизованное, независимое и централизованное обучение с децентрализованным исполнением. Эти алгоритмы базируются на четырех разных архитектурах нейронных сетей: полносвязной, сверточной, рекуррентной, исполнитель-критик.

С теоретической точки зрения алгоритмы мультиагентного нейросетевого обучения расширяют глубокое обучение на множество одновременно обучающихся агентов. При этом обоснованные гарантии сходимости алгоритмов практически отсутствуют, на передний план выходит стабильность обучения. Дисперсия глубокого обучения от успешной игры до полного поражения в случае мультиагентного обучения экспоненциально возрастает при увеличении числа агентов. Инновации алгоритма DQN и всех его мультиагентных последователей, заключающиеся в применении буфера воспроизведения и целевой нейронной сети, были направлены именно на улучшение стабильности обучения.

Несмотря на объективные успехи и превосходство над табличными алгоритмами по качеству достигнутых результатов и по снижению нагрузки на программиста благодаря автоматическому формированию описаний характерных признаков, нейросетевые методы существенно зависят от технологии. И не просто от используемой библиотеки глубокого обучения, но и от языка и стиля программирования, выбора структур данных, решений, которые постоянно принимает программист при реализации того или иного алгоритма. Все это чрезвычайно сильно влияет на успешность реализации

и качество работы даже одного и того же алгоритма. Эту проблему пытаются решить, используя эталонные реализации базовых алгоритмов нейросетевого обучения, однако остается множество нерешенных технических вопросов.

Де-факто принято, что популярные архитектуры нейросетевого обучения одинаково хорошо работают в разных средах. И это действительно так, но гиперпараметры необходимо подбирать заново. Архитектура нейронной сети, лидирующая в одной среде, не обязательно останется лидером в другой среде, даже без учета непрерывности и недетерменированности сред. И вообще, принадлежат ли нейросетевой алгоритм IQL с 10 млн весов и алгоритм IQL с 10 тыс. весов к одному классу MDRL-алгоритмов? Или, может быть, нейросетевой алгоритм MADDPG с 10 тыс. весов, несмотря на принципиально иную схему обучения, становится более похож на алгоритм IQL соответсвующего нейросетевого размера?

В то же время в новейших алгоритмах нейросетевого обучения начинают эксплуатироваться особенности технической реализации того или иного метода. Так, в алгоритме MADDPG отрицательные градиенты одной нейронной сети используются для обучения других нейронных сетей. Возможно, что именно эти нестандартные технические решения позволят еще больше расширить и углубить нейронные сети, сделать их более интеллектуальными.

В следующей главе будут рассмотрены возможности совершенствования мультиагентного обучения на несколько иных принципах, имеющих большой потенциал в построении сильного искусственного интеллекта: на алгоритмах эволюционной оптимизации.

3.9. Задачи для самоконтроля

1. Используя концепцию толерантности, решите проблему затенения действий (a, a) для двух агентов, обучающихся с помощью алгоритма IQL, в кооперативной, повторяющийся матричной игре «Затенение действий»:

		Агент 2		
		a	**b**	**c**
Агент 1	**a**	10	0	0
	b	0	5	6
	c	0	6	7

2. Добавьте в алгоритм IQL мультиагентный алгоритмический прием «гистерезис системы» в виде двух параметров скорости обучения α_1, α_2. Проведите эксперимент и выберите наилучшие значения α_1, α_2 при условии $\alpha_1 >$

α_2 и вычислении функции ценности действий $Q_i(s, a)$ и ошибки временны́х различий δ для агента i по формулам

$$Q_i(s,a)=\begin{cases}(1-\alpha_1)Q_i(s,a)+\alpha_1\delta, & \text{если } \delta \geq Q_i(s,a);\\ (1-\alpha_2)Q_i(s,a)+\alpha_2\delta, & \text{если } \delta < Q_i(s,a),\end{cases}$$

$$\delta = r+\gamma\max_{a'}Q_i(s',a').$$

3. Обучите агентов, используя алгоритм CDQN с разными оптимизаторами — optim.RMSprop, optim.LBFGS, optim.ASGD и функциями потерь nn.CrossEntropyLoss, nn.KLDivLoss, nn.SmoothL1Loss. Какая комбинация оптимизаторов и функций потерь эффективнее? Отличается ли какой-либо агент более агрессивным поведением? Проявляется ли проблема «ленивого агента»?

4. В архитектуре нейронной сети алгоритма VDN замените третий рекуррентный слой LSTM на рекуррентный слой GRU. Улучшатся или ухудшатся результаты обучения? Изменится ли поведение какого-либо агента?

5. Постарайтесь улучшить результаты работы алгоритмов VDN и CDQN путем стабилизации мультиагентного обучения на основе принципа взвешенной функции потерь:

1. Инициализировать

   $i \leftarrow 1, ..., n;\ a \in A_i;\ s \in S;\ \pi(s,a) \leftarrow \dfrac{1}{|A_i|};$ ...

2. Цикл

... ...

... Вычислить $\pi(s, a)$.

... ...

... Сохранить кортеж $(s, a, \pi(s,a), r, s')$ в буфере воспроизведения.

... ...

... Вычислить функцию потерь $L^j(\theta) \leftarrow \dfrac{1}{K}\sum_{p=1}^{K}\dfrac{\pi_p^{old}(s,a)}{\pi_p^{new}(s,a)}\left(y_p^j - Q_p^j(s,a;\theta)\right)^2.$

... ...

Принцип взвешенной функции потерь заключается в добавлении в формулу для вычисления функции потерь определенного коэффициента, зависящего от стратегии действий из мини-выборки $\pi^{old}(s, a)$ и текущей стратегии действий $\pi^{new}(s, a)$. Чем сильнее различаются стратегии, тем больше коэффициент влияет на функцию потерь, уменьшая или увеличивая ее значение.

6. Найдите в Интернете описание процесса Орнштейна — Уленбека. Запрограммируйте процесс и добавьте в алгоритм MADDPG генерацию шума для исследования действий. Сравните получившиеся результаты с результатами при использовании случайного шума.

7. Главным конкурентом алгоритма MADDPG является алгоритм COMA — реализуйте его и сравните результаты обучения на карте 3ps1zgWallFOX.SC2Map. Схема мультиагентного обучения с использованием алгоритма COMA приведена на рис. 3.32. Ключевым отличием алгоритма COMA является использование одной нейронной сети критика и рекуррентных слоев GRU в архитектурах нейронных сетей исполнителей.

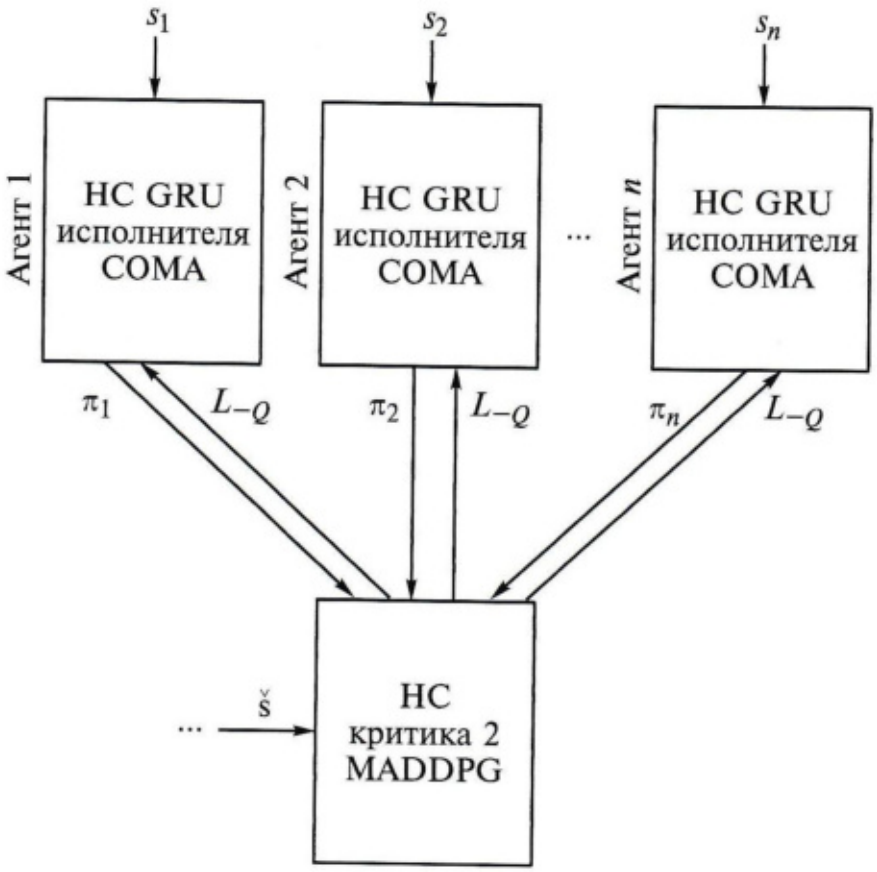


Рис. 3.32. Схема мультиагентного обучения с использованием алгоритма COMA

8. Разработайте сценарий, подготовьте карту и обучите агентов с помощью алгоритма MADDPG стратегиям микроменеджмента компьютерной игры StarCraft II:

– фокусирование — агенты одновременно атакуют одного противника;

– группирование — агенты образуют группы по типу защитных и атакующих действий;

– дистанцирование — агенты постоянно перемещаются и сохраняют такое расстояние до противника, на котором он не может атаковать;

– координация — агенты одновременно атакуют противника с разных сторон;

– ориентирование — агенты используют характер рельефа и защитные свойства местности для получения тактического преимущества.

9. Создайте оригинального юнита в компьютерной игре StarCraft II и раскройте его потенциал для кооперативного нейросетевого мультиагентного обучения с использованием приведенного ниже алгоритма создания юнита:

1. В редакторе StarCraft II Editor на панели задач справа откройте редактор Data.
2. В открывшимся редакторе слева вверху откройте вкладку Units.
3. В окне слева нажмите правую кнопку мыши и выберете вкладку Add Unit...
4. В поле Name: введите название нового юнита.
5. В поле Parent: оставьте значение по умолчанию Default Settings (Unit).
6. В поле Field Values: поставьте переключатель в позицию Copy from: и нажмите кнопку Choose...
7. В открывшимся окне во вкладке (All-Race) выберите расу юнита и в открывшимся внизу дереве в разделе Unit выберите тип юнита, на базе которого создается новый юнит. Нажмите кнопку OK.
8. Закройте меню Unit properties нажатием кнопки OK.
9. Слева вверху нажмите вкладку «+». В появившемся меню выберите вкладку Edit Actor data и затем — вкладку Actors.
10. В окне слева нажмите правую кнопку мыши и выберите вкладку Add Actor...
11. В поле Name: введите уже выбранное название нового юнита.
12. В поле Actor Type: поменяйте тип на Unit.
13. В поле Field Values: поставьте переключатель в позицию Copy from: и нажмите кнопку Choose...
14. В открывшимся окне выберите тип юнита, на базе которого создается новый юнит. Нажмите кнопку OK.
15. В списке акторов выберите нового актора.
16. В правом нижнем окне в поле Unit Name поменяйте название на название нового юнита.
17. Слева вверху откройте вкладку Units и из списка выберите нового юнита.
18. Справа в окне в поле Supplies поставьте значение 0;
    в поле Default Acquire Level поставьте значение Passive;
    в поле Response поставьте значение No Response.

10. Создайте, протестируйте и выберите лучший новый алгоритм мультиагентного нейросетевого обучения методом заполнения морфологического куба (рис. 3.33).

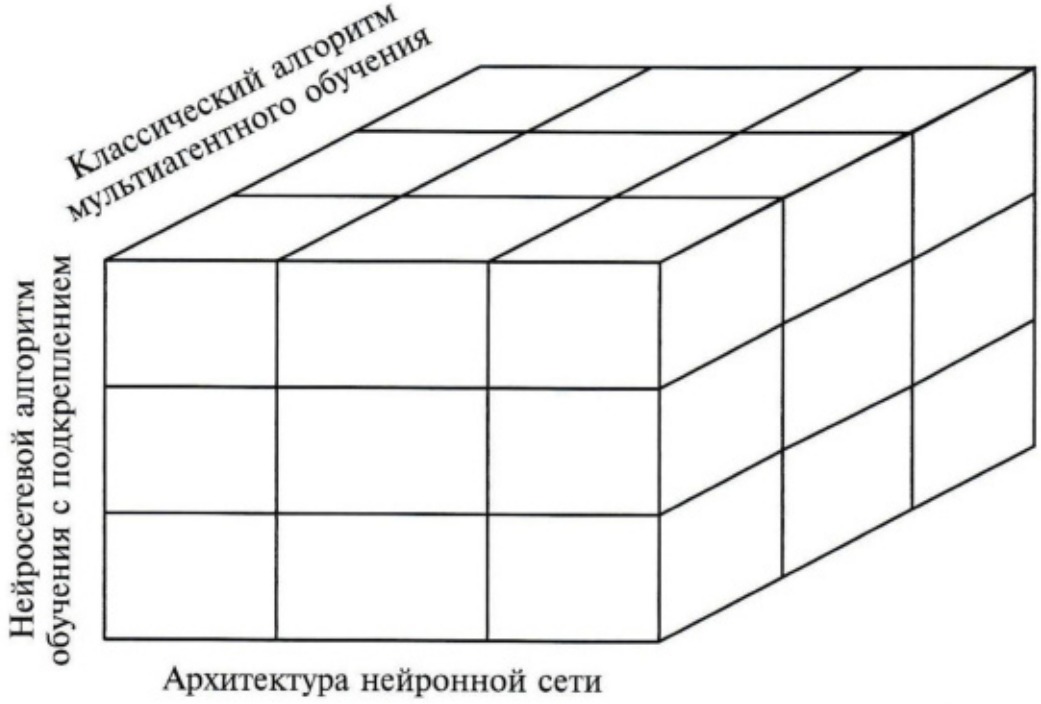


Рис. 3.33. Морфологический куб для создания нового алгоритма мультиагентного нейросетевого обучения

Для решения задачи воспользуйтесь поисковой системой «Академия Google»: https://scholar.google.ru/

Глава 4

Эволюционное обучение

Жизнь есть жизнь. Простое уравнение,
но в тот момент она еще не была математикой,
а командование так и не поняло условных обозначений.

Юн Ха Ли. Гамбит девятихвостого лиса

4.1. Классификация

Эволюция (*лат.* evolutio) в биологии — необратимое развитие организма на основе постепенного изменения живой материи в последовательности поколений организмов. В информационных технологиях эволюционное моделирование (*англ.* evolutionary computation) представляет собой создание моделей с использованием алгоритмов глобальной оптимизации, основанных на принципах биологической эволюции. В эволюционных алгоритмах (*англ.* evolutionary algorithms) в полной мере используются процедуры оценки и отбора наиболее приспособленных особей.

Эволюционные алгоритмы позволяют находить высокооптимизированные решения для широкого круга задач, в том числе могут быть использованы для машинного обучения агентов. Для этого вначале случайным образом генерируется популяция агентов. Каждый агент популяции выполняет действия в среде и затем оценивается по некоторому критерию с присвоением фитнесса — значения функции приспособленности. После этого агенты упорядочиваются по значению фитнесса, подвергаются мутации и скрещиванию. Характеристики лучших агентов наследуются более поздними популяциями до тех пор, пока не будет решена задача обучения или не завершится перебор поколений.

В табл. 4.1 представлены основные алгоритмы мультиагентного эволюционного обучения, классифицированные по типу используемого эволюционного алгоритма: генетические алгоритмы, эволюционные стратегии, эволюционная теория игр, коэволюционные алгоритмы, а также по принципу обучения с подкреплением: вычисление ценности состояний-действий, вычисление стратегий. В гл. 4 подробно рассматриваются принципы мультиагентного эволюционного обучения на основе независимого генетического алгоритма (InGA) и алгоритма коэволюционного обучения (CoE).

Таблица 4.1

Классификация алгоритмов мультиагентного эволюционного обучения

Принцип обучения с подкреплением	Тип эволюционного алгоритма			
	Генетические алгоритмы	Эволюционные стратегии	Эволюционная теория игр	Коэволюционные алгоритмы
Вычисление ценности состояний-действий	InGA iEMAS NEAT+Q MAGA-Net MAGA	–	–	CoE CE-MAGA ParetoCE CoSyNE
Вычисление стратегий	–	MERL PBT-MARL ES	RD	–

К настоящему времени разработано огромное число различных алгоритмов эволюционного обучения, но большинство из них основывается на каноническом генетическом алгоритме. Мультиагентный генетический алгоритм MAGA (*англ.* Multi-Agent Genetic Algorithm) за 300 поколений позволял обучать популяцию агентов кооперироваться в мире-сетке, имеющем 10 тыс. измерений. При этом агенты обладали информацией только о локальных объектах, поэтому обмен знаниями между агентами происходил с помощью операции скрещивания. На основе алгоритма MAGA был разработан алгоритм MAGA-Net, используемый для стабильного распознавания структур сообществ, которые скрыты в сложных коммуникационных сетях, имеющих более 5000 узлов.

Одним из фундаментальных недостатков генетических алгоритмов является отсутствие разнообразия в популяции агентов при длительном обучении. Довольно быстро выделяются характеристики агента, которые представляют собой лишь локальный максимум, но все другие агенты популяции уступают этому агенту в процессе отбора, и через некоторое время вся популяция состоит только из копий одного агента. Существуют различные способы борьбы с этим недостатком, например, выбор для размножения не самых приспособленных агентов, а случайных агентов из популяции или использование коэволюционных алгоритмов.

Коэволюционной модификацией алгоритма MAGA является алгоритм CE-MAGA (*англ.* Coevolutionary Multi-Agent Genetic Algorithm). Он был разработан в целях трехмерного планирования маршрутов с несколькими ограничениями для беспилотных летательных аппаратов. В свое время алгоритм CE-MAGA показал лучшую производительность по исследованию местности площадью 50 км^2, лучшие показатели по уклонению от возможных угроз и скорости построения маршрутов полета в течение 2 с за 400 поколений агентов. В эволюции использовалась не отдельная популяция агентов, а несколько малоразмерных субпопуляций. Механизм

коэволюции основывался на процедуре скрещивания различных характеристик агентов.

В алгоритме ParetoCE (*англ.* Pareto Coevolution) использовалась совместная эволюция нескольких агентов для нахождения оптимального по Парето решения задачи мультиагентного обучения, при котором стратегия одного агента не может быть улучшена относительно стратегии второго агента без ее ухудшения относительно стратегий всех других агентов. В алгоритме ParetoCE применялись также разные роли агентов, как создателей, так и потребителей градиентов, в процессе обучения. Областью применения алгоритма являлась генерация клеточных автоматов, которые удавалась создавать за 1300 поколений при коэволюции двух популяций агентов, имеющих по 150 правил создания клеточных автоматов.

В коэволюционном алгоритме CoSyNE (*англ.* Cooperative Synapse Neuroevolution) под коэволюцией понималось разделение моновзвешенных ребер нейронных сетей на субпопуляции, которые в процессе обучения и отбора создавали новую нейронную сеть. Алгоритм CoSyNE продемонстрировал лучшую сходимость и более высокую скорость работы по сравнению с алгоритмами, в которых используются принцип временны́х различий и вычисление стратегий балансировки столба в классической задаче одноагентного обучения с подкреплением.

В алгоритме NEAT+Q (*англ.* Neuroevolution of Augmenting Topologies+ Q-learning) в отличие от алгоритма CoSyNE проводилась эволюция архитектур нейронных сетей целиком, без разделения их на структурные единицы. В данном алгоритме для эволюции нейронных сетей Q-обучение использовалось вместе с алгоритмом обратного распространения ошибки. В задаче планирования загрузки сервера, взятой из области автономных вычислений, алгоритм NEAT+Q продемонстрировал уникальные возможности по масштабированию решаемой задачи, функционирование с непрерывным пространством действий, превосходство над алгоритмами без машинного обучения и на 75 % более высокую точность по сравнению с точностью базовых алгоритмов случайного перебора.

В эволюции архитектур нейронных сетей применяется также алгоритм iEMAS (*англ.* Immunological Evolutionary Multi-Agent System). Он позволяет моделировать поведение иммунной системы живого организма введением в эволюционное обучение дополнительной процедуры — удаления неэффективных агентов с помощью агентов-лимфоцитов. В решении классических оптимизационных задач алгоритм iEMAS позволил достичь равноценных результатов обычных эволюционных алгоритмов при существенно меньших вычислительных ресурсах и получить значение фитнесса для агентов в популяции на 50 % точнее.

Следующая большая группа алгоритмов мультиагентного эволюционного обучения — алгоритмы, в которых используются не только принципы биологической эволюции, но и явное вычисление градиентов для нахождения стратегий агентов. Алгоритм ES (*англ.* Evolution Strategies) в компьютерных играх Atari продемонстрировал точность, сравнимую с точностью

нейросетевых алгоритмов при длительности обучения 1 ч. Кроме того, алгоритм ES инвариантен к частотности выполняемых агентами действий, обладает робастными гиперпараметрами, способен обучаться при отложенной награде и легко распараллеливается.

Главной альтернативой лидирующему нейросетевому алгоритму MADDPG является алгоритм MERL (*англ.* Multiagent Evolutionary Reinforcement Learning), разработанный лабораторией Intel. Гибридный алгоритм MERL включает как градиентные, так и безградиентные подходы к оптимизации. Безградиентная часть алгоритма MERL основывается на биологической эволюции и максимизирует командную награду с помощью нейроэволюции. Градиентная часть алгоритма MERL строится на двойном глубоком детерминированном градиенте стратегии с задержкой (TD3), который вычисляется независимо для каждого агента, но периодически копируется в безградиентную часть этого алгоритма. В качестве главных преимуществ алгоритма MERL выделяют независимость от среды, одновременное вычисление мультиагентной и одноагентной стратегии, высокую скорость обучения. Так, в среде «хищники-жертва» алгоритм MERL достигает цели обучения на 30 % быстрее нейросетевых аналогов.

Изначально разработанный для одногоагентного обучения и расширенный для мультиагентного обучения алгоритм PBT-MARL (*англ.* Population-based Training for Multi-Agent RL) в компьютерной игре «футбол 2×2» продемонстрировал, что специальная эволюционная оптимизация дополнительных наград популяции из 32 агентов напрямую не мотивирует агентов к совместному поведению. В долгосрочной перспективе это приводит к кооперативному поведению множества одновременно обучающихся агентов. В алгоритме PBT-MARL используются также элементы теории игр в виде матричной игры, в которой находилось равновесие Нэша для агрегирования результатов 10 обучающихся агентов популяции.

Отдельным направлением в мультиагентном эволюционном обучении с использованием элементов теории игр является эволюционная теория игр, наиболее яркий ее представитель — алгоритм RD (*англ.* Replicator Dynamics). В классической теории игр предполагается, что полная информация об игре доступна всем агентам, но такое предположение вместе с допущением об индивидуальной рациональности агентов не всегда позволяет правильно моделировать динамический характер взаимодействий в реальном мире. В эволюционной теории игр предположение о рациональности обучающихся агентов ослабляется и заменяется принципами биологической эволюции — мутацией и отбором. В некоторых случаях мутация может отклонять процесс обучения от нахождения чистой стратегии, но при этом улучшается сходимость к смешанным стратегиям.

Основным преимуществом алгоритма RD и его модификаций являются возможности автоматического анализа выученных стратегий и последующего создания на основе реверс-инжиниринга полноценных аналитических моделей сложной среды.

4.2. Модель

4.2.1. Нейроэволюция

Нейроэволюция (*англ.* neuroevolution) — процесс машинного обучения, основанный на эволюционных алгоритмах и нейронных сетях. В отличие от машинного обучения с учителем, для которого требуются точно совпадающие входные-выходные обучающие данные, в нейроэволюции необходим только результат оценки работы нейронной сети в определенной среде. Именно поэтому нейроэволюция часто используется для мультиагентного обучения в компьютерных играх. Нейроэволюцию относят к одной из технологий обучения с подкреплением и противопоставляют технологиям глубокого обучения, в которых используются методы градиентного спуска для обучения нейронных сетей с фиксированной топологией. Наиболее популярным алгоритмом нейроэволюции является генетический алгоритм.

Генетический алгоритм (*англ.* Genetic Algorithm, GA) — эвристический алгоритм поиска, используемый для решения задач оптимизации путем случайного подбора, комбинирования и вариации искомых параметров на основе принципов биологической эволюции. Нейроэволюционный генетический алгоритм позволяет решать задачу оптимизации для нахождения экстремума оптимальной функции награды $\mathcal{R}_i^*(\theta_i)$.

На рис. 4.1. представлена обобщенная схема мультиагентного эволюционного обучения с подкреплением. Агент i ($i = 1, ..., n$) от среды E получает состояния s_i или наблюдения o_i и выполняет действия a_i в некоторый момент времени t. Множество M агентов i^m ($m = 1, ..., M$) одного вида и принадлежащих одному *поколению* G_j ($j = 1, ..., K$), т. е. имеющих одинаковое время создания t_j, составляет *популяцию* P_i. Каждый агент i^m из популяции P_i независимо взаимодействует со своей версией среды $\varepsilon_{P_i^m} \sim E$. Агент i имеет функцию наград R_i, которая ставит в соответствие истории H_i, представляющей собой последовательность «состояние s_i — действие a_i» скалярные значения наград r_i. *Функция приспособленности* (*фитнесс-функция*) $\mathcal{F}_i$ — это вещественная или целочисленная функция одной или нескольких переменных, которая направляет генетический алгоритм в сторону оптимального решения путем присвоения истории H_i некоторого оценочного скалярного значения $\mathcal{F}_i(H_i)$.

Если каждый агент i^m в популяции P_i реализуется нейронной сетью, параметризированной весами θ_i, называемыми *генотипом*, то генетический алгоритм выполняет поиск экстремума оптимальной функции награды $\mathcal{R}_i^*(\theta_i)$ посредством итеративной максимизации или минимизации ожидаемого значения функции приспособленности $\mathcal{F}_i(\theta_i)$ в средах $\varepsilon_{P_i^m}$:

$$\mathcal{R}_i^*(\theta_i) = \underset{r_i \in R_i}{\operatorname{argmax}} \mathbb{E}_{\varepsilon_{P_i^m} \sim E}\left[\mathcal{F}_i(\theta_i)\right].$$

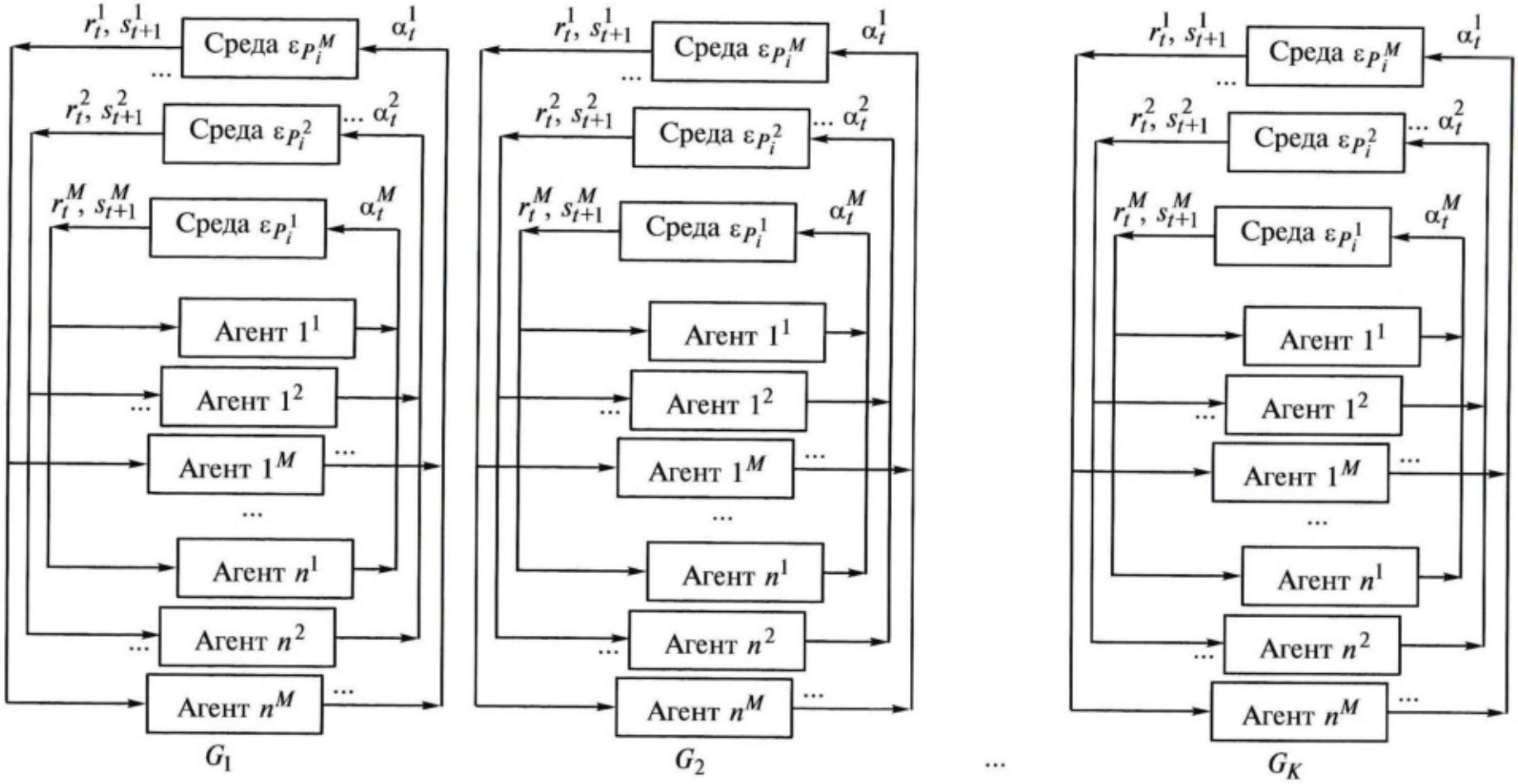


Рис. 4.1. Обобщенная схема мультиагентного эволюционного обучения с подкреплением

В среде SMAC используется единая награда для всех агентов, так как среда нацелена на машинное обучение кооперативных агентов. Соответственно, на основе этой награды может быть разработана только единая функция приспособленности $\mathcal{F}(\theta_i)$, которая больше пригодна для коэволюции. В связи с этим в независимом генетическом алгоритме InGA в среде SMAC используется специальная функция приспособленности $\mathcal{F}_i(\theta_i)$ для агентов i^m в популяции P_i:

$$\mathcal{F}_i(\theta_i) = \begin{cases} w + r_i^{\theta}, \text{ если } \text{win} = \text{True}; \\ r_i + r_i^{\theta}, \text{ если } \text{win} = \text{False}. \end{cases}$$

Смысл функции приспособленности $\mathcal{F}_i(\theta_i)$ заключается в выделении из популяции P_i агентов i^m, которые приводят к победе (win = True) за счет присвоения специальной награды w вместо награды от среды SMAC за эпизод игры r_i. В среде SMAC основные награды агент получает за целый сыгранный эпизод, поэтому и функция приспособленности $\mathcal{F}_i(\theta_i)$ вычисляется не за каждый шаг игры, а только за эпизод. Кроме того, функция приспособленности $\mathcal{F}_i(\theta_i)$ дополняет награду среды SMAC r_i индивидуальной наградой за действия каждого агента r_i^{θ}. В рассматриваемом в разд. 4.3 независимом генетическом алгоритме InGA индивидуальная награда r_i^{θ} присваивается за атакующие действия агентов, которые приближают их к победе в эпизоде игры.

4.2.2. Коэволюция

Коэволюция (*англ.* coevolution) — процесс машинного обучения двух и более популяций агентов, основанный на эволюционных алгоритмах, в которых значение функции приспособленности одного агента зависит от его взаимодействий с другими агентами. *Кооперативным коэволюционным алгоритмом* (*англ.* cooperative coevolutionary algorithm) называется коэволюционный алгоритм, в котором агенты получают награды за успешные совместные действия. *Конкурентный коэволюционный алгоритм* (*англ.* competitive coevolutionary algorithm) — коэволюционный алгоритм, в котором агенты получают награды за превосходство над другими агентами в совместно эволюционирующих популяциях.

Доказательство сходимости к глобальному минимуму, определение производительности или формализованная проверка свойств коэволюционных алгоритмов могут основываться на модели упорядоченных множеств, динамической модели, модели эволюционной теории игр или марковской модели.

Разные математические модели приводят к созданию различных коэволюционных алгоритмов с разными свойствами. Рассмотрим применение марковской модели для описания коэволюционного генетического алгоритма. Пусть Ω_i — пространство генотипов агентов i, входящих в популяцию P_i.

Тогда Ω_i является пространством поиска для коэволюционного генетического алгоритма. Пусть $\mathcal{P}_i$ — множество всех возможных популяций P_i; $\mathcal{F}_i : \Omega_i \to \mathbb{R}$ — функция приспособленности и $\sigma_{\mathcal{F}_i} : \mathcal{P}_i \to \Omega_i$ — функция стохастического отбора. Тогда генетический оператор *скрещивания* (размножения) представляет собой функцию $\chi_i : \Omega_i \times \Omega_i \to \Omega_i$, которая на основе генотипов двух агентов из популяций P_i получает нового агента, имеющего части генотипов обоих родителей. Генетический оператор *мутации*, используя некоторое правило, незначительно изменяет генотип исходного агента $\mu_i : \Omega_i \to \Omega_i$.

Вероятность $\mathcal{P}$ создания агента i^m с генотипом $\theta_i^m \in \Omega_i$ из популяции P_i определяется как

$$\mathcal{P}\left(P_i \to i^m\right) = \mathcal{P}\left(\mu_i\left(\chi_i\left(\sigma_{\mathcal{F}_i}\left(P_i\right), \sigma_{\mathcal{F}_i}\left(P_i\right)\right)\right)\right).$$

Таким образом, вероятность создания генотипов новой популяции P_i' зависит от состояния генотипов предыдущей популяции P_i, что удовлетворяет марковскому свойству. В связи с этим функционирование генетического алгоритма можно представить в виде марковской цепи с матрицей переходов $T_{P_i, P_i'}$:

$$T_{P, P'} = \frac{M!}{\prod_{\theta_i^m \in \Omega_i}\left(\sum_{\theta_j'^m \in P_i'}\left[\theta_i^m = \theta_j^m\right]\right)! \prod_{\theta_i'^m \in P_i'}} \mathcal{P}\left(P_i \to i^m\right).$$

Добавление коэволюционной составляющей в данную марковскую модель генетического алгоритма выполняется посредством определения взаимозависимости в функции приспособленности $\mathcal{F}_i : \Omega_i \times \Omega_i \to \mathbb{R}$. В процессе машинного обучения марковская модель позволяет довольно точно описать стохастические свойства генетического алгоритма и усредненную динамику коэволюции агентов.

Кроме того, использование марковской модели при сравнении производительности коэволюционных алгоритмов создает некоторую «нулевую гипотезу», в зависимости от которой могут быть оценены те или иные успехи в обучении. Так, если два агента обучаются в матричной игре «Орлянка» (см. разд. 2.2) с помощью конкурентного коэволюционного генетического алгоритма и средние значения фитнесса обоих популяций агентов будут равны 0.5, то это соответствует лучшей стратегии из равновесия Нэша: «равновероятный выбор стороны монеты».

В целом коэволюция может быть реализована с помощью различных способов дизайна функции приспособленности, архитектуры нейронной сети, топологии представления решаемой задачи (см. разд. 4.1). В коэволюционном алгоритме CoE применяются принципы совместного скрещивания генотипов агентов и используется единая функция приспособленности. В среде SMAC в коэволюционном алгоритме CoE функция

приспособленности $\mathcal{F}(\theta_i)$ едина для всех агентов $i = 1, ..., n$ и вычисляется по формуле

$$\mathcal{F}(\theta_i) = \begin{cases} w, \text{ если } \text{win} = \text{True}; \\ r, \text{ если } \text{win} = \text{False}, \end{cases}$$

где w — специальная награда за победу агентов в эпизоде; r — награда от среды SMAC за эпизод игры.

4.3. Алгоритмы

4.3.1. Независимый генетический алгоритм (InGA)

Независимый генетический алгоритм (*англ.* Independent Genetic Algorithm, InGA) основан на генетическом алгоритме лаборатории AI компании Uber.

Если сравнивать реализованный алгоритм InGA с дарвиновской, райтовской и ламарковской моделями эволюции, то он ближе к третьей модели в том смысле, что выученные данные — так называемый фенотип, сформированный на основе генотипа агента и под действием внешнесредовых факторов, — включается в геном следующего поколения агентов. Особенностью алгоритма InGA является использование принципов эволюционного обучения: элитизма, мутации и безградиентного подхода (см. гл. 3).

Под элитизмом в алгоритме InGA понимается отбор определенного числа элитных агентов *Ea*, имеющих самое большое значение фитнесса, чтобы сохранить их в неизменном виде для следующего поколения (шаг 12). В алгоритме InGA значение параметра обучения *Ea* равно единице (шаг 1). Невыбранные особи не уничтожаются, а подвергаются мутации или копируются без изменений (шаг 13).

Таким образом, элитные агенты для следующего поколения выбираются методом ранжирования. Кроме того, в различных версиях генетического алгоритма для селекции агентов могут использоваться следующие методы: турнирный метод — вначале выбираются случайным образом группы агентов и затем из каждой группы отбираются агенты с наибольшим значением фитнесса; метод рулетки — чем больше значение фитнесса агента, тем больше вероятность его отбора; сигма-отсечение — для предотвращения преждевременной сходимости генетического алгоритма вероятность выбора агента зависит от значения масштабируемой функции приспособленности.

Алгоритм InGA для агента i приведен ниже:

1. Инициализировать
   $Ne = G_j$, $j \leftarrow 1, ..., K$; $i \leftarrow 1, ..., n$; $a_i \in A_i$; $\mathcal{K} \leftarrow |A_i|$; $m \leftarrow 1, \ldots, M$; $\varepsilon_{P_i^m} \sim E$;
   $\theta \leftarrow \xi$, $\theta' \leftarrow \xi$, $\epsilon \sim \mathcal{N}(0,1)$, $\forall s \in S$;
   $\delta \in (0, 1)$; $Ea \in [1, EA]$; $\mathcal{M} \in [1, MA]$; $w \in [1, W]$.
2. Цикл по эпизодам
3. Цикл по агентам в популяции
4. Из среды $\varepsilon_{P_i^m} \sim E$ для агента i^m в популяции P_i получить состояние s_i^m.
5. Определить возможные действия $a_i^m \in A_i^m$ в состоянии s_i^m.
6. Передать состояние s_i^m в нейронную сеть агента i^m и выбрать возможное действие a_i^m в соответствии с максимальным значением $Q_1\left(s_i^m, a_i^m; \theta_i^m\right), Q_2\left(s_i^m, a_i^m; \theta_i^m\right), \ldots, Q_{\mathcal{K}}\left(s_i^m, a_i^m; \theta_i^m\right)$.
7. Выполнить возможное действие a_i^m.
8. Из среды $\varepsilon_{P_i^m} \sim E$ получить награду r_i^m и параметр победы win.
9. Вычислить независимую награду $r_i^{\theta_i^m}$.
10. Вычислить значение функции приспособленности:
$$\mathcal{F}_i^m\left(\theta_i^m\right) \leftarrow \begin{cases} w + r_i^{\theta_i^m}, & \text{если win} = \text{True}; \\ r_i^m + r_i^{\theta_i^m}, & \text{если win} = \text{False}. \end{cases}$$
11. Создать новое поколение агентов на основе предыдущего: $\theta_i' \leftarrow \theta_i$.
12. Упорядочить по убыванию и отобрать Ea нейронных сетей θ_i' с максимальными значениями функции приспособленности.
13. Мутировать $\mathcal{M}$ случайных нейронных сетей, не входящих в элиту:
    $\theta_i' \leftarrow \theta_i' + \delta\epsilon_i$.

Процедура мутации выполняется после процедуры отбора элитных агентов и заключается в случайном выборе из оставшихся агентов определенного числа $\mathcal{M} = 10$ агентов-мутантов, которые подвергаются изменению и передаче в следующее поколение (шаг 13). Поскольку алгоритм InGA основан на нейроэволюции, изменениям в процедуре мутации подвергается генотип агентов, хранящийся в весах нейронных сетей θ_i каждого агента i. Суть мутационных изменений состоит в добавлении к значениям весов нейронных

сетей случайного гауссовского шума $\epsilon \sim \mathcal{N}(0,1)$ определенного уровня, который контролируется параметром обучения $\delta = 0.01$ (шаг 11).

Ну и наконец, алгоритм InGA не использует градиенты для мультиагентного обучения. Современные исследования показывают, что нейросетевые алгоритмы обучения с подкреплением, использующие градиенты, пригодны не для всех сред. В некоторых средах, в том числе в средах компьютерных игр, использование градиента приводит к увеличению продолжительности обучения и угрозе попадания в локальный минимум.

В целом для обучения с подкреплением в средах с редкой наградой чрезвычайно актуальна проблема попадания в локальный минимум. В таком случае эволюционное обучение, в котором используется плотный поиск нового решения в пространстве, близком к предыдущему успешному решению, может быть намного эффективнее, чем следование некоторому локальному градиенту. Кроме того, отказ от алгоритма обратного распространения ошибки дает экономию до 70 % вычислительных ресурсов и расхода памяти в каждом эпизоде обучения.

В представленном выше алгоритме InGA цикл на шаге 2 для сохранения общности схемы обучения с подкреплением называется циклом по эпизодам, хотя с точки зрения эволюционного обучения этот цикл также можно назвать циклом по поколениям агентов $Ne = G_j$.

На рис. 4.2 представлена архитектура нейронной сети алгоритма InGA для рассматриваемой в гл. 4 задачи мультиагентного обучения двух агентов в среде SMAC. В отличие от нейронной сети генетического алгоритма компании Uber, которая впервые провела эволюционное обучение большой нейронной сети с 4 млн весов, для нейронной сети алгоритма InGA требуется только два полносвязных линейных слоя с входными и выходными размерами соответственно 32×64, 64×10. На вход нейронная сеть получает состояние среды s, заданное 32 отсчетами, на выходе возвращает 10 Q-значений.

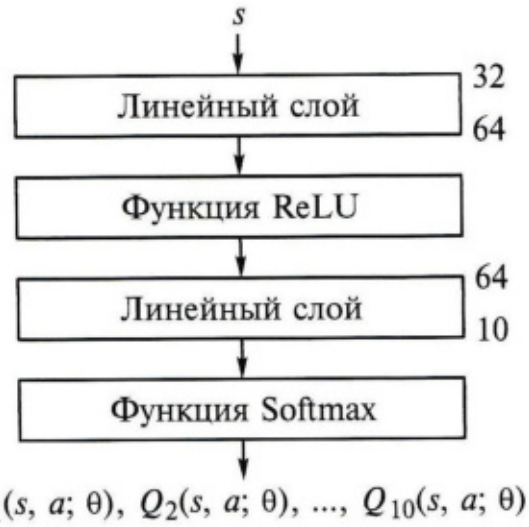


Рис. 4.2. Архитектура нейронной сети алгоритма InGA

Схема мультиагентного обучения с использованием алгоритма InGA аналогична схеме мультиагентного нейросетевого обучения IQL, представленной на рис. 3.6.

4.3.2. Коэволюционный алгоритм (CoE)

Коэволюционные алгоритмы по сути мультиагентны, их принцип совместного обучения соответствует концепции мультиагентного обучения. Более того, механизмы коэволюции чрезвычайно централизованы. Однако,

как было показано в гл. 3, сильная централизация мультиагентного обучения проигрывает другим алгоритмическим приемам. Эксперименты эволюционного обучения подтверждают, что коэволюция популяций нескольких независимых агентов эффективнее, чем коэволцюция популяции одного централизованного агента. Поэтому в разработанном коэволюционном алгоритме CoE (*англ.* CoEvolution), основанном на алгоритме InGA, из всех возможных принципов коэволюции используются только единая награда (шаг 9) и скрещивание (шаг 12).

Алгоритм CoE для агента i приведен ниже:

1. Инициализировать
   $Ne = \mathrm{G}_j$, $j \leftarrow 1, ..., K$; $i \leftarrow 1, ..., n$; $a_i \in A_i$; $\mathcal{K} \leftarrow |A_i|$; $m \leftarrow 1, ..., M$; $\varepsilon_{P_i^m} \sim E$;
   $\theta \leftarrow \xi$, $\theta' \leftarrow \xi$, $\epsilon \sim \mathcal{N}(0,1)$, $\forall s \in S$;
   $\delta \in (0, 1)$; $Ea \in [1, EA]$; $Ca \in [1, CA]$; $\mathcal{M} \in [1, MA]$; $w \in [1, W]$.
2. Цикл по эпизодам
3. Цикл по агентам в популяции
4. Из среды $\varepsilon_{P_i^m} \sim E$ для агента i^m в популяции P_i^m получить состояние s_i^m.
5. Определить возможные действия $a_i^m \in A_i^m$ в состоянии s_i^m.
6. Передать состояние s_i^m в нейронную сеть агента i^m и выбрать возможное действие a_i^m в соответствии с максимальным значением
   $Q_1\left(s_i^m, a_i^m; \theta_i^m\right), Q_2\left(s_i^m, a_i^m; \theta_i^m\right), \ldots, Q_{\mathcal{K}}\left(s_i^m, a_i^m; \theta_i^m\right)$.
7. Выполнить возможное действие a_i^m.
8. Из среды $\varepsilon_{P_i^m} \sim E$ получить награду r^m и параметр победы win.
9. Вычислить значение функции приспособленности:
   $$\mathcal{F}_i^m\left(\theta_i^m\right) \leftarrow \begin{cases} w, & \text{если win = True;} \\ r^m, & \text{если win = False.} \end{cases}$$
10. Создать новое поколение агентов на основе предыдущего: $\theta_i' \leftarrow \theta_i$.
11. Упорядочить по убыванию и отобрать Ea нейронных сетей θ_i' с максимальными значениями функции приспособленности.
12. Скрестить генотип Ca случайных нейронных сетей, не входящих в элиту.
13. Мутировать $\mathcal{M}$ случайных нейронных сетей, не входящих в элиту:
    $\theta_i' \leftarrow \theta_i' + \delta\epsilon_i$.

В эволюционной биологии, согласно «гипотезе Черной Королевы» (*англ.* Red Queen hypothesis), живые организмы постоянно участвуют в эволюционной борьбе конкурирующих наборов совместно развивающихся генотипов, которые создают взаимные адаптации и контр-адаптации. Скрещивание (половое размножение) позволяет передать следующему поколению генотипы, которые в настоящее время не приносят пользы, но в будущем усилят защитные силы организма.

Если выгодная мутация происходит среди особей, размножающихся бесполым способом, то для этой мутации нет возможности распространиться, а для особи — получить гены из других линий своего вида, в которых могли возникнуть собственные полезные мутации. Более того, некоторые случаи наследственной изменчивости могут быть выгодными, только если одна мутация вступала в соединение с другой мутацией, а скрещивание увеличивает вероятность, что такое соединение произойдет. Скрещиванием гарантируется генетическое разнообразие, что в случае биологической коэволюции паразита с организмом-хозяином позволяет противостоять паразитной адаптации.

Скрещивание приносит пользу и популяции в целом — стимулирует проявление в ней коллективных эффектов благодаря эгоистичным действиям независимых организмов. В популяции, где используется скрещивание генов, порождается большее разнообразие фенотипов и наиболее приспособленные фенотипы смогут пережить изменение внешних условий окружающей среды.

В эволюционном обучении на основе генетического алгоритма выделяют несколько методов выбора агентов-родителей для скрещивания: панмиксия — оба агента-родителя выбираются случайно, каждый агент популяции имеет одинаковую вероятность быть выбранным; инбридинг — первый агент-родитель выбирается случайно, а вторым выбирается наиболее похожий на первого агента-родителя; аутбридинг — первый агент-родитель выбирается случайно, а вторым выбирается наименее похожий на первого агента-родителя.

На рис. 4.3 представлен возможный вариант скрещивания генотипов агентов в коэволюционном алгоритме CoE с использованием межпопуляционной панмиксии — оба гомогенных агента-родителя выбираются случайно из разных популяций. Генотип каждого агента представлен весами двухуровневой полносвязной нейронной сети (см. рис. 4.2). Поэтому при полностью случайной передаче генотипа существуют 16 вариантов заполнения весов нейронной сети агентов-потомков весами нейронных сетей агентов-родителей.

В качестве главного преимущества коэволюционных алгоритмов отмечают, что агенты обучаются не доминированию, а нахождению равновесия, по сути схожего с равновесием Нэша, что в точности соответствует целям кооперативного мультиагентного обучения. Это свойство оказывается особенно полезным при обучении в играх, имеющих интранзитивные отношения:

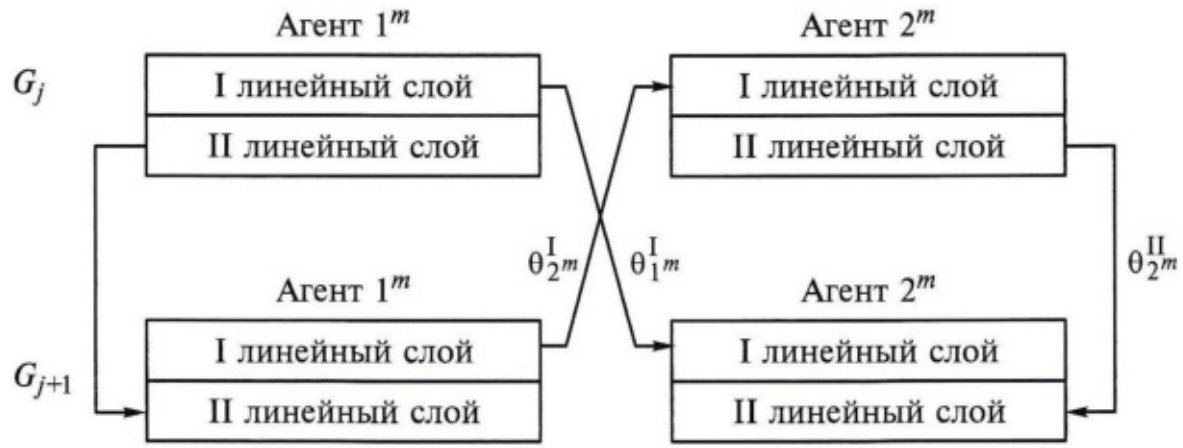


Рис. 4.3. Скрещивание генотипов в алгоритме CoE

стратегия π_1 побеждает стратегию π_2, стратегия π_2 побеждает стратегию π_3 и стратегия π_3 побеждает стратегию π_1.

Кроме того, считается, что коэволюционное обучение превосходит традиционное эволюционное обучение в сложных высокоструктурированных иерархических задачах. В то же время коэволюционное обучение намного сложнее отлаживать, чем эволюционное обучение. Диагностические проблемы случаются, когда внутренние меры по оценке изменения значений функций приспособленности не позволяют получить общее представление о динамике машинного обучения или же в процессе обучения наблюдается «посредственная стабильность», которая может быть связана с нахождением коэволюционным алгоритмом решений, субоптимальных с точки зрения общей архитектуры системы.

В коэволюционном обучении возникает также проблема «абсолютного лидерства», когда отстающие в развитии агенты популяции не имеют достаточно информации для обучения. Аналогично в шахматной игре при противоборстве с гроссмейстером новичку очень тяжело чему-либо научиться. Еще одна известная проблема — «циклическое поведение», когда агенты слишком часто пытаются адаптироваться к незначительным изменениям среды, что в итоге приводит к практически исходному состоянию генотипа.

4.4. Карта

Мультиагентное эволюционное обучение в гл. 4 проводилось на карте 2p4zFOX.SC2Map, представленной на рис. 4.4. В начале каждого эпизода на карте слева, в точке Zealot Point, автоматически создаются два агента юнита зилота расы протоссов, которыми управляет обучающий алгоритм, справа, в точке Zergling Point, автоматически создаются четыре агента юнита зерглинга расы зергов, которыми управляет компьютер, используя стандартные правила. Длительность одного полного эпизода игры составляет 6 с.

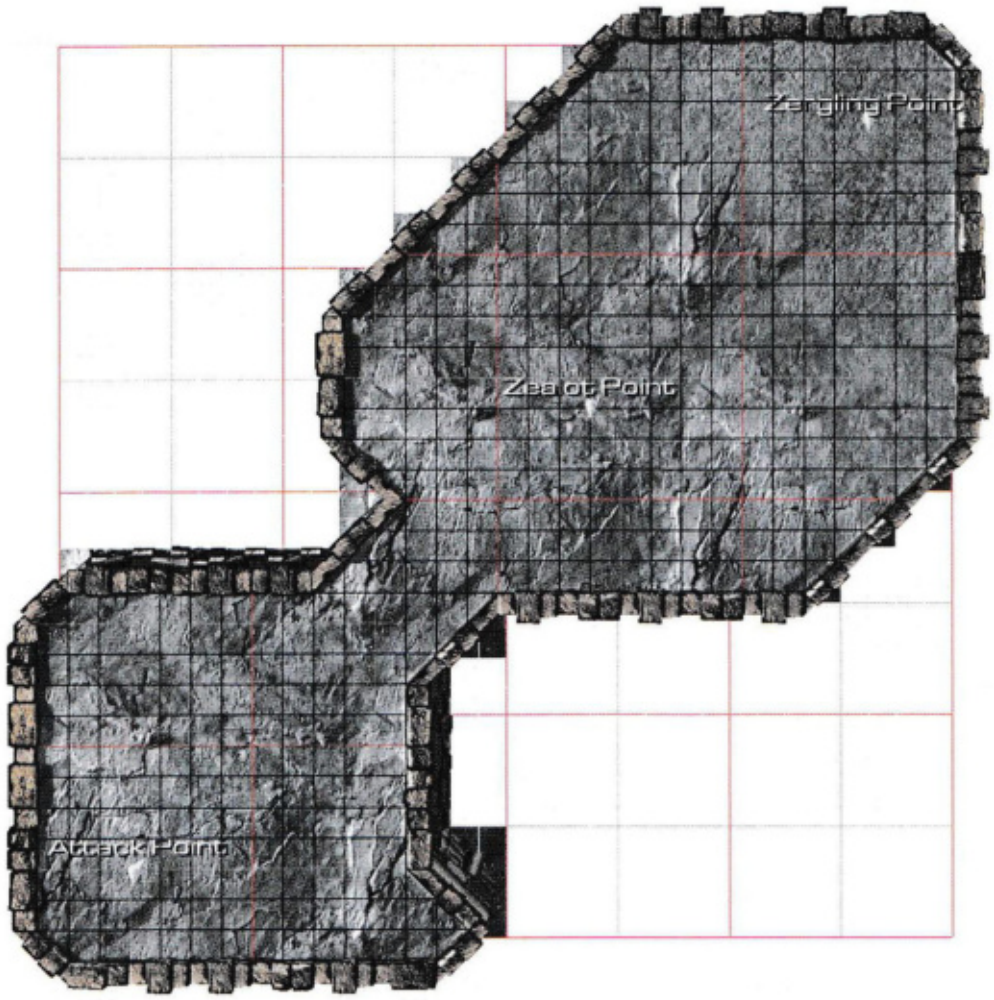


Рис. 4.4. Внешний вид карты 2p4zFOX.SC2Map в редакторе StarCraft II Editor

На карте, в отличие от карт, рассмотренных в гл. 1–3 (см. рис. 1.4, 2.5, 3.13), появилась контрольная точка Attack Point, куда пытаются прорваться агенты компьютера. Карта имеет нестандартную форму — по центру сформирован узкий проход. При обучении агенты могут выучить стратегию микроменеджмента компьютерной игры StarCraft II — ориентирование, для получения тактического преимущества они используют характер рельефа и защитные свойства местности.

Зилот — наземный боевой юнит протоссов, обладающий только атакой ближнего боя. Наносит восемь единиц урона, радиус атаки 0.1. Имеет 80 единиц здоровья и одну единицу брони. По значениям урона и здоровья зилот — один из лучших юнитов в начальной стадии игры. При столкновении с противниками эффективно используется против зерглингов и мародеров.

На рис. 4.5–4.9 проиллюстрированы различные ситуации в среде SMAC: начало эпизода игры, противник атакует справа сверху вне зоны атаки агентов (см. рис. 4.5); противник прорвался на контрольную точку Attack Point (см. рис. 4.6); агенты реализовали неправильную стратегию — «четверо против одного» (см. рис. 4.7); агенты выучили стратегию перекрытия прохода по центру карты (см. рис. 4.8); все агенты выучили оптимальную стратегию (см. рис. 4.9).

Рис. 4.5. Начало эпизода

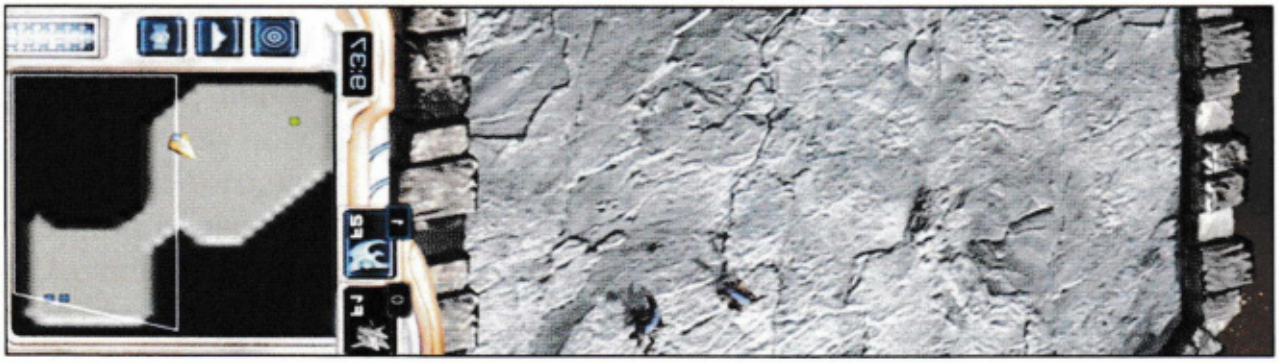


Рис. 4.6. Противник прорвался на контрольную точку Attack Point

Рис. 4.7. Неправильная стратегия — «четверо против одного»

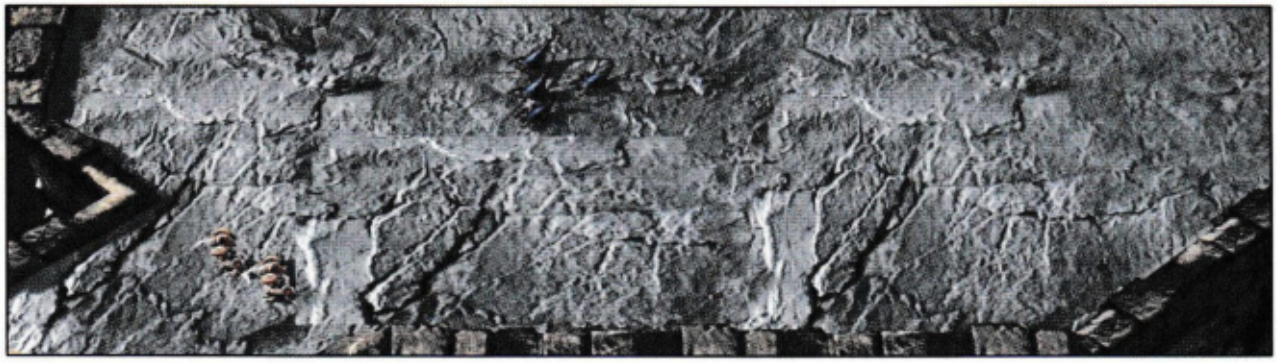


Рис. 4.8. Агенты выучили стратегию перекрытия прохода по центру карты

Рис. 4.9. Все агенты выучили оптимальную стратегию

Результаты экспериментального обучения в среде SMAC будут рассмотрены в разд. 4.7.

4.5. Технология

Рассматриваемые в гл. 4 алгоритмы реализованы на языке программирования Python версии 3.6. В представленном в разд. 4.6 программном коде, кроме библиотеки кооперативного мультиагентного обучения SMAC, используются стандартные библиотеки: numpy для работы с массивами, time для измерения времени, collections для создания словаря с упорядоченными ключами, random для выбора случайных элементов из списка, matplotlib для построения графиков.

Карта 2p4zFOX.SC2Map создана на базе карты SMAC в редакторе StarCraft II Editor. Ниже представлены изменения, которые необходимо внести в файл smac_maps.py, расположенный в папке «...\smac\env\starcraft2\maps», при создании новой карты или редактировании существующей:

```
«2p4zFOX»: {
    «n_agents»: 2,
    «n_enemies»: 4,
    «limit»: 220,
    «a_race»: «P»,
    «b_race»: «Z»,
    «unit_type_bits»: 0,
    «map_type»: «zealots»,}
```

Эксперименты с алгоритмами проводились в операционной системе Windows 10. Код, реализующий мультиагентные эволюционные алгоритмы обучения, представлен в подразд. 4.6.1 и 4.6.2. Код, тестирующий обученных агентов, вынесен в отдельный файл, его можно найти на платформе Github. Примеры реализации алгоритмов в полном объеме представлены на Github: https://github.com/Alekat13/FoxCommander

Видеопримеры работы алгоритмов приведены на YouTube: https://www.youtube.com/channel/UC33QPEjSlfA8P9rG0HtiH6g

4.6. Код

4.6.1. Алгоритм InGA

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
import matplotlib.pyplot as plt
import time

import torch
import torch.nn as nn

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Определяем архитектуру нейронной сети
class Q_network(nn.Module):
    #На вход нейронная сеть получает состояние среды
    #На выходе нейронная сеть возвращает оценку действий в виде Q-значений
    def __init__(self, obs_size, n_actions):
        super(Q_network, self).__init__()
        self.Q_network = nn.Sequential(
            #Первый линейный слой обрабатывает входные данные состояния среды
            nn.Linear(obs_size, 64),
            nn.ReLU(),
            #Второй линейный слой обрабатывает выходные данные
            nn.Linear(64, n_actions)
        )
        #Применение к выходным данным функции Softmax
        self.sm_layer = nn.Softmax(dim=1)
    #Вначале данные x обрабатываются полносвязной сетью с функцией ReLU
    #На выходе происходит обработка функцией Softmax
    def forward(self, x):
        q_network_out = self.Q_network(x)
        sm_layer_out = self.sm_layer(q_network_out)
        #Финальный выход нейронной сети
        return sm_layer_out

#Выбираем возможное действие с максимальным Q-значением
def select_actionFox(action_probabilities, avail_actions_ind):
    p = np.random.random(1).squeeze()
    #Находим возможное действие:
    #Проверяем, есть ли действие в доступных действиях агента
    for ia in action_probabilities:
        action = np.argmax(action_probabilities)
        if action in avail_actions_ind:
            return action
        else:
            action_probabilities[action] = 0

#Основная функция программы
def main():

    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="2p4zFOX", reward_only_positive=False,
                        reward_scale_rate=200, difficulty="1")
    #Получаем и выводим на печать информацию о среде
    env_info = env.get_env_info()
```

```
    print ('env_info=',env_info)
    #Получаем и выводим на печать размер наблюдений агента
    obs_size =  env_info.get('obs_shape')
    print ("obs_size=",obs_size)
    #Число действий агента
    n_actions = env_info["n_actions"]
    #Число дружественных агентов
    n_agents = env_info["n_agents"]

    #Определяем основные параметры эволюционного обучения
##############################################################################
    population_amount = 50 #Число агентов в популяции
    noise_level = 0.01 #Уровень шума для мутации
    mutant_amount = 10 #Число агентов, которые будут мутированы
    elite_amount = 1 #Число элитных особей для следующего поколения
    n_episodes = 10 #Число эпизодов игры или поколений эволюции

    global_step = 0  #Подсчитываем общее число шагов в игре
##############################################################################

    #Используем только центральный процессор
    device = "cpu"

    #Создаем списки популяций для двух агентов
    q_network_pop0 = []
    q_network_pop1 = []
    q_network_pop_next0 = []
    q_network_pop_next1 = []

    for agent_id in range(n_agents):
        for pai in range(population_amount):
            #Создаем основную нейронную сеть
            #заново для каждого агента популяции, чтобы иметь разные веса
            q_network = Q_network(obs_size, n_actions).to(device)
            if agent_id == 0:
                #Создаем список нейронных сетей популяции
                q_network_pop_next0.append(q_network)
            elif agent_id == 1:
                q_network_pop_next1.append(q_network)

    #Выводим на печать архитектуру базовой нейронной сети популяции
    print ('q_network_pop_next0 =', q_network_pop_next0[0])

    #Определяем вспомогательные параметры
    reward_history = []
    winrate_history = []
    agent0fire_history = []
    agent1fire_history = []
    bestindex_historyS = []
    bestindex_historyA = np.zeros([population_amount])


    #Основной цикл по эпизодам игры или поколениям эволюции
    ################_цикл for по эпизодам_#####################################
    for e in range(n_episodes):
        print ("----------------------Эпизод--------------------", e)

        global_reward = 0

        #Задаем нулевые значения фитнесс-функции в начале эпизода
        fitness_pai = np.zeros([n_agents, population_amount])

        #Массив для хранения независимых дополнительных наград агентов
```

```
local_fitness = np.zeros([n_agents, population_amount])

#Обновляем поколение
q_network_pop0 = q_network_pop_next0
q_network_pop1 = q_network_pop_next1
################_цикл for pai по популяции агента в поколении_#######
for pai in range(population_amount):
    print ("В популяции агент №", pai)
    #Перезагружаем среду
    env.reset()
    #Флаг окончания эпизода
    terminated = False
    #Награда за эпизод
    episode_reward = 0

    #Шаги игры внутри эпизода
    ######################_цикл while_#################################
    while not terminated:

        #Обнуляем промежуточные переменные
        actions = []
        action = 0

        #Сохраняем историю состояний среды на одном шаге для разных
        #агентов
        obs_agent = np.zeros([n_agents], dtype=object)

        #########_Цикл по агентам для выполнения действий в игре_####
        for agent_id in range(n_agents):

            #Получаем состояние среды для независимого агента
            obs_agent[agent_id] = env.get_obs_agent(agent_id)

            #Конвертируем данные в тензор
            obs_agentT = torch.FloatTensor([obs_agent[agent_id]])
                                                    .to(device)

            #Передаем состояние среды в основную нейронную сеть
            #и получаем Q-значения для каждого действия
            if agent_id == 0:
                action_probabilitiesT =
                                q_network_pop0[pai](obs_agentT)
            elif agent_id == 1:
                action_probabilitiesT =
                                q_network_pop1[pai](obs_agentT)

            #Конвертируем данные в numpy
            action_probabilitiesT = action_probabilitiesT.to("cpu")
            action_probabilities =
                            action_probabilitiesT.data.numpy()[0]

            #Находим возможные действия агента в данный момент
            avail_actions = env.get_avail_agent_actions(agent_id)
            avail_actions_ind = np.nonzero(avail_actions)[0]

            #Выбираем возможное действие агента с учетом
            #максимального Q-значения
            action = select_actionFox(action_probabilities,
                                avail_actions_ind)

            #Обрабатываем исключение при ошибке в возможных действиях
            if action is None:
                action = np.random.choice (avail_actions_ind)
```

```
                #Создаем независимую дополнительную награду для агентов
                if (action == 6) or (action == 7):
                    local_fitness[agent_id][pai] =
                                        local_fitness[agent_id][pai] + 1

                #Собираем действия от разных агентов
                actions.append(action)

            ###_конец цикла по агентам для выполнения действий в игре_###

            #Передаем действия агентов в среду, получаем награду
            #и прерывание игры от среды
            reward, terminated, info = env.step(actions)
            #Суммируем награды за этот шаг для вычисления общей награды
            episode_reward += reward
            #Обновляем счетчик общего числа шагов
            global_step += 1

        ######################_конец цикла while_#########################

        #Вычисляем значение фитнесс-функции для нейронной сети популяции
        for agent_id in range(n_agents):
            if (terminated == True) and (info['battle_won'] == True):
                #Если была победа, то присваиваем максимальное значение
                #фитнесса
                fitness_pai[agent_id][pai] = 300 +
                                        local_fitness[agent_id][pai]
            else:
                #Или в качестве фитнесса берем награду за эпизод
                fitness_pai[agent_id][pai] = episode_reward +
                                        local_fitness[agent_id][pai]

        #Выводим счетчик шагов игры и общую награду за эпизод
        global_reward = global_reward + episode_reward
        agent0fire_history.append(local_fitness[0][pai])
        agent1fire_history.append(local_fitness[1][pai])

    ############_конец цикла for pai по популяции агента в эпизоде_######

    #Собираем данные для графиков
    reward_history.append(global_reward/population_amount)
    status = env.get_stats()
    winrate_history.append(status["win_rate"])

    ################_Начинаем обучать_#####################################

    #Новое поколение появляется на основе старого поколения
    q_network_pop_next0 = q_network_pop0
    q_network_pop_next1 = q_network_pop1

    #Но для эволюции некоторые особи нового поколения будут изменены
    #Выбираем нейронную сеть с лучшим фитнессом
    #Вначале находим индекс с максимальным значением фитнесса
    for agent_id in range(n_agents):
        best_index = 0
        max_fitness = -100
        for i in range(population_amount):
            if fitness_pai[agent_id][i]>=max_fitness:
                best_index = i
                max_fitness = fitness_pai[agent_id][i]
```

```
        #Первая особь нового поколения - это лучшая особь из предыдущего
        #поколения
        q_network_pop_next0[0] = q_network_pop0[best_index]
        q_network_pop_next1[0] = q_network_pop1[best_index]
        bestindex_historyA[best_index] = bestindex_historyA[best_index] + 1

        #Мутируем некоторое число нейронных сетей
        #Вначале выбираем кандидатов для мутирования и создаем случайные
        #индексы
        index_rand0 = np.zeros([mutant_amount])
        index_rand1 = np.zeros([mutant_amount])
        for i in range(mutant_amount):
            index_rand0[i] = np.random.randint(1, population_amount)
            index_rand1[i] = np.random.randint(1, population_amount)

        #Скрещиваем особей с индексом от elite_amount до mutant_amount
        k=elite_amount
        for i in range(mutant_amount):
            #Добавляем шум в веса нейронных сетей первого агента
            ind0 = int (index_rand0[i])
            for par in q_network_pop0[ind0].parameters():
                noise = np.random.normal(size=par.data.size())
                noiseT = torch.FloatTensor(noise)
                par.data += noise_level * noiseT
            #Передаем мутированных особей новому поколению первого агента
            q_network_pop_next0[k] = q_network_pop0[ind0]

            #Добавляем шум в веса нейронных сетей второго агента
            ind1 = int (index_rand1[i])
            for par in q_network_pop1[ind1].parameters():
                noise = np.random.normal(size=par.data.size())
                noiseT = torch.FloatTensor(noise)
                par.data += noise_level * noiseT
            #Передаем мутированных особей новому поколению второго агента
            q_network_pop_next1[k] = q_network_pop1[ind1]
            #Увеличиваем счетчик
            k += 1

        ################_конец обучения_#####################################

    ################_конец цикла по эпизодам игры_############################

    #Закрываем среду StarCraft II
    env.close()

    #Сохраняем параметры лучшей нейронной сети из популяции
    torch.save(q_network_pop_next0[0].state_dict(),"qnet_0.dat")
    torch.save(q_network_pop_next1[0].state_dict(),"qnet_1.dat")

    #Выводим на печать графики
    #Средняя награда
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(reward_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Число награды за эпизод')
    plt.show()
    #Процент побед
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(winrate_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Процент побед')
```

```
    plt.show()
    #Число атакующих действий агентов
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(agent0fire_history)
    plt.xlabel('Итерация')
    plt.ylabel('Число атакующих действий агента 1')
    plt.show()
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(agent1fire_history)
    plt.xlabel('Итерация')
    plt.ylabel('Число атакующих действий агента 2')
    plt.show()
    #Частота использования определенной нейронной сети
    for pai in range(population_amount):
        bestindex_historyS.append(bestindex_historyA[pai])

    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(bestindex_historyS)
    plt.xlabel('Номер нейронной сети в популяции')
    plt.ylabel('Частота попадания в элиту')
    plt.show()

#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    #Время обучения
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

4.6.2. Алгоритм CoE

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
import matplotlib.pyplot as plt
import time
from collections import OrderedDict

import torch
import torch.nn as nn


#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Определяем архитектуру нейронной сети
class Q_network(nn.Module):
    #На вход нейронная сеть получает состояние среды
    #На выходе нейронная сеть возвращает оценку действий в виде Q-значений
    def __init__(self, obs_size, n_actions):
        super(Q_network, self).__init__()
        self.Qlayers = nn.Sequential(OrderedDict([
            #Именуем слои для прямого обращения к их весам
            ('fl1', nn.Linear(obs_size, 64)),
            ('relu1', nn.ReLU()),
            ('fl2',nn.Linear(64, n_actions))
        ]))
        #Применение к выходным данным функции Softmax
```

```
        self.sm_layer = nn.Softmax(dim=1)
    #Вначале данные x обрабатываются полносвязной сетью с функцией ReLU
    #На выходе происходит обработка функцией Softmax
    def forward(self, x):
        q_network_out = self.Qlayers(x)
        sm_layer_out = self.sm_layer(q_network_out)
        #Финальный выход нейронной сети
        return sm_layer_out

#Выбираем возможное действие с максимальным Q-значением
def select_actionFox(action_probabilities, avail_actions_ind):
    p = np.random.random(1).squeeze()
    #Находим возможное действие
    #Проверяем, есть ли действие в доступных действиях агента
    for ia in action_probabilities:
        action = np.argmax(action_probabilities)
        if action in avail_actions_ind:
            return action
        else:
            action_probabilities[action] = 0
#Основная функция программы
def main():

    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="2p4zFOX", reward_only_positive=False,
                        reward_scale_rate=200, difficulty="1")
    #Получаем и выводим на печать информацию о среде
    env_info = env.get_env_info()
    print ('env_info=',env_info)
    #Получаем и выводим на печать размер наблюдений агента
    obs_size =  env_info.get('obs_shape')
    print ("obs_size=",obs_size)
    #Число действий агента
    n_actions = env_info["n_actions"]
    #Число дружественных агентов
    n_agents = env_info["n_agents"]

    #Определяем основные параметры эволюционного обучения
    ##########################################################################
    population_amount = 50 #Число агентов в популяции
    noise_level = 0.01 #Уровень шума для мутации
    mutant_amount = 20 #Число агентов, которые будут мутированы
    elite_amount = 10 #Число элитных особей для следующего поколения
    crossover_amount = 10 #Число агентов для оператора скрещивания
    global_step = 0  #Подсчитываем общее число шагов в игре
    n_episodes = 50 #Число эпизодов игры или поколений эволюции
    ##########################################################################

    #Используем для обучения только центральный процессор
    device = "cpu"

    #Создаем списки популяций для двух агентов
    q_network_pop0 = []
    q_network_pop1 = []
    q_network_pop_next0 = []
    q_network_pop_next1 = []

    for agent_id in range(n_agents):
        for pai in range(population_amount):
            #Создаем основную нейронную сеть
            #заново для каждого агента популяции, чтобы иметь разные веса
            q_network = Q_network(obs_size, n_actions).to(device)
```

```
        if agent_id == 0:
            #Создаем список нейронных сетей популяции
            q_network_pop_next0.append(q_network)
        elif agent_id == 1:
            q_network_pop_next1.append(q_network)

#Выводим на печать архитектуру базовой нейронной сети популяции
print ('q_network_pop_next0 =', q_network_pop_next0[0])

#Определяем вспомогательные параметры
reward_history = []
winrate_history = []
bestindex_historyS = []
bestindex_historyA = np.zeros([population_amount])
#Параметр выбора уровня нейронной сети для скрещивания
layer_choice = [0, 1]

#Основной цикл по эпизодам игры или поколениям эволюции
################_цикл for по эпизодам_###################################
for e in range(n_episodes):
    print ("----------------------Episode--------------------", e)

    global_reward = 0

    #Задаем нулевые значения фитнесс-функции в начале эпизода
    fitness_pai = np.zeros([population_amount])

    #Обновляем поколение
    q_network_pop0 = q_network_pop_next0
    q_network_pop1 = q_network_pop_next1

    ################_цикл for pai по популяции агента в эпизоде_#########
    for pai in range(population_amount):
        print ("Population №", pai)
        #Перезагружаем среду
        env.reset()
        #Флаг окончания эпизода
        terminated = False
        #Награда за эпизод
        episode_reward = 0

        #Шаги игры внутри эпизода
        ######################_цикл while_#################################
        while not terminated:

            #Обнуляем промежуточные переменные
            actions = []
            action = 0

            #Сохраняем историю состояний среды на одном шаге
            #для разных агентов
            obs_agent = np.zeros([n_agents], dtype=object)

            #########_Цикл по агентам для выполнения действий в игре_####
            for agent_id in range(n_agents):

                #Получаем состояние среды для независимого агента
                obs_agent[agent_id] = env.get_obs_agent(agent_id)

                #Конвертируем данные в тензор
                obs_agentT =
                 torch.FloatTensor([obs_agent[agent_id]]).to(device)
```

```
                #Передаем состояние среды в основную нейронную сеть
                #и получаем Q-значения для каждого действия
                if agent_id == 0:
                    action_probabilitiesT =
                     q_network_pop0[pai](obs_agentT)
                elif agent_id == 1:
                    action_probabilitiesT =
                     q_network_pop1[pai](obs_agentT)

                #Конвертируем данные в numpy
                action_probabilitiesT = action_probabilitiesT.to("cpu")
                action_probabilities =
                 action_probabilitiesT.data.numpy()[0]
                #Находим возможные действия агента в данный момент
                avail_actions = env.get_avail_agent_actions(agent_id)
                avail_actions_ind = np.nonzero(avail_actions)[0]

                #Выбираем возможное действие агента с учетом
                #максимального Q-значения
                action = select_actionFox(action_probabilities,
                                          avail_actions_ind)

                #Обрабатываем исключение при ошибке в возможных действиях
                if action is None:
                    action = np.random.choice (avail_actions_ind)

                #Собираем действия от разных агентов
                actions.append(action)

            ####_конец цикла по агентам для выполнения действий в игре_##

            #Передаем действия агентов в среду, получаем награду
            #и прерывание игры от среды
            reward, terminated, info = env.step(actions)
            #Суммируем награды за этот шаг для вычисления общей награды
            episode_reward += reward
            #Обновляем счетчик общего числа шагов
            global_step += 1

        ######################_конец цикла while##########################

        #Вычисляем значение фитнесс-функции для нейронной сети популяции
        #Обрабатываем исключение KeyError
        if info.get('battle_won') is None:
            #В качестве фитнесса берем награду за эпизод
            fitness_pai[pai] = episode_reward
        else:
            if (info['battle_won'] == True):
                #Если была победа, то присваиваем максимальный фитнесс
                fitness_pai[pai] = 300 + episode_reward
            else:
                #В качестве фитнесса берем награду за эпизод
                fitness_pai[pai] = episode_reward

        #Выводим счетчик шагов игры и общую награду за эпизод
        print('Среднее число шагов=',
              (global_step/population_amount)/(e+1), "Награда для НС {} =
               {}".format(pai, episode_reward))
        print('Фитнесс = ', fitness_pai[pai])
        global_reward = global_reward + episode_reward

    ############_конец цикла for pai по популяции агента в эпизоде_######
```

```
#Собираем данные для графиков
reward_history.append(global_reward/population_amount)
status = env.get_stats()
winrate_history.append(status["win_rate"])

################_Начинаем обучать_#####################################

#Новое поколение создается на основе предыдущего поколения
q_network_pop_next0 = q_network_pop0
q_network_pop_next1 = q_network_pop1

#Но для эволюции некоторые особи нового поколения будут изменены
#Выбираем нейронную сеть с лучшим фитнессом
#Вначале находим индекс с максимальным значением фитнесса
fitness_sort = np.sort(fitness_pai)[::-1]
bestindex_mas = np.zeros([elite_amount])
for eai in range(elite_amount):
    for i in range(population_amount):
        if (fitness_sort[eai] == fitness_pai[i]):
            bestindex_mas [eai] = i

#Элитное множество особей нового поколения оставляем без изменений
best_index = 0
for eai in range(elite_amount):
    best_index = int (bestindex_mas[eai])
    bestindex_historyA[best_index] = bestindex_historyA[best_index] + 1
    q_network_pop_next0[eai] = q_network_pop0[best_index]
    q_network_pop_next1[eai] = q_network_pop1[best_index]

#Скрещиваем некоторое число нейронных сетей
#Вначале выбираем кандидатов для скрещивания и создаем случайные
#индексы
index_rand0 = np.zeros([crossover_amount])
index_rand1 = np.zeros([crossover_amount])
for i in range(crossover_amount):
    index_rand0[i] = np.random.randint(elite_amount,
                                       population_amount)
    index_rand1[i] = np.random.randint(elite_amount,
                                       population_amount)

#Скрещиваем особей с индексом от elite_amount до mutant_amount
for i in range(crossover_amount):

    ind0 = int (index_rand0[i])
    ind1 = int (index_rand1[i])

    #Заполняем веса нулевого уровня первой сети
    rand_1_0 = np.random.choice (layer_choice)
    if (rand_1_0==0):
        q_network_pop_next0[ind0].Qlayers.fl1.weight =
         q_network_pop0[ind0].Qlayers.fl1.weight
    elif (rand_1_0==1):
        q_network_pop_next0[ind0].Qlayers.fl1.weight =
         q_network_pop1[ind1].Qlayers.fl1.weight

    #Заполняем веса второго уровня первой сети
    rand_1_2 = np.random.choice (layer_choice)
    if (rand_1_2==0):
        q_network_pop_next0[ind0].Qlayers.fl2.weight =
         q_network_pop0[ind0].Qlayers.fl2.weight
    elif (rand_1_2==1):
        q_network_pop_next0[ind0].Qlayers.fl2.weight =
         q_network_pop1[ind1].Qlayers.fl2.weight
```

```
            #Заполняем веса нулевого уровня второй сети
            rand_2_0 = np.random.choice (layer_choice)
            if (rand_2_0==0):
                q_network_pop_next1[ind1].Qlayers.fl1.weight =
                 q_network_pop0[ind0].Qlayers.fl1.weight
            elif (rand_2_0==1):
                q_network_pop_next1[ind1].Qlayers.fl1.weight =
                 q_network_pop1[ind1].Qlayers.fl1.weight

            #Заполняем веса второго уровня второй сети
            rand_2_2 = np.random.choice (layer_choice)
            if (rand_2_2==0):
                q_network_pop_next1[ind1].Qlayers.fl2.weight =
                 q_network_pop0[ind0].Qlayers.fl2.weight
            elif (rand_2_2==1):
                q_network_pop_next1[ind1].Qlayers.fl2.weight =
                 q_network_pop1[ind1].Qlayers.fl2.weight

        #Мутируем некоторое число нейронных сетей
        #Вначале выбираем кандидатов для мутирования и создаем случайные
        #индексы
        index_rand0 = np.zeros([mutant_amount])
        index_rand1 = np.zeros([mutant_amount])
        for i in range(mutant_amount):
            index_rand0[i] = np.random.randint(elite_amount,
                                               population_amount)
            index_rand1[i] = np.random.randint(elite_amount,
                                               population_amount)

        #Скрещиваем особей с индексом от elite_amount до mutant_amount
        k=elite_amount
        for i in range(mutant_amount):
            #Добавляем шум в веса нейронных сетей первого агента
            ind0 = int (index_rand0[i])
            for par in q_network_pop0[ind0].parameters():
                noise = np.random.normal(size=par.data.size())
                noiseT = torch.FloatTensor(noise)
                par.data += noise_level * noiseT
            #Передаем мутированных особей новому поколению первого агента
            q_network_pop_next0[k] = q_network_pop0[ind0]

            #Добавляем шум в веса нейронных сетей второго агента
            ind1 = int (index_rand1[i])
            for par in q_network_pop1[ind1].parameters():
                noise = np.random.normal(size=par.data.size())
                noiseT = torch.FloatTensor(noise)
                par.data += noise_level * noiseT
            #Передаем мутированных особей новому поколению второго агента
            q_network_pop_next1[k] = q_network_pop1[ind1]
            #Увеличивыаем счетчик
            k += 1

        ################_конец обучения_#####################################

    ################_конец цикла по эпизодам игры_############################

    #Закрываем среду StarCraft II
    env.close()

    #Сохраняем параметры лучшей нейронной сети из популяции
    torch.save(q_network_pop_next0[0].state_dict(),"qnet_0.dat")
    torch.save(q_network_pop_next1[0].state_dict(),"qnet_1.dat")
```

```
    #Выводим на печать графики
    #Средняя награда
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(reward_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Количество награды за эпизод')
    plt.show()
    #Процент побед
    plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
               edgecolor='k')
    plt.plot(winrate_history)
    plt.xlabel('Номер эпизода')
    plt.ylabel('Процент побед')
    plt.show()

    #Частота использования определенной нейронной сети
    for pai in range(population_amount):
        bestindex_historyS.append(bestindex_historyA[pai])

    nnpops = [f"{i}" for i in range(population_amount)]
    plt.figure(num=None, figsize=(15, 9), dpi=150, facecolor='w',
               edgecolor='k')
    plt.bar(nnpops, bestindex_historyS)
    plt.xlabel('Популяция нейронных сетей')
    plt.ylabel('Частота попадания в элиту')
    plt.show()

#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    #Время обучения
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

4.7. Эксперимент

4.7.1. Алгоритм InGA

На рис. 4.10 и 4.11 представлены результаты обучения двух независимых агентов с помощью алгоритма InGA при заданных параметрах обучения $\delta = 0.01$, $\mathcal{M} = 10$, $Ea = 1$ на 10 поколениях популяций из 50 агентов. Все агенты успешно выучивают стратегию «бежать вправо вверх и атаковать противника», демонстрируя правильное кооперативное поведение. Процент побед в процессе обучения скачкообразно увеличивается. Число разнообразных нейронных сетей, отбираемых в следующее поколение, не очень велико, однако это связано с небольшим числом и поколений, используемых для обучения и элитных нейронных сетей *Ea*. Визуализация процесса игры также демонстрирует постепенное приближение агентов к оптимальным действиям и итоговое их успешное использование (см. рис. 4.9).

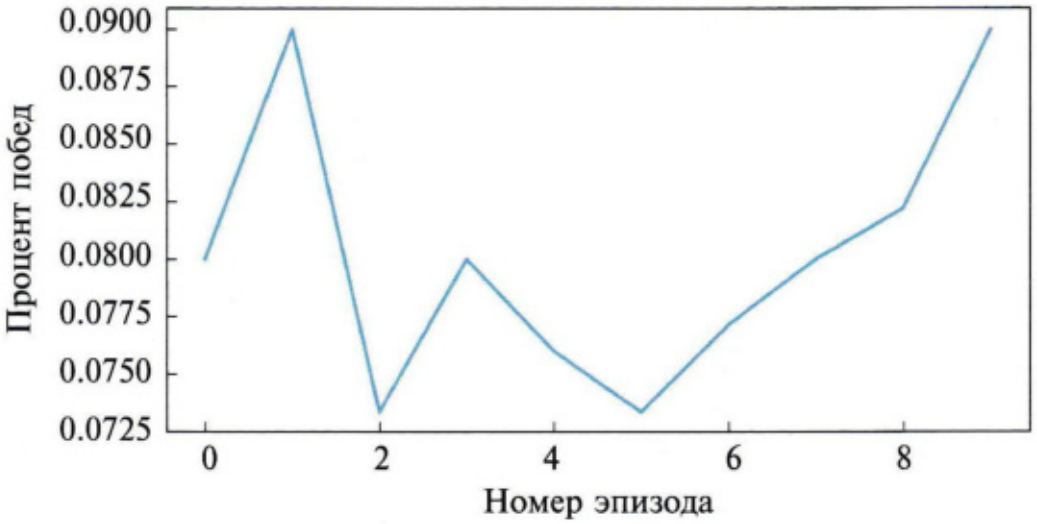


Рис. 4.10. Процент побед при обучении двух независимых агентов с помощью алгоритма InGA в среде SMAC

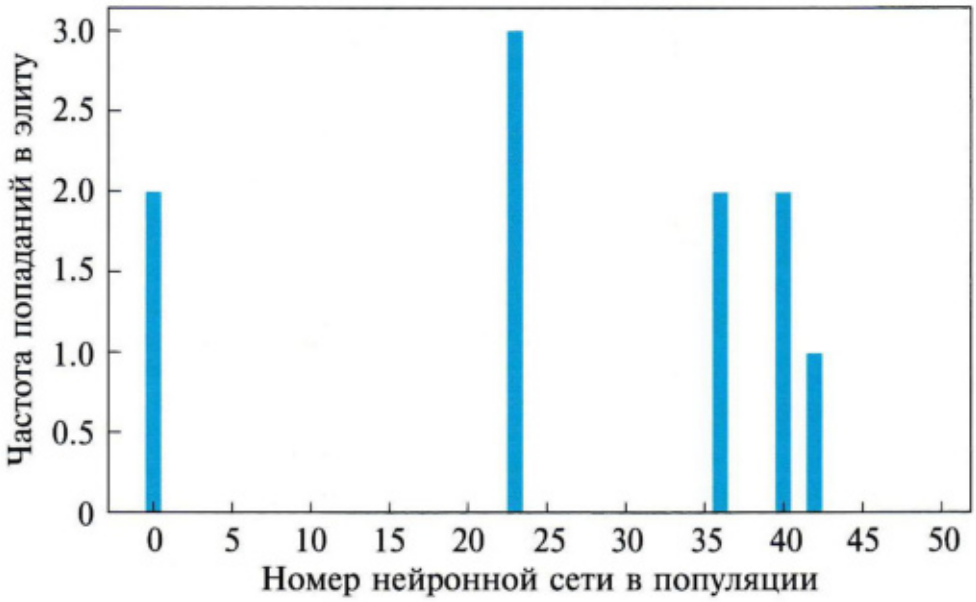


Рис. 4.11. Частота попадания в элиту нейронных сетей из популяции при обучении двух независимых агентов с помощью алгоритма InGA в среде SMAC

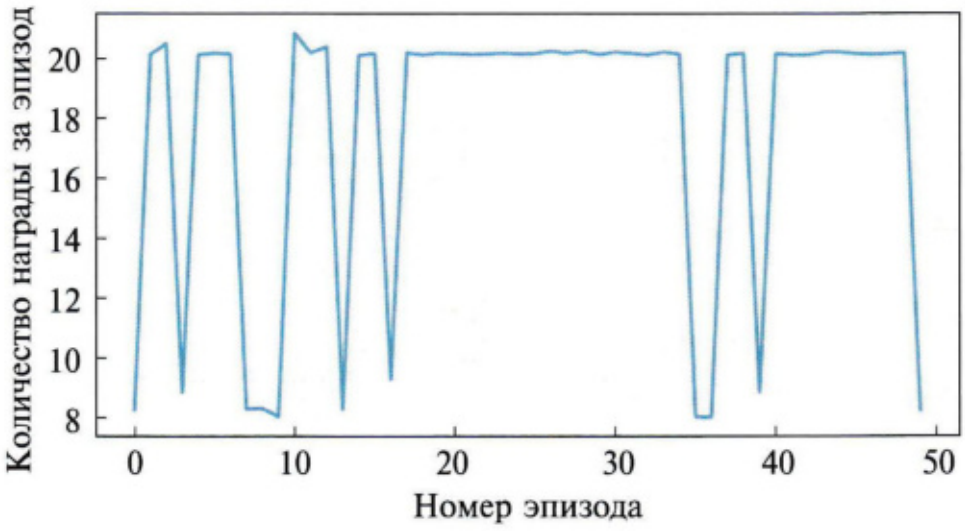


Рис. 4.12. Результаты тестирования алгоритма InGA в среде SMAC

На рис. 4.12 представлены результаты тестирования алгоритма InGA. Тестирование проводилась при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 75 %, среднее количество награды — 16.31.

4.7.2. Алгоритм CoE

На рис. 4.13 и 4.14 представлены результаты обучения двух агентов с помощью коэволюционного алгоритма CoE при заданных параметрах обучения $\delta = 0.01$, $\mathcal{M} = 10$, $Ea = 10$, $Ca = 10$ на 50 поколениях популяций из 50 агентов. Процент побед в процессе обучения практически постоянно монотонно увеличивается. Число разнообразных нейронных сетей, отбираемых в следующее поколение, равномерно распределено. Номера первых 10 нейронных

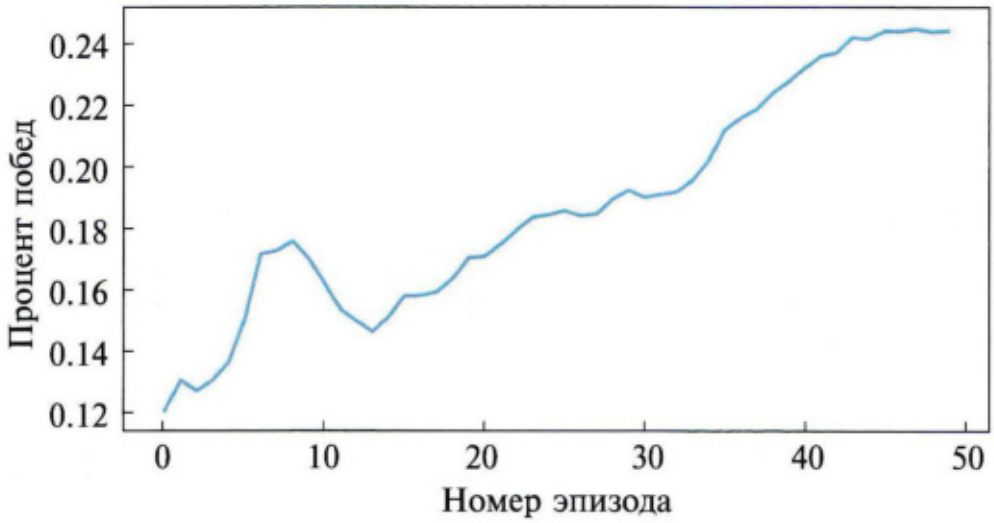


Рис. 4.13. Процент побед при обучении двух агентов с помощью алгоритма CoE в среде SMAC

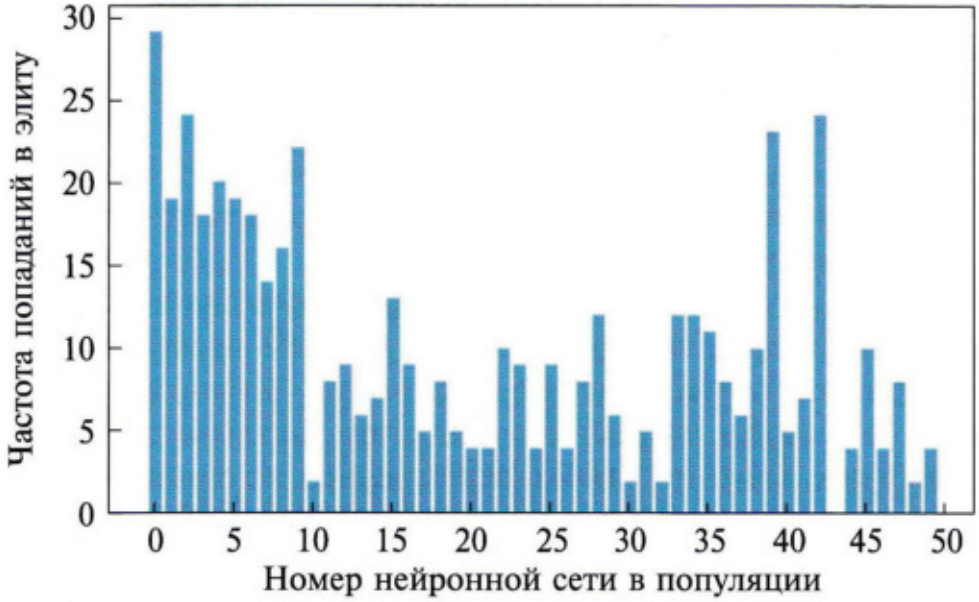


Рис. 4.14. Частота попадания в элиту нейронных сетей из популяции при обучении двух агентов с помощью алгоритма CoE в среде SMAC

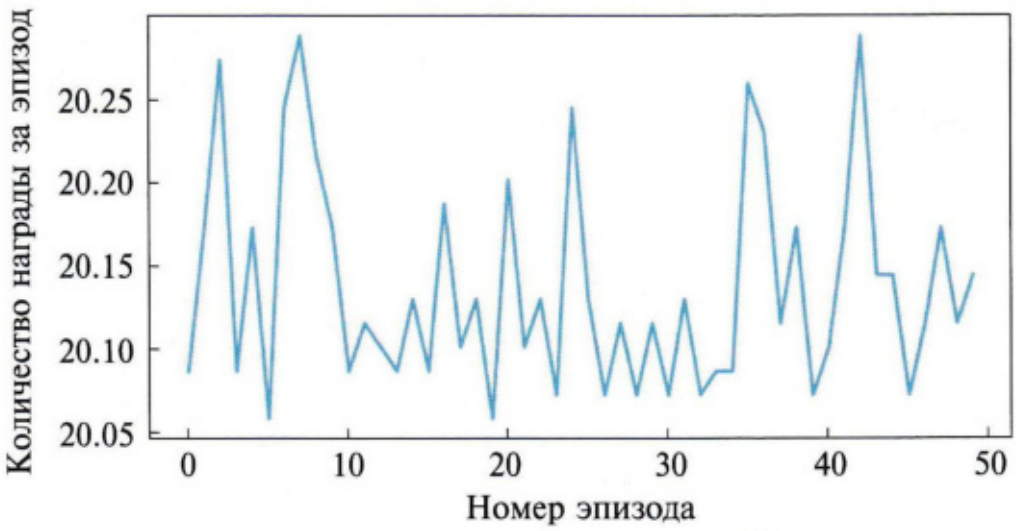


Рис. 4.15. Результаты тестирования алгоритма CoE в среде SMAC

сетей, имеющих отличие в отборе, соответствуют управляющему параметру обучения *Ea*. Визуализация процесса игры демонстрирует также корректные действия агентов.

На рис. 4.15 представлены результаты тестирования алгоритма CoE.

Тестирование проводилась при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 100 %, среднее количество награды — 20.13.

4.8. Выводы

В гл. 4 были рассмотрены алгоритмы мультиагентного эволюционного обучения InGA и CoE. Алгоритм InGA является упрощенной версией генетического алгоритма, адаптированного для мультиагентного обучения, и базируется на трех основных принципах эволюционного обучения: элитизма, мутации, безградиентного подхода. Коэволюционный алгоритм CoE, основанный на алгоритме InGA, по сути мультиагентен и реализует идею совместной эволюции одновременно обучающихся агентов на основе принципов скрещивания и единой награды.

Основным преимуществом эволюционного обучения по сравнению с современным лидирующим нейросетевым обучением является отказ в большинстве алгоритмов от вычисления градиентов. Это позволяет экономить вычислительные ресурсы, эффективнее избегать локального минимума, находить нестандартные решения и гибко переключаться с одного перспективного решения на другое в пространстве поиска. Возможно, в будущих гибридных алгоритмах на первых этапах мультиагентного обучения будут использоваться именно эволюционные принципы, для того чтобы моментально оценить все пространство возможных решений и лишь затем применять нейросетевые алгоритмы для окончательного нахождения оптимальной стратегии агентов.

Главным недостатком эволюционного обучения является отсутствие теоретических гарантий при решении задачи определенной вычислительной сложности. Так, алгоритмы Q-обучения позволяют гарантированно находить оптимальную стратегию для заданного марковского процесса принятия решений за полиномиальное время относительно числа состояний и действий МППР. Однако эволюционным алгоритмам в худшем случае может понадобиться обработать экспоненциальное количество возможных стратегий, прежде чем найти оптимальное решение.

В следующей, заключительной, главе рассматривается мультиагентное обучение, основанное на принципах роевого интеллекта, которое часто ошибочно относят к эволюционному обучению, имея в виду их схожие, но концептуально различные модели, вдохновленные живой природой.

4.9. Задачи для самоконтроля

1. Проведите дизайн наград и разработайте новую функцию приспособленности в алгоритме InGA для предотвращения прорыва зергов в контрольную точку Attack Point на карте 2p4zFOX.SC2Map. Что нужно изменить в функции приспособленности, для того чтобы агенты воспользовались свойством местности и получили тактическое преимущество?

2. Реализуйте методы выбора элитных агентов для следующего поколения: турнирный метод, метод рулетки, сигма-отсечение в алгоритме InGA. Сравните эффективность эволюционного обучения с использованием этих методов выбора элитных агентов.

3. Реализуйте методы выбора агентов-родителей — инбридинг и аутбридинг — в коэволюционном алгоритме CoE. Сравните эффективность коэволюционного обучения с использованием этих методов.

4. Реализуйте алгоритм эволюционных стратегий (*англ.* Evolution Strategies, ES) компании OpenAI:

1. Инициализировать $\alpha \in (0,1)$; $\sigma \in (0,1)$; $\theta \leftarrow \xi$.
2. Цикл $t = 0, 1, 2, \ldots$
3. Получить $\epsilon_0, \ldots, \epsilon_n \sim \mathcal{N}(0,1)$.
4. Вычислить $\mathcal{F}_i = \mathcal{F}(\theta_t + \sigma\epsilon_i)$ для $i = 1, \ldots, n$.
5. Обновить $\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{n\sigma} \sum_{i=1}^{n} \mathcal{F}_i \epsilon_i$.

Адаптируйте алгоритм ES для мультиагентного эволюционного обучения и сравните его эффективность на карте 2p4zFOX.SC2Map с эффективностью алгоритмов InGA и CoE.

5. Разработайте описание игры и реализуйте мир-сетку «Лабиринт» 100×100 (рис. 4.16). Обучите популяцию агента достигать цели с помощью эволюционных алгоритмов ES, InGA и нейросетевого алгоритма IQL. Где в мире-сетке располагаются клетки локального минимума? Обученный каким алгоритмом агент способен доходить до цели наиболее быстро?

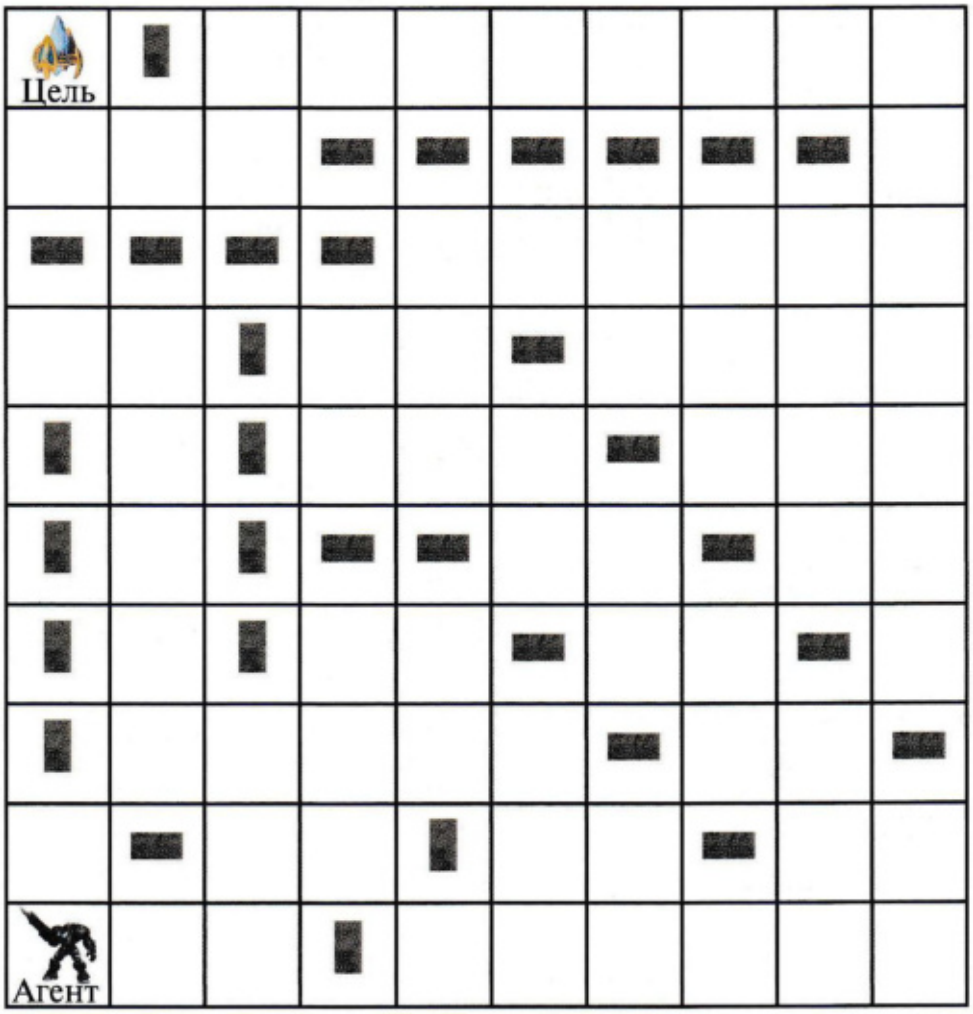


Рис. 4.16. Мир-сетка «Лабиринт» в масштабе 1:10

6. Разработайте описание игры и реализуйте мир-сетку «Крест навыков» (рис. 4.17). В каждой клетке *R*? агент либо получает, либо не получает награду. Для преодоления клетки Sk_i агент должен выучить один из навыков № 1–4. В каждый момент времени агент способен обладать только одним навыком. Обучите популяцию агентов достигать цели с помощью эволюционных алгоритмов ES, InGA и нейросетевого алгоритма IQL. В чем заключается проблема «катастрофической забывчивости» агента? Обученный каким алгоритмом агент способен регулярно получать максимальное количество награды?

7. Разработайте описание и реализуйте мир-сетку «Хищники-жертва» 100×100 (рис. 4.18). Жертва двигается немного быстрее хищников, и для того чтобы поймать жертву (Caught = True), хищники должны действовать кооперативно. Обучите агентов-хищников достигать цели с помощью кэволюционного алгоритма CoE, используя функцию приспособленности $\mathcal{F}(\theta_i)$:

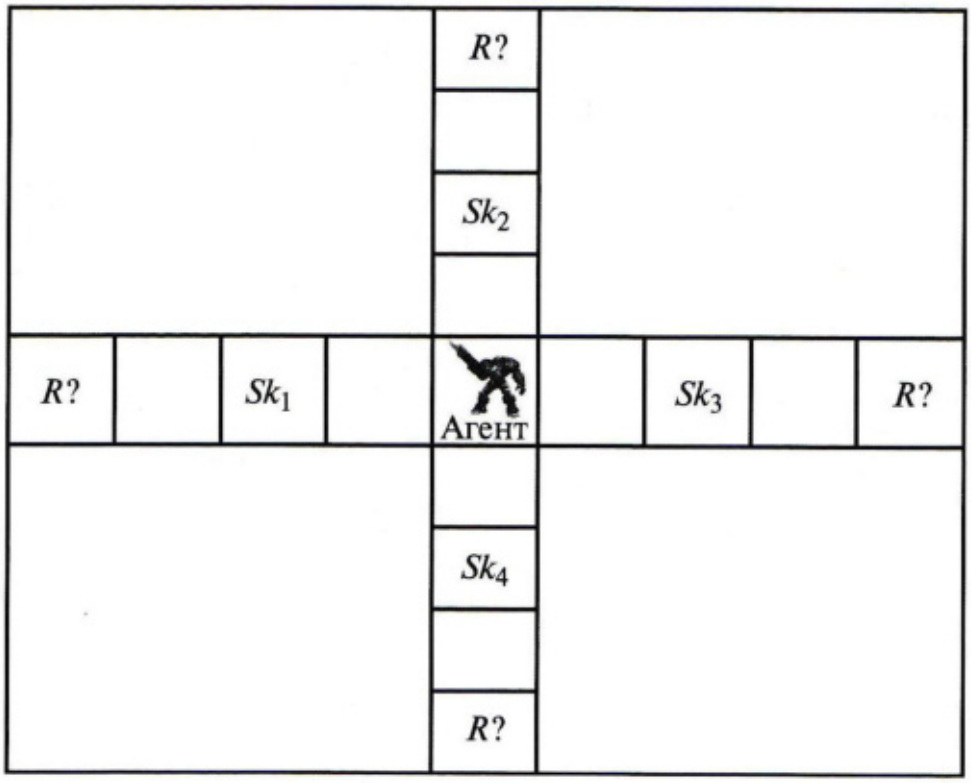


Рис. 4.17. Мир-сетка «Крест навыков»

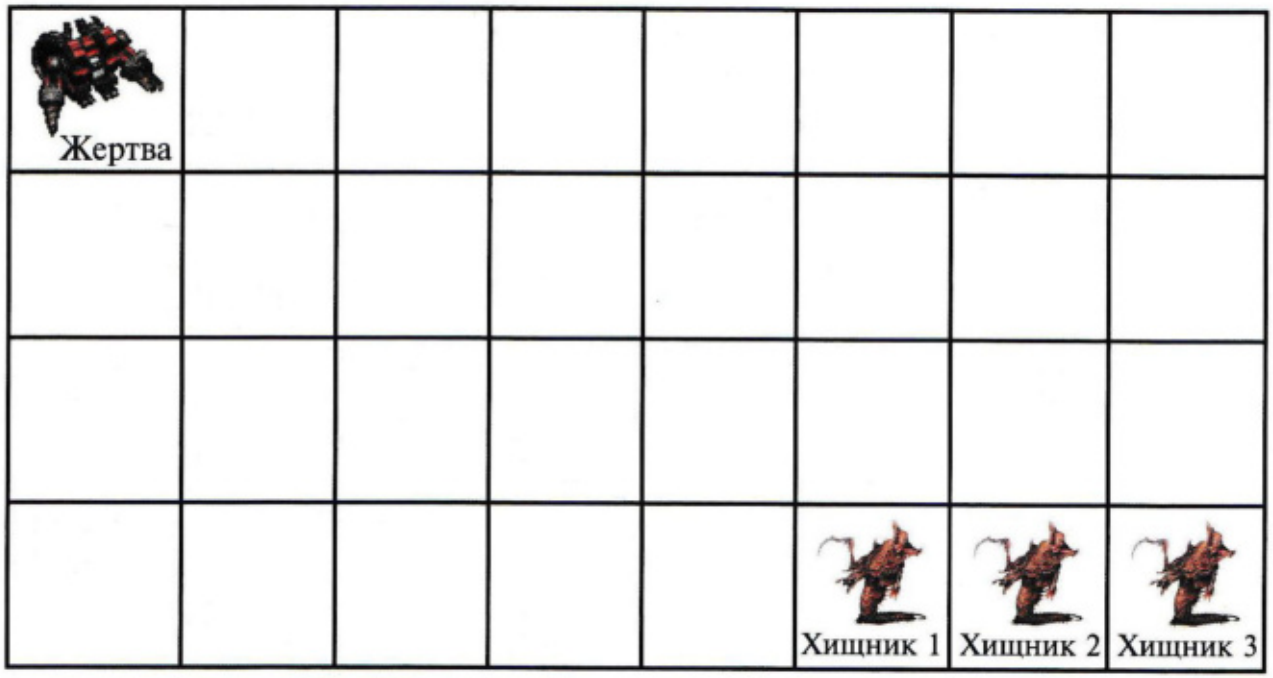


Рис. 4.18. Мир-сетка «Хищники-жертва» (в масштабе)

$$\mathcal{F}(\theta_i) = \begin{cases} \dfrac{d0 - de}{10}, & \text{если Caught} = \text{False}; \\ \dfrac{200 - de}{10}, & \text{если Caught} = \text{True}. \end{cases}$$

где $d0$ — среднее начальное расстояние до жертвы; de — среднее финальное расстояние до жертвы.

8. Разработайте описание игры и реализуйте мир-сетку «Пастухи» 37×37 (рис. 4.19). Агенты должны научиться координировать действия для перемещения защищаемого объекта в целевую клетку без столкновения с противником, который пытается навредить объекту и при этом избежать встречи

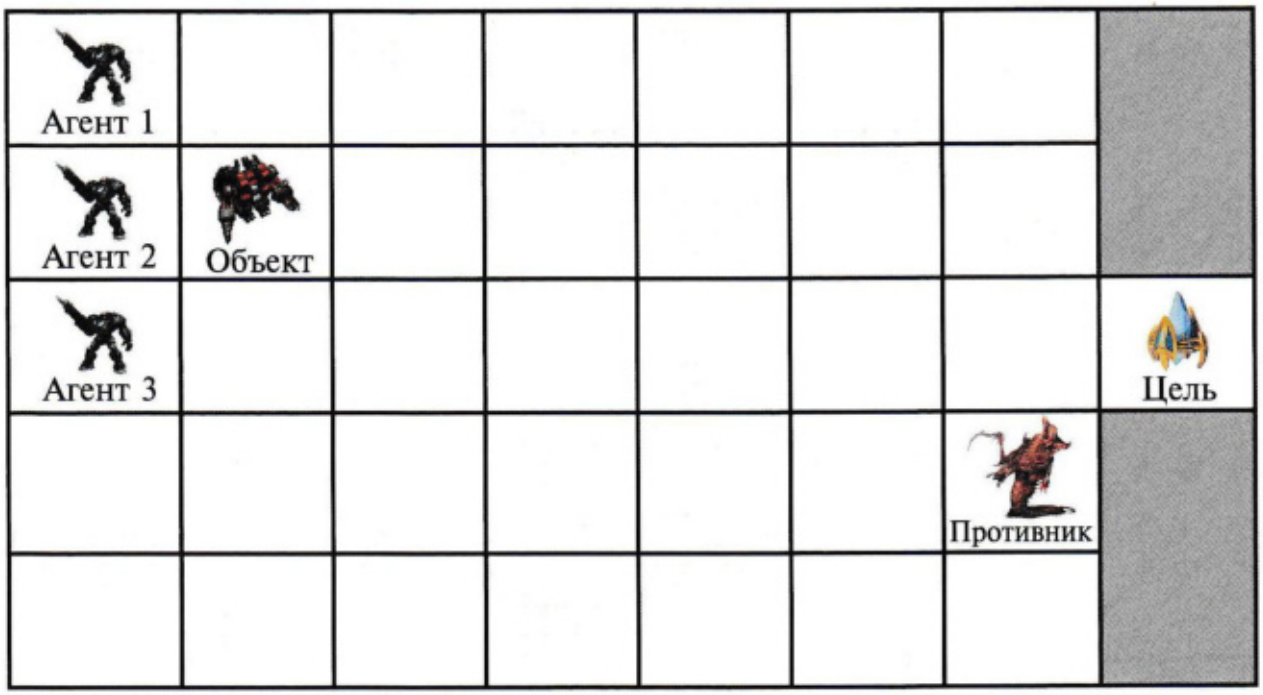


Рис. 4.19. Мир-сетка «Пастухи» (в масштабе)

с агентами. Обучите агентов с помощью коэволюционного алгоритма CoE, используя функцию приспособленности $\mathcal{F}(\theta_i)$:

$$\mathcal{F}(\theta_i)=\begin{cases}2-\dfrac{\tau}{T}, & \text{если Goal} = \text{True};\\ \max\left(0{,}1-\dfrac{de}{d0}\right), & \text{если Goal} = \text{False},\end{cases}$$

где τ — число сделанных шагов до достижения целевой клетки; T — максимальное число шагов в эпизоде; de — финальное расстояние от объекта до целевой клетки; $d0$ — начальное расстояние от объекта до целевой клетки.

9. Найдите в Интернете описание алгоритма MERL и реализуйте его. Сравните его эффективность с эффективностью алгоритмов InGA и CoE на карте 2p4zFOX.SC2Map.

10. Разработайте сценарий, карту и оригинального юнита компьютерной игры StarCraft II для полного раскрытия возможностей алгоритма мультиагентного коэволюционного обучения CoE.

Глава 5

Роевое обучение

Рой должен быть готов к подчинению
и должен верить в наши методы, какими бы те ни были...
в любом случае, чтобы объединить рой,
надо указать всем на общего врага.
Юн Ха Ли. Гамбит девятихвостого лиса

5.1. Классификация

Понятие *роевого интеллекта* (*англ.* swarm intelligence) описывает коллективное поведение децентрализованной самоорганизующейся системы, которая состоит из множества простых гомогенных агентов, локально взаимодействующих друг с другом и с окружающей средой. Локальные и зачастую случайные взаимодействия приводят к возникновению интеллектуального группового поведения, не контролируемого отдельными агентами.

Роевое обучение — это совокупность алгоритмов машинного обучения, вдохновленных живой природой и нацеленных на обучение роя агентов. В отличие от классического обучения с подкреплением — обучения индивидуальных агентов сложным стратегиям, — при роевом обучении, как и при кооперативном мультиагентном, можно одновременно обучать множество агентов умению находить и выполнять простые стратегии. Роевое обучение также применяется для обучения роя роботов и роя сенсоров.

В табл. 5.1 представлены основные качественные различия в формировании роя мультиагентной системы, роботов и сенсоров по следующим критериям: размер роя; способ управления роем; однородность агентов в рое; масштабируемость роя; тип среды, в которой обычно обучается рой; способность агентов роя к перемещению в среде; области применения систем.

Главной характеристикой роя, определяющей его ключевые свойства, является размер.Однако неопределено,при каком числе агентов обычная мультиагентная система превращается в рой агентов. Размер роя всегда зависит от решаемой задачи и архитектуры агентов. Так, 10 плавающих роботов, которые исследуют морскую территорию, или 20 роботов-зооидов, обеспечивающих человеко-компьютерное взаимодействие, уже называют роем роботов. При этом самый большой на данный момент рой роботов, разработанный компанией Amazon для складских помещений, включает 100 тыс. роботов Kiva. В природе рой мух имеет до 200 особей, рой пчел

в среднем начитывает 100 тыс. особей, а рой муравьев — до 1 млн особей. Самым большим считается рой кузнечиков, для его измерения используют значение массы роя на занимаемой роем местности — до 500 т кузнечиков на 1 км^2. Максимальный рой кузнечиков, который был обнаружен, весил около 100 тыс. т и занимал площадь 1000 км^2.

Таблица 5.1

Качественные различия в формировании роя

Характеристики роя	Интеллектуальные системы		
	Мультиагентная система	Рой роботов	Рой сенсоров
Размер	Средний	Маленький	Большой
Способ управления	Централизованное, децентрализованное, автономное	Децентрализованное, автономное	Централизованное, автономное
Однородность агентов	Гомогенные, гетерогенные	Гомогенные	Гомогенные
Масштабируемость	Средняя	Низкая	Высокая
Тип среды	Известная, неизвестная	Известная, неизвестная	Известная
Динамичность	Средняя	Высокая	Низкая
Область применения	Компьютерные игры, менеджмент ресурсов, распределенный контроль	Аварийные, чрезвычайные и специальные ситуации	Распределенный контроль, медицина, экология

Непревзойденное совершенство живой природы мотивирует разработчиков алгоритмов учиться у нее и пытаться копировать ее свойства. В алгоритмах, вдохновленных природой, для решения задач оптимизации применяется эвристика, основанная на поведении живых организмов, чаще всего насекомых.

Муравьи, например, используют окружающую среду как средство коммуникации. Они обмениваются информацией косвенным путем, оставляя на пути своего движения продукты внешней секреции — феромоны. Обмен информацией в муравьином рое носит локальный характер: только муравьи, которые находятся в непосредственной близости от феромоновых троп, могут получить информацию о них. При нахождении пищи муравьи могут оставлять больше феромонов, чем обычно, что неявно указывает рою на ценность того или иного пути. В то же время автоматическое испарение феромона, без присутствия дополнительного подкрепляющего сигнала в виде феромонов других муравьев, помогает рою «забыть» о неудачных или слишком сложных путях и решениях. Такая эвристика называется *стигмергией* (от *греч.* stigma — метка, ergon — действие), и именно на ней базируется алгоритм муравьиной оптимизации DRACO, который будет рассмотрен

в этой главе применительно к мультиагентному роевому обучению. Другие популярные алгоритмы роевого обучения, которые могут быть использованы для мультиагентного роевого обучения, представлены в табл. 5.2.

В первую группу алгоритмов (см. табл. 5.2) входят классические алгоритмы роевого обучения, вдохновленные живой природой и копирующие особенности поведения роев живых организмов. Таких алгоритмов разработано более ста, наиболее представительными среди них являются следующие: алгоритм ACO (*англ.* Ant Colony Optimization), моделирующий стигмергию муравьиной колонии; алгоритм ABC (*англ.* Artificial Bee Colony), в котором вместо феромоновой эвристики используется собирательное поведение пчелиной колонии; алгоритм PSO (*англ.* Particle Swarm Optimization), моделирующий движение организмов в группе при поиске пищи; алгоритм EHO (*англ.* Elephant Herding Optimization), описывающий сложные эмоциональные и поведенческие отношения в стаде слонов; алгоритм FA (*англ.* Firefly Algorithm), имитирующий характеристики мигающего поведения тропических светлячков; алгоритм BFO (*англ.* Bacterial Foraging Optimization), основанный на пищевых моделях условно-патогенных бактерий — кишечных палочек — и специально ориентированный на решение задачи эксплуатации-исследования в процессе обучения.

Таблица 5.2

Классификация алгоритмов мультиагентного роевого обучения

Принцип обучения с подкреплением	Тип алгоритма		
	Вдохновленные живой природой	Глубокое обучение с подкреплением	Динамические модели формаций
Вычисление ценности состояний-действий	ACO ABC PSO EHO FA BFO	–	–
Вычисление стратегий	–	DIMAPG ME-TRPO	MAF MAGF

Все алгоритмы первой группы изначально были созданы для табличных миров-сеток или графовых моделей, существование которых в реальном мире требует слишком больших допущений и абстракций. В связи с этим в последние годы исследователи сосредоточили усилия на адаптации новейшей схемы глубокого обучения с подкреплением для мультиагентного роевого обучения. В алгоритме DIMAPG (*англ.* Distributed Multi Agent Policy Gradient) для гомогенного кооперативного обучения от 3 до 630 агентов в мультиагентной задаче «Хищники-жертвы» используется централизованная стратегия, позволяющая учесть способность мультиагентной системы к гибкому масштабированию.

В алгоритме ME-TRPO (*англ.* Mean Embedding-Trust Region Policy Optimization) использованы известный в одноагентном обучении с подкреплением алгоритм TRPO и алгоритм компактного представления состояний среды в виде числовых векторов — эмбеддингов — для гомогенного кооперативного обучения 20 агентов в традиционных мультиагентных задачах «Рандеву» и «Защита территории». В итоге алгоритм ME-TRPO позволил получить довольно неожиданный практический результат, который состоит в том, что задача мультиагентного роевого обучения по существу сводится к задаче представления состояний среды для роя агентов или даже к псевдоподсчету состояний, т. е. к измерению не состояний, а визуальной плотности среды или к хэшированию состояний. Алгоритмы DIMAPG и ME-TRPO образуют вторую группу (см. табл. 5.2).

Третью группу составляют алгоритмы, основанные на динамических моделях и посвященные интересному направлению роевого обучения — образованию формаций. *Формация* — это геометрическая форма в двух- или трехмерном пространстве, всеми агентами-участниками составленная и сохраняемая ими при передвижении. В более общем случае — это совокупность определенных ограничений на топологии состояний агентов. Образование формации позволяет эффективно задействовать гетерогенные, но простейшие умения всех агентов роя и имеет большое прикладное значение при обучении роя роботов согласованно действовать в чрезвычайных ситуациях. Так, в алгоритме MAF (*англ.* Multiple Agent Formation) для создания треугольной формации применяются как физические агенты в среде, так и виртуальные агенты, помогающие вычислять центр тяжести формации при движении агентов.

В алгоритме MAGF (*англ.* Multi-Agent Group Formation) для создания шестиугольной формации использована динамическая модель управления, позволяющая решить проблему консенсуса. Проблема консенсуса — это фундаментальная проблема управления мультиагентными системами, которая заключается в согласовании всеми агентами некоторого значения данных, необходимого в процессе вычислений. Безмодельный подход к формированию консенсуса состоит в том, что все агенты должны согласовать значение данных, принятое большинством агентов.

5.2. Модель

5.2.1. Комбинаторная оптимизация

Задача комбинаторной оптимизации (*англ.* combinatorial optimization problem) представляет собой кортеж $\mathcal{M} = (S, \Omega, f)$, где S — пространство поиска, состоящее из конечного множества переменных решения X_p, $p = 1, ..., n$, каждая из которых имеет область значений D_p; Ω — множество ограничений

для переменных решения X_p; $f : S \to \mathbb{R}_0^+$ — целевая функция, которую необходимо минимизировать.

Подстановка значения в переменную X_p заключается в присвоении некоторого значения $v_p^l \in D_p$ переменной X_p, т. е. $X_p = v_p^l$. *Допустимое* решение $s \in S$ задачи комбинаторной оптимизации $\mathcal{M}$ состоит в подстановке в каждую переменную значения v_p^l из области значений D_p таким образом, чтобы выполнялись все ограничения Ω. Если множество Ω пустое, то задача комбинаторной оптимизации $\mathcal{M}$ не ограничена и каждая переменная X_p может независимо иметь любое значение $v_p^l \in D_p$. В этом случае задача комбинаторной оптимизации $\mathcal{M}$ называется неограниченной, в обратном случае — ограниченной.

Допустимое решение $s^* \in S$ задачи комбинаторной оптимизации $\mathcal{M}$ называется глобальным минимумом, если и только если $f(s^*) \le f(s)$, $\forall s \in S$. При этом множество всех глобальных минимумов $S^* \in s$.

Для нахождения оптимального решения s^* задачи комбинаторной оптимизации $\mathcal{M}$ могут применяться различные алгоритмы поиска решения на графах, но для роевого обучения наиболее важным является алгоритм оптимизации подражанием муравьиной колонии ACO.

В классическом алгоритме ACO искусственный муравей (*англ.* ant) — это простой, независимый, асинхронный агент, который вместе с другими агентами ищет оптимальное решение s^* задачи комбинаторной оптимизации $\mathcal{M}$. Инстанцирование переменной X_p некоторым значением $v_p^l \in D_p$ назовем компонентом решения и введем обозначение $c_p^l \in C$, где C — множество всех компонентов решений. Муравьи должны правильно скомбинировать компоненты решения c_p^l, чтобы найти допустимое решение $s \in S$. Для этого каждому компоненту решения c_p^l ставится в соответствие переменная феромона $T_{pl} \in T$, где T — множество всех переменных феромона. Каждая переменная феромона T_{pl} имеет значение феромона τ_{pl}, которое показывает желательность выбора компонента решения c_p^l. Переменные феромона T_{pl} зависят от времени и поэтому являются функциями, зависящими от итерации алгоритма t.

В алгоритме ACO множество m муравьев начинает искать допустимые решения $s \in S$ задачи комбинаторной оптимизации $\mathcal{M}$ с изначально пустым множеством решений каждого муравья $s_i = 0$. На каждом шаге муравей расширяет свое множество решений s_i путем выбора одного компонента решения $c_p^l \in N(s_i) \subseteq C$, где $N(s_i)$ — множество компонентов решения, которые могут быть использованы с учетом сохранения допустимости получаемого решения $s \in S$. Обычно $N(s_i)$ определяется неявно непосредственно в процессе построения решения роем муравьев.

Выбор компонента решения c_p^l является стохастическим и основывается на следующем правиле:

$$p\left(c_p^l|s_i\right)=\frac{\tau_{pl}^{\alpha}\left[\eta\left(c_p^l\right)\right]^{\beta}}{\sum_{c_p^k\in N(s_i)}\tau_{pk}^{\alpha}\left[\eta\left(c_p^k\right)\right]^{\beta}},\quad \forall c_p^l\in N\left(s_i\right),$$

где $\eta(\cdot)$ — функция, которая ставит в соответствие каждому допустимому компоненту решения $c_p^l \in N\left(s_i\right)$ некоторое эвристическое значение, которое обычно называют эвристической информацией.

Так, для некоторого графа функция $\eta(u,v)$ может возвращать эвристическое значение видимости пути (u,v), обычно принимаемое для графовых моделей с длиной пути $d(u,v)$, равным $1/d(u,v)$. Параметры α и β определяют степень влияния феромонов и эвристической информации на работу алгоритма ACO. Если $\alpha = 0$, то вероятность выбора компонента решения c_p^l пропорциональна коэффициенту $\left[\eta\left(c_p^l\right)\right]^{\beta}$ и компонент решения c_p^l с наибольшим эвристическим значением будет выбран, что соответствует «жадному» поведению алгоритма. Если $\beta = 0$, то на вероятность выбора компонента решения c_p^l влияют только значения феромонов.

Еще одним ключевым правилом в алгоритме ACO является правило обновления феромонов:

$$\tau_{pl}=\rho\tau_{pl}+\sum_{s\in S_{upd}|c_p^l\in s} g(s),$$

где $\rho \in (0,1]$ — параметр, определяющий степень испарения феромона; S_{upd} — множество решений, которые были использованы для размещения феромона; функция оценки $g(\cdot): S\to\mathbb{R}^+$ такая, что $f(s)<f(s')\Rightarrow g(s)\geq g(s')$, определяет качество найденного решения s. Для упрощения функция g(·) может быть заменена на некоторое статическое значение уровня феромона $\Delta\tau_{pl}$. Таким образом, в правиле обновления феромона значение $\rho\tau_{pl}$ будет отвечать за автоматическое уменьшение ценности редких решений, а значение $\Delta\tau_{pl}$ — выступать в качестве подкрепляющего сигнала успешного решения.

Рассмотрим реализацию модели $\mathcal{M}$ на примере задачи коммивояжера (*англ.* Travelling Salesman Problem, TSP). В задаче TSP некоторая проблема реального мира представляется в виде полносвязного взвешенного неориентированного графа $G = (V, E)$, где V — непустое множество вершин графа; E — множество ребер графа (рис. 5.1). Каждому ребру соответствует значение $d(u, v)$, которое представляет собой длину пути из вершины u в вершину v, или стоимость перемещения по ребру $(u, v) \in E$. Для решения задачи TSP необходимо найти минимально взвешенный гамильтонов путь на графе G, т. е. такой циклический путь, который включает каждую вершину графа ровно по одному разу.

Таким образом, в соответствии с моделью $\mathcal{M}$ компоненты решения c_p^l представляют собой перемещения от одной вершины графа к другой (рис. 5.2). Каждая переменная решения X_p соответствует некоторой вершине

графа и может иметь $n - 1$ значений v_p^l, $l = 1, ..., n$; $l \neq p$. Каждому ребру графа ставится в соответствие значение феромона τ_{pl} (рис. 5.3). Компонент решения $c_p^l \in N(s_i) \subseteq C$, возникающий при инстанцировании переменной X_p некоторым значением $v_p^l \in D_p$, будет соответствовать допустимому решению $s \in S$ тогда и только тогда, когда множество пройденных вершин формируют гамильтонов путь. При этом целевая функция $f(\cdot)$ позволяет вычислить для каждого допустимого решения s сумму длин пройденных ребер и определить тем самым длину гамильтонова пути.

Как следует из рис. 5.1–5.3, в процессе решения задачи TSP с помощью алгоритма ACO взвешенному стоимостному графу ставится в соответствие феромоновый граф, на основании которого муравей будет выбирать путь. Очевидно, что феромоновый граф повторяет идею Q-таблицы, определяя для агента выбор того или иного действия, того или иного пути. Так, используя феромоновый граф на следующей итерации алгоритма ACO, муравей, стартуя из вершины I, выберет с большей вероятностью переходы с максимальными значениями феромонов: I → IV; IV → III; III → II; II → I (см. рис. 5.3). Значения феромонов в графе, вычисленные на основе стоимостного графа и совместного первого прохода муравьев по двум разным гамильтоновым путям:

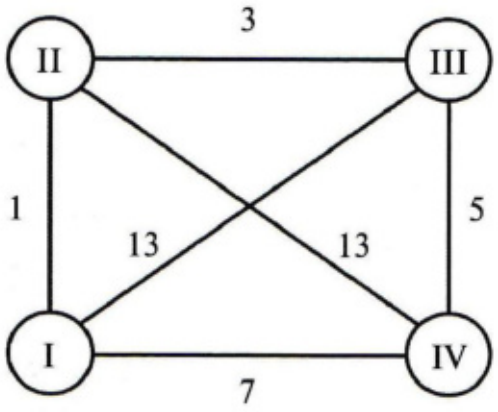


Рис. 5.1. Пример стоимостного графа, моделирующего некоторую проблему реального мира

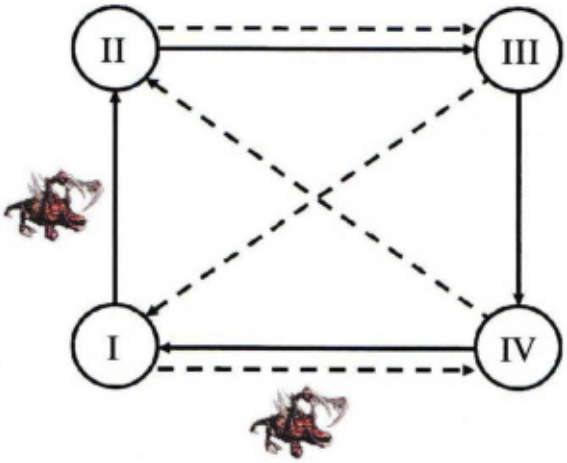


Рис. 5.2. Два разных гамильтоновых пути

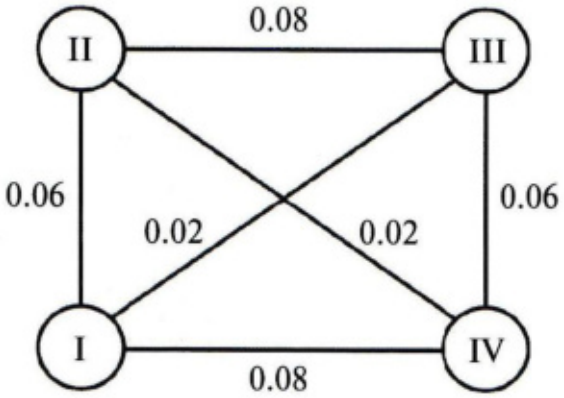


Рис. 5.3. Феромоновый граф

$$Q_1 = (1 + 3 + 5 + 7) = 16;$$

$$Q_2 = (7 + 13 + 3 + 13) = 36;$$

$$\Delta\tau_{(I, II)} = \frac{1}{16} = 0.06;$$

$$\Delta\tau_{(\mathrm{I},\,\mathrm{III})} = \frac{1}{36} = 0.02;$$

$$\Delta\tau_{(\mathrm{II},\,\mathrm{III})} = 0.06 + 0.02 = 0.08;$$

$$\Delta\tau_{(\mathrm{III},\,\mathrm{IV})} = 0.06;$$

$$\Delta\tau_{(\mathrm{IV},\,\mathrm{II})} = 0.02;$$

$$\Delta\tau_{(\mathrm{IV},\,\mathrm{I})} = 0.06 + 0.02 = 0.08.$$

Перейдем от графовой модели роевого обучения к марковской.

5.2.2. Роевая марковская модель принятия решений

Для разработки алгоритма мультиагентного роевого обучения, в котором совмещены преимущества муравьиного алгоритма и глубокого обучения с подкреплением, необходимо от классической модели комбинаторной оптимизации перейти к роевой марковской модели принятия решений.

Роевая марковская модель принятия решений (*англ.* Swarm Markov Decision Process, SMDP), основанная на модели децентрализованных частично наблюдаемых марковских процессов принятия решений (Dec-POMDP), является двухуровневой. На первом, локальном, уровне она определяет *шаблон агента*, задающий локальные свойства агента в рое как кортеж $\mathcal{A} = (S, \mathcal{O}, A, \pi)$, где S — множество локальных состояний агента; $\mathcal{O}$ — множество возможных наблюдений агента; A — множество возможных действий агента; $\pi : \mathcal{O} \times A \to [0,1]$ — стохастическая стратегия агента.

На втором, глобальном, уровне модель SMDP задается как кортеж $(n, \mathcal{A}, T, R, O)$, в котором n — множество агентов в рое; $\mathcal{A}$ — шаблон агента; $T : S_n \times A_n \times S_n \to [0,\infty)$ — глобальная функция переходов; $R : S_n \times A_n \to \mathbb{R}$ — глобальная функция наград; $O : S_n \times \{1, \ldots, n\} \to \mathcal{O}$ — функция наблюдений, которая определяет локальные наблюдения $o_i \in \mathcal{O}$ агента i в состоянии $s \in S_n$, т. е. $o_i = O(s, i)$.

Двухуровневая структура модели SMDP позволяет явно определить гомогенность агентов роя, созданных на базе идентичного шаблона. Кроме того, функция переходов и функция наблюдений инвариантны к замене одних агентов роя на других, что позволяет дополнительно контролировать гомогенность агентов роя. Агенты в модели SMDP имеют только локальные наблюдения о глобальном состоянии среды, что соответствует моделированию большинства практических задач.

В рассматриваемом в разд. 5.3 алгоритме DRACO глобальная функция наград R в модели SMDP заменяется на локальную феромоновую функцию $\mathcal{P}\hbar_i$, которая формируется на основе следующего правила:

$$\mathcal{PH}_i(s_i, a_i) = \rho\tau_{s_i} + r_i.$$

Феромоновая функция $\mathcal{PH}_i$ обеспечивает стигмергию между агентами роя, мотивируя их решать задачу обучения на основе дополнения кооперативной награды среды SMAC r_i значением феромона τ_{s_i} в состоянии s_i, в котором находится агент i. Автоматическое испарение феромона τ_{s_i} контролируется с помощью коэффициента $\rho \in (0,1]$, применяемого на каждой итерации игры, само значение феромона задается в виде двухмерной таблицы (визуализация — см. разд. 5.4) и вычисляется по формуле

$$\tau_{s_i} = \begin{cases} \Delta\tau_1, \text{ если } s(x,y)_i = \text{True}; \\ \Delta\tau_2, \text{ если } a_i = \text{Attack}, \end{cases}$$

где $\Delta\tau_1$, $\Delta\tau_2$ — коэффициенты, определяющие уровень феромона в зависимости от нахождения агента i в некоторой позиции (x, y) и выполнения атакующих действий a_i.

В глубоком Q-обучении с подкреплением для нелинейной аппроксимации Q-значений используются глубокие нейронные сети, параметризированные весами θ. Дополнение глубокого Q-обучения с подкреплением принципами стигмергии муравьиного алгоритма приводит к вычислению функцию потерь на основе AQ-значений:

$$L(\theta) = \mathbb{E}\left[\left(\mathcal{PH}(s,a) + \gamma \max_{a'} AQ(s', a'; \theta') - AQ(s, a; \theta)\right)^2\right],$$

где $L(\theta)$ — функция потерь; $\mathcal{PH}(s, a)$ — феромоновая функция.

5.3. Алгоритм

Глубокий алгоритм муравьиной оптимизации с подкреплением (*англ.* Deep Reinforcement Ant Colony Optimization, DRACO) основан на вдохновленном природой классическом алгоритме ACO и мультиагентном алгоритме глубокого независимого обучения с подкреплением IQL.

В отличие от существующих гибридов глубокого и роевого обучения, в алгоритме DRACO не синтезируется архитектура нейронной сети с помощью муравьиного алгоритма и не строится примитивный каскад обработки результатов муравьиного алгоритма с помощью нейронной сети. В алгоритме DRACO принципы глубокого и муравьиного обучения встроены друг в друга по аналогии с алгоритмом Ant-Q, при этом используется мультиагентный алгоритмический прием централизованного обучения и децентрализованного исполнения.

Алгоритм DRACO для агента i приведен ниже:

1. Инициализировать
   $\alpha \in (0,1]$; $\gamma \in [0,1]$; $\varepsilon \in [0.1,1]$; $Ne \in [1,N]$;
   $\Delta\tau_1, \Delta\tau_2 \in [1,T]$; $\mathrm{Map}[X, Y] \leftarrow 0, X, Y \in [1,\mathrm{M}]$; $\rho \in (0,1]$; $\mathcal{PH}_i \leftarrow 0$;
   $Eb \leftarrow 0$; $Bl \leftarrow 10\,000$; $Bs \leftarrow 32$; $Cs \leftarrow 100$;
   $i \leftarrow 1, \ldots, n$; $a \in A_i$; $j \leftarrow 1, \ldots, Bs$; $K \leftarrow |A_i|$;
   $\theta \leftarrow \xi$, $\theta' \leftarrow \xi$, $\forall s \in S$.
2. Цикл
3. Получить состояние среды s и координаты (x, y) агента.
4. Обновить карту феромонов $\mathrm{Map}[x, y] \leftarrow \mathrm{Map}[x, y] + \Delta\tau_1$.
5. Определить возможные действия $a \in A_i$ в состоянии s.
6. Передать состояние s в основную нейронную сеть и выбрать возможное действие a в соответствии с максимальным значением $AQ_1(s, a; \theta), AQ_2(s, a; \theta), \ldots, AQ_K(s, a; \theta)$ и с учетом параметра ε.
7. Выполнить возможное действие a.
8. Обновить карту феромонов $\mathrm{Map}[x, y] \leftarrow \mathrm{Map}[x, y] + \Delta\tau_2$.
9. Получить награду от среды r и следующее состояние s'.
10. Вычислить уровень феромона $\mathcal{PH}_i \leftarrow \tau_{\mathrm{Area}_{\mathrm{Map}[x,y]}} + r$.
11. Сохранить кортеж $(s, a, \mathcal{PH}_i, s')$ в буфере воспроизведения Eb. Если объем Bl превышен, заменить значение по принципу очереди FIFO.
12. Случайно получить мини-выборку $(s, a, \mathcal{PH}_i, s')$ объемом Bs из буфера воспроизведения Eb.
13. Для каждого j-го кортежа $(s, a, \mathcal{PH}_i, s')$ из мини-выборки вычислить $AQ^j(s', a'; \theta')$, передав в целевую нейронную сеть s'.
14. Вычислить $y^j \leftarrow \begin{cases} \mathcal{PH}, & \text{если } t = T; \\ \mathcal{PH} + \gamma \max\limits_{a'} AQ^j(s', a'; \theta'), & \text{если } t \neq T. \end{cases}$
15. Для каждого j-го кортежа (s, a, r, s') из мини-выборки вычислить $AQ^j(s, a; \theta)$, передав в основную нейронную сеть состояние s.
16. Вычислить функцию потерь $L^j(\theta) \leftarrow \frac{1}{K}\sum_{p=1}^{K}(y_p^j - AQ_p^j(s, a; \theta))^2$.
17. Обновить веса θ основной нейронной сети с использованием $L_j(\theta)$ и скорости обучения α.
18. Заменить веса θ' целевой нейронной сети на веса θ основной нейронной сети, если закончился очередной период Cs.
19. Применить коэффициент испарения феромона $\mathrm{Map} \leftarrow \rho\mathrm{Map}$.

Однако в отличие от алгоритма Ant-Q, в котором AQ-значения являются простым двойником Q-значений, в алгоритме DRACO при сохранении общей идеи ценности состояний-действий, базовый принцип, на основании которого вычисляются ценности, заменяется совокупностью феромоновых значений (шаг 11). Преимуществом алгоритма DRACO является использование современных принципов глубокого обучения с подкреплением, что дает возможность не применять ручную разметку табличных состояний и не строить вручную графовую модель среды.

В алгоритме DRACO по аналогии с алгоритмом EAS (*англ.* Elitist Ant System) феромоны добавляются на общий путь — в доступную всем муравьям общую карту феромонов Map[X, Y] размером $M, X, Y \in [1,M]$ (шаг 4). Как и в алгоритме ACS (*англ.* Ant Colony System), уровень феромона зависит от успешности действий муравьев (шаг 8). В данной реализации алгоритма DRACO уровень феромона вычисляется в смежной с позицией агента (x, y) области 3×3 (шаг 10). Но в отличие от алгоритма MMAS (*англ.* Max-Min Ant System) феромоновый след лучшего решения не ограничивается максимальными значениями и в отличие от алгоритма RBACO (*англ.* Rang Based ACO) все муравьи, а не только муравьи с высочайшими рангами, оставляют феромоновый след.

При решении задачи эксплуатации-исследования почти во всех рассматриваемых в данном учебном пособии алгоритмах используется концепция ε-«жадного» выбора действий. Она же используется и в алгоритме DRACO (шаг 6). Альтернативами могут быть зашумление весов нейронной сети алгоритма CoE, зашумление стратегий выбора действий алгоритма MADDPG или методы внутренней мотивации, основанные на псевдосчете состояний и направляющие поиск в сторону новых состояний, не совпадающих с прогнозом.

На рис. 5.4 представлена архитектура нейронной сети алгоритма DRACO для рассматриваемой в гл. 5 задачи мультиагентного роевого обучения

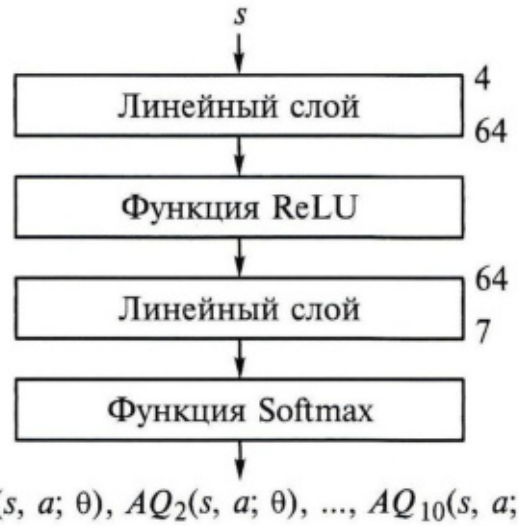


Рис. 5.4. Архитектура нейронной сети алгоритма DRACO

75 агентов. В соответствии с базовыми принципами роевого обучения все агенты роя гомогенны и будут иметь одинаковые нейронные сети. Кроме того, архитектура агентов должна быть чрезвычайно проста, что означает создание простой нейронной сети для каждого агента. В соответствии с современными тенденциями в исследовании методов представления состояний среды для агентов роя, в данной реализации не применяются стандартные локальные наблюдения среды SMAC, а в качестве входных данных используются только двухмерные значения координат (x, y) позиций агента и его противника.

В связи с этим архитектура нейронной сети алгоритма DRACO включает два полносвязных линейных слоя с входными и выходными размерами соответственно 4×64, 64×7. На вход нейронная сеть получает состояние среды s, заданное четырьмя отсчетами, на выходе возвращает семь AQ-значений. Схема мультиагентного обучения с использованием алгоритма DRACO соответствует схеме алгоритма IQL (см. рис. 3.5), однако мультиагентность учтена неявно в виде феромоновой карты, которую совместными действиями создают агенты роя при решении задачи обучения в среде (см. разд. 5.4).

5.4. Карта

Мультиагентное роевое обучение, описанное в гл. 5, проводилось на карте 75z1cFOX.SC2Map, представленной на рис. 5.5. В начале каждого эпизода на карте внизу, в точке Zerglings, автоматически создается рой из 75 агентов юнитов зерглингов расы зергов, которыми управляет обучающий алгоритм, справа, в точке Colossi, автоматически создается один агент — юнит колосс расы протоссов, которым управляет компьютер, используя стандартные правила. Длительность одного полного эпизода игры составляет 6 с. Точка Colossi находится на возвышенности, добраться до которой агенты могут только по пяти подъемам.

Колосс — наземный боевой юнит протоссов, обладающий атакой дальнего боя, способный поражать как наземные, так и воздушные юниты на расстоянии шести квадратов игрового поля, нанося 15 единиц урона. Колосс имеет высоту обзора 4 квадрата игрового поля и радиус обзора 10 квадратов игрового поля, 200 единиц здоровья и 150 единиц щита. Щит при отсутствии атаки довольно быстро восстанавливается — две единицы в секунду. Колосс — шагающий боевой робот, который способен перешагивать препятствия и быстро перемещаться по карте с возвышенностями. При столкновении с зергами колосс чрезвычайно эффективно используется против зерглингов и гидралисков.

На рис. 5.6–5.10 проиллюстрированы различные ситуации в среде SMAC: начало эпизода игры, рой начинает исследовать среду (см. рис. 5.6);

колосс атакует, уничтожая площадным ударом сразу несколько агентов (см. рис. 5.7); несколько выживших агентов подобрались ближе к колоссу в зону доступной им атаки (см. рис. 5.8); рой рассеян по карте и методично уничтожается колоссом (см. рис. 5.9); рой выучил оптимальную стратегию и, окружив, уничтожает колосса единой атакой (см. рис. 5.10).

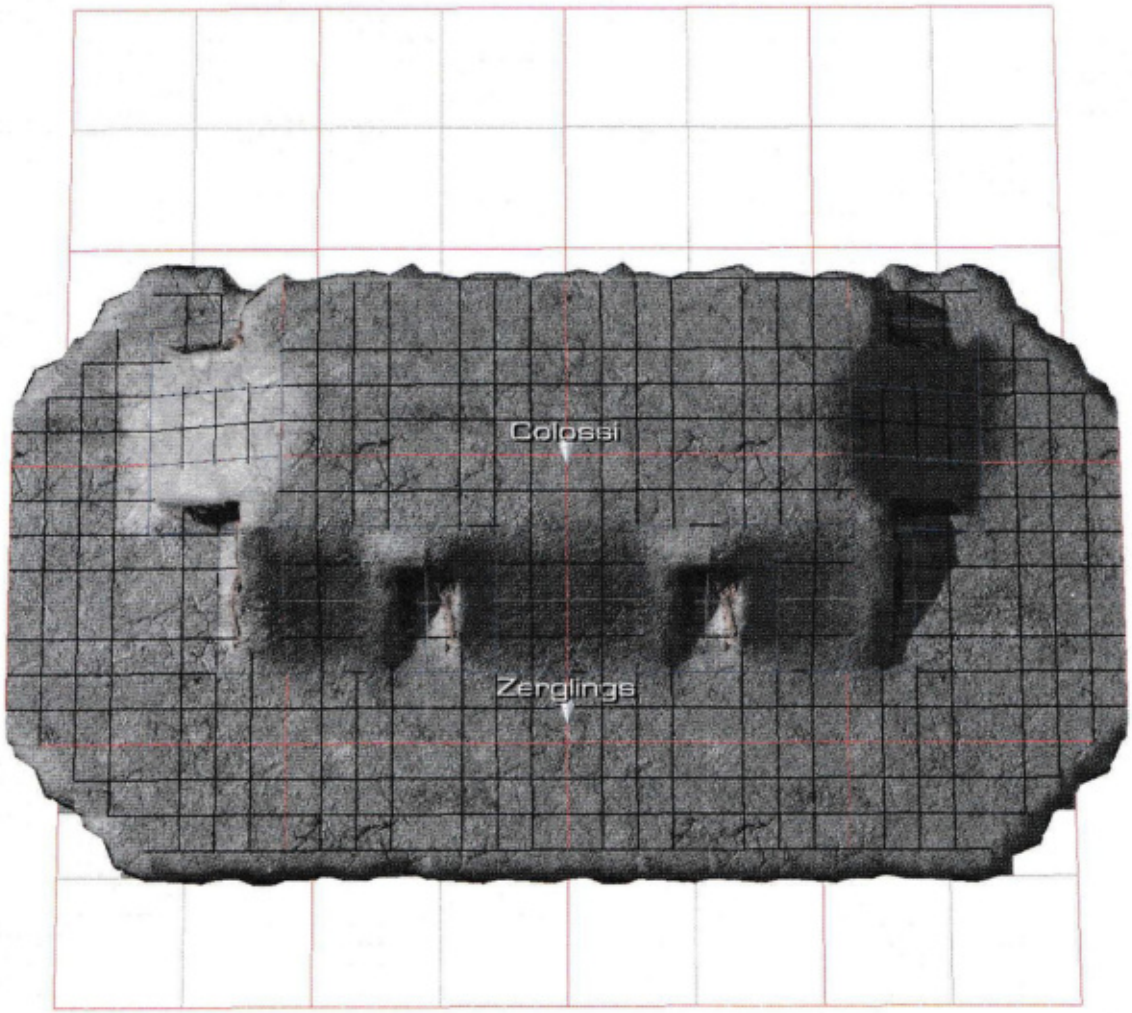


Рис. 5.5. Внешний вид карты 75z1cFOX.SC2Map в редакторе StarCraft II Editor

На рис. 5.11 представлено изображение карты феромонов, которая используется для централизованного обучения роя агентов с помощью алгоритма DRACO. Для целей визуализации цифровое изображение карты феромонов отмасштабировано к размеру 320×320 пикселей.

Уровень феромонов отображается насыщенностью красного цвета — чем больше красного цвета, тем большее количество феромонов создано роем в определенном месте карты.

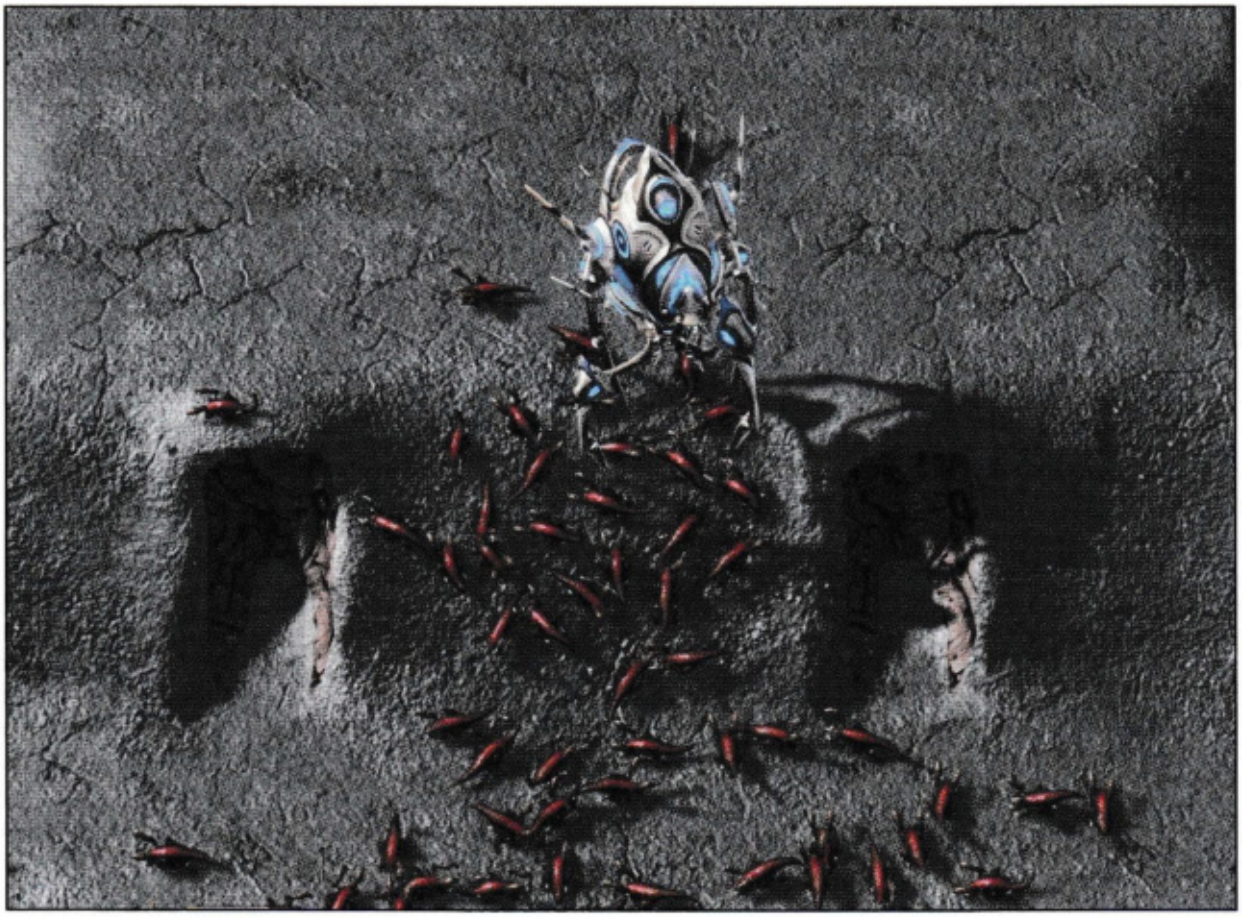


Рис. 5.6. Начало эпизода

Рис. 5.7. Колосс атакует

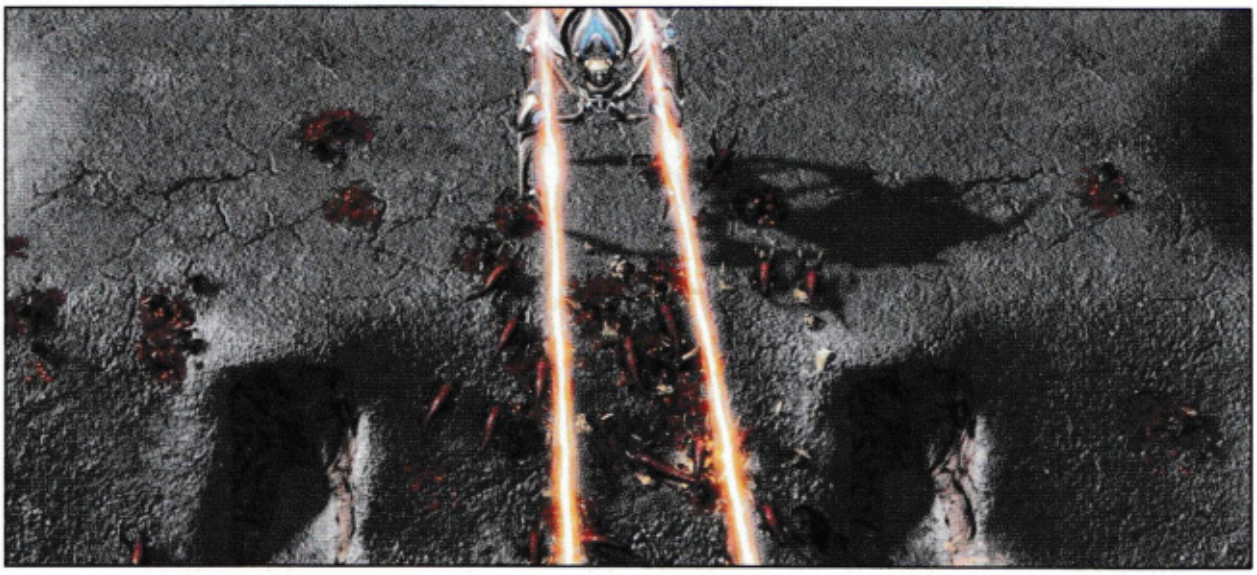
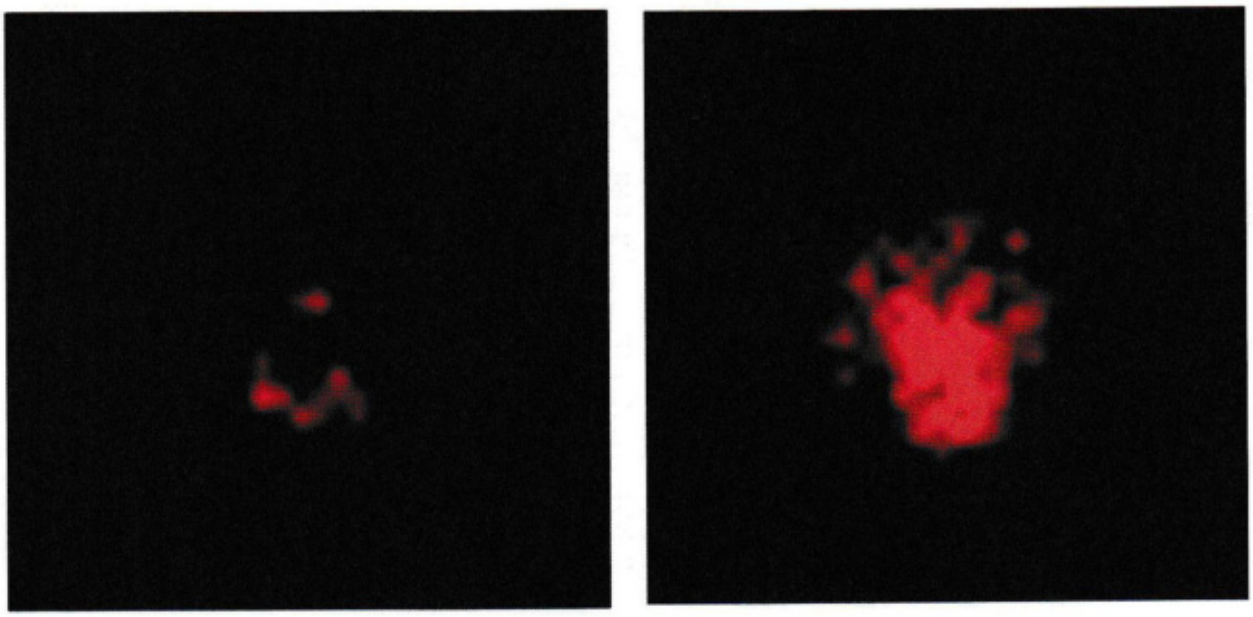


Рис. 5.8. Несколько выживших подобрались ближе для атаки

Рис. 5.9. Рой рассеян по карте

Рис. 5.10. Рой выучил оптимальную стратегию



а *б*

Рис. 5.11. Карта феромонов:
а — начало эпизода; *б* — конец успешного эпизода

Результаты экспериментального обучения в среде SMAC будут рассмотрены в разд. 5.7.

5.5. Технология

Рассматриваемые в гл. 5 алгоритмы реализованы на языке программирования Python версии 3.6. В представленном в разд. 5.6 программном коде, кроме библиотеки кооперативного мультиагентного обучения SMAC, используются стандартные библиотеки: numpy для работы с массивами, collections для использования специализированного типа данных обобщенной очереди deque, time для измерения времени, matplotlib для построения графиков, cv2 для цифровой обработки изображений.

```
«75z1cFOX»: {
    «n_agents»: 75,
    «n_enemies»: 1,
    «limit»: 240,
    «a_race»: «Z»,
    «b_race»: «P»,
    «unit_type_bits»: 0,
    «map_type»: «bane», }
```

Карта 75z1cFOX.SC2Map создана на базе карты SMAC в редакторе StarCraft II Editor. Ниже представлены изменения, которые необходимо внести в файл smac_maps.py, расположенный в папке «...\smac\env\

starcraft2\maps», при создании новой карты или редактировании существующей. Эксперименты с алгоритмами проводились в операционной системе Windows 10. Код, реализующий роевой алгоритм обучения, представлен в подразд. 5.6.1. Код, тестирующий обученные нейронные сети, вынесен в отдельный файл, его можно найти на платформе Github.

Примеры реализации алгоритмов в полном объеме представлены на Github: https://github.com/Alekat13/FoxCommander

Видеопримеры работы алгоритмов приведены на YouTube: https://www.youtube.com/channel/UC33QPEjSlfA8P9rG0HtiH6g

5.6. Код

```
#Подключаем библиотеки
from smac.env import StarCraft2Env
import numpy as np
from collections import deque
import matplotlib.pyplot as plt
import time

import torch
import torch.nn as nn
import torch.optim as optim

import cv2

#Флаг вывода массива целиком
np.set_printoptions(threshold=np.inf)

#Размер карты феромонов, совпадающий с размером игровой карты
MAPX = 32
MAPY = 32
#Объем буфера воспроизведения
BUF_LEN = 10000

#Определяем архитектуру нейронной сети
class AQ_network(nn.Module):
    #На вход нейронная сеть получает состояние среды в виде координат (x,y)
    #На выходе нейронная сеть возвращает оценку действий в виде AQ-значений
    def __init__(self, obs_size, n_actions):
        super(AQ_network, self).__init__()
        self.AQ_network = nn.Sequential(
            #Первый линейный слой обрабатывает входные данные состояния среды
            nn.Linear(obs_size, 64),
            nn.ReLU(),
            #Второй линейный слой обрабатывает внутренние данные
            nn.Linear(64, n_actions)
        )
        #Применение к выходным данным функции Softmax
        self.sm_layer = nn.Softmax(dim=1)
    #Вначале данные x обрабатываются полносвязной сетью с функцией ReLU
    #На выходе происходит обработка функцией Softmax
    def forward(self, x):
        aq_network_out = self.AQ_network(x)
        sm_layer_out = self.sm_layer(aq_network_out)
        #Финальный выход нейронной сети
        return sm_layer_out
```

```
#Класс для реализации буфера воспроизведения
class Exp_Buf():
    #Буфер воспроизведения на основе обобщенной очереди
    expbufVar = deque(maxlen=BUF_LEN)

#Вычисляем количество феромона в точке как среднее по области
def compute_pherintensity(pheromone_map, X, Y, pa):
    if (X >=1) and (Y>=1):
        pheromone_inpoint =
         pheromone_map[X-pa][Y+pa] + pheromone_map[X][Y+pa] +
         pheromone_map[X+pa][Y+pa] + pheromone_map[X-pa][Y] +
         pheromone_map[X][Y]       + pheromone_map[X+pa][Y] +
         pheromone_map[X-pa][Y-pa] + pheromone_map[X][Y-pa] +
         pheromone_map[X+pa][Y-pa])/9
    else: pheromone_inpoint = 0
    #Возвращаем количество феромона в точке
    return pheromone_inpoint

#Выбираем возможное действие с максимальным AQ-значением в зависимости
#от эпсилон
def select_actionFox(action_probabilities, avail_actions_ind, epsilon):
    p = np.random.random(1).squeeze()
    #Исследуем пространство действий
    if np.random.rand() < epsilon:
        return np.random.choice (avail_actions_ind)
    else:
        #Находим возможное действие
        #Проверяем, есть ли действие в доступных действиях агента
        for ia in action_probabilities:
            action = np.argmax(action_probabilities)
            if action in avail_actions_ind:
                return action
            else:
                action_probabilities[action] = 0

#Создаем мини-выборку определенного объема из буфера воспроизведения
def sample_from_expbuf(experience_buffer, batch_size):
    #Функция возвращает случайную последовательность заданной длины из его
    #элементов
    perm_batch = np.random.permutation(len(experience_buffer))[:batch_size]
    #Мини-выборка
    experience = np.array(experience_buffer)[perm_batch]
    #Возвращаем значения мини-выборки по частям
    return experience[:,0], experience[:,1], experience[:,2],
           experience[:,3], experience[:,4]

#Основная функция программы
def main():
    #Загружаем среду StarCraft II, карту, сложность противника и расширенную
    #награду
    env = StarCraft2Env(map_name="75z1cFOX", reward_only_positive=False,
                        reward_scale_rate=200, difficulty="1")
    #Получаем и выводим на печать информацию о среде
    env_info = env.get_env_info()
    print ('env_info=',env_info)
    #Получаем и выводим на печать размер наблюдений агента
    obs_size =  env_info.get('obs_shape')
    print ("obs_size=",obs_size)
    #Число действий агента
    n_actions = env_info["n_actions"]
    #Число дружественных агентов
    n_agents = env_info["n_agents"]
    #Определяем основные параметры роевого обучения
```

```
#############################################################################
#Определим динамический эпсилон с затуханием
eps_max = 1.0 #Начальное значение эпсилон
eps_min = 0.1 #Финальное значение эпсилон
eps_decay_steps = 20000 #Шаг затухания эпсилон
#Основные переходы в алгоритме DRACO зависят от управляющих параметров
global_step = 0  #подсчитываем общее число шагов в игре
copy_steps = 100 #через каждые 100 шагов синхронизируем нейронные сети
start_steps = 1000 #начинаем обучать через 1000 шагов
steps_train = 4  #после начала обучения продолжаем обучать через
каждые четыре шага
#Размер мини-выборки
batch_size = 32
#Общее число эпизодов игры
n_episodes = 151
#Параметр дисконтирования
gamma = 0.99
#Скорость обучения
alpha = 0.01
#Коэффициент выветривания феромона
evap = 0.98
#Количество феромона, выделяемого каждым муравьем
pher_volume = 0.1
#Усиление феромона за успешное действие
pher_intens = 50
#Размер смежной области для вычисления количества феромона в точке
pher_area = 1
#Искусственный размер наблюдений агента в виде четырех координат
obs_sizeXY = 4
#############################################################################

#Pytorch определяет возможность использования графического процессора
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
#Определяем выходной размер нейронной сети
qofa_out = n_actions
#Создаем основную нейронную сеть
aq_network = AQ_network(obs_sizeXY, n_actions).to(device)
#Создаем целевую нейронную сеть
tgt_network = AQ_network(obs_sizeXY, n_actions).to(device)
#Создаем списки для мультиагентного случая
aq_network_list = []
tgt_network_list = []
optimizer_list = []
objective_list = []
exp_buf_L = []

for agent_id in range(n_agents):
    #Создаем список буферов
    exp_buf_L.append(Exp_Buf())
    #Создаем список основных нейронных сетей для каждого агента роя
    aq_network_list.append(aq_network)
    #Создаем список целевых нейронных сетей для каждого агента роя
    tgt_network_list.append(tgt_network)
    #Создаем список оптимизаторов нейронных сетей для каждого агента роя
    optimizer_list.append(optim.Adam(params=aq_network_list[agent_id]
                                     .parameters(), lr=alpha))
    #Создаем список функций потерь для каждого агента роя
    objective_list.append(nn.MSELoss())
#Выводим на печать пример одной основной нейронной сети
print ('aq_network_list[0]=', aq_network_list[0])
#Определяем вспомогательные параметры
Loss_History = []
Reward_History = []
```

```
winrate_history = []
total_loss = []
m_loss = []

#Основной цикл по эпизодам игры
################_цикл for по эпизодам#####################################
for e in range(n_episodes):
    #Перезагружаем среду
    env.reset()
    #Флаг окончания эпизода
    terminated = False
    #Награда за эпизод
    episode_reward = 0
    #Обновляем и выводим динамический эпсилон
    epsilon = max(eps_min, eps_max - (eps_max-eps_min) *
                  global_step/eps_decay_steps)
    print ('epsilon=',epsilon)
    #Определяем карту феромонов
    pheromone_map = np.zeros([MAPX, MAPY])
    #Определим псевдокарту 32x32x1 как визуализацию феромонов
    map_data = np.zeros((MAPY, MAPX, 3), np.uint8)

    #Шаги игры внутри эпизода
    ######################_цикл while#####################################
    while not terminated:

        #Обнуляем промежуточные переменные
        actions = []
        action = 0
        #Сохраняем историю действий на одном шаге для разных агентов
        actionsFox = np.zeros([n_agents])
        #Сохраняем историю состояний среды на одном шаге для разных агентов
        obs_agentXY = np.zeros([n_agents, obs_sizeXY])
        obs_agent_nextXY = np.zeros([n_agents, obs_sizeXY])

        #Заполним карту феромонов и визуализируем карту
        #по правилу нахождения агента в определенной точке
        for agent_id in range(n_agents):
            #Получаем текущие характеристики юнита
            unit = env.get_unit_by_id(agent_id)
            #Заполняем карту феромонов
            pheromone_map[int(unit.pos.x)][int(unit.pos.y)] =
             pheromone_map[int(unit.pos.x)][int(unit.pos.y)] +
             pher_volume
            #Визуализируем феромоны на псевдокарте BGR
            cv2.line(map_data, (int(unit.pos.x), int(unit.pos.y)),
                    (int(unit.pos.x), int(unit.pos.y)), (0, 0,
                     pheromone_map[int(unit.pos.x)][int(unit.pos.y)]), 1)

        ############_Цикл по агентам для выполнения действий в игре_#####
        for agent_id in range(n_agents):

            #Получаем текущие характеристики юнита
            unit = env.get_unit_by_id(agent_id)
            obs_agentXY[agent_id][0] = unit.pos.x
            obs_agentXY[agent_id][1] = unit.pos.y

            #Соберем данные противников для псевдокарты
            for e_id, e_unit in env.enemies.items():
                obs_agentXY[agent_id][2] = e_unit.pos.x
                obs_agentXY[agent_id][3] = e_unit.pos.y

            #Конвертируем данные в тензор
```

```
        obs_agentT =
                torch.FloatTensor([obs_agentXY[agent_id]]).to(device)
        #Передаем состояние среды в основную нейронную сеть
        #и получаем AQ-значения для каждого действия
        action_probabilitiesT = aq_network_list[agent_id](obs_agentT)
        #Конвертируем данные в numpy
        action_probabilitiesT = action_probabilitiesT.to("cpu")
        action_probabilities = action_probabilitiesT.data.numpy()[0]
        #Находим возможные действия агента в данный момент времени
        avail_actions = env.get_avail_agent_actions(agent_id)
        avail_actions_ind = np.nonzero(avail_actions)[0]
        #Выбираем возможное действие агента с учетом
        #максимального AQ-значения и параметра эпсилон
        action = select_actionFox(action_probabilities,
                                  avail_actions_ind, epsilon)
        #Обрабатываем исключение при ошибке в действиях
        if action is None: action = np.random.choice
                                    (avail_actions_ind)
        #Создаем дополнительный феромон за полезное действие в игре
        if (action == 6):
            #Получаем текущие характеристики юнита
            unit = env.get_unit_by_id(agent_id)
            #Заполняем карту феромонов
            pheromone_map[int(unit.pos.x)][int(unit.pos.y)] =
             pheromone_map[int(unit.pos.x)][int(unit.pos.y)] +
             pher_intens + pher_volume

        #Собираем действия от разных агентов
        actions.append(action)
        actionsFox[agent_id] = action
    ######_конец цикла по агентам для выполнения действий в игре_####

    #Передаем действия агентов в среду, получаем награду
    #и прерывание игры от среды
    reward, terminated, _ = env.step(actions)
    #Суммируем награды за этот шаг для вычисления награды за эпизод
    episode_reward += reward

    #Поворачиваем изображение псевдокарты
    flipped = cv2.flip(map_data, 0)
    #Увеличиваем изображение псевдокарты для отображения
    resized = cv2.resize(flipped, dsize=None, fx=10, fy=10)
    #Выводим на экран псведокарту
    cv2.imshow('Pheromone map', resized)
    cv2.waitKey(1)

    ####################_Цикл по агентам для обучения_##############
    for agent_id in range(n_agents):
        #Получаем характеристики переместившегося юнита
        unit = env.get_unit_by_id(agent_id)
        obs_agent_nextXY[agent_id][0] = unit.pos.x
        obs_agent_nextXY[agent_id][1] = unit.pos.y

        #Соберем данные противников для псевдокарты
        for e_id, e_unit in env.enemies.items():
            obs_agent_nextXY[agent_id][2] = e_unit.pos.x
            obs_agent_nextXY[agent_id][3] = e_unit.pos.y

        #Вычисляем количество феромона в точке
        pheromone_inpoint = compute_pherintensity (pheromone_map,
         int(unit.pos.x), int(unit.pos.y), pher_area)
        #Создаем подкрепляющий сигнал на основе количества феромона
        #и награды
```

```
pher_reinf = reward + pheromone_inpoint
#Сохраняем переход в буфере воспроизведения для каждого
#агента
exp_buf_L[agent_id].expbufVar.append([obs_agentXY[agent_id],
 actionsFox[agent_id], obs_agent_nextXY[agent_id],
 pher_reinf, terminated])

#Если буфер воспроизведения наполнен, начинаем обучать сеть
########################начало if обучения#################
if (global_step % steps_train == 0) and (global_step >
                                         start_steps):
    #Получаем мини-выборку из буфера воспроизведения
    exp_obs, exp_act, exp_next_obs, exp_pher_reinf, exp_termd
      = sample_from_expbuf(exp_buf_L[agent_id].expbufVar,
                           batch_size)
    #Конвертируем данные в тензор
    exp_obs = [x for x in exp_obs]
    obs_agentT = torch.FloatTensor([exp_obs]).to(device)
    #Подаем мини-выборку в основную нейронную сеть,
    #чтобы получить AQ(s,a)
    action_probabilitiesT =
     aq_network_list[agent_id](obs_agentT)
    action_probabilitiesT = action_probabilitiesT.to("cpu")
    action_probabilities =
     action_probabilitiesT.data.numpy()[0]

    #Конвертируем данные в тензор
    exp_next_obs = [x for x in exp_next_obs]
    obs_agentT_next =
     torch.FloatTensor([exp_next_obs]).to(device)

    #Подаем мини-выборку в основную нейронную сеть,
    #чтобы получить AQ'(s',a')
    action_probabilitiesT_next =
     tgt_network_list[agent_id](obs_agentT_next)
    action_probabilitiesT_next =
     action_probabilitiesT_next.to("cpu")
    action_probabilities_next =
     action_probabilitiesT_next.data.numpy()[0]

    #Вычисляем целевое значение y
    y_batch = exp_pher_reinf + gamma *
     np.max(action_probabilities_next, axis=-1)*
           (1 - exp_termd)

    #Переформатируем y_batch размером batch_size
    y_batch64 = np.zeros([batch_size, qofa_out])
    for i in range (batch_size):
        for j in range (qofa_out):
            y_batch64[i][j] = y_batch[i]
    #Конвертируем данные в тензор
    y_batchT = torch.FloatTensor([y_batch64])

    #Обнуляем градиенты
    optimizer_list[agent_id].zero_grad()
    #Вычисляем функцию потерь
    loss_t = objective_list[agent_id](action_probabilitiesT,
                                      y_batchT)
    #Сохраняем данные для графиков
    Loss_History.append(loss_t)
    loss_n=loss_t.data.numpy()
    total_loss.append(loss_n)
    m_loss.append(np.mean(total_loss[-1000:]))
```

```
                #Выполняем обратное распространение ошибки
                loss_t.backward()

                #Выполняем оптимизацию нейронных сетей
                optimizer_list[agent_id].step()

            ######################_конец if обучения_###################

            #Синхронизируем веса основной и целевой нейронных сетей,
            #через каждые 100 шагов
            if (global_step+1) % copy_steps == 0 and
                global_step > start_steps:
             tgt_network_list[agent_id].load_state_dict
              (aq_network_list[agent_id].state_dict())

        ###################_Конец цикла по агентам для обучения_#########

        #Обновляем счетчик общего числа шагов
        global_step += 1

        #Применяем коэффициент испарения феромона
        for i in range(MAPX):
            for j in range(MAPY):
                pheromone_map[i][j] = evap*pheromone_map[i][j]

    ######################_конец цикла while_##############################
    #Выводим счетчик шагов игры и общую награду за эпизод
    print('global_step=', global_step, "Total reward in episode {} =
          {}".format(e, episode_reward))

    #Собираем данные для графиков
    Reward_History.append(episode_reward)
    status = env.get_stats()
    winrate_history.append(status["win_rate"])

################_конец цикла по эпизодам игры_############################
#Закрываем среду StarCraft II
env.close()
#Убираем с экрана псевдокарту
cv2.destroyAllWindows()

#Сохраняем параметры обученных нейронных сетей
for agent_id in range(n_agents):
    torch.save(aq_network_list[agent_id].state_dict(),
               "aqnet_%.0f.dat"%agent_id)
#Выводим на печать графики
#Средняя награда
plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
           edgecolor='k')
plt.plot(Reward_History)
plt.xlabel('Номер эпизода')
plt.ylabel('Количество награды за эпизод')
plt.show()
#Процент побед
plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
           edgecolor='k')
plt.plot(winrate_history)
plt.xlabel('Номер эпизода')
plt.ylabel('Процент побед')
plt.show()
#Значения функции потерь
plt.figure(num=None, figsize=(6, 3), dpi=150, facecolor='w',
```

```
                edgecolor='k')
    plt.plot(m_loss)
    plt.xlabel('Номер каждой 1000-й итерации')
    plt.ylabel('Функция потерь')
    plt.show()
#Точка входа в программу
if __name__ == "__main__":
    start_time = time.time()
    main()
    #Время обучения
    print("--- %s минут ---" % ((time.time() - start_time)/60))
```

5.7. Эксперимент

На рис. 5.12 и 5.13 представлены результаты обучения роя из 75 агентов с помощью алгоритма DRACO при заданных параметрах обучения $\gamma = 0.99$, $\alpha = 0.01$, $\rho = 0.98$, $\Delta\tau_1 = 0.1$, $\Delta\tau_2 = 50$ на 150 эпизодах игры. Рой агентов успешно выучивает оптимальную стратегию. При анализе визуализации игры становится очевидным, что рой действует как суперорганизм, выполняя единый бросок на противника из нижней части карты в верхнюю (см. рис. 5.10).

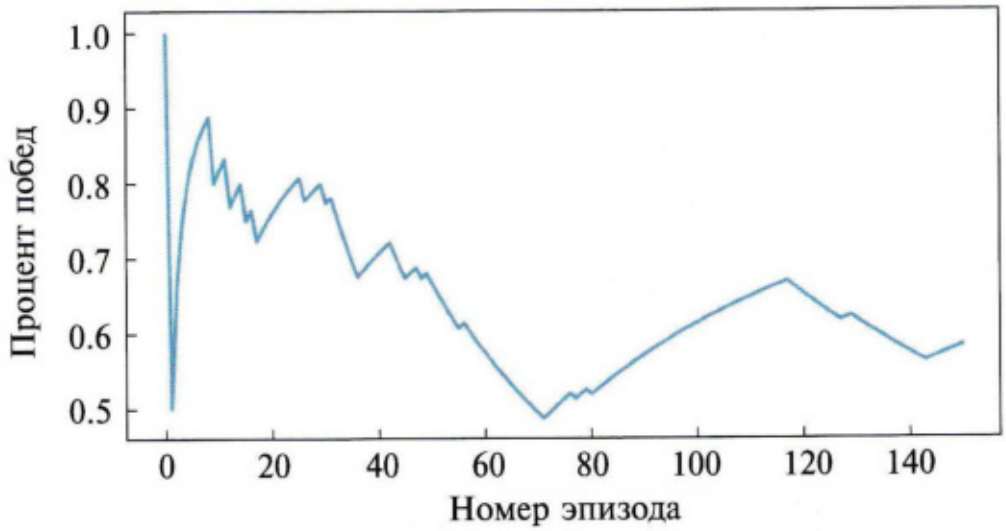


Рис. 5.12. Процент побед при обучении роя агентов с помощью алгоритма DRACO в среде SMAC

На рис. 5.14 представлены результаты тестирования алгоритма DRACO. Тестирование проводилось при стандартной награде среды SMAC. Процент побед на 50 тестовых эпизодах игры равен 100 %, среднее количество награды — 20.0. Нестандартный вид графика подтверждает преимущества роевого обучения, при котором множество агентов практически идеально решают

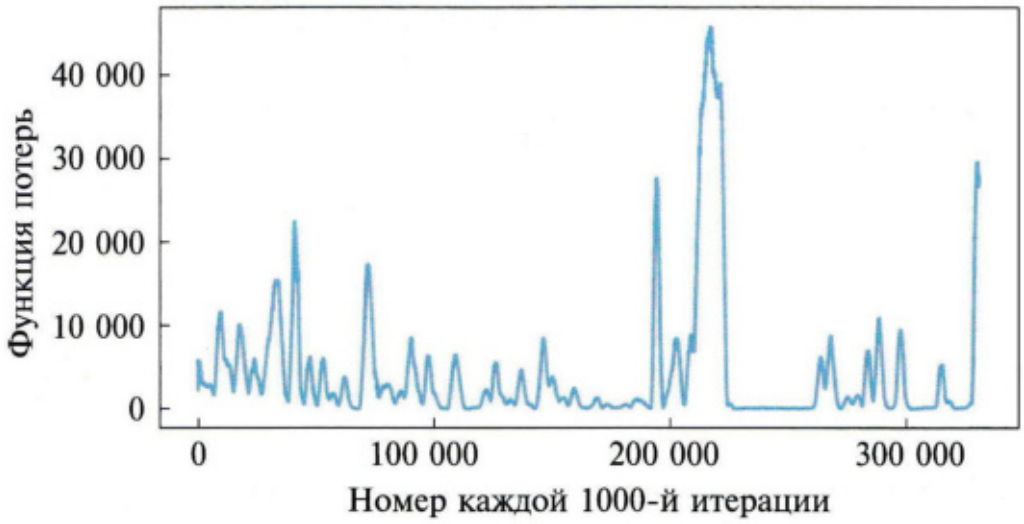


Рис. 5.13. Функция потерь при обучении роя агентов с помощью алгоритма DRACO в среде SMAC

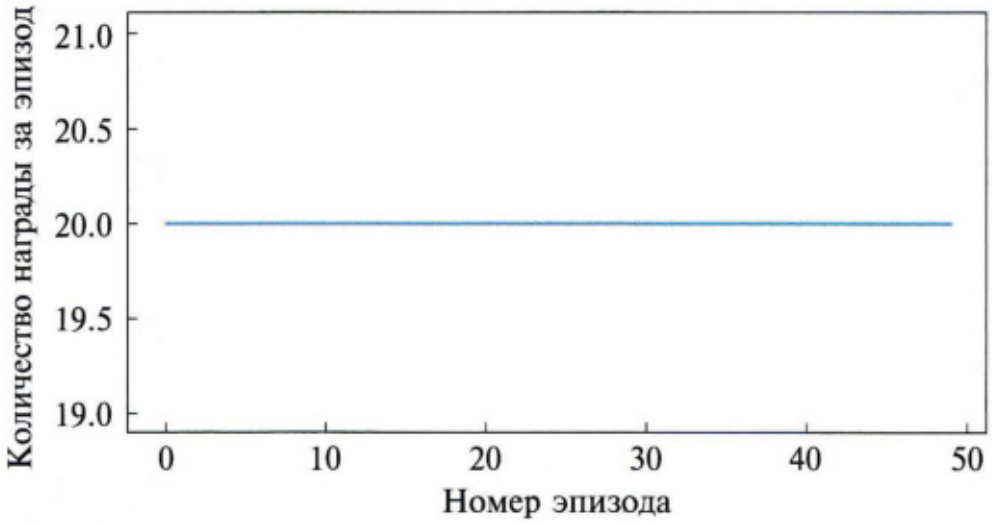


Рис. 5.14. Результаты тестирования алгоритма IQL в среде SMAC

поставленную задачу, не давая возможности обстоятельствам и факторам среды как-то повлиять на исход игры.

5.8. Выводы

В гл. 5 был рассмотрен алгоритм мультиагентного роевого обучения DRACO, наследующий принципы централизованного обучения муравьиного алгоритма ACO и дополненный децентрализованными принципами глубокого обучения с подкреплением мультиагентого алгоритма IQL.

Преимуществом роевого обучения является получение ценных свойств роя, характерных и для кооперативной мультиагентной системы: естественное распараллеливание процессов; возможность решать коллективные задачи, по своей природе недоступные для выполнения единственным агентом;

повышенная надежность и устойчивость к ошибкам благодаря гомогенной взаимозаменяемости агентов роя; одновременный охват большой сенсорной зоны, позволяющий рою быстрее исследовать среду и осуществлять точную навигацию; экономическая и энергетическая эффективность, которая проявляется при физической реализации роя на взаимозаменяемой и дешевой элементной базе с простыми источниками питания.

Неразрешенной проблемой роевого обучения является потребность в теоретических моделях, которые объединили бы исследования из разных областей компьютерных наук и позволили проводить формальные проверки свойств сходимости, робастности и стабильности алгоритмов роевого обучения. Необходимы также тестовые программные платформы, на которых можно было бы объективно и повторяемо проводить эксперименты с алгоритмами роевого обучения. Кроме того, требуются инструменты для устранения неопределенности при интерпретации стратегии поведения большого обученного роя.

Возможно, следующим этапом в развитии роевого обучения станет не только качественный, но и количественный рост алгоритмов обучения — мультиагентного обучения на уровне множества роев. В этом случае исследователям придется найти ответы на вопросы грандиозной масштабируемости алгоритмов роевого обучения и их вычислительной эффективности.

5.9. Задачи для самоконтроля

1. В гл. 5 реализованы феромоны одмихнионы — метки пути, указывающие дорогу к дому или к найденной добыче, метки на границах индивидуальной территории. Для алгоритма DRACO и коэволюционного алгоритма CoE (см. гл. 4) предложите алгоритмические решения, позволяющие использовать следующие феромоны: торибоны — феромоны страха и тревоги; гонофионы — феромоны, индуцирующие смену пола. Сравните эффективность предложенных решений.

2. Усовершенствуйте алгоритм DRACO для победы над колоссом на карте 50z1cFOXwall.SC2Map (рис. 5.15).

3. Адаптируйте алгоритм ABC для мультиагентного роевого обучения в среде SMAC на карте 75z1cFOX.SC2Map.

4. Адаптируйте алгоритм PSO для мультиагентного роевого обучения в среде SMAC на карте 75z1cFOX.SC2Map.

5. Адаптируйте алгоритм BFO для мультиагентного роевого обучения в среде SMAC на карте 75z1cFOX.SC2Map.

6. По аналогии с нейронной сетью PixelCNN, автоматически формирующей модель плотности цифрового изображения, проведите совместное обучение сверточной нейронной сети для автоматического формирования уровня феромонов и полносвязной нейронной сети, вычисляющей AQ-значения (рис. 5.16). Сравните эффективность работы с алгоритмом DRACO.

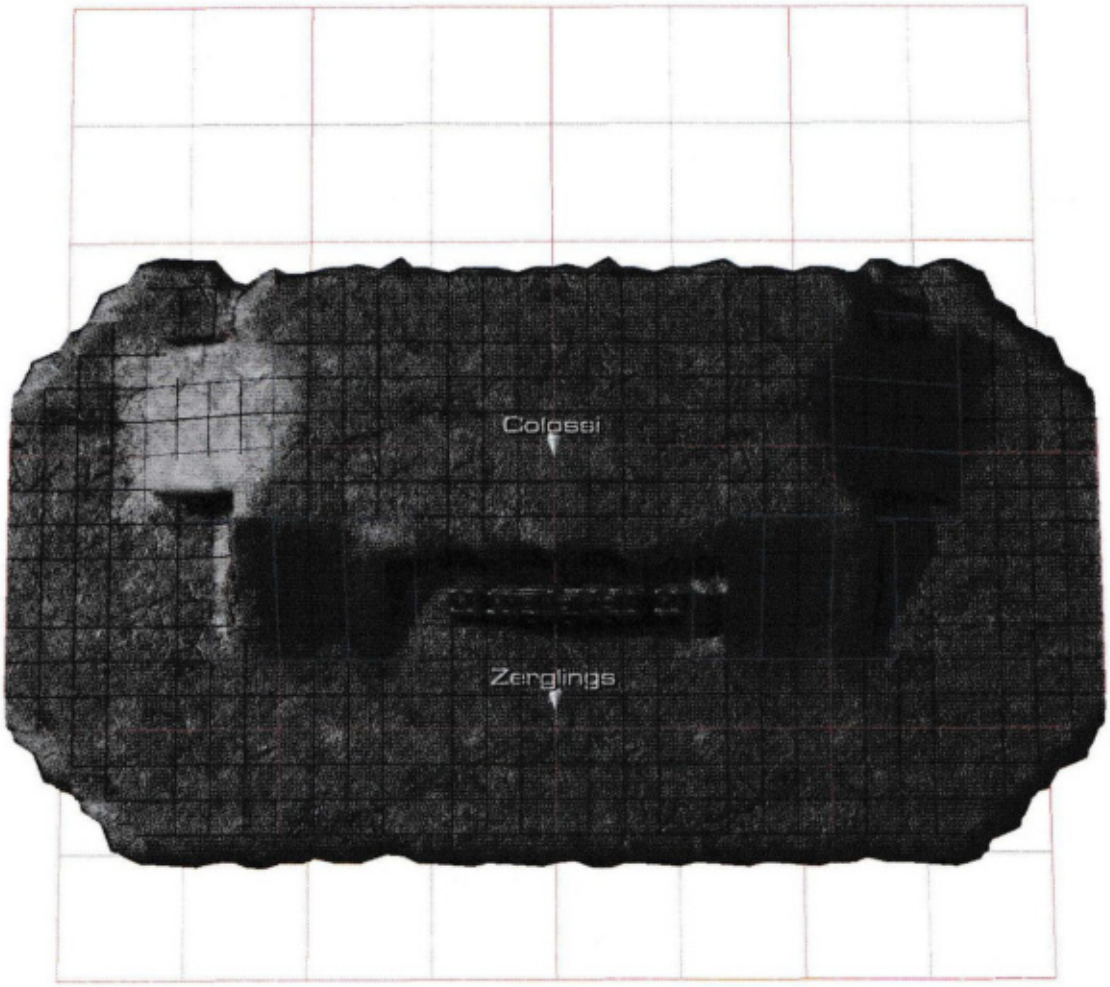


Рис. 5.15. Внешний вид карты 50z1cFOXwall.SC2Map в редакторе StarCraft II Editor

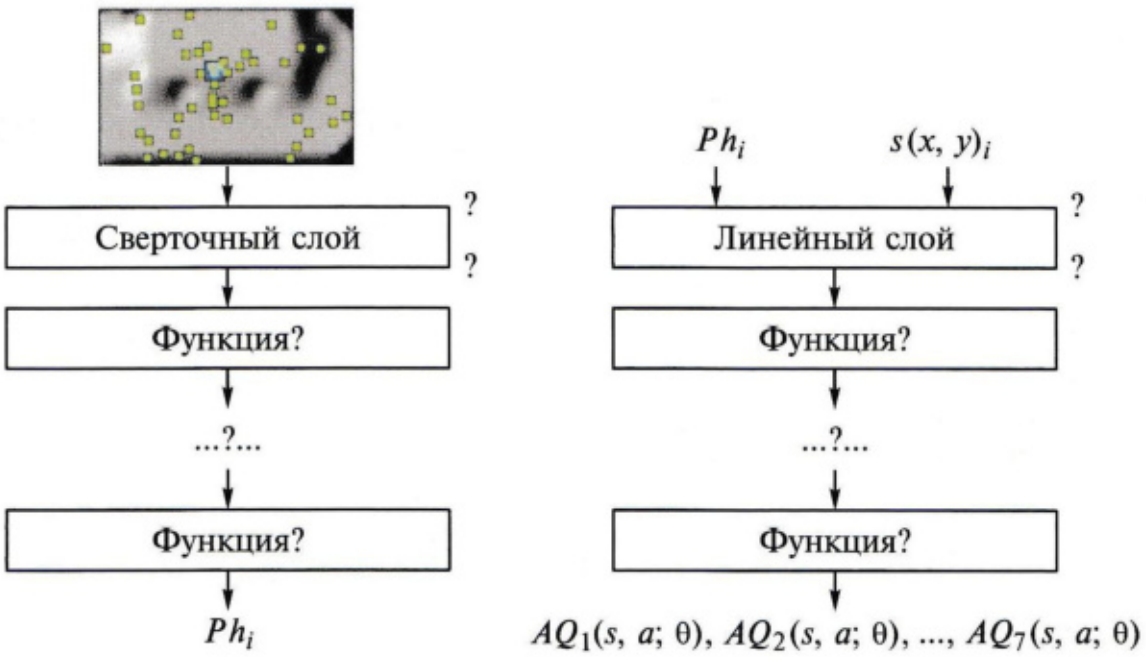


Рис. 5.16. Архитектура нейронной сети нового алгоритма мультиагентного роевого обучения

7. Усовершенствуйте алгоритм мультиагентного роевого обучения, разработанный в задаче 6, используя формулу псевдосчета $\hat{N}_n(s)$ числа состояний. Пусть $\dot{S}$ — модель плотности состояний в конечном пространстве S; $\dot{s}_n(s)$ — вероятность, устанавливаемая моделью $\dot{S}$ для каждого состояния s, после обучения модели $\dot{S}$ на последовательности состояний $s_1, s_2, \ldots, s_n$. Предполагается, что $\dot{s}_n(s) > 0, \forall s, n$. Вероятность перекодирования $\dot{s}'_n(s)$ — это вероятность, устанавливаемая моделью $\dot{S}$ для состояния s после дополнительного обучения модели $\dot{S}$ с учетом состояния s. Тогда псевдосчет $\hat{N}_n(s)$:

$$\hat{N}_n(s) = \frac{\dot{s}_n(s)\left(1-\dot{s}'_n(s)\right)}{\dot{s}'_n(s)-\dot{s}_n(s)}.$$

8. Одна из классических мультиагентных задач «Рандеву» заключается в том, что агенты роя, не используя коммуникацию, а лишь применяя наблюдения за позициями соседних агентов, должны обучиться встречаться в определенной, ранее неизвестной точке карты. Формализуйте задачу «Рандеву» с использованием функции наград $R(s,a)$:

$$R(s,a) = \alpha \sum_{i=1}^{N} \sum_{j=i+1}^{N} \min\left(d_{i,j}, d_c\right) + \beta a,$$

где $\alpha = -\left(\frac{N(N-1)}{2} d_c\right)^{-1}$ — коэффициент нормализации награды; $d_{i,j}$ — расстояние между агентами; $d_c = \max\left(x_{\max}, y_{\max}\right)$ — пороговое расстояние, определяющее максимально возможное расстояние $d_{i,j}$; $\beta = -1 \cdot 10^{-3}$ — коэффициент нормализации действий. Промоделируйте задачу «Рандеву» в среде SMAC.

9. Используя отдельную программную библиотеку MAgent, обучите рой агентов создавать и сохранять при перемещении следующие формации: «Веер Пир», имеющую вид клина; «Первая Формация», имеющую вид квадрата с выступами с каждой стороны наподобие рогов; «Атакующий Тигр», имеющую вид звезды.

10. В битве при Свечной Арке генерал Шуос Джедао одержал победу при превосходстве сил противника в соотношении 8:1. Разработайте сценарий гетерогенного кооперативного мультиагентного обучения, карту и оригинальных юнитов компьютерной игры StarCraft II для моделирования битвы при Свечной Арке в среде SMAC.

Заключение

В последний раз бросаю кости, я научил тебя всему, чему смог.
Не повторяй моих ошибок. Прощай, генерал.
Юн Ха Ли. Гамбит девятихвостого лиса

Технология создания сильного искусственного интеллекта может основываться на различных идеях, моделях, алгоритмах. Если взять за образец наиболее совершенный на данный момент биологический интеллект — интеллект человека, то известно, что в день эта нейросетевая система обрабатывает до 7 тыс. мыслей. Мозг человека условно может находиться в нескольких тысячах состояний при выполнении различных действий. Базируясь на современных информационных технологиях, совсем не сложно построить Q-таблицу, которая описала бы такой процесс одноагентного взаимодействия биологического организма со средой. Однако в реальной жизни не существует постоянного подкрепляющего сигнала, который в каждый момент времени подсказывал бы, какие действия были выполнены верно, а какие — нет.

В то же время биологический интеллект можно рассмотреть как «сообщество разума», которое состоит из индивидуальных простых процессов, имеющих дополнительно к внешнему подкрепляющему сигналу внутреннюю мотивацию, и именно взаимодействие этих процессов создает множество способностей, которые обычно приписывают универсальному интеллекту. Мультиагентное обучение с подкреплением позволяет взглянуть на задачу создания сильного искусственного интеллекта именно с позиции распределенных коллективных систем, в которых используется кооперативное взаимодействие автономных обучающихся агентов.

В учебном пособии были подробно проанализированы 11 базовых алгоритмов мультиагентного обучения с подкреплением. Алгоритм независимого табличного Q-обучения позволяет одновременно обучать несколько агентов для решения небольшой задачи в простой среде, но требует детальной экспертной настройки параметров обучения, дизайна наград и описания среды. В алгоритмах PHC, WoLF-PHC и Nash-Q, основанных на принципах теории игр, явно учитывается взаимозависимость агентов в кооперативных и некооперативных играх для достижения равновесного мультиагентного взаимодействия. Но этими алгоритмами обеспечиваются теоретически обоснованные и практически эффективные решения только небольших задач в простых средах.

Большие задачи обучения с множеством участвующих агентов и значительными объемами обрабатываемой информации в сложной среде решаются с помощью мультиагентных нейросетевых алгоритмов IQL, CDQN, VDN и MADDPG. Для особенных сред, где неэффективны градиентные подходы, могут быть использованы алгоритмы эволюционного InGA и коэволюционного CoE обучения. Для обучения экстремально большого числа простых гомогенных агентов в сложной среде требуются специализированные алгоритмы роевого обучения типа DRACO.

В алгоритмах мультиагентного обучения с подкреплением в основном используются марковская модель принятия решений и различные ее модификации: для нейросетевого случая — в виде децентрализованных частично наблюдаемых марковских процессов принятия решений; для теоретико-игрового случая — в виде модели стохастических игр, для роевого обучения — в виде двухуровневой роевой марковской модели принятия решений. Однако только в табличном мире марковская модель принятия решений имеет преимущества перед эвристикой, что позволяет объективно оценить стабильность алгоритма, предсказать результат обучения с подкреплением, предложить направление совершенствования алгоритма и схемы мультиагентного обучения.

Наиболее эффективным современным алгоритмическим приемом мультиагентного обучения является централизованное обучение с децентрализованным исполнением. В мультиагентном обучении с подкреплением успешно используются буфер воспроизведения и целевые нейронные сети, разделение параметров и практически все актуальные нейросетевые архитектуры. Однако это повышает требования к используемым вычислительным ресурсам. Открытыми остаются вопросы автоматической настройки параметров обучения и функционирование при редкой или отложенной награде.

В области мультиагентного обучения с подкреплением предлагается много интересных подходов для решения прикладных задач и открываются дополнительные пути достижения технологической сингулярности, которая позволит будущему интеллекту выйти из биологической колыбели в стремлении к созиданию через взаимодействие в новых средах.

Литература

К главе 1

Девятков В.В. Системы искусственного интеллекта: учеб. пособие для вузов. М.: Изд-во МГТУ им. Н.Э. Баумана, 2001. 352 с.

Тарасов В.Б. От многоагентных систем к интеллектуальным организациям. М.: Едиториал УРСС, 2002. 353 с.

Buşoniu L., Babuška R., De Schutter B. Multi-agent reinforcement learning: An overview // Innovations in multi-agent systems and applications. Berlin: Springer, 2010. P. 183–221.

Claus C., Boutilier C. The dynamics of reinforcement learning in cooperative multiagent systems // AAAI/IAAI. 1998. Vol. 2. P. 746–752.

Greenwald A., Hall K., Serrano R. Correlated Q-learning // ICML. 2003. Vol. 20. No. 1. P. 242.

Hernandez-Leal P. et al. A survey of learning in multiagent environments: Dealing with non-stationarity // URL: https://arxiv.org/abs/1707.09183 (дата обращения 01.11.2020).

Klein D., Abbeel P. CS 188: Artificial Intelligence // URL: https://inst.eecs.berkeley.edu/~cs188/sp20 (дата обращения 01.11.2020).

Könönen V. Asymmetric multiagent reinforcement learning // Web Intelligence and Agent Systems: An international journal. 2004. Vol. 2. No. 2. P. 105–121.

Lauer M., Riedmiller M. An algorithm for distributed reinforcement learning in cooperative multi-agent systems // Proceedings of the Seventeenth International Conference on Machine Learning. 2000.

Laurent G.J. et al. The world of independent learners is not Markovian // International Journal of Knowledge-based and Intelligent Engineering Systems. 2011. Vol. 15. No. 1. P. 55–64.

Leike J. et al. AI safety gridworlds. URL: https://arxiv.org/abs/1711.09883 (дата обращения 01.11.2020).

Littman M.L. Value-function reinforcement learning in Markov games // Cognitive systems research. 2001. Vol. 2. No. 1. P. 55–66.

Matignon L., Laurent G.J., Le Fort-Piat N. Independent reinforcement learners in cooperative Markov games: a survey regarding coordination problems // Knowledge Engineering Review. 2012. Vol. 27, No. 1. P. 1–31.

Pollack M.E., Ringuette M. Introducing the Tileworld: Experimentally evaluating agent architectures // AAAI. 1990. Vol. 90. P. 183–189.

Schwartz H.M. Multi-agent machine learning: A reinforcement approach. NY: John Wiley & Sons, 2014. 264 p.

Sutton R.S., Barto A.G. Reinforcement learning: An introduction. Cambridge: MIT press, 2018. 548 p.

Tan M. Multi-agent reinforcement learning: Independent vs. cooperative agents // Proceedings of the tenth international conference on machine learning. 1993. P. 330–337.

Tesauro G. Extending Q-learning to general adaptive multi-agent systems // Advances in neural information processing systems. 2003. Vol. 16. P. 871–878.

Tuyls K., Weiss G. Multiagent learning: Basics, challenges, and prospects // AI Magazine. 2012. Vol. 33. No. 3. P. 41–41.

Verma T., Varakantham P., Lau H.C. Entropy based independent learning in anonymous multi-agent settings // Proceedings of the International Conference on Automated Planning and Scheduling. 2019. Vol. 29. No. 1. P. 655–663.

Whiteson S. Learning with Opponent — Learning Awareness // Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems. 2018.

Zhao Y. et al. Winning Isn't Everything: Enhancing Game Development with Intelligent Agents // IEEE Transactions on Games. 2020. Vol. 12. No. 2. P. 199–212.

Zinkevich M. Online convex programming and generalized infinitesimal gradient ascent // Proceedings of the 20th international conference on machine learning. 2003. P. 928–936.

К главе 2

Abouheaf M. Optimization and reinforcement learning techniques in multi-agent graphical games and economic dispatch: PhD thesis. The University of Texas at Arlington, 2012. 213 p.

Beck-Courcelle D. et al. Study of Multiple Multiagent Reinforcement Learning Algorithms in Grid Games: MASc thesis. Carleton University, 2013. 109 p.

Berger U. Brown's original fictitious play // Journal of Economic Theory. 2007. Vol. 135. No. 1. P. 9.

Bowling M. Multiagent learning in the presence of agents with limitations: PhD thesis. Carnegie Mellon University, 2003. P. 172.

Bowling M., Veloso M. Multiagent learning using a variable learning rate // Artificial Intelligence. 2002. Vol. 136. No. 2. P. 215–250.

Bowling M., Veloso M. Rational and Convergent Learning in Stochastic Games // International joint conference on artificial intelligence. 2001. Vol. 17. No. 1. P. 1021–1026.

Brau B.C. An Exploration of Multi-agent Learning Within the Game of Sheephead: MSc thesis. Minnesota: State University, 2011. 79 p.

Busoniu L. et al. Multi-agent reinforcement learning: A survey // 9th International Conference on Control, Automation, Robotics and Vision. 2006. P. 7.

Busoniu L. et al. Multi-agent Reinforcement Learning: An Overview // Innovations in Multi-Agent Systems and Applications. 2010. No. 1. P. 183–221.

Casgrain P. et al. Deep Q-Learning for Nash Equilibria: Nash-DQN. URL: https://arxiv.org/abs/1904.10554 (дата обращения 01.11.2020).

Claus C., Boutilier C. The dynamics of reinforcement learning in cooperative multiagent systems // AAAI/IAAI. 1998. Vol. 1. P. 7.

Conitzer V., Sandholm T. AWESOME: A general multiagent learning algorithm that converges in self-play and learns a best response against stationary opponents // Machine Learning. 2007. No. 67. P. 23–43.

Hernandez-Leal P. et al. A survey of learning in multiagent environments: Dealing with non-stationarity. URL: https://arxiv.org/abs/1707.09183. 2019 (дата обращения 01.11.2020).

Hu J., Wellman M.P. Multiagent reinforcement learning: Theoretical framework and an algorithm // ICML. 1998. Vol. 98. P. 242–250.

Hu J., Wellman M.P. Nash Q-Learning for General-Sum Stochastic Games // Journal of machine learning research. 2003. No. 4. P. 1039–1069.

Kapetanakis S., Kudenko D. Reinforcement Learning of Coordination in Cooperative Multi-agent Systems // AAAI/IAAI. 2002. Vol. 1. P. 6.

Lemke C.E., Howson J.T. Equilibrium Points of Bimatrix Games // Journal of the Society for Industrial and Applied Mathematics. 1964. Vol. 12. No. 2. P. 413–423.

Littman M.L. Markov games as a framework for multi-agent reinforcement learning // Machine learning proceedings. 1994. P. 157–163.

Littman M.L., Stone P. Implicit Negotiation in Repeated Games // International Workshop on Agent Theories, Architectures, and Languages. 2001. P. 393–404.

Liu T. et al. Game Theoretic Control of Multiagent Systems // SIAM Journal on Control and Optimization. 2019. Vol. 57. No. 3. P. 1691–1709.

Lu X. Multi-Agent Reinforcement Learning in Games: PhD thesis. Carleton University, 2012. 188 p.

Nowé A. et al. Game Theory and Multi-agent Reinforcement Learning // Reinforcement Learning. 2012. P. 441–470.

Park Y.J. et al. Multi-agent reinforcement learning with approximate model learning for competitive games // PLoS ONE. 2019. Vol. 14. No. 9. P. 21.

Powers R., Shoham Y. New Criteria and a New Algorithm for Learning in Multi-Agent Systems // Advances in Neural Information Processing Systems. 2004. P. 8.

Qiao H. Multiagent learning with bargaining — a game theoretic approach: PhD thesis. University of Arizona, 2007. P. 112.

Roberson B. The Colonel Blotto game // Economic Theory. Vol. 29. No. 1. 2006. P. 24.

Rokhlin D.B. Q-learning in a stochastic stackelberg game between an uninformed leader and a naïve follower // Theory Probability Application. 2019. Vol. 64. No. 1. P. 41–58.

Roughgarden T. et al. Algorithmic game theory // Communications of the ACM. 2010. Vol. 53. No. 7. P. 775.

Schwartz H. Multi-Agent Machine Learning: A Reinforcement Approach. NY: John Wiley & Sons, 2014. 264 p.

Sheppard J.W. Multi-agent reinforcement learning in markov games: PhD thesis. The Johns Hopkins University, 1998. 268 p.

Shwartz A., Makowski M.A. Comparing Policies in Markov Decision Processes: Mandl's Lemma Revisited // Mathematics of Operations Research. 1990. Vol. 15. No. 1. P. 155–174.

Singh S.P. et al. Nash Convergence of Gradient Dynamics in General-Sum Games // Uncertainty in artificial intelligence proceedings. 2000. P. 541—548.

Tadelis S. Game Theory An Introduction. Princeton: Princeton University Press, 2013. 416 p.

Tuyls K., Weiss G. Multiagent Learning: Basics, Challenges, and Prospects // AI Magazine. 2012. Vol. 33. No. 3. P. 41–52.

Uther W., Veloso M. Adversarial Reinforcement Learning // Proceedings of the AAAI Fall Symposium on Model Directed Autonomous Systems. 1997. P. 22.

Wei E. Learning to play cooperative games via reinforcement learning: PhD thesis. George Mason University. 2018. 164 p.

Wheeler Jr., R. Narendra K. Decentralized Learning in Finite Markov Chains // IEEE Transactions on Automatic Control. 1986. Vol. 31. No. 6. P. 519–526.

Xu D. An integrated simulation, learning and game-theoretic framework for supply chain competition: PhD thesis. The University of Arizona, 2014. P. 233.

Yang Z. et al. A theoretical analysis of deep Q-learning. URL: https://arxiv.org/abs/1901.00137v3 (дата обращения 01.11.2020).

К главе 3

Baker B., Kanitscheider I., Markov T., Wu Y. Emergent Tool Use From Multi-Agent Autocurricula. URL: https://arxiv.org/abs/1909.07528 (дата обращения 01.11.2020).

Cassandra A. A Survey of POMDP Applications // AAAI. 1998. Vol. 1724. P. 1–9.

Foerster J. Deep multi-agent reinforcement learning DeepMARL: PhD thesis. University of Oxford. 2018. URL: https://ora.ox.ac.uk/objects/uuid:a55621b3-53c0-4e1b-ad1c-92438b57ffa4 (дата обращения 01.11.2020).

Foerster J., Assael I., Freitas N., Whiteson S. Learning to Communicate with Deep Multi-Agent Reinforcement Learning // NIPS. 2017. Vol. 29. P. 1—13.

Foerster J., Chen R., Al-Shedivat M., Whiteson S. Learning with Opponent-Learning Awareness. URL: https://arxiv.org/abs/1709.04326 (дата обращения 01.11.2020).

Foerster J., Farquhar G., Afouras T., Nardelli N. Counterfactual multi-agent policy gradients. URL: https://arxiv.org/abs/1705.08926 (дата обращения 01.11.2020).

Foerster J., Nardelli N., Farquhar G., Afouras T. Stabilising experience replay for deep multi-agent reinforcement learning. URL: https://arxiv.org/abs/1702.08887 (дата обращения 01.11.2020).

Gupta J., Egorov M., Kochenderfer M. Cooperative multi-agent control using deep reinforcement learning // International Conference on Autonomous Agents and Multiagent Systems. 2017. P. 66–83.

Hausknecht M.J. Cooperation and communication in multiagent deep reinforcement learning: PhD thesis. The University of Texas at Austin, 2016. 169 p.

Hausknecht M., Stone P. Deep Recurrent Q-Learning for Partially Observable MDPs. URL: https://arxiv.org/abs/1507.06527 (дата обращения 01.11.2020).

He H., Boyd-Graber J., Kwok K., Daume H. Opponent Modeling in Deep Reinforcement Learning. URL: https://arxiv.org/abs/1609.05559 (дата обращения 01.11.2020).

Hernandez-Leal P., Kartal B., Taylor M. A survey and critique of multiagent deep reinforcement learning // Autonomous Agents and Multi-Agent Systems. 2019. Vol. 33. P. 750–797.

Hernandez-Leal P., Kartal B., Taylor M. Is multiagent deep reinforcement learning the answer or the question? A brief survey. URL: https://arxiv.org/abs/1810.05587v2 (дата обращения 01.11.2020).

Hessel M., Modayil J., Hasselt H.V., Schaul T. Rainbow: Combining Improvements in Deep Reinforcement Learning. URL: https://arxiv.org/abs/1710.02298 (дата обращения 01.11.2020).

Hong Z., Su S., Shann T., Chang Y. A Deep Policy Inference Q-Network for Multi-Agent Systems. URL: https://arxiv.org/abs/1712.07893 (дата обращения 01.11.2020).

Jaakkola T., Singh S., Jordan M. Reinforcement learning algorithm for partially observable Markov decision problems // NIPS. 1994. Vol. 4. P. 345–352.

Konda V., Tsitsiklis J. Actor-Critic Algorithms // NIPS. 2000. Vol. 13. P. 1008–1014.

Lapan M. Deep Reinforcement Learning Hands-On: Apply modern RL methods, with deep Q-networks, value iteration, policy gradients, TRPO, AlphaGo Zero and more. Birmingham: Packt Publishing Ltd, 2020. 827 p.

Lazaridou A., Peysakhovich A., Baroni M. Multi-Agent Cooperation and the Emergence of (Natural) Language. URL: https://arxiv.org/abs/1612.07182 (дата обращения 01.11.2020).

Lerer A., Peysakhovich A. Maintaining cooperation in complex social dilemmas using deep reinforcement learning. URL: https://arxiv.org/abs/1707.01068 (дата обращения 01.11.2020).

Lillicrap T., Hunt J., Pritzel A., Heess N. Continuous control with deep reinforcement learning. URL: https://arxiv.org/abs/1509.02971 (дата обращения 01.11.2020).

Liu Y., Wang W., Hu Y., Hao J. Multi-Agent Game Abstraction via Graph Attention Neural Network. URL: https://arxiv.org/abs/1911.10715v1 (дата обращения 01.11.2020).

Lowe R., Wu Y., Tamar A., Harb J. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. URL: https://arxiv.org/abs/1706.02275 (дата обращения 01.11.2020).

Matignon L., Laurent G.J., Le Fort-Piat N. Independent reinforcement learners in cooperative Markov games a survey regarding coordination problems // CUP. 2012. Vol. 27. No. 1. P. 1–31.

Mnih V., Kavukcuoglu K., Silver D., Graves A. Playing Atari with Deep Reinforcement Learning. URL: https://arxiv.org/abs/1312.5602 (дата обращения 01.11.2020).

Mnih V., Kavukcuoglu K., Silver D., Rusu A. Human-Level Control Through Deep Reinforcement Learning // Nature. 2015. Vol. 518. P. 529–533.

Oliehoek F., Amato C. A concise introduction to decentralized POMDPs. Berlin: Springer, 2016. 141 p.

Omidshafiei S., Pazis J., Amato C., How J. Deep decentralized multi-task multi-agent reinforcement learning under partial observability. URL: https://arxiv.org/abs/1703.06182 (дата обращения 01.11.2020).

Oroojlooy Jadid A., Hajinezhad D. A Review of Cooperative Multi-Agent Deep Reinforcement Learning. URL: https://arxiv.org/abs/1908.03963 (дата обращения 01.11.2020).

Palmer G., Tuyls K., Bloembergen D., Savani R. Lenient multi-agent deep reinforcement learning. URL: https://arxiv.org/abs/1707.04402 (дата обращения 01.11.2020).

Panait L., Sullivan K., Luke S. Lenient learners in cooperative multiagent systems // Proceedings of the fifth international joint conference on Autonomous agents and multiagent systems. 2006. P. 801–803.

Pham H.N.A., Triantaphyllou E. The Impact of Overfitting and Over-generalization on the Classification Accuracy in Data Mining: PhD thesis. Louisiana State University, 2011. 127 p.

Puigdomènech A., Piot B., Kapturowski S., Sprechmann P. Agent57: Outperforming the Atari Human Benchmark. URL: https://arxiv.org/abs/2003.13350 (дата обращения 01.11.2020).

Rashid T., Samvelyan M., Witt C., Farquhar G. QMIX: Monotonic Value Function Factorisation for DeepMulti-Agent Reinforcement Learning. URL: https://arxiv.org/abs/1803.11485 (дата обращения 01.11.2020).

Schmidhuber J., Hochreiter S. Long short-term memory // Neural Pomputation. 1997. Vol. 9. No. 8. P. 1735–1780.

Schrittwieser J., Antonoglou I., Hubert T., Simonyan K. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. URL: https://arxiv.org/abs/1911.08265 (дата обращения 01.11.2020).

Schulman J., Levine S., Abbeel P., Jordan M. Trust region policy optimization // PMLR. 2015. Vol. 37. P. 1889–1897.

Silver D., Hubert T., Schrittwieser J., Antonoglou I. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play // Science. 2018. Vol. 362. P. 1140–1144.

Silver D., Lever G., Heess N., Degris T. Deterministic policy gradient algorithms // ICML. 2014. Vol. 32. P. 387–395.

Silver D., Schrittwieser J., Simonyan K., Antonoglou I. Mastering the game of Go without human knowledge // Nature. 2017. Vol. 550. P. 354–359.

Son K., Kim D., Kang W.J., Hostallero D.E., Yi Y. QTRAN: Learning to Factorize with Transformation for Cooperative Multi-Agent Reinforcement Learning. URL: https://arxiv.org/abs/1905.05408 (дата обращения 01.11.2020).

Sukhbaatar S., Szlam A., Fergus R. Learning Multiagent Communication with Backpropagation. URL: https://arxiv.org/abs/1605.07736 (дата обращения 01.11.2020).

Sunehag P., Lever G., Gruslys A., Czarnecki W.M. Value-decomposition networks for cooperative multi-agent learning based on team reward // AAMAS. 2018. Vol. 17. P. 2085–2087.

Tampuu A., Matiisen T., Kodelja D., Kuzovkin I. Multiagent cooperation and competition with deep reinforcement learning // PLoS One. 2017. Vol. 12. No. 4.

Usunier N., Synnaeve G., Lin Z., Chintala S. Episodic Exploration for Deep Deterministic Policies: An Application to StarCraft Micromanagement Tasks. URL: https://arxiv.org/abs/1609.02993 (дата обращения 01.11.2020).

Vinyals O., Babuschkin I., Czarnecki W., Mathieu M. Grandmaster level in StarCraft II using multi-agent reinforcement learning // Nature. 2019. Vol. 575. P. 350–354.

Wang W., Yang T., Liu L., Hao J. From Few to More: Large-Scale Dynamic Multiagent Curriculum Learning. URL: https://arxiv.org/abs/1909.02790v2 (дата обращения 01.11.2020).

Wei E., Luke S. Lenient learning in independent-learner stochastic cooperative games // The Journal of MLR. 2016. Vol. 17. No. 1.

Werbos P.J. Backpropagation through time: what it does and how to do it // IEEE. 1990. Vol. 78. No. 10. P. 1550–1560.

Zheng Y., Meng Z., Hao J., Zhang Z., Yang T. A deep bayesian policy reuse approach against non-stationary agents // NIPS. 2018. Vol. 1. P. 962–972.

К главе 4

Карпенко А.П. Современные алгоритмы поисковой оптимизации. Алгоритмы, вдохновленные природой: учебное пособие. М.: Изд-во МГТУ им. Н.Э. Баумана, 2017. 446 с.

Almufti S., Marqas R., Ashqi V. Taxonomy of bio-inspired optimization algorithms // Journal of Advanced Computer Science & Technology. 2019. Vol. 8. No. 2. P. 23–31.

Baar W., Bauso D. Networked Bio-Inspired Evolutionary Dynamics on a Multi-Population // 18th European Control Conference. 2019. P. 1023–1028.

De Jong K. Evolutionary computation: a unified approach // Proceedings of the Genetic and Evolutionary Computation Conference Companion. 2020. P. 327–342.

Ficici S.G., Pollack J.B. Pareto optimality in coevolutionary learning // European Conference on Artificial Life. Berlin: Springer, 2001. P. 316–325.

Gill S.S., Buyya R. Bio-inspired algorithms for big data analytics: a survey, taxonomy, and open challenges // Big Data Analytics for Intelligent Healthcare Management. NY: Academic Press, 2019. P. 1–17.

Gomes J., Mariano P., Christensen A.L. Cooperative coevolution of partially heterogeneous multiagent systems // Proceedings of the International Conference on Autonomous Agents and Multiagent Systems. 2015. P. 297–305.

Gomes J., Mariano P., Christensen A.L. Dynamic team heterogeneity in cooperative coevolutionary algorithms // IEEE Transactions on Evolutionary Computation. 2017. Vol. 22. No. 6. P. 934–948.

Jacob C. et al. Illustrating evolutionary computation with Mathematica. Voltem: Morgan Kaufmann, 2001. 547 p.

Jaderberg M. et al. Population based training of neural networks. URL: https://arxiv.org/abs/1711.09846 (дата обращения 01.11.2020).

Khalid M.A., Yusof U., Aman K.K. A survey on bio-inspired multi-agent system for versatile manufacturing assembly line // ICIC Express Letters. 2016. Vol. 10. No. 1. P. 1–7.

Knudson M., Tumer K. Coevolution of heterogeneous multi-robot teams // Proceedings of the 12th annual conference on Genetic and evolutionary computation. 2010. P. 127–134.

Leibo J.Z. et al. Malthusian reinforcement learning. URL: https://arxiv.org/abs/1812.07019 (дата обращения 01.11.2020).

Levin S.A., Udovic J.D. A mathematical model of coevolving populations // The American Naturalist. 1977. Vol. 111. No. 980. P. 657–675.

Li Z., Liu J. A multi-agent genetic algorithm for community detection in complex networks // Physica A: Statistical Mechanics and its Applications. 2016. Vol. 449. P. 336–347.

Liekens A.M.L., ten Eikelder H.M.M., Hilbers P.A.J. Finite population models of co-evolution and their application to haploidy versus diploidy // Genetic and Evolutionary Computation Conference. Berlin: Springer, 2003. P. 344–355.

Liu S. et al. Emergent coordination through competition. URL: https://arxiv.org/abs/1902.07151(дата обращения 01.11.2020).

Miikkulainen R. et al. Multiagent learning through neuroevolution // IEEE World Congress on Computational Intelligence. Berlin: Springer, 2012. P. 24–46.

Moriarty D.E., Miikkulainen R. Forming neural networks through efficient and adaptive coevolution // Evolutionary computation. 1997. Vol. 5. No. 4. P. 373–399.

Omidshafiei S. et al. α-rank: Multi-agent evaluation by evolution // Scientific reports. 2019. Vol. 9. No. 1. P. 1–29.

Panait L., Luke S. Cooperative multi-agent learning: The state of the art // Autonomous agents and multi-agent systems. 2005. Vol. 11. No. 3. P. 387–434.

Panait L., Sullivan K., Luke S. Lenience towards teammates helps in cooperative multiagent learning // Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multi Agent Systems. 2006. P. 1–10.

Paredis J. Coevolutionary computation // Artificial life. 1995. Vol. 2. No. 4. P. 355–375.

Peng Z., Wu J., Chen J. Three-dimensional multi-constraint route planning of unmanned aerial vehicle low-altitude penetration based on coevolutionary multi-agent genetic algorithm // Journal of Central South University of Technology. 2011. Vol. 18. No. 5. P. 1502.

Rockefeller G., Khadka S., Tumer K. Multi-level Fitness Critics for Cooperative Coevolution // Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems. 2020. P. 1143–1151.

Rossi F. et al. Review of multi-agent algorithms for collective behavior: a structural taxonomy // IFAC-PapersOnLine. 2018. Vol. 51. No. 12. P. 112–117.

Schmitt L.M. Theory of coevolutionary genetic algorithms // International Symposium on Parallel and Distributed Processing and Applications. Berlin: Springer, 2003. P. 285–293.

Schmitt L.M. Theory of genetic algorithms // Theoretical Computer Science. 2001. Vol. 259. No. 1–2. P. 1–61.

Seredynski F. Coevolutionary multi-agent systems: the application to mapping and scheduling problems // Proceedings of the IEEE International Conference on Industrial Technology. 1996. P. 431–435.

Seredynski F. Competitive coevolutionary multi-agent systems: The application to mapping and scheduling problems // Journal of Parallel and Distributed Computing. 1997. Vol. 47. No. 1. P. 39–57.

Singh S. et al. Intrinsically motivated reinforcement learning: An evolutionary perspective // IEEE Transactions on Autonomous Mental Development. 2010. Vol. 2. No. 2. P. 70–82.

Singh S., Lewis R.L., Barto A.G. Where do rewards come from // Proceedings of the annual conference of the cognitive science society. 2009. P. 2601–2606.

Srivastava V., Leonard N.E. Bio-inspired decision-making and control: From honeybees and neurons to network design // American Control Conference. 2017. P. 2026–2039.

Stella L. Bio-Inspired Collective Decision-Making in Game Theoretic Models and Multi-Agent Systems: PhD thesis. University of Sheffield, 2019. 141 p.

Stone P., Veloso M. Multiagent systems: A survey from a machine learning perspective // Autonomous Robots. 2000. Vol. 8. No. 3. P. 345–383.

Uriot T., Izzo D. Safe Crossover of Neural Networks Through Neuron Alignment. URL: https://arxiv.org/abs/2003.10306 (дата обращения 01.11.2020).

Wang Y., Qi Y., Li Y. Memory-based multiagent coevolution modeling for robust moving object tracking // The Scientific World Journal. 2013. Vol. 2013. P. 1—13.

Whiteson S., Stone P. Evolutionary function approximation for reinforcement learning // Journal of Machine Learning Research. 2006. Vol. 7. P. 877–917.

Wiegand R.P., Liles W.C., De Jong K.A. Modeling Variation in Cooperative Coevolution Using Evolutionary Game Theory // 7th International Workshop Foundations of Genetic Algorithms. 2002. P. 203–220.

Yang Z., Tang K., Yao X. Large scale evolutionary optimization using cooperative coevolution // Information sciences. 2008. Vol. 178. No. 15. P. 2985–2999.

Yong C.H., Miikkulainen R. Coevolution of role-based cooperation in multiagent systems // IEEE Transactions on Autonomous Mental Development. 2009. Vol. 1. No. 3. P. 170–186.

К главе 5

Aubret A., Matignon L., Hassas S. A survey on intrinsic motivation in reinforcement learning. URL: https://arxiv.org/abs/1908.06976(дата обращения 01.11.2020).

Bellemare M. et al. Unifying count-based exploration and intrinsic motivation // NIPS. 2016. Vol. 29. P. 1471–1479.

Chakraborty A., Kar A.K. Swarm intelligence: A review of algorithms // Nature-Inspired Computing and Optimization. Berlin: Springer, 2017. P. 475–494.

Coppola M. et al. A Survey on Swarming With Micro Air Vehicles: Fundamental Challenges and Constraints // Frontiers in Robotics and AI. 2020. Vol. 7. P. 18.

Dorigo M., Blum C. Ant colony optimization theory: A survey // Theoretical computer science. 2005. Vol. 344. No. 2–3. P. 243–278.

Dorigo M., Stützle T. Ant colony optimization: overview and recent advances // Handbook of metaheuristics. Berlin: Springer, 2019. P. 311–351.

Gambardella L.M., Dorigo M. Ant-Q: A reinforcement learning approach to the traveling salesman problem // Machine Learning Proceedings. 1995. P. 252–260.

GhasemAghaei R. et al. Ant colony-based reinforcement learning algorithm for routing in wireless sensor networks // Instrumentation & Measurement Technology Conference. 2007. P. 1–6.

Hüttenrauch M. et al. Deep reinforcement learning for swarm systems // Journal of Machine Learning Research. 2019. Vol. 20. No. 54. P. 1–31.

Hüttenrauch M., Šošić A., Neumann G. Local communication protocols for learning complex swarm behaviors with deep reinforcement learning // International Conference on Swarm Intelligence. Berlin: Springer, 2018. P. 71–83.

Khan A. et al. Collaborative multiagent reinforcement learning in homogeneous swarms. URL: https://openreview.net/pdf?id=ByeDojRcYQ (дата обращения 01.11.2020).

Kho L.C. et al. Ant colony optimization for 2 satisfiability in restricted neural symbolic integration // AIP Conference Proceedings. 2020. Vol. 2266. No. 1. P. 050006.

Matta M. et al. Q-RTS: a real-time swarm intelligence based on multi-agent Q-learning // Electronics Letters. 2019. Vol. 55. No. 10. P. 589–591.

Oh K.K., Park M.C., Ahn H.S. A survey of multi-agent formation control // Automatica. 2015. Vol. 53. P. 424–440.

Ostrovski G. et al. Count-based exploration with neural density models. URL: https://arxiv.org/abs/1703.01310 (дата обращения 01.11.2020).

Schranz M. et al. Swarm Robotic Behaviors and Current Applications // Frontiers in Robotics and AI. 2020. Vol. 7. P. 36.

Singh P. et al. Swarm Intelligence Algorithms: A Tutorial. Boca Raton: CRC press, 2020. 363 p.

Socha K., Blum C. An ant colony optimization algorithm for continuous optimization: application to feed-forward neural network training // Neural Computing and Applications. 2007. Vol. 16. No. 3. P. 235–247.

Šošic A. et al. Inverse reinforcement learning in swarm systems // Proceedings of the 1st Workshop on Transferin Reinforcement Learning at the 16th International Conference on Autonomous Agents and Multiagent Systems. Sao Paulo, Brazil, 2017. P. 17.

Rizk Y., Awad M., Tunstel E.W. Decision making in multiagent systems: A survey // IEEE Transactions on Cognitive and Developmental Systems. 2018. Vol. 10. No. 3. P. 514–529.

Tarassov V.B., Gapanyuk Y.E. Complex Graphs in the Modeling of Multi-agent Systems: From Goal-Resource Networks to Fuzzy Metagraphs // Russian Conference on Artificial Intelligence. Berlin: Springer, 2020. P. 177–198.

Yang X.S. (ed.). Nature-Inspired Computation and Swarm Intelligence: Algorithms, Theory and Applications. NY: Academic Press, 2020.

Zhou S., Yan S. The stability analysis for a class of multi-agent group formation with input saturation constraints // Proceedings of Chinese Guidance, Navigation and Control Conference. 2014. P. 1618–1623.

Оглавление

Предисловие 3

Введение 5

Глава 1. Независимое табличное обучение 9
1.1. Классификация 9
1.2. Модель 11
1.3. Алгоритм 14
1.4. Карта 16
1.5. Технология 22
1.6. Код 23
1.6.1. Алгоритм независимого табличного Q-обучения 23
1.6.2. Тестирование Q-таблицы 27
1.7. Эксперимент 28
1.8. Выводы 31
1.9. Задачи для самоконтроля 32

Глава 2. Обучение в матричных и стохастических играх 34
2.1. Классификация 34
2.2. Модель 37
2.2.1. Матричные игры 37
2.2.2. Стохастические игры 39
2.3. Алгоритм 42
2.3.1. Поиск экстремума стратегий (PHC) 42
2.3.2. «Выигрывай или учись быстро» (WoLF-PHC) 44
2.3.3. Q-обучение Нэша (Nash-Q) 46
2.4. Карта 48
2.5. Технология 51
2.6. Код 52
2.6.1. Алгоритм WoLF-PHC 52
2.6.2. Алгоритм Nash-Q 57
2.7. Эксперимент 66
2.7.1. Матричные игры 66
2.7.2. Стохастические игры 69
2.8. Выводы 70
2.9. Задачи для самоконтроля 70

Глава 3. Нейросетевое обучение 73
3.1. Классификация 73
3.2. Модель 78
3.2.1. Глубокое Q-обучение 78
3.2.2. Децентрализованные частично наблюдаемые марковские процессы принятия решений (Dec-POMDP) 79
3.2.3. Двойная декомпозиция Q-значений 81
3.2.4. Глубокий детерминированный градиент стратегий 83
3.3. Алгоритмы 85
3.3.1. Независимое глубокое обучение с использованием полносвязной нейронной сети (IQN) 85
3.3.2. Централизованное обучение с использованием сверточной нейронной сети (CDQN) 90
3.3.3. Декомпозиция Q-значений с использованием рекуррентной нейронной сети (VDN) 92
3.3.4. Мультиагентный глубокий детерминированный градиент стратегий (MADDPG) 96
3.4. Карта 101
3.5. Технология 104
3.6. Код 105
3.6.1. Алгоритм IQN 105
3.6.2. Алгоритм VDN 111
3.6.3. Алгоритм MADDPG 121
3.7. Эксперимент 130
3.7.1. Алгоритм IQN 130
3.7.2. Алгоритм CDQN 132
3.7.3. Алгоритм VDN 133
3.7.4. Алгоритм MADDPG 135
3.8. Выводы 137
3.9. Задачи для самоконтроля 138

Глава 4. Эволюционное обучение 143
4.1. Классификация 143
4.2. Модель 147
4.2.1. Нейроэволюция 147
4.2.2. Коэволюция 149
4.3. Алгоритмы 151
4.3.1. Независимый генетический алгоритм (InGA) 151
4.3.2. Коэволюционный алгоритм (CoE) 153
4.4. Карта 156
4.5. Технология 159
4.6. Код 160
4.6.1. Алгоритм InGA 160
4.6.2. Алгоритм CoE 165

4.7. Эксперимент ... 171
4.7.1. Алгоритм InGA ... 171
4.7.2. Алгоритм CoE ... 173
4.8. Выводы ... 174
4.9. Задачи для самоконтроля ... 175

Глава 5. Роевое обучение ... 179
5.1. Классификация ... 179
5.2. Модель ... 182
5.2.1. Комбинаторная оптимизация ... 182
5.2.2. Роевая марковская модель принятия решений ... 186
5.3. Алгоритм ... 187
5.4. Карта ... 190
5.5. Технология ... 194
5.6. Код ... 195
5.7. Эксперимент ... 202
5.8. Выводы ... 203
5.9. Задачи для самоконтроля ... 204

Заключение ... 207

Литература ... 209
К главе 1 ... 209
К главе 2 ... 210
К главе 3 ... 212
К главе 4 ... 215
К главе 5 ... 218

Учебное издание

Алфимцев Александр Николаевич

Мультиагентное обучение с подкреплением

Редактор *Е.К. Кошелева*
Художник *Я.М. Асинкритова*
Корректор *Л.В. Забродина*
Компьютерная графика *О.В. Левашовой*
Компьютерная верстка *И.Д. Звягинцевой*

Оригинал-макет подготовлен
в Издательстве МГТУ им. Н.Э. Баумана.

В оформлении использованы шрифты
Студии Артемия Лебедева.

Подписано в печать 18.05.2021. Формат 70×100/16.
Усл. печ. л. 18,2. Тираж 181 экз. Изд. № 936-2020. Заказ 394

Издательство МГТУ им. Н.Э. Баумана.
105005, Москва, 2-я Бауманская ул., д. 5, к. 1.
press@baumanpress.ru
https://bmstu.press

Отпечатано в типографии МГТУ им. Н.Э. Баумана.
105005, Москва, 2-я Бауманская ул., д. 5, к. 1.
baumanprint@gmail.com

Рассмотрены современные и классические алгоритмы одновременного машинного обучения множества агентов, основанные на теории игр, табличных, нейросетевых, эволюционных и роевых технологиях. Представлено последовательное развитие теоретической модели алгоритмов, базирующееся на марковских процессах принятия решений. Реализация алгоритмов выполнена на языке программирования Python с использованием библиотеки глубокого обучения PyTorch. Средой машинного обучения является компьютерная игра StarCraft II с интерфейсом кооперативного мультиагентного обучения SMAC.

ISBN 978-5-7038-5616-1

bmstu.press